

Intro to MPI

John Urbanic

Parallel Computing Scientist
Pittsburgh Supercomputing Center

Distinguished Service Professor
Carnegie Mellon University

Quick Note

It turns out that *scp* has been banned from the login nodes for "reasons", and we should use *data.bridges2.psc.edu* for such transfers.

I use a GUI myself (via MobaXterm) for file transfers, so I didn't notice until now.

This is a great opportunity to point out that the Bridges User Guide is a great place to look whenever you have such issues:

[*https://www.psc.edu/resources/bridges-2/user-guide/*](https://www.psc.edu/resources/bridges-2/user-guide/)

End of Moore's Law Will Lead to New Architectures

Non-von
Neumann

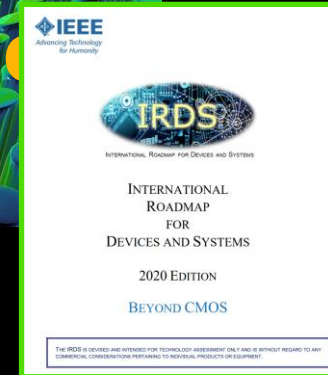
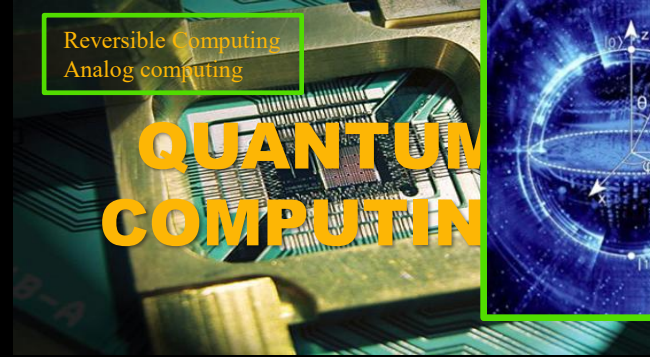
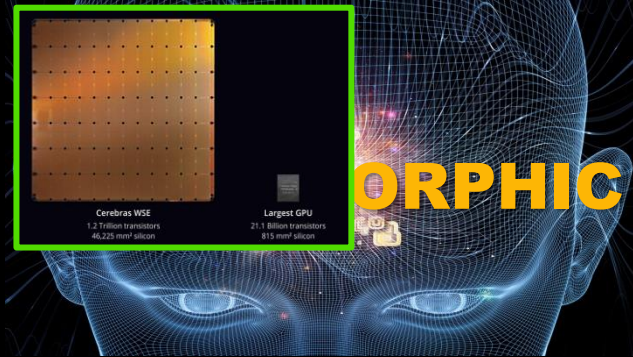
ARCHITECTURE

von Neumann

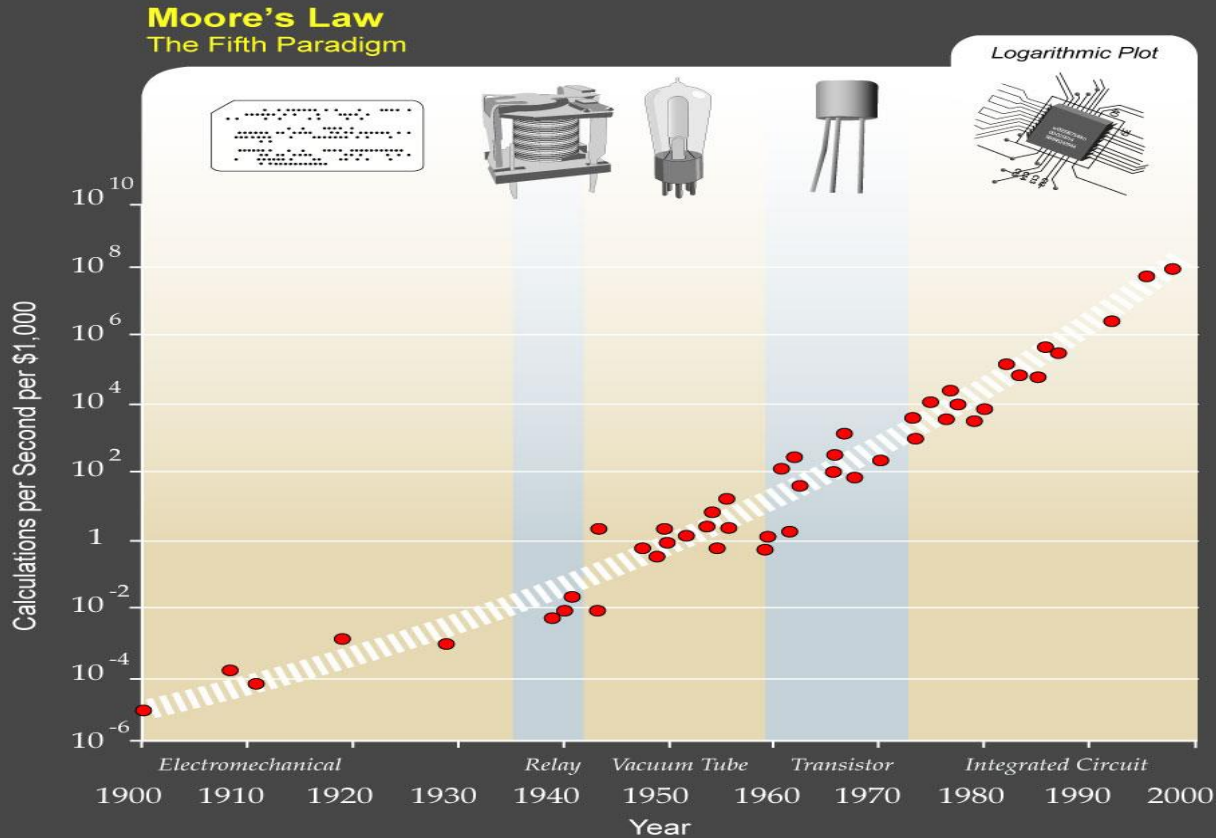
CMOS

Beyond CMOS

TECHNOLOGY



It would only be the 6th paradigm.



We can do better. We have a role model.

- We hope to "simulate" a human brain in real time on one of these Exascale platforms with about 1 - 10 Exaflop/s and 4 PB of memory
- These newest Exascale computers use 20+ MW
- The human brain runs at 20W
- Our brain is a million times more power efficient!

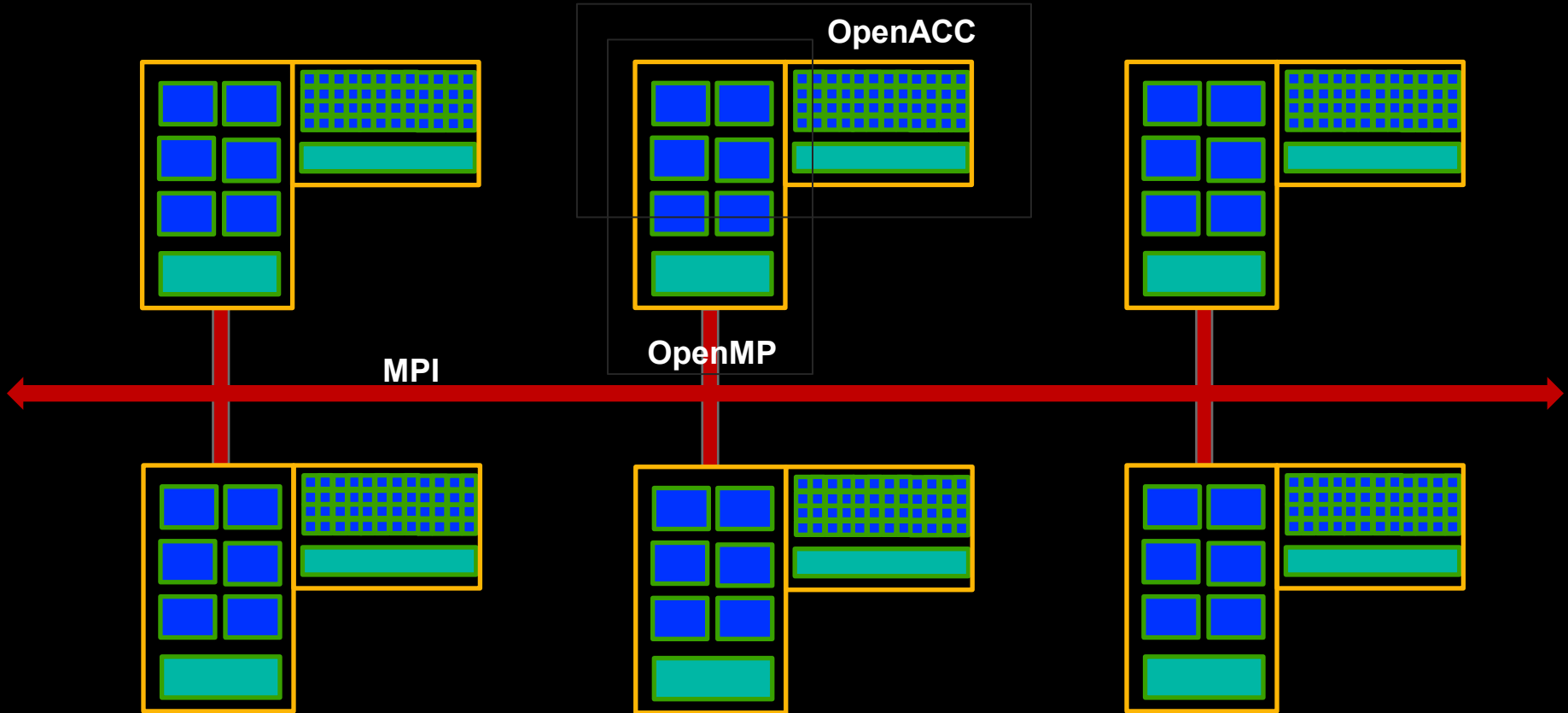


Why you should be (extra) motivated.

- This parallel computing thing is no fad.
- The laws of physics are drawing this roadmap.
- If you get on board (the right bus), you can ride this trend for a long, exciting trip.

Let's learn how to use these things!

In Conclusion...



The Cloud



"A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable."

Leslie Lamport in 1987

"It's really more of a Fog."

Me, just now.

Starting to accelerate.

2024-08-27	Intro	Intro			
2024-08-29	Parallel Programming	Programming Environment			
2024-09-03		Parallel Computing			
2024-09-05		Intro to MPI			
2024-09-10		Laplace	15	Laplace	
2024-09-12		Advanced MPI			
2024-09-17		More Advanced MPI			
2024-09-19	Data Science	Pandas			
2024-09-24		SQL			
2024-09-26		Advanced SQL	5	SQL	
2024-10-01		Big Data			
2024-10-03		Spark			
2024-10-08	Machine Learning	ML with Spark			
2024-10-10		Clustering	10	Astro Signal	
2024-10-22		Recommender			
2024-10-24		Recommender continued			
2024-10-29		6D Data Hunt Assignment / IDEs	10	6D Data Hunt	
2024-10-31	Exam	Exam	20		
2024-11-07		Exam Review			
2024-11-12		6D Data Hunt Review			
2024-11-14	Deep Learning	Intro to TensorFlow			
2024-11-19		TensorFlow	10	Fashion MNIST	
2024-11-21		TensorFlow			
2024-11-26		TensorFlow	10	Newton vs. Machine	
2024-12-03		GPUs			
2024-12-05		Newton vs. Machine Review			
	Final Exam		20		

Start slow

Finish Fast

Introduction to MPI



Pre-Introduction: Why Use MPI?

- Has been around a long time (25+ years)
- Dominant
- Will be around a long time (on all new platforms/roadmaps)
- Lots of libraries
- Lots of algorithms
- Very scalable (3,000,000+ cores right now)
- Portable
- Works with hybrid models
- Explicit parallel routines force the programmer to address parallelization from the beginning, not as an afterthought. This can enable both debugging and memory placement optimization.
- Therefore:
 - A good long term learning investment
 - Useful/possible to understand whether you are coder or a manager

Introduction

What is MPI? The Message-Passing Interface Standard(MPI) is a *library* that allows you to do problems in parallel using *message-passing* to *communicate* between *processes*.

- Library**

It is not a language (like X10 or UPC), or even an extension to a language. Instead, it is a library that your native, standard, serial compiler, or even interpreter (f77, f90, cc, CC, python, etc.) uses.

- Message Passing**

Message passing is sometimes referred “paradigm”. But it is really just any method of explicitly passing data between processes and it is flexible enough to implement most other paradigms (Data Parallel, Work Sharing, etc.). It can be defined as having independent processors, with no shared data, communicate through subroutine calls.

- Communicate**

This communication may be via a dedicated MPP torus network, or merely an office LAN. To the MPI programmer, it looks much the same.

- Processes**

These can be 200,000 cores on Summit, or 40 processes on your laptop. Think “Unix process” and you won’t be far off, although we usually want only 1 process per processor/core so that we actually get a speed up.

Basic MPI

In order to do parallel programming, you require some basic functionality, namely, the ability to:

- Start Processes
- Send Messages
- Receive Messages
- Synchronize

With these four capabilities, you can construct any program. We will look at the basic versions of the MPI routines that implement this capability. MPI-3 offers over 400 functions. These are largely just more convenient and efficient for certain tasks. However, with the handful that we are about to learn, we will be able to implement just about any algorithm, and you will be well-prepared for when we cover much of the remaining routines in the Advanced talk.

First Example (Starting Processes): Hello World

The easiest way to see exactly how a parallel code is put together and run is to write the classic "Hello World" program in parallel. In this case it simply means that every PE will say hello to us. Something like this:

```
mpirun -n 8 python hello.py  
Hello world from PE 0  
Hello world from PE 1  
Hello world from PE 2  
Hello world from PE 3  
Hello world from PE 4  
Hello world from PE 5  
Hello world from PE 6  
Hello world from PE 7
```

Hello World: Python MPI

How complicated is the code to do this? Not very:

```
from mpi4py import MPI  
  
print("Hello world from PE", MPI.COMM_WORLD.Get_rank())
```

"Real" MPI vs. mpi4py

- mpi4py is a clever "wrapper" around the most popular MPI routines, which are implemented and defined for C and Fortran.
- mpi4py is wonderful and convenient. But it is not the massively supported and standardized package that the base MPI specification is. Nor is it nearly as popular or as well documented. And, mpi4py does not implement much of the full MPI API.
- So it will behoove us very much to track with the official MPI specification. We will use the limited mpi4py documentation for our work, and it will be plenty sufficient. But, if you develop more sophisticated applications, require the performance of a compiler-based language, or get involved with mature projects, you will likely migrate to the standard MPI.
- With this approach you can confidently do so.

MPI_COMM_RANK

Get_rank()

MPI_COMM_RANK

This routine returns to every PE its rank, or unique address from 0 to PEs-1. This is the only thing that sets each PE apart from its companions. In this case, the number is merely used to have each PE print a slightly different message. In general, though, the PE number will be used to load different data files or take different branches in the code. It has another argument, the communicator, that we will ignore for a few slides.

What's Happening?

```
from mpi4py import MPI  
print("Hello world from PE",MPI.COMM_WORLD.Get_rank())
```

```
from mpi4py import MPI  
print("Hello world from PE",MPI.COMM_WORLD.Get_rank())
```

```
from mpi4py import MPI  
print("Hello world from PE",MPI.COMM_WORLD.Get_rank())
```

```
from mpi4py import MPI  
print("Hello world from PE",MPI.COMM_WORLD.Get_rank())
```

```
from mpi4py import MPI  
print("Hello world from PE",MPI.COMM_WORLD.Get_rank())
```

```
from mpi4py import MPI  
print("Hello world from PE",MPI.COMM_WORLD.Get_rank())
```

```
from mpi4py import MPI  
print("Hello world from PE",MPI.COMM_WORLD.Get_rank())
```

```
from mpi4py import MPI  
print("Hello world from PE",MPI.COMM_WORLD.Get_rank())
```

```
Hello world from PE 5  
Hello world from PE 2  
Hello world from PE 7  
Hello world from PE 1  
Hello world from PE 0  
Hello world from PE 3  
Hello world from PE 4  
Hello world from PE 6
```


Keep This Picture

```
from mpi4py import MPI  
  
print("Hello world from PE", MPI.COMM_WORLD.Get_rank())
```

```
mpirun -n 8 python hello.py
```



Do all nodes really run the *same* code?

Yes, they do run the same code independently. You might think this is a serious constraint on getting each PE to do unique work. Not at all. They can use their PE numbers to diverge in behavior as much as they like.

The extreme case of this is to have different PEs execute entirely different sections of code based upon their PE number.

```
if (my_PE_num = 0)
    Routine_SpaceInvaders
else if (my_PE_num = 1)
    Routine_CrackPasswords
else if (my_PE_num =2)
    Routine_WeatherForecast
    .
    .
    .
```

So, we can see that even though we have a logical limitation of having each PE execute the same program, for all practical purposes we can really have each PE running an entirely unrelated program by bundling them all into one executable and then calling them as separate routines based upon PE number.

Manager and Worker PEs

The much more common case is to have a single PE that is used for some sort of coordination purpose, and the other PEs run code that is the same, although the data will be different. This is how one would implement a manager/worker paradigm.

```
if (my_PE_num = 0)
    ManagerCodeRoutine
else
    WorkerCodeRoutine
```

Of course, the above Hello World code is the trivial case of

```
EverybodyRunThisRoutine
```

and consequently the only difference will be in the output, as it at least uses the PE number.

An Analogy To Remember

- Think of an MPI program as a book our group is reading. We might all start at the same time, but I wouldn't expect us to all be on the same line at any point.
- Furthermore, some of our books are "choose your own adventures". We might each be taking a different path.
- So, it doesn't make sense to ask what line an MPI program is on.
- I might also ask each of you to rate each chapter. These are analogous to variables; you each have your own copy.
- So it doesn't make sense to ask what the value of any variable is. There are multiple copies.



Communicators

Our last little detail in Hello World is the "communicator" in

```
MPI.COMM_WORLD.Get_rank()
```

This parameter is known as the "communicator" and can be found in many of the MPI routines. In general, it is used so that one can divide up the PEs into subsets for various algorithmic purposes. For example, if we had an array - distributed across the PEs – for which we wished to find the determinant, we could define some subset of the PEs that holds a certain column of the array so that we could address only that column conveniently. Or, we might wish to define a communicator for just the odd PEs. Or just the “ocean” PEs in a weather model. You get the idea.

However, this is a convenience that can be dispensed with in simpler algorithms. As such, one will often see the value MPI_COMM_WORLD when a communicator is required. This is simply the global set and states we don't really care to deal with any particular subset here. We will use it in all of our examples. We will delve into its considerable usefulness in the Advanced talk.

mpirun

To run an MPI executable we must tell the machine how many copies we wish to run at runtime. On Bridges you can choose any number up to the size of your job request. We used 8. The command is `mpirun`:

```
mpirun -n 8 python hello.py
Hello world from PE 0
Hello world from PE 1
Hello world from PE 2
Hello world from PE 3
Hello world from PE 4
Hello world from PE 5
Hello world from PE 6
Hello world from PE 7
```

Which is (almost) what we desired when we started.

Second Example: Sending and Receiving Messages

Hello World might be illustrative, but we haven't really done any message passing yet.

Let's write about the simplest possible message passing program:

It will run on 2 PEs and will send a simple message (the number 42) from PE 1 to PE 0. PE 0 will then print this out.

Sending a Message

Sending a message is a simple procedure. The "standard" pages are in C, so you should get used to seeing this format. Python makes many of these parameters default/optional.

```
MPI_Send( numbertosend, 1, MPI_INT, 0, 10, MPI_COMM_WORLD )
```

numbertosend	whatever we wish to send. In this case it is simply an integer. It could be anything from a character string to a column of an array or a structure. It is even possible to pack several different data types in one message.
1	the number of items we wish to send. If we were sending a vector of 10 int's, we would point to the first one in the above parameter and set this to the size of the array.
MPI_INT	the type of object we are sending. Possible values are: MPI_CHAR, MPI_SHORT, MPI_INT, MPI_LONG, MPI_UNSIGNED_CHAR, MPI_UNSIGNED_SHORT, MPI_UNSIGNED, MPI_UNSIGNED_LONG, MPI_FLOAT, MPI_DOUBLE, MPI_LONG_DOUBLE, MPI_BYTE, MPI_PACKED. We will save that for the advanced talk.
0	Destination of the message. In this case PE 0.
10	Message tag. All messages have a tag attached to them that can be very useful for sorting messages. For example, one could give high priority control messages a different tag than data messages. When receiving, the program would check for messages that use the control tag first. We just picked 10 at random.
MPI_COMM_WORLD	We don't really care about any subsets of PEs here. So, we just chose this "default".

Receiving a Message

Receiving a message is equally simple and very symmetric with MPI_Send (hint: *cut and paste is your friend here*). In our case it will look like:

```
MPI_Recv( numbertoreceive, 1, MPI_INT, MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &status)
```

numbertoreceive	Variable that will receive the item. In our case it is simply an integer that has some undefined value until now.
1	Number of items to receive. Just 1 here.
MPI_INT	Datatype. Better be an int, since that's what we sent.
MPI_ANY_SOURCE	The node to receive from. We could use 1 here since the message is coming from there, but we'll illustrate the "wild card" method of receiving a message from anywhere.
MPI_ANY_TAG	We could use a value of 10 here to filter out any other messages (there aren't any) but, again, this was a convenient place to show how to receive any tag.
MPI_COMM_WORLD	Just using default set of all PEs.
status	A structure that receives the status data which includes the source and tag of the message.

Send and Receive Code

```
from mpi4py import MPI

my_pe_num = MPI.COMM_WORLD.Get_rank()

if my_pe_num==0:
    message = MPI.COMM_WORLD.recv(source=1, tag=0)
    print("Number received is ", message)

else:
    data_to_send = 42
    MPI.COMM_WORLD.send(data_to_send, dest=0, tag=0)
```

Third Example: Synchronization

We are going to write another code which will employ the remaining tool that we need for general parallel programming: synchronization. Many algorithms require that you be able to get all of the nodes into some controlled state before proceeding to the next stage. This is usually done with a synchronization point that requires all of the nodes (or some specified subset at the least) to reach a certain point before proceeding. Sometimes the manner in which messages block will achieve this same result implicitly, but it is often necessary to explicitly do this and debugging is often greatly aided by the insertion of synchronization points which are later removed for the sake of efficiency.

Third Example: Synchronization

Our code will perform the rather pointless operations of:

- 1) Have PE 0 send a number to the other 3 PEs
Foreshadowing:
Broadcast
MPI_Bcast()
- 2) have them multiply that number by their own PE number
- 3) they will then print the results out, in order (remember the hello world program?)
Synchronize
MPI_Barrier()
- 4) and send them back to PE 0
Reduction
MPI_Reduce()
- 5) which will print out the sum.

Synchronization: Python Code

```
from mpi4py import MPI

number_to_send = 4
total = 0

my_pe_num = MPI.COMM_WORLD.Get_rank()

if my_pe_num == 0:
    for destination in range(1,4):
        MPI.COMM_WORLD.send(number_to_send, dest=destination, tag=0)
else:
    number_received = MPI.COMM_WORLD.recv(source=0, tag=0)
    result = number_received * my_pe_num

for pe in range(1,4):
    MPI.COMM_WORLD.barrier()
    if pe == my_pe_num:
        print("PE ", my_pe_num, "'s result is ", result)

if my_pe_num == 0:
    for sender in range(1,4):
        result = MPI.COMM_WORLD.recv(source=sender, tag=0)
        total += result
    print("Total is ", total, "\n")
else:
    MPI.COMM_WORLD.send(result, dest=0, tag=0)
```

Step 1 – Manager, Worker

```
from mpi4py import MPI

number_to_send = 4
total = 0

my_pe_num = MPI.COMM_WORLD.Get_rank()

if my_pe_num == 0:
    for destination in range(1,4):
        MPI.COMM_WORLD.send(number_to_send, dest=destination, tag=0)
else:
    number_received = MPI.COMM_WORLD.recv(source=0, tag=0)
    result = number_received * my_pe_num

for pe in range(1,4):
    MPI.COMM_WORLD.barrier()
    if pe == my_pe_num:
        print("PE ", my_pe_num, "'s result is ", result)

if my_pe_num == 0:
    for sender in range(1,4):
        result = MPI.COMM_WORLD.recv(source=sender, tag=0)
        total += result
    print("Total is ", total, "\n")
else:
    MPI.COMM_WORLD.send(result, dest=0, tag=0)
```

Step 2 – Manager, Worker

```
from mpi4py import MPI

number_to_send = 4
total = 0

my_pe_num = MPI.COMM_WORLD.Get_rank()

if my_pe_num == 0:
    for destination in range(1,4):
        MPI.COMM_WORLD.send(number_to_send, dest=destination, tag=0)
else:
    number_received = MPI.COMM_WORLD.recv(source=0, tag=0)
    result = number_received * my_pe_num

for pe in range(1,4):
    MPI.COMM_WORLD.barrier()
    if pe == my_pe_num:
        print("PE ", my_pe_num, "'s result is ", result)

if my_pe_num == 0:
    for sender in range(1,4):
        result = MPI.COMM_WORLD.recv(source=sender, tag=0)
        total += result
    print("Total is ", total, "\n")
else:
    MPI.COMM_WORLD.send(result, dest=0, tag=0)
```

Step 3: Print in order...

Remember Hello World's random order? What if we did:

```
IF myPE=0 PRINT "Hello from 0."  
IF myPE=1 PRINT "Hello from 1."  
IF myPE=2 PRINT "Hello from 2."  
IF myPE=3 PRINT "Hello from 3."  
IF myPE=4 PRINT "Hello from 4."  
IF myPE=5 PRINT "Hello from 5."  
IF myPE=6 PRINT "Hello from 6."  
IF myPE=7 PRINT "Hello from 7."
```

Would this print in order?

What if one PE was 1,000,000 times faster than another?

Step 3: Print in order...

No. How about?

```
IF myPE=0 PRINT "Hello from 0."  
BARRIER  
IF myPE=1 PRINT "Hello from 1."  
BARRIER  
IF myPE=2 PRINT "Hello from 2."  
BARRIER  
IF myPE=3 PRINT "Hello from 3."  
BARRIER  
IF myPE=4 PRINT "Hello from 4."  
BARRIER  
IF myPE=5 PRINT "Hello from 5."  
BARRIER  
IF myPE=6 PRINT "Hello from 6."  
BARRIER  
IF myPE=7 PRINT "Hello from 7."
```

If you liked our Book Club analogy, this is like saying “everyone stop when you get to Chapter 2, then we can continue.”

Would this print in order?

What if one PE was 1,000,000 times faster than another?

Step 3: Print in order...

Now let's be lazy:

```
FOR X = 0 to 7
  IF MyPE = X
    PRINT "Hello from MyPE."
BARRIER
```

Important Debugging Note!

You may find that Python likes to buffer output. This means that print output might not immediately appear. This can be frustrating when trying to use prints to debug your MPI logic.

The most effective answer is to "use a debugger". We will talk about that later.

For now, if you run into the mysterious missing print output problem, your easiest solution is to request that Python not buffer your output with the flush option:

```
print("Hello", flush=True)
```

Step 3 – Manager, Worker

```
from mpi4py import MPI

number_to_send = 4
total = 0

my_pe_num = MPI.COMM_WORLD.Get_rank()

if my_pe_num == 0:
    for destination in range(1,4):
        MPI.COMM_WORLD.send(number_to_send, dest=destination, tag=0)
else:
    number_received = MPI.COMM_WORLD.recv(source=0, tag=0)
    result = number_received * my_pe_num

for pe in range(1,4):
    MPI.COMM_WORLD.barrier()
    if pe == my_pe_num:
        print("PE ", my_pe_num, "'s result is ", result)

if my_pe_num == 0:
    for sender in range(1,4):
        result = MPI.COMM_WORLD.recv(source=sender, tag=0)
        total += result
    print("Total is ", total, "\n")
else:
    MPI.COMM_WORLD.send(result, dest=0, tag=0)
```

Step 4 – Manager, Worker

```
from mpi4py import MPI

number_to_send = 4
total = 0

my_pe_num = MPI.COMM_WORLD.Get_rank()

if my_pe_num == 0:
    for destination in range(1,4):
        MPI.COMM_WORLD.send(number_to_send, dest=destination, tag=0)
else:
    number_received = MPI.COMM_WORLD.recv(source=0, tag=0)
    result = number_received * my_pe_num

for pe in range(1,4):
    MPI.COMM_WORLD.barrier()
    if pe == my_pe_num:
        print("PE ", my_pe_num, "'s result is ", result)

if my_pe_num == 0:
    for sender in range(1,4):
        result = MPI.COMM_WORLD.recv(source=sender, tag=0)
        total += result
    print("Total is ", total, "\n")
else:
    MPI.COMM_WORLD.send(result, dest=0, tag=0)
```

Step 5 – Manager, Worker

```
from mpi4py import MPI

number_to_send = 4
total = 0

my_pe_num = MPI.COMM_WORLD.Get_rank()

if my_pe_num == 0:
    for destination in range(1,4):
        MPI.COMM_WORLD.send(number_to_send, dest=destination, tag=0)
else:
    number_received = MPI.COMM_WORLD.recv(source=0, tag=0)
    result = number_received * my_pe_num

for pe in range(1,4):
    MPI.COMM_WORLD.barrier()
    if pe == my_pe_num:
        print("PE ", my_pe_num, "'s result is ", result)

if my_pe_num == 0:
    for sender in range(1,4):
        result = MPI.COMM_WORLD.recv(source=sender, tag=0)
        total += result
    print("Total is ", total, "\n")
else:
    MPI.COMM_WORLD.send(result, dest=0, tag=0)
```

Results of “Synchronization”

The output you get when running this codes with 4 PEs is:

```
PE 1's result is 4.  
PE 2's result is 8.  
PE 3's result is 12.  
Total is 24
```

What would happen if you ran with more than 4 PEs? Less?

A Few Words About *MPI_Barrier*

Barriers are often necessary.

Barriers are often convenient. They can help with debugging, and are sometimes put in just to avoid thinking about possible race conditions. These unnecessary barriers can cause wasteful idle time and should be eventually eliminated.

Think of them like a race car driver thinks about brakes. You don't want to use them more than required, but you aren't going to get anywhere without them.

Analysis of “Synchronization”

The best way to make sure that you understand what is happening in the code above is to look at things from the perspective of each PE in turn. THIS IS THE WAY TO DEBUG ANY MESSAGE-PASSING CODE.

Follow from the top to the bottom of the code as PE 0, and do likewise for PE 1. See exactly where one PE is dependent on another to proceed. Look at each PEs progress as though it is 100 times faster or slower than the other nodes. Would this affect the final program flow? It shouldn't unless you made assumptions that are not always valid.

Final Example: Beyond the Basics

You now have the 4 primitives that you need to write any algorithm. However, there are much more efficient ways to accomplish certain tasks, both in terms of typing and computing.

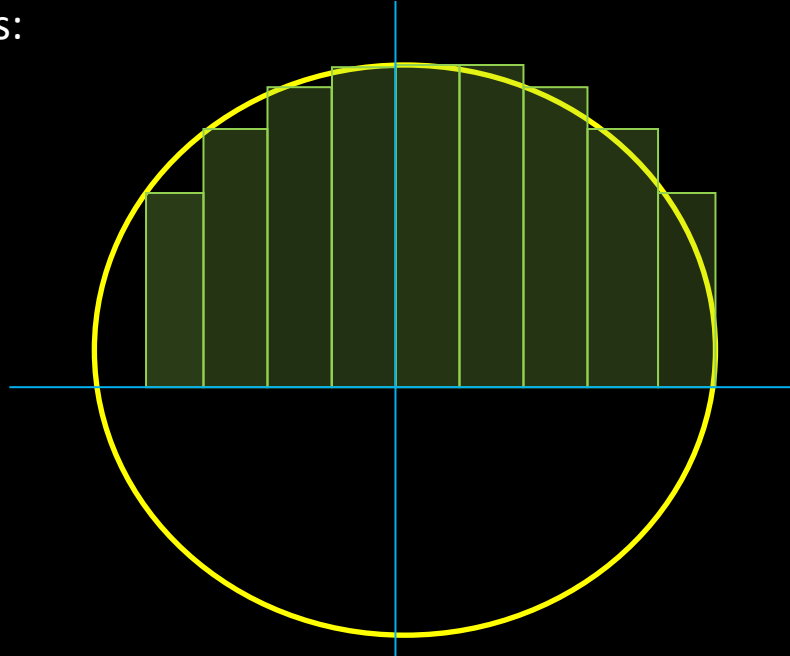
We will look at a few very useful and common routines (reduction, broadcasts, and `Comm_Size`) as we do a final example.

You will then be a full-fledged (or maybe fledgling) MPI programmer.

Final Example: Finding Pi

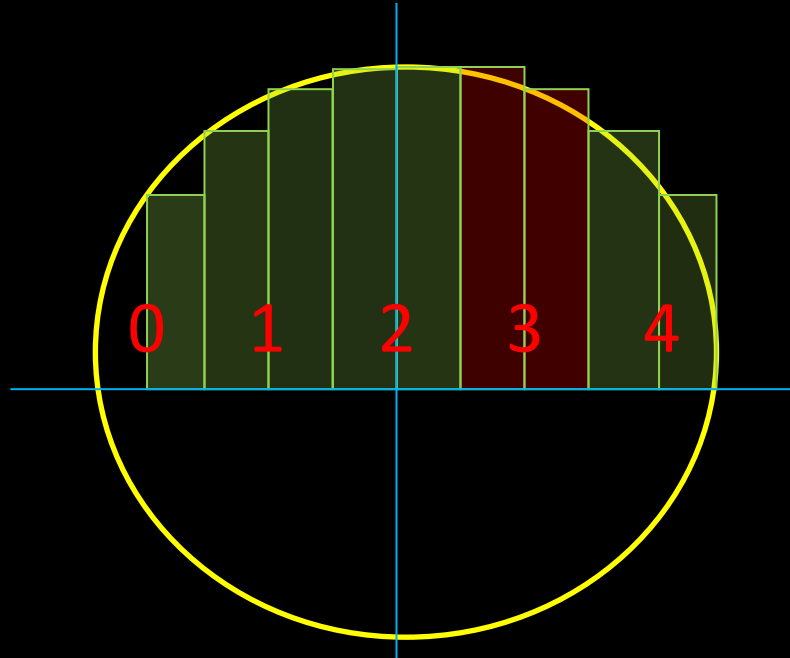
Our last example will find the value of pi by integrating over the area of a circle of radius 1. We just use the fact that the Area = πR^2 and that the equation for such a circle is $x^2 + y^2 = 1$.

We will use the standard (and classic serial) method of finding the area under a curve by adding up a bunch of rectangular slices:



Final Example: Finding Pi

We can parallelize this very effectively by just having a number of PE's work on subsections and add up their final results. With 5 PE's, we should expect it to take roughly $1/5$ as long to reach an answer.



Broadcast

We could easily do this with the commands we have in hand. However, this is a good time to introduce a few of the more common and powerful additional MPI commands: broadcast and reduce.

`MPI_Bcast()` is very useful when we want to communicate some common data to every PE. We know we can do this with a loop, as in the previous example, but with Bcast we not only save some typing and make the code more readable, but we enable the hardware to use a much more efficient broadcast mode. Picture the difference between me giving you each this lecture point-to-point, or “broadcasting” to you all as I am.

We use `MPI_Bcast()` here to let all of the nodes know how many intervals the user would like to employ for the approximation. You will find yourself using it frequently.

Reduce

We also need to add together each PE's partial sum. We just did something like this in the previous example, but once again we find that MPI provides us with a command that saves typing and is makes more efficient use of the hardware.

`MPI_Reduce()` allows us to do this loop with one command. However, it can be used for more than just adding numbers. Other common operations are mins and maxes, but there are plenty more.

Reduce!

This may seem like it is an odd way of getting a sum, or a minimum, but this is a very key concept in data science, and throughout computing in general.

You might think of it as an elegant way to specify many scalable operations by merely specifying how any two elements combine.

But, the advantage it offers as data scales up is why it is an absolutely necessity that you get familiar with the concept.

Note that both Bcast and Reduce require all of the PE's to participate, and require you to designate one particular PE as the sender or collector. This will often be 0 if it is the manager PE.



Laplace Assignment

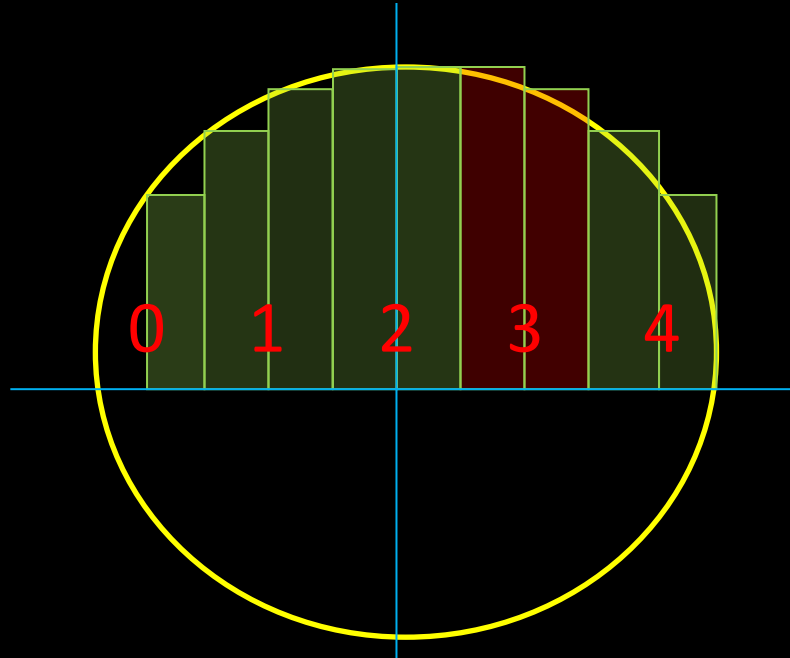
John Urbanic

Parallel Computing Scientist
Pittsburgh Supercomputing Center

Distinguished Service Professor
Carnegie Mellon University

Final Example: Finding Pi

We can parallelize this very effectively by just having a number of PE's work on subsections and add up their final results. With 5 PE's, we should expect it to take roughly $1/5$ as long to reach an answer.



Broadcast

We could easily do this with the commands we have in hand. However, this is a good time to introduce a few of the more common and powerful additional MPI commands: broadcast and reduce.

`MPI_Bcast()` is very useful when we want to communicate some common data to every PE. We know we can do this with a loop, as in the previous example, but with Bcast we not only save some typing and make the code more readable, but we enable the hardware to use a much more efficient broadcast mode. Picture the difference between me giving you each this lecture point-to-point, or “broadcasting” to you all as I am.

We use `MPI_Bcast()` here to let all of the nodes know how many intervals the user would like to employ for the approximation. You will find yourself using it frequently.

Reduce

We also need to add together each PE's partial sum. We just did something like this in the previous example, but once again we find that MPI provides us with a command that saves typing and is makes more efficient use of the hardware.

`MPI_Reduce()` allows us to do this loop with one command. However, it can be used for more than just adding numbers. Other common operations are mins and maxes, but there are plenty more.

Reduce!

This may seem like it is an odd way of getting a sum, or a minimum, but this is a very key concept in data science, and throughout computing in general.

You might think of it as an elegant way to specify many scalable operations by merely specifying how any two elements combine.

But, the advantage it offers as data scales up is why it is an absolutely necessity that you get familiar with the concept.

Note that both Bcast and Reduce require all of the PE's to participate, and require you to designate one particular PE as the sender or collector. This will often be 0 if it is the manager PE.



Finding Pi

```
from mpi4py import MPI
from numpy import arange
from math import sqrt

comm = MPI.COMM_WORLD
my_pe_num = comm.Get_rank()

intervals = 0
if my_pe_num == 0:
    intervals = int(input("How many intervals? "))

intervals = comm.bcast(intervals, root=0)

mypi = 0
h = 2/intervals
start = ( my_pe_num *2 / comm.Get_size()) -1
end = ((my_pe_num+1)*2 / comm.Get_size()) -1

for x in arange(start, end, h):
    mypi = mypi + h * 2* sqrt(1-x*x)

pi = comm.reduce(mypi, op=MPI.SUM, root=0)

if my_pe_num == 0:
    print("Pi is approximately %f" % pi)
    print("Error is %f" % (pi-3.14159265358979323846));
```

Just what you would guess.



Python Dependencies

A combination of language features, community philosophy and typical programmer expertise results in the attitude that grabbing somebody's library from somewhere must always be a good idea. You may even have been taught this, under the banner of "code reuse" or "don't reinvent the wheel".

The end results is that many of the modules that you will install have many layers of dependency. If you have ever done a "pip install" only to have it go off and install dozens of mystery packages, you know what I mean. If you have not, you will.

Every one of these packages has the potential to have a version conflict and there are often cyclic dependencies as well.

So, clearly the answer is a package manager.

How many is a lot?

While it is something of an apples to oranges comparison, there are currently over 62,000 source files in the Linux kernel. And those files and libraries are interrelated with dependencies. So these things are manageable at large scale with proper engineering.

Package Management

"All problems in computer science can be solved by another level of indirection."

— Butler Lampson

"Except for too many layers of indirection."

Lampson's Law strikes many places in the software world. Increasingly so. But maybe few places as severely as the Python ecosystem. There are many solutions to "dependency hell". Pip itself hopes to help with some dependency resolution.

Then, there are *virtualenv* environments, which you might use to create "clean slate" environments for each project. Of course this means that everything is siloed. This is undesirable for serious environments.

Anaconda

One popular solution is to have someone else (Anaconda Inc.) collect a bunch of compatible versions of all the popular libraries for you to draw upon.

Conda has ~250 base packages, and another 7,500 it can fetch from *PyPI* (with varying reliability). The idea is that some combination of vetting and advanced dependency analysis will install packages without conflict.

In practice, there are plenty of opportunities to go awry. One common mistake is to try to use *pip* simultaneously (although this *should* work, in theory). However, there are plenty of examples of where *conda* trips on itself.

Nevertheless, it is probably the best solution if you are going to install any number of non-trivial Python packages. There is also a nifty GUI option.

Creating an mpi4py environment

In our case, we need only start with an Anaconda base environment and just add in mpi4py (and maybe numpy).

```
[urbanic@bridges2-login012 ~]$ module load anaconda3/2022.10
(base) [urbanic@bridges2-login012 ~]$ conda create -n mpi4py_env mpi4py numpy
(base) [urbanic@bridges2-login012 ~]$ source activate mpi4py_env
(mpi4py_env) [urbanic@bridges2-login012 ~]$
```

You only have to create the mpi4py_env once. However you have to source the environment every time you want to use it. I personally have added "module load anaconda3/2022.10", and maybe "source activate mpi4py_env" to my .bashrc login startup file. You might want to do this as well.

.bashrc

Really, this is too useful not to try. Not just here, but for all your future command line use. Give it a shot!

Running mpi4py on Bridges

The simplest way is to run interactively. Just ask for the maximum number of PEs that you intend to use for this session, make sure you activate your MPI environment, and you are ready to run. Attempting this on a login node will result in confusion.

```
(base) [urbanic@bridges2-login011 ~]$ interact -n 8 -t 4:00:00
...
...
(base) [urbanic@r001 ~]$ source activate mpi4py_env
(mpi4py_env) [urbanic@r001 ~]$ mpirun -n 4 python hello.py
Hello world from PE 0
Hello world from PE 1
Hello world from PE 2
Hello world from PE 3
```

If you didn't do the smart/lazy thing that I did and add the "module load anaconda3/2020.11" command into your .bashrc file, then you will have to type that command too, just before you source the environment.

References

The best starting place is the document maintained by the library author at

<https://mpi4py.readthedocs.io/en/stable/>

The Tutorial section there is helpful for beginners.

There are several references from these pages, the most important of which is GitHub:

<https://github.com/mpi4py/mpi4py>

which will provide you with other leads, and several routes to the most comprehensive document:

<https://buildmedia.readthedocs.org/media/pdf/mpi4py/stable/mpi4py.pdf>

For serious work, you will want to refer to the real MPI specification, with all of the MPI routines. We will cover most of these later. You can find this, as well as many other MPI manuals at the MPI Forums:

<http://www.mpi-forum.org/docs/>

Not-Assignment #00: Test your MPI basics.

Write a code that runs on 8 PEs and does a “circular shift.” This means that every PE sends some data to its nearest neighbor either “up” (one PE higher) or “down.” To make it circular, PE 7 and PE 0 are treated as neighbors. Make sure that whatever data you send is received.

Due date: Never. This is optional and worth 0 points.

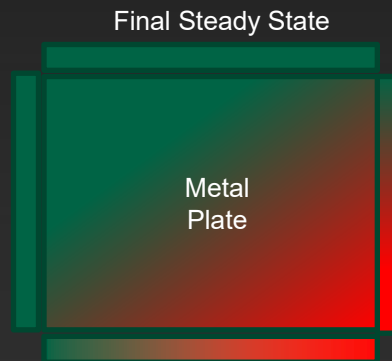
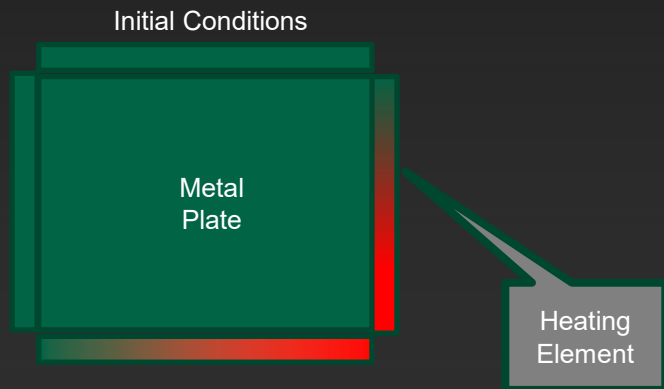
However, it is a good warm up for our next assignment, and I am happy to help you work through it.

Laplace Assignment



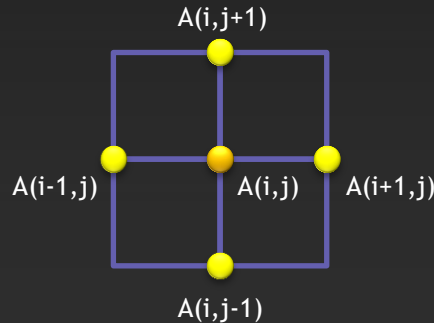
Our Foundation Exercise: Laplace Solver

- I've been using this for MPI, OpenMP and now OpenACC. It is a great simulation problem, not rigged for MPI.
- In this most basic form, it solves the Laplace equation: $\nabla^2 f(x, y) = 0$
- The Laplace Equation applies to many physical problems, including:
 - Electrostatics
 - Fluid Flow
 - Temperature
- For temperature, it is the Steady State Heat Equation:



Exercise Foundation: Jacobi Iteration

- The Laplace equation on a grid states that each grid point is the average of its neighbors.
- We can iteratively converge to that state by repeatedly computing new values at each point from the average of neighboring points.
- We just keep doing this until the difference from one pass to the next is small enough for us to tolerate.



$$A_{k+1}(i, j) = \frac{A_k(i-1, j) + A_k(i+1, j) + A_k(i, j-1) + A_k(i, j+1)}{4}$$

Serial Python Jacobi Solver

```
for i in range( 1 , ROWS+1 ):
    for j in range( 1 , COLUMNS+1 ):
        temperature[ i , j ] = 0.25 * ( temperature_last[i+1,j] + temperature_last[i-1,j] +
                                         temperature_last[i,j+1] + temperature_last[i,j-1] )
```

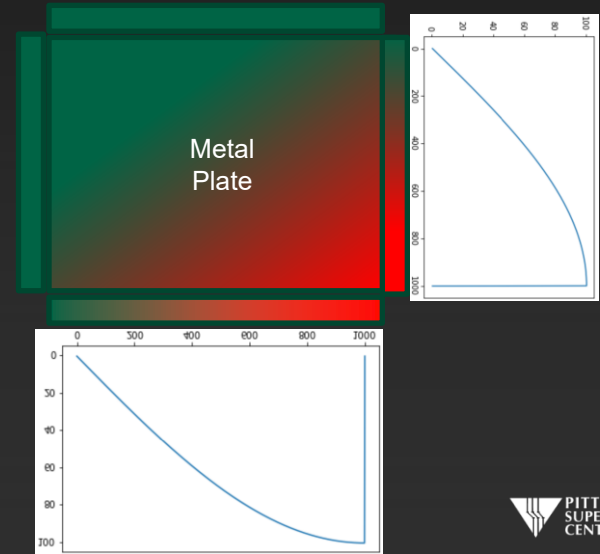
Kernel of Laplace

```
while ( dt > MAX_TEMP_ERROR ) and ( iteration < max_iterations ):  
    for i in range( 1 , ROWS+1 ):  
        for j in range( 1 , COLUMNS+1 ):  
            temperature[ i , j ] = 0.25 * ( temperature_last[i+1,j] + temperature_last[i-1,j] +  
                                             temperature_last[i,j+1] + temperature_last[i,j-1] )  
  
    dt = 0  
  
    for i in range( 1 , ROWS+1 ):  
        for j in range( 1 , COLUMNS+1 ):  
            dt = max( dt, temperature[i,j] - temperature_last[i,j])  
            temperature_last[ i , j ] = temperature [ i , j ]  
  
    print(iteration)  
    iteration += 1
```


Serial Python Initialize Subroutine

```
def initialize_temperature(temp):  
    temp[:, :] = 0  
  
    #Set right side boundary condition  
    for i in range(ROWS+1):  
        temp[ i , COLUMNS+1 ] = 100 * math.sin( ( (3.14159/2) /ROWS ) * i )  
  
    #Set bottom boundary condition  
    for i in range(COLUMNS+1):  
        temp[ ROWS+1 , i ] = 100 * math.sin( ( (3.14159/2) /COLUMNS ) * i )
```

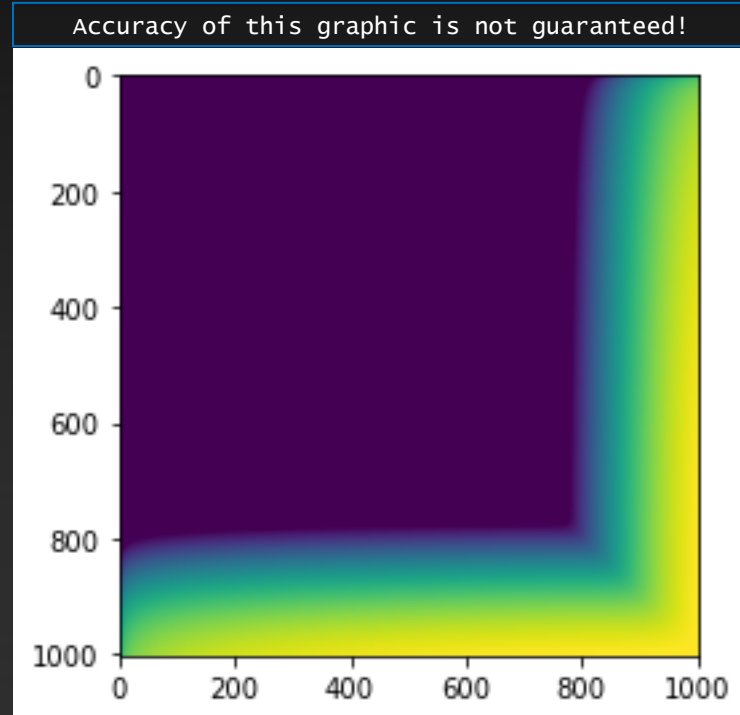
- Just a quarter sine wave.
- Goes from 0 to 100 degrees.
- All of these graphics of the final state are suggestive. You should run and visualize the serial code to see an accurate answer.



Serial Python Output Subroutine

```
def output(data):  
    #import matplotlib  
    #matplotlib.pyplot.imshow(data)  
    data.tofile("plate.out")
```

- This is just a couple possibilities. You should do whatever you are confident with.
- You may choose to bring the binary output back to your laptop for graphical processing.
- You can visualize directly from Bridges, but this takes some thought.



Whole Python Code

```
import numpy as np
import math

ROWS , COLUMNS = 1000 , 1000
MAX_TEMP_ERROR = 0.01
temperature      = np.empty(( ROWS+2 , COLUMNS+2 ))
temperature_last = np.empty(( ROWS+2 ,COLUMNS+2 ))

def initialize_temperature(temp):

    temp[:,:] = 0

    #Set right side boundary condition
    for i in range(ROWS+1):
        temp[ i , COLUMNS+1 ] = 100 * math.sin( ( (3.14159/2) /ROWS ) * i )

    #Set bottom boundary condition
    for i in range(COLUMNS+1):
        temp[ ROWS+1 , i ] = 100 * math.sin( ( (3.14159/2) /COLUMNS ) * i )

def output(data):
    #import matplotlib
    #matplotlib.pyplot.imshow(data)
    data.tofile("plate.out")

initialize_temperature(temperature_last)

max_iterations = int (input("Maximum iterations: "))

dt = 100
iteration = 1

while ( dt > MAX_TEMP_ERROR ) and ( iteration < max_iterations ):

    for i in range( 1 , ROWS+1 ):
        for j in range( 1 , COLUMNS+1 ):
            temperature[ i , j ] = 0.25 * ( temperature_last[i+1,j] + temperature_last[i-1,j] +
                                             temperature_last[i,j+1] + temperature_last[i,j-1] )

    dt = 0

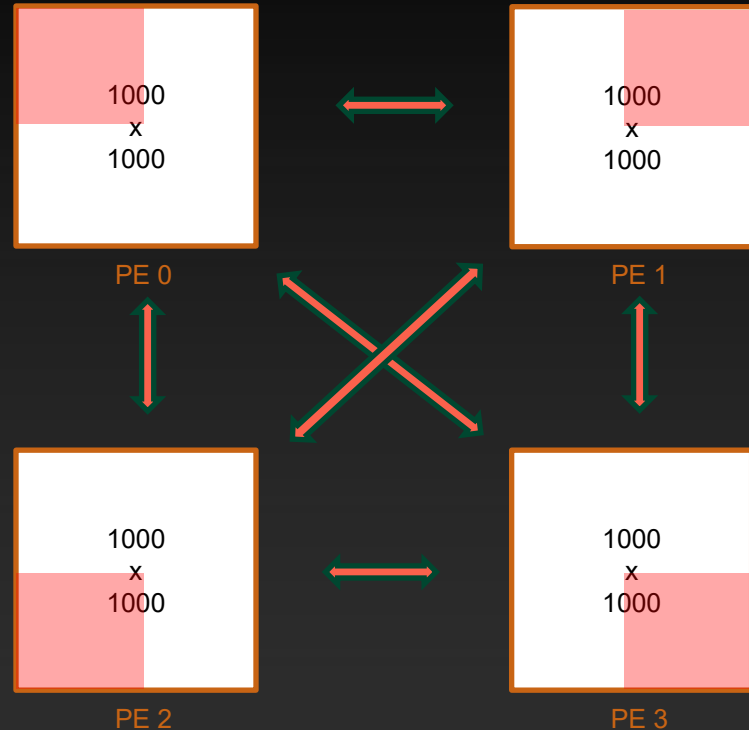
    for i in range( 1 , ROWS+1 ):
        for j in range( 1 , COLUMNS+1 ):
            dt = max( dt, temperature[i,j] - temperature_last[i,j])
            temperature_last[ i , j ] = temperature [ i , j ]

    print(iteration)
    iteration += 1

output(temperature_last)
```

First Things First: Domain Decomposition

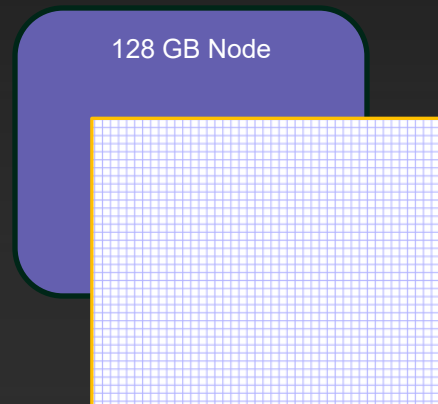
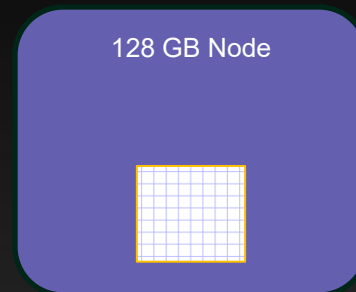
- All processors have an entire Temperature array.
- Each processor works on its part of Temperature.
- After every iteration, all processors broadcast their pieces to all other processors.
- **No! Preserving any large shared data structures is bad.**
- **Increased memory. Not Scalable.**
- **Large messages. Not Scalable.**



Scalability Demands No Global Arrays

1000 x 1000 Grid
8 bytes * 1M elements * 2 arrays
= 16MB memory

100,000 x 100,000 Grid
8 bytes * 10 Billion elements * 2 arrays
= 160 GB memory



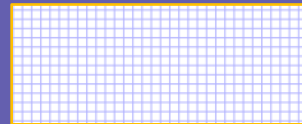
Divide Up Data As Needed

100,000 x 100,000 Grid

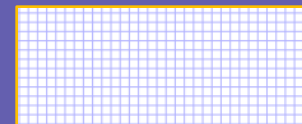
8 bytes * 10 Billion elements * 2 arrays

= 160 GB memory

128 GB Node



128 GB Node



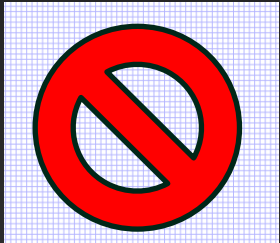
Keep Partitioning Data

1,000,000 x 1,000,000 Grid

8 bytes * 1 Trillion elements * 2 arrays

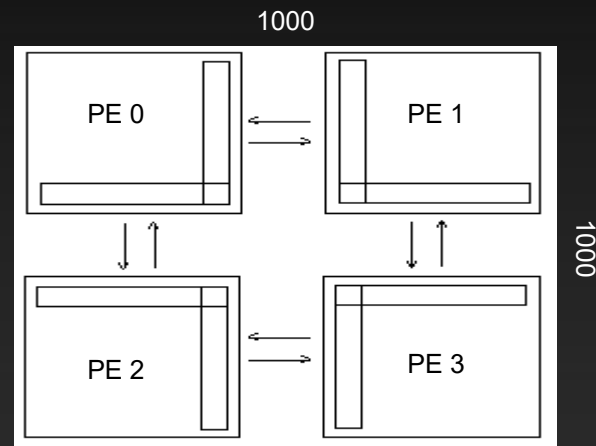
= 16 TB memory

= 126 Regular Bridges Nodes



Try Again: Domain Decomposition II

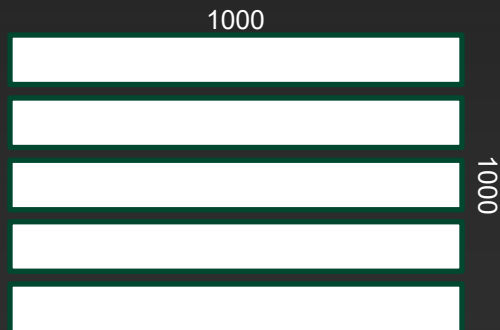
- Each processor has only its own sub-grid.
- 1000x1000 array not longer exists (except on paper).
- Communicate boundary values only.
- Reduces memory.
- Reduces communications.
- Have to keep track of neighbors in two directions.
- But not bad.



Simplest: Domain Decomposition III



- Only have to keep track of up/down neighbors, and no corner case.
- Easiest with our simple message passing experience.
- Scales, as below. How would we handle 5 PEs with the “square decomposition”?

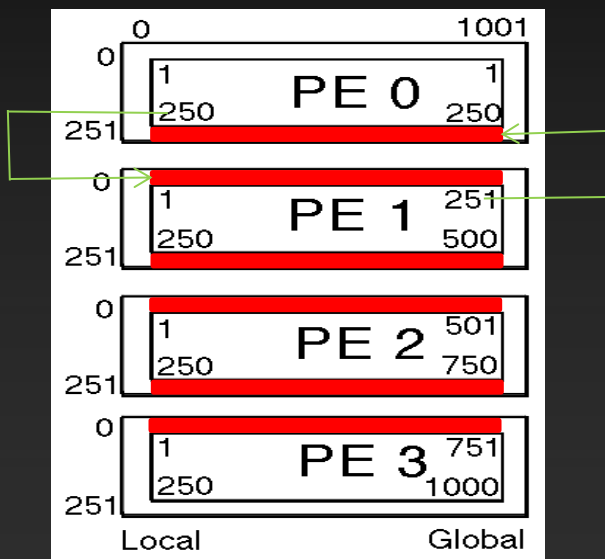


Simplest Decomposition for 2D Code

0		0																
	<table><tr><td>1</td><td>1</td></tr><tr><td>250</td><td>250</td></tr><tr><td>1</td><td>251</td></tr><tr><td>250</td><td>500</td></tr><tr><td>1</td><td>501</td></tr><tr><td>250</td><td>750</td></tr><tr><td>1</td><td>751</td></tr><tr><td>250</td><td>1000</td></tr></table>	1	1	250	250	1	251	250	500	1	501	250	750	1	751	250	1000	
1	1																	
250	250																	
1	251																	
250	500																	
1	501																	
250	750																	
1	751																	
250	1000																	
Local	Global	1001																

Simplest Decomposition for Python Code

In the parallel case, we will break this up into 4 processors. There is only one set of boundary values. But when we distribute the data, each processor needs to have an extra row for data distribution, these are commonly called the “ghost cells”.



The program has a local view of data. The programmer has to have a global view of data. The ghost cells don't exist in the global dataset. They are only copies from the “real” data in the adjacent PE.

Sending Multiple Elements & Using NumPy

- The routines we have been using use Python *pickling* to format the data. This is friendly and allows us to ignore the data types and other details. But, it is terribly inefficient.
- For the first time we want to send multiple elements. In this case, a whole row of data.
- We are also using NumPy. In general it makes our code cleaner and more efficient.
- It also allows us to work directly with array slices (strips of the arrays).
- We can use this with the much more efficient "in place" single segment memory buffer routines that mpi4py offers. These are capitalized ("comm.Recv" vs. "comm.recv") and use an in-place numpy array as the buffer.
- But, our data must be contiguous.

mpi4py NumPy array usage

```
from mpi4py import MPI
import numpy as np

send_array = np.empty(100)
recv_array = np.empty(100)

for index in range(100): # Number array elements by index
    send_array[index] = index

my_pe_num = MPI.COMM_WORLD.Get_rank()

if my_pe_num==0:
    MPI.COMM_WORLD.Send(send_array[10:20], dest = 1)
else:
    MPI.COMM_WORLD.Recv(recv_array[40:50], source = MPI.ANY_SOURCE, tag= MPI.ANY_TAG)
    print(recv_array[40:50])
```

```
(mpi4py_env) [urbanic@r072 MPI_Example_Codes]$ mpirun -n 2 python send_slice.py
[10. 11. 12. 13. 14. 15. 16. 17. 18. 19.]
```

Make sure you understand this example as it will prove invaluable in this assignment.

Sending Multiple Elements

C and Python are row-major:

`int A[8][12];` ← This last index is the one contiguous in memory.

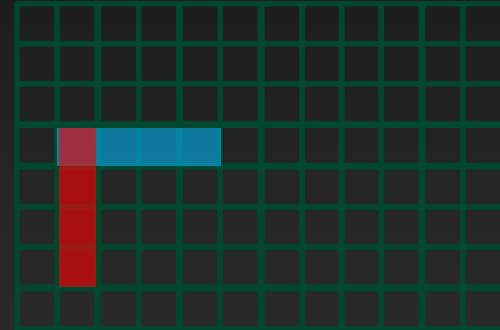
`MPI_Send(&A[3][1], 4, MPI_INT, pe, tag, MPI_COMM_WORLD);`

Fortran is column-major:

`integer A(0:7,0:11)`

`MPI_Send(A(3,1), 4, MPI_INT, pe, tag, MPI_COMM_WORLD, err);`

← This first index is the one contiguous in memory.



Array Ordering Performance Implications

- Once again, we will make this issue less critical from the MPI perspective once we learn about user defined datatypes in the advanced talk, but...
- this is extremely important to understand for general performance reasons. Notice how the C code iterates over the last (j) variable in the inner loop while the Fortran uses the first (i) in our serial code? This would kill our performance if it was the other way around. And I mean in the serial code. Go ahead and try...

```
for(i = 1; i <= ROWS; i++) {  
    for(j = 1; j <= COLUMNS; j++) {  
        Temperature[i][j] = 0.25 * (Temperature_last[i+1][j] + Temperature_last[i-1][j] +  
                                     Temperature_last[i][j+1] + Temperature_last[i][j-1]);  
    }  
}
```

```
do j=1,columns  
    do i=1,rows  
        temperature(i,j)= 0.25 * (temperature_last(i+1,j)+temperature_last(i-1,j) + &  
                                   temperature_last(i,j+1)+temperature_last(i,j-1) )  
    enddo  
enddo
```

Main Loop Structure

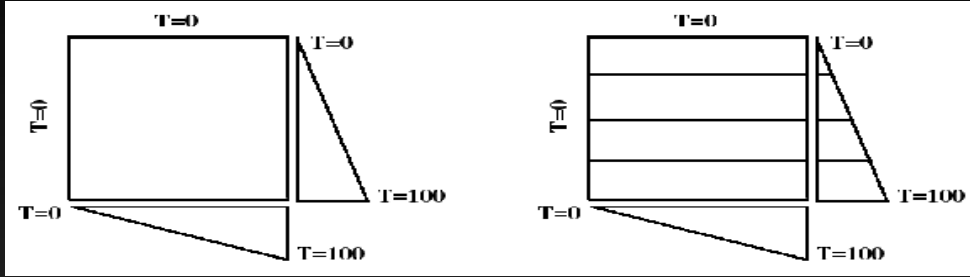
```
while ( dt > MAX_TEMP_ERROR && iteration <= max_iterations )  
    Calculate new Temperature  
    Copy into temp array  
    ...
```

**Compute
Phase**
(almost unchanged)

```
Send values down (Temperature or Temperature_last?)  
Send values up  
Receive values from above into ghost zone (Temperature or Temperature_last?)  
Receive values from below into ghost zone  
  
Find max change  
Synchronize?
```

**Communicate
Phase**
(all new)

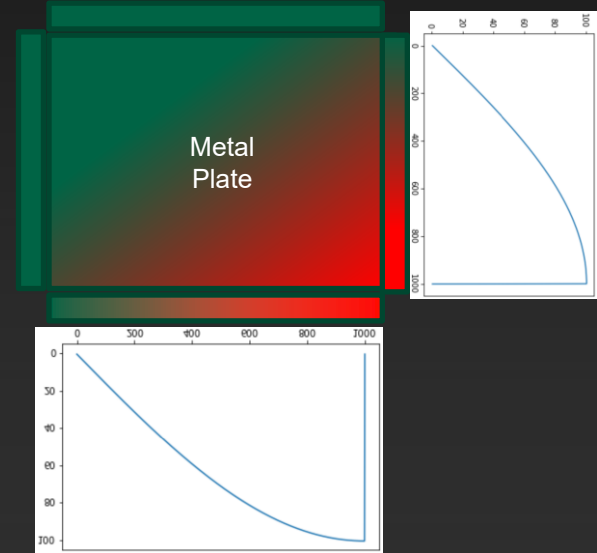
Boundary Conditions



The above is for linearly increasing boundary conditions.

Ours are just a little trickier, with a sinusoidal character.

It is straightforward, and visualization is a good verification that you have nailed your logic.



Some ways you might go wrong...

You have two main data structures

- Temperature
- Temperature_last

Each has

- Boundary Conditions (unchanged through entire run)
- Ghost zones (changing every timestep)

Each iteration

- Copying/calculating Temperature to/from Temperature_last
- Sending/receiving into/from ghost zones and data

It is easy to mix these things up. I suggest you step through at least the initialization and first time step for each of the above combinations of elements.

There are multiple reasonable solutions. Each will deal with the above slightly differently.

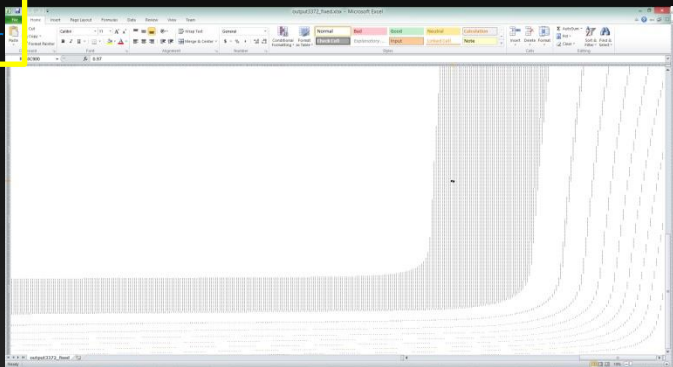
How do you know you are correct?



Your solution converges
at 3578 timesteps!

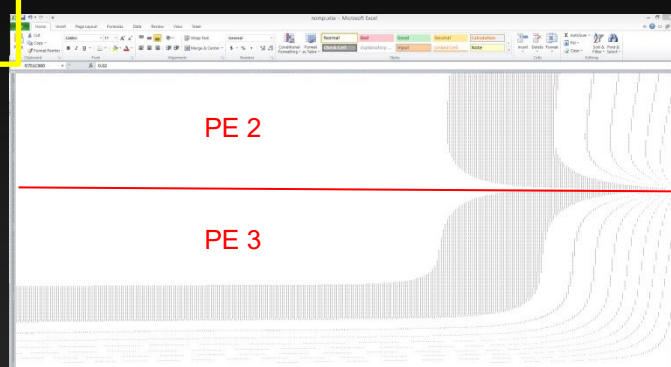
How do you know you are correct?

Bottom
Right
Corner



Working MPI Solution

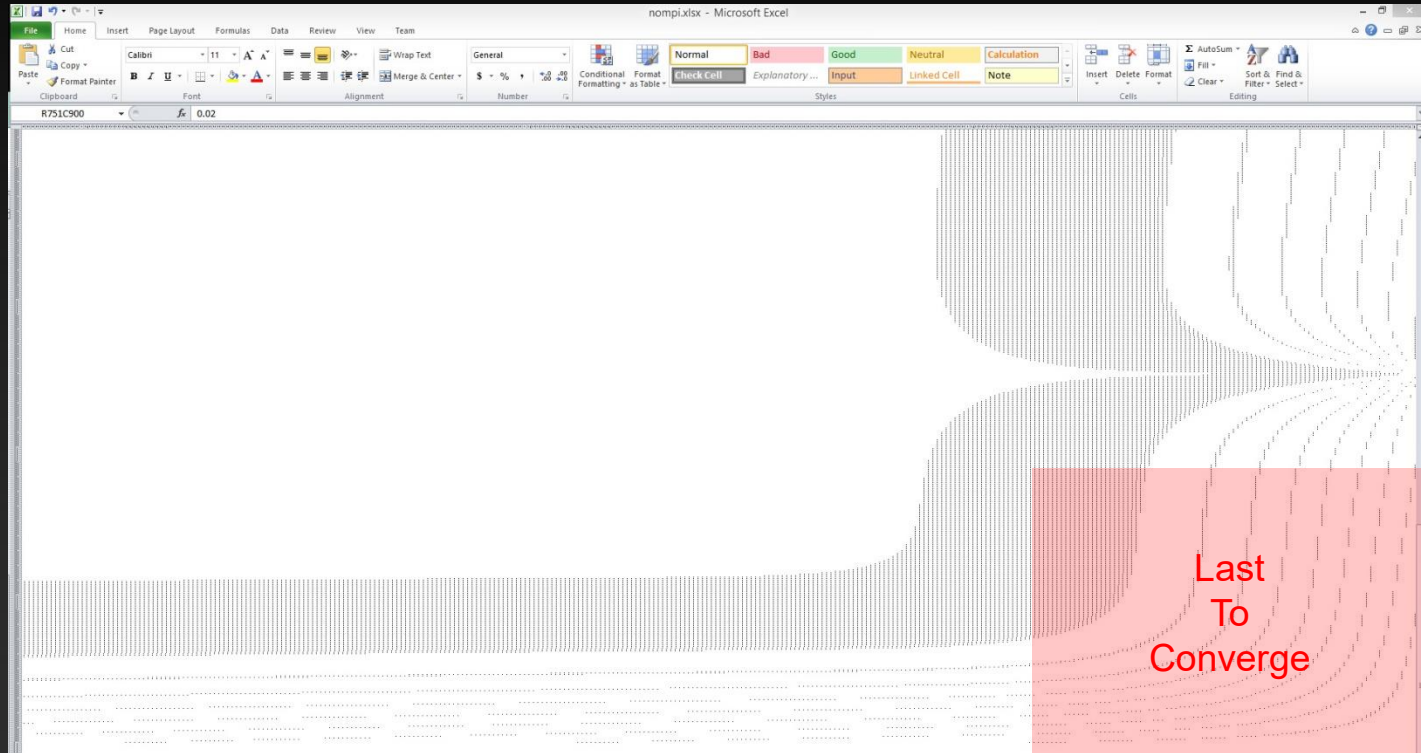
Bottom
Right
Corner



MPI Routines Disabled

Both converge at 3578 steps!

All the action is here.



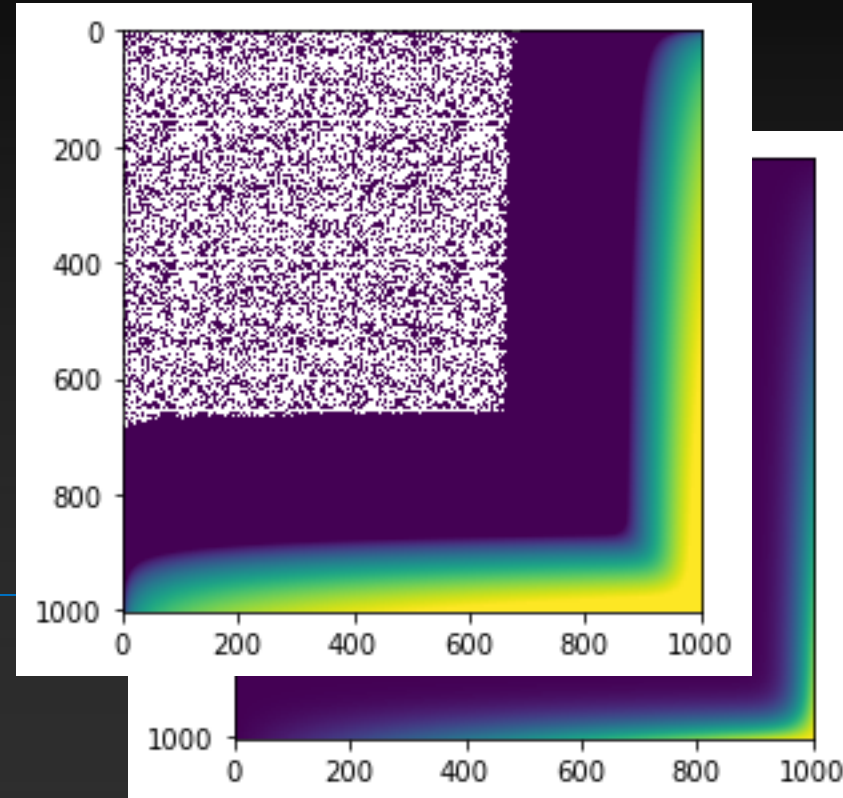
Visualization is critical

Simulation Output

```
def output(data):  
    #import matplotlib  
    #matplotlib.pyplot.imshow(data)  
    data.tofile("plate.out")
```

Simple Plots

```
import matplotlib  
import numpy  
  
plate = numpy.fromfile("plate.out", dtype=float).reshape((1002,1002))  
  
#matplotlib.pyplot.imshow(plate)  
matplotlib.pyplot.imshow(plate, norm=matplotlib.colors.LogNorm(0.1,50,clip=True))
```



Assignment #1: Parallelizing Laplace with MPI

In `~urbanic/LargeScaleComputing/Laplace/laplace.py` you will find the serial python code.

You must parallelize this to run on 4 PE's on Bridges-2. It should run about 4 times faster.

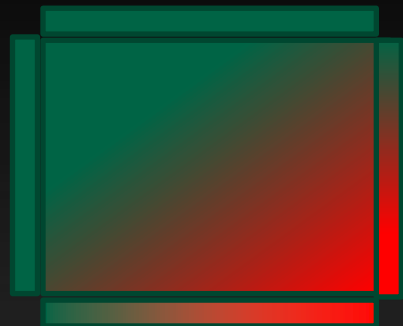
Your final submission must include:

- A Python code (a .py file attachment)
- a simple visualization of the final answer (a .png or .jpg)
- and timings for the full run to completion vs. the serial code (just in the body of the mail).

Note that the full convergence takes a while. *You should do most of your testing and development with much shorter runs, or you could create a 100x100 size problem for development purposes.* Hint: just focus on hot edges and corners and you can see if your code is working with very few iterations.

This project is **due September 18th**. We will review that day, so no late credit is possible. This is 15 points of your grade, and the effort required reflects that.

Send me all three items in a single email.



Advanced MPI

John Urbanic

Parallel Computing Scientist
Pittsburgh Supercomputing Center

Distinguished Service Professor
Carnegie Mellon University

Assignment #1: Parallelizing Laplace with MPI

In `~urbanic/LargeScaleComputing/Laplace/laplace.py` you will find the serial python code.

You must parallelize this to run on 4 PE's on Bridges-2. It should run about 4 times faster.

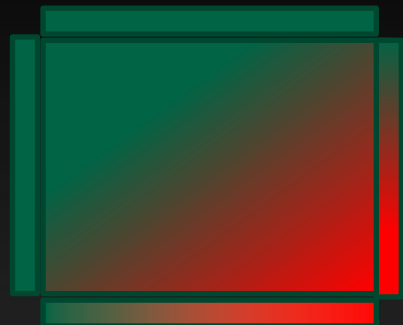
Your final submission must include:

- A Python code (a .py file attachment)
- a simple visualization of the final answer (a .png or .jpg)
- and timings for the full run to completion vs. the serial code (just in the body of the mail).

Note that the full convergence takes a while. *You should do most of your testing and development with much shorter runs, or you could create a 100x100 size problem for development purposes.* Hint: just focus on hot edges and corners and you can see if your code is working with very few iterations.

This project is **due September 18th**. We will review that day, so no late credit is possible. This is 15 points of your grade, and the effort required reflects that.

Send me all three items in a single email.



Hints for Assignment #1

- The next slides discuss some common anti-patterns that you want to avoid. Keep them in mind!
- You will find both visualization and print statement very helpful in debugging. Use them both often.
- With print statements you may find that the system buffering confuses the order of messages. As per the earlier suggesting, flushing the output is an easy way to avoid this:

```
print("Hello", flush=True)
```

- We will learn how to use an actual debugger later. But, that would be an unnecessary distraction now.

MPI Anti-patterns

There are some common MPI beginner *anti-patterns* or *code smells* that you should beware of.

First is failing to recognize when all of the PEs are doing the same thing, just working with different data. If you find yourself writing code that looks like this:

```
if my_pe == 1:
    do something

if my_pe == 2:
    do something similar

if my_pe == 3:
    do something similar

if my_pe == 4:
    do something similar

if my_pe == 5:
    do something similar
```

you are probably falling into this trap.

MPI Reality Check

Remember that we are writing *scalable* code here. Ask yourself if this would work with 10,000 PEs.

```
if my_pe == 1:
    do something
if my_pe == 2:
    do something similar
if my_pe == 3:
    do something similar
if my_pe == 4:
    do something similar
if my_pe == 5:
    do something similar
if my_pe == 6:
    do something
if my_pe == 7:
    do something similar
if my_pe == 8:
    do something similar
if my_pe == 9:
    do something similar
if my_pe == 10:
    do something similar
if my_pe == 11:
    do something
if my_pe == 12:
    do something similar
if my_pe == 13:
    do something similar
if my_pe == 14:
    do something similar
if my_pe == 15:
    do something similar
if my_pe == 16:
    .
    .
    .
    .
    .
    .
    .
```

Obviously not. Nor would it work with a runtime variable number of PEs, which should always be your goal (like our "finding Pi" example).

Acceptable PE specific logic

There are a few exceptions to this. The manager/worker paradigm is very common, and recognizable. And you may find you need some special logic for the "boundaries". We've already seen both of these cases:

Worker / Manager (from Pi code)

```
if my_pe_num == 0:  
    printf("How many intervals? ")  
    ...
```

Boundary Code

```
if my_PE_num == 7:  
    MPI.COMM_WORLD.Send(...)  
else:  
    MPI.COMM_WORLD.Recv(...)
```

Note that these also behave well when answering the question "What would this look like on 10,000 PEs?"

Same is true of variables

Apply the same thought process to variables. If you find yourself creating multiple definitions of a variable, you are probably not thinking parallel. Asking yourself if this will scale to 10,000 PEs will also keep you on track.

```
dt1 = 0
```

```
dt2 = 0
```

```
dt3 = 0
```

```
dt4 = 0
```

```
dt5 = 0
```

```
dt6 = 0
```

```
dt7 = 0
```

```
dt8 = 0
```

```
.
```

```
.
```

```
.
```

```
.
```

```
.
```

```
.
```

```
.
```

```
.
```

MPI Advanced Features

In the MPI Basics talk we only touched upon the core MPI routines. Here we will discuss some of the advanced features that you are likely to use if you progress. We've grouped them thematically as just "looking through the index" can be intimidating.

- Datatypes
- Collective Communication
- Topology (Communicators)
- Non-Blocking Communications
- Single-sided Communications (RMA)
- Shared Memory
- Hybrid Programming
- Performance Options
- Attributes
- Dynamic Processes
- MPI-IO
- Modern Fortran (and other languages)
- Library Support
- MPI Implementations

MPI Evolution

1993: Supercomputing 93 - draft MPI standard presented.

1994: Final version of MPI-1.0 released

1995: MPI-1.1

1996: MPI-2

1997: MPI-1.2

2008: MPI-1.3, MPI-2.1

2009: MPI-2.2

2012: MPI-3.0

2021: MPI-4.0

2023: MPI-4.1

Although there are some general themes to various releases (3.0 has a lot of “ultra-scalability” features), we won’t review routines chronologically. I’ve gathered each topic across all releases as it makes much more sense that way.

User Defined Data Types

User defined data types are often somewhere between organizationally useful, and indispensably convenient. They serve two important purposes:

- **Translation between formats**
 - Automatic conversion of data sent from a big-endian to little-endian node
 - Not very common these days
- **Noncontiguous messages**
 - Many cases, from rows and columns of arrays to dynamically allocated lists
 - Serious performance implications

User Defined Data Types

There are two steps to using a defined data type:

1) Create the type using one of the multiple creation routines

`MPI_Type_contiguous(count...)`

Could be used in our current Laplace.

`MPI_Type_vector(count,...,stride...)`

Adds a stride. Could be used for row/column.

`MPI_Type_indexed()`

Varying strides and block sizes.

`MPI_Type_create_subarray()`

What you would guess and very flexible.

`MPI_Type_create_struct()`

Most general.

... more

These routines accept existing, possibly user-defined, types. So they can be layered to build even more complex types.

2) Commit the type

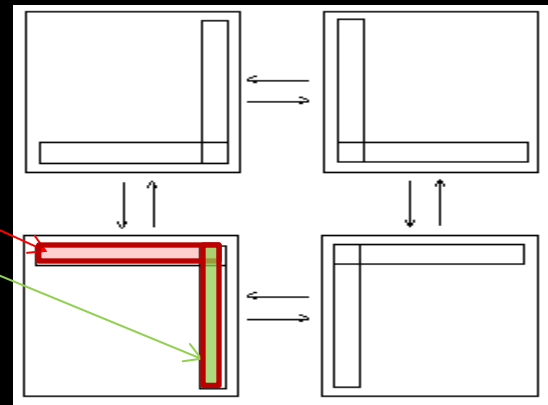
`MPI_Type_commit()`

Vector Type

(stencil code example)

`MPI_Type_vector(int count,int blocklength, int stride, MPI_Datatype oldtype,MPI_Datatype *newtype)`

```
MPI_Datatype row, column ;
MPI_Type_vector ( COLUMNS, 1, 1, MPI_DOUBLE, &row );
MPI_Type_vector ( ROWS, 1, COLUMNS, MPI_DOUBLE , &column );
MPI_Type_commit ( &row );
MPI_Type_commit ( &column );
.
.
//Send top row to up neighbor
MPI_Send(Temperature[1,1], 1, row, dest, tag, MPI_COMM_WORLD);
.
//Send last column to right hand neighbor (in the pictured layout)
MPI_Send(Temperature[1,COLUMNS], 1, column, dest, tag, MPI_COMM_WORLD);
```



Note that we are only sending 1 of each type, and that column and row sends look the same except for their type. We also could have used the `MPI_Type_contiguous()` routine to define the row type: `MPI_Type_contiguous(ROWS, MPI_DOUBLE, &ROW)`

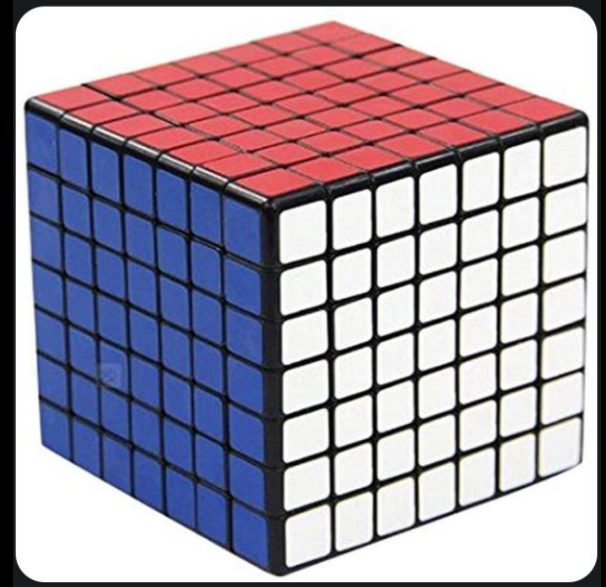
3D Halo Codes

To send whole face, you could use

- Contiguous on one face to send 49 elements
- Vector on another to send 49 elements with a stride of 7
- Vector on another face with a stride of $7 \times 7 = 49$
- Opposing faces use same type to send.
- Matching faces receives use same type.

But for Halo codes we want to exclude the ghost zones. So for the first face you need to have block lengths and count of 5, with the same stride of 7, but make sure you start on the first "real" grid point.

You can also use the `MPI_Type_create_subarray` routine to define these. It is usually simpler and more symmetric for these cases as you explicitly state the start points, dimensions and sizes and leave the stride calculations as implicit. You can get by with 3 different types of message (for the X, Y and Z directions) that cover all the sends and receives.



There is a nice walkthrough example at:

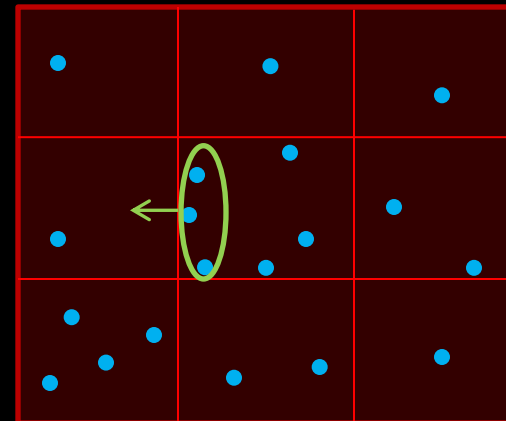
<https://stackoverflow.com/questions/28182076/number-of-subarray-data-types-for-exchanging-2d-halos-in-3d-process-decompositio>

Indexed Type

(MD code example)

```
MPI_Type_create_indexed_block(int count, int blocklength,int displacements[], MPI_Datatype oldtype, MPI_Datatype *newtype)
```

```
MPI_Datatype particle_type, buffer_type;  
.  
//when we create particle type it groups whatever the elements of  
//particle_list really are: array of floats, C struct, ...  
.  
//find particles on border and put their particle_list[]  
//indices into displacements[]  
:  
.  
MPI_Type_create_indexed_block(num_to_send, 1, displacements[],  
                             particle_type, &buffer_type);  
  
MPI_Type_commit (&buffer_type);  
  
MPI_Send(particle_list, 1, buffer_type, left, tag, comm);  
  
//And repeat every time!  
MPI_Type_free(&buffer_type);
```



particle_list



displacements



Collective Communications

The Broadcast and reduce operations have more complex analogs that are very useful in a variety of algorithms. Just like Bcast and Reduce, the benefit of using these routines, instead of point-to-point messages, is not only syntactical convenience, but also much better efficiency.

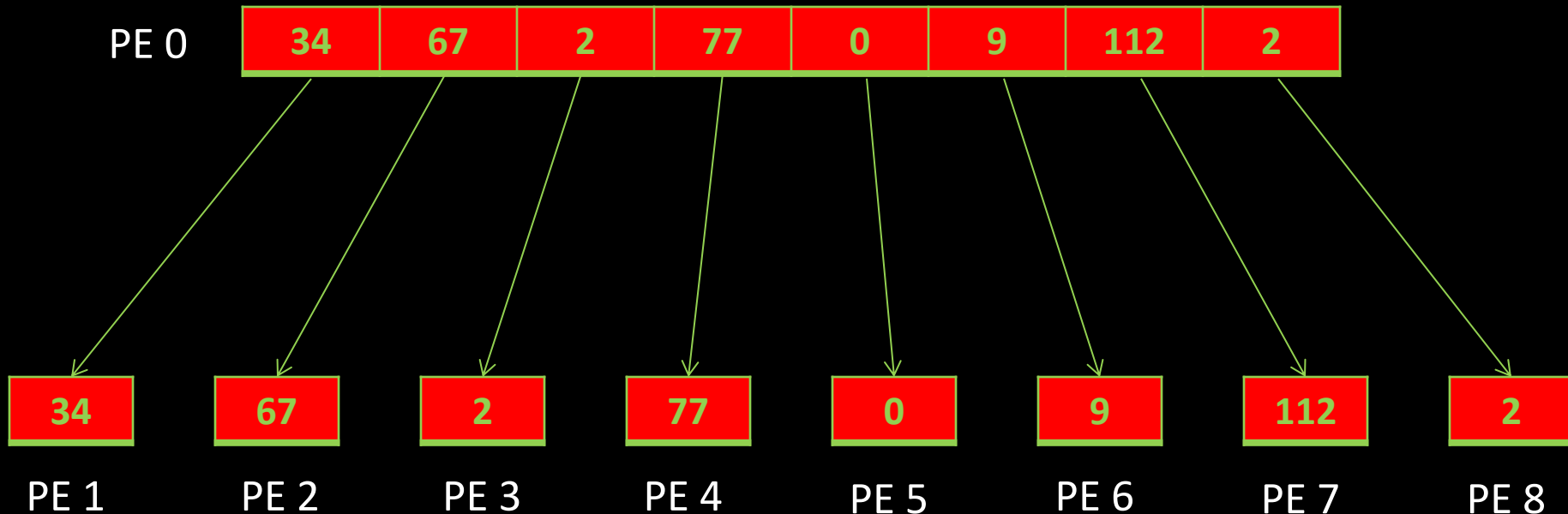
Keep in mind that these are called “collective” because every PE in the communicator must call these routines at the same time.

Let’s take a look at the basic idea, and then we’ll see the wide variety of related routines.

Scatter

One PE has some data that should be distributed differently to other PEs. Every PE in the communicator participates in this call:

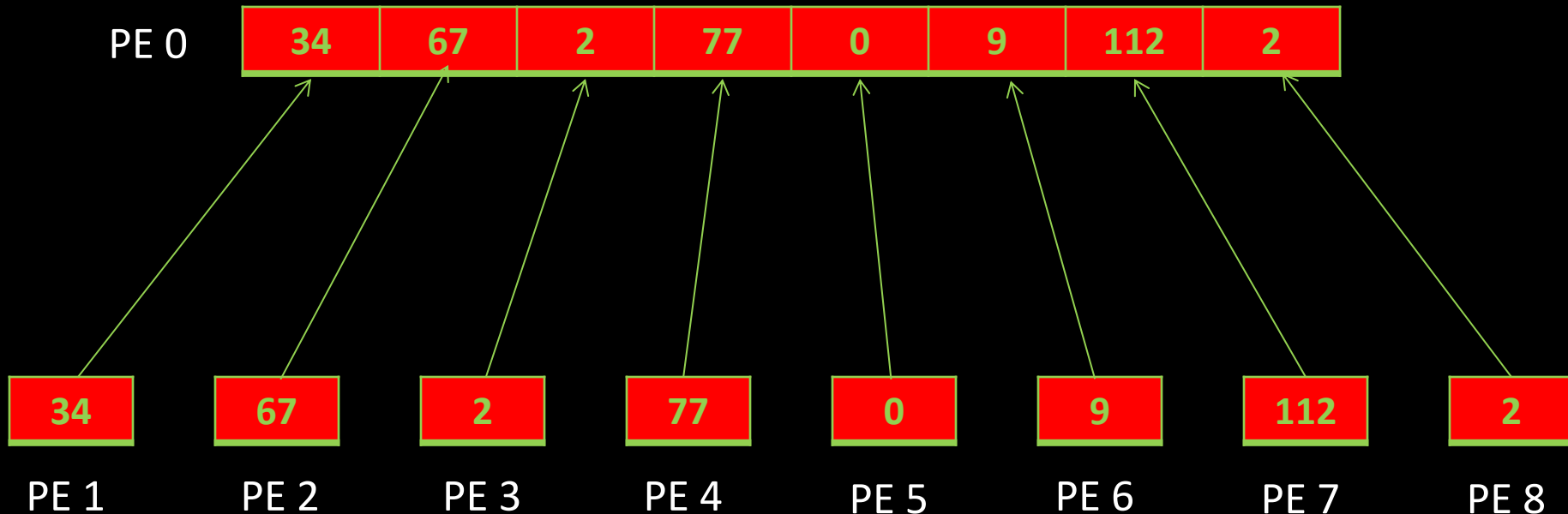
```
MPI_Scatter(void* send_data, int send_count, MPI_Datatype send_datatype, void* recv_data, int recv_count,  
           MPI_Datatype recv_datatype, int root, MPI_Comm communicator)
```



Gather

The exact opposite. The other PEs have some data that you wish to collect on one PE.

```
MPI_Gather(void* send_data, int send_count, MPI_Datatype send_datatype, void* recv_data, int recv_count, MPI_Datatype recv_datatype, int root, MPI_Comm communicator)
```



Scatter / Gather

There are several significantly different variations of these routines. When you need them, they are very helpful:

<code>MPI_Scatterv()</code>	Count can vary for each PE (uses an array for send_counts)
<code>MPI_Gatherv()</code>	“”

The “All” versions mean results are returned to all processors. No root parameter.

<code>MPI_Allgather()</code>	Like Gather, but all processes receive result
<code>MPI_Allgatherv()</code>	Allgather with varying count
<code>MPI_Alltoall()</code>	Each process sends distinct data to each receiver. Perfect for transpose or FFT.
<code>MPI_Alltoallv()</code>	Count can vary
<code>MPI_Alltoallw()</code>	Count, displacement and even datatype can vary (crazy flexible)

These are not as scalable and the MPI-3 standards group even considered deprecating vector collectives. Don't use them for “exa-scale” codes.

Communicators and Groups

The most basic way to think of communicators is as a way to break PEs into separate groups. You will most likely first encounter this when you want to have some PEs participate in an action while others do not. This immediately makes collective operations problematic. When using the `COMM_WORLD` communicator, all PE's have to participate in every collective. So, we will want to make our own sub-group.

Indeed, groups are one way MPI operates when creating sets of PEs. First we create a group, then we manipulate it until it represents the PEs that we want, then we create a communicator representing that group.

This following example illustrates how a group consisting of all but the 0 process of the World group is created. Then a communicator (`comm_workers`) is formed for that new group. The new communicator is used in a collective call just for the workers. Then all processes execute a collective call in the `MPI_COMM_WORLD` context.

Communicators to enable control flow

```
main(int argc, char **argv)  {

    int my_PE, send_data=1, recv_data1, recv_data2;
    MPI_Group group_world, group_workers;
    MPI_Comm comm_workers;
    int ranks[] = {0}; //Ranks to exclude, just 0 here

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &my_PE);

    MPI_Comm_group(MPI_COMM_WORLD, &group_world);
    MPI_Group_excl(group_world, 1, ranks, &group_workers);
    MPI_Comm_create(MPI_COMM_WORLD, group_worker, &comm_workers);

    //Reduce just for workers
    if(my_PE != 0){
        MPI_Reduce(&send_data, &recv_data1, 1, MPI_INT, MPI_SUM, 0, comm_workers);
    }

    MPI_Reduce(&send_data, &recv_data2, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

    //Proper clean up
    if (comm_workers != MPI_COMM_NULL) MPI_Comm_free(&comm_workers); //comm_workers is NULL on PE 0
    MPI_Group_free(&group_world);
    MPI_Group_free(&group_workers);

    MPI_Finalize();

}
```

recv_data1 = number of workers PE (PEs-1) on PE 1

recv_data2 = number of world PEs on PE 0

Everything else is unknown

Topologies

Communicators can be tools to make our data decomposition easier.

- Most MPI codes have a data decomposition that reflects the physical layout of the model.
- Often there is a locality relationship to the PE's, such as “nearest neighbor”.
- If this logical topology can be properly mapped to the physical network and PE layout, it can greatly aid performance.
- At Petascale exploiting this is almost a necessity.

MPI's topology routines will help us with these issues. It has two general approaches to the subject:

- Cartesian topologies
- Graph relationships

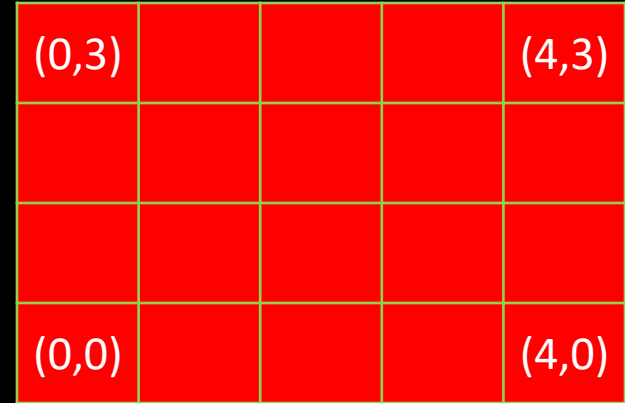
We will look at them both.

Cartesian Topologies

The first thing to do is to create a communicator that captures our topology.

```
MPI_Cart_create(MPI_Comm comm_old, int ndims, const int dims[], int periods[], int reorder, MPI_Comm *comm_cart)
```

```
int dims[2] = { 5, 4 };  
int periods = { 0, 0 };  
MPI_Cart_create(comm_old, 2, dims[], periods[], 1, &comm_2D);
```



Use `reorder = 1 = true` so that MPI can reorder the PEs in the most optimal relationship.

Excess processes get a `MPI_COMM_NULL`. If we ran the above on 24 PEs, 4 would get that. Too few processes is an error.

The new communicator now contains this relationship and we can use it in various ways going forward.

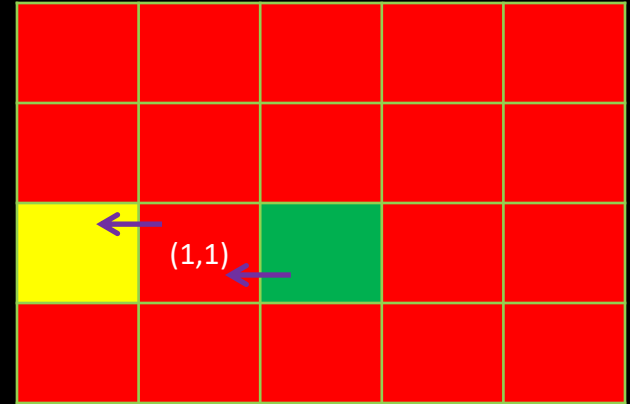
Cartesian Topologies Routines

How do we use this? Perhaps the most important thing in a cartesian layout is to know who your immediate neighbors are.

```
MPI_Cart_shift(MPI_Comm comm, int direction, int disp, int *rank_source, int *rank_dest)
```

For example, suppose that the PE that is in the (1,1) location for this new communicator wants to know where to send and receive for a left shift of data. It would do this:

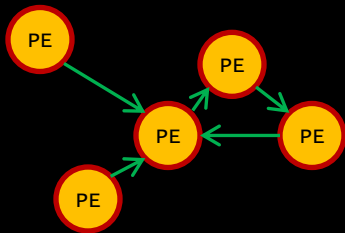
```
MPI_Cart_shift(comm_2D, 0, -1, &rank_source, &rank_dest);
```



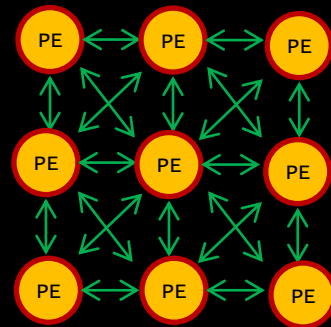
PEs on the edge will get an MPI_PROC_NULL, unless there are periodic conditions, and then they would get the appropriate opposite PE.

Graph Topologies

To capture topologies that aren't strictly Cartesian, MPI has a graph API as well. You might think it is mostly applicable to crazy, convoluted graph relationships.



And, it could be. But, there are a lot of FEA or dynamic load balanced data decompositions that don't always look like simple Cartesian relationships. Even something as simple as our “5 point stencil” 2D example can easily become a “9 point stencil”, which requires diagonal neighbors.

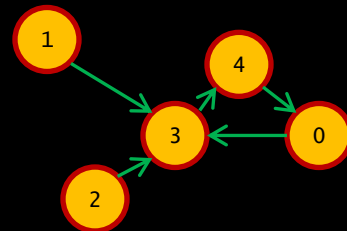


Adjacent Graph Interface

This keeps all creation local, and hence is the preferred method. Use whenever you know who your neighbors should be.

```
MPI_Dist_graph_create_adjacent(MPI_Comm comm_old, int indegree, int sources[], int sourceweights[],  
                               int outdegree, int destinations[], int destweights[],  
                               MPI_Info info, int reorder, MPI_Comm *comm_dist_graph)
```

PE	Indegree	Sources	Outdegree	Dest
0	1	4	1	3
1	0		1	3
2	0		1	3
3	3	0,1,2	1	4
4	1	3	1	0



Weights might be useful to the implementation. info might provide further hints to the system. Allowing reordering will allow the implementation to chose a better (NP-hard!) mapping.

Neighborhood Collectives

This is the super scalable payoff for our “neighborhood aware” communicators. We have operations which perform collective communications within our close neighborhood. The two most important are:

```
MPI_Neighbor_allgather(void *sendbuf, int sendcount, MPI_Datatype sendtype,  
                      void *recvbuf, int recvcount, MPI_Datatype recvtpe, MPI_Comm comm)
```

Each PE i gathers data items from each PE j if an edge (j,i) exists in the topology graph, and each PE i sends the same data items to all PEs j where an edge (i,j) exists. The send buffer is sent to each neighboring PE and the l -th block in the receive buffer is received from the l -th neighbor. This could be thought of as a neighborhood Bcast, from the sending PEs perspective.

```
MPI_Neighbor_alltoall(void *sendbuf, int sendcount, MPI_Datatype sendtype,  
                    void *recvbuf, int recvcount, MPI_Datatype recvtpe, MPI_Comm comm)
```

Each PE i receives data items from each PE j if an edge (j,i) exists in the topology graph or Cartesian topology. Similarly, each PE i sends data items to all processes j where an edge (i,j) exists. This call is more general than MPI_NEIGHBOR_ALLGATHER in that different data items can be sent to each neighbor. The k -th block in send buffer is sent to the k -th neighboring process and the l -th block in the receive buffer is received from the l -th neighbor.

The order of the send and receive buffers for graph topologies is the order of PEs returned by the neighbor query routine.

For Cartesian topologies the order is the order of dimensions. Buffers for boundary PROC_NULL PEs must still exist, but aren't read or written.

More Neighborhood Collectives

There are vector version of these collectives, to allow varying counts:

```
MPI_Neighbor_allgatherv(void *sendbuf, int sendcount, MPI_Datatype sendtype,  
                        void *recvbuf, const int recvcounts[], const int displs[],  
                        MPI_Datatype recvtype, MPI_Comm comm)
```

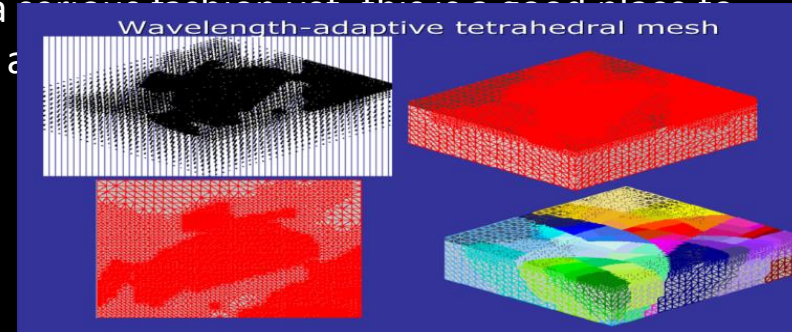
```
MPI_Neighbor_alltoallv( void *sendbuf, int sendcounts[], int sdispls[], MPI_Datatype sendtype,  
                        void *recvbuf, int recvcounts[], int rdispls[], MPI_Datatype recvtype, MPI_Comm comm)
```

The extra flexible w version allows us to mix different types. This can be useful to allow an efficient in-place transfer (think about exchanging columns/row datatypes in this one call).

```
MPI_Neighbor_alltoallw( void *sendbuf, int sendcounts[], MPI_Aint sdispls[], MPI_Datatype sendtypes[],  
                        void *recvbuf, int recvcounts[], MPI_Aint rdispls[], MPI_Datatype recvtypes[], MPI_Comm comm)
```

and although we haven't delved into non-blocking routines in a serious fashion yet, this is a good place to mention that we will have non-blocking versions of all of these a

```
MPI_Ineighbor_allgather()  
MPI_Ineighbor_allgatherv()  
MPI_Ineighbor_alltoall()  
MPI_Ineighbor_alltoallv()  
MPI_Ineighbor_alltoallw()
```



A Peek Behind The Curtain



You can ignore the ne

However, many of you
"Why are there these

Since I always get tho

Do not get intimidate
follow along and we v

e is passed?", or

gh these details.

use MPI. Just

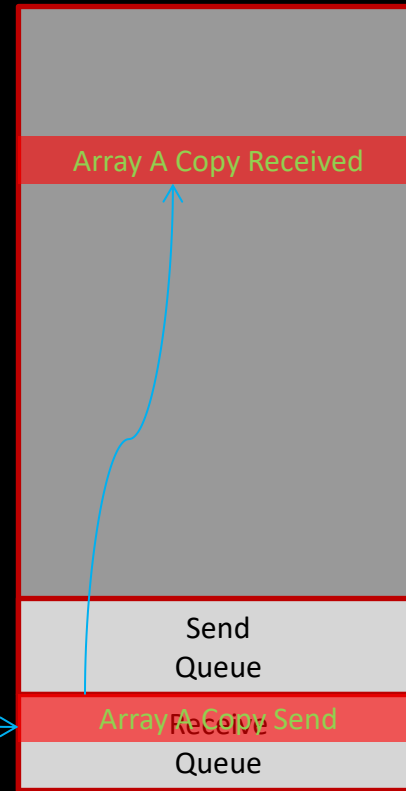
Default MPI Messaging

PE 0



"MPI_Send"

PE 1



"MPI_Recv"

Send makes a copy

PE 0



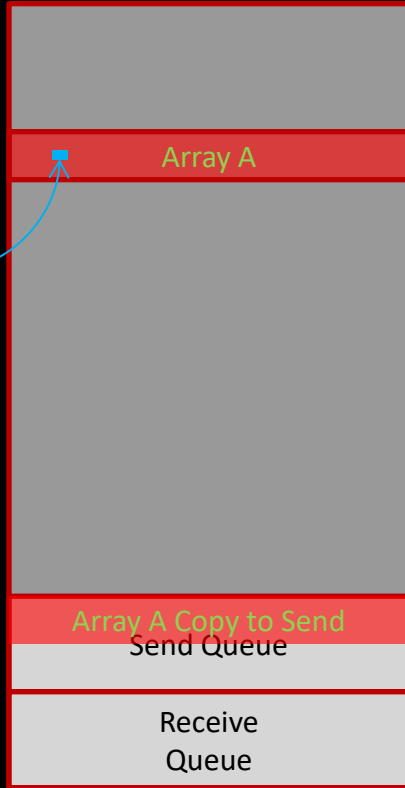
PE 1



“MPI_Send(A, PE 1)”

Why make a copy?

PE 0

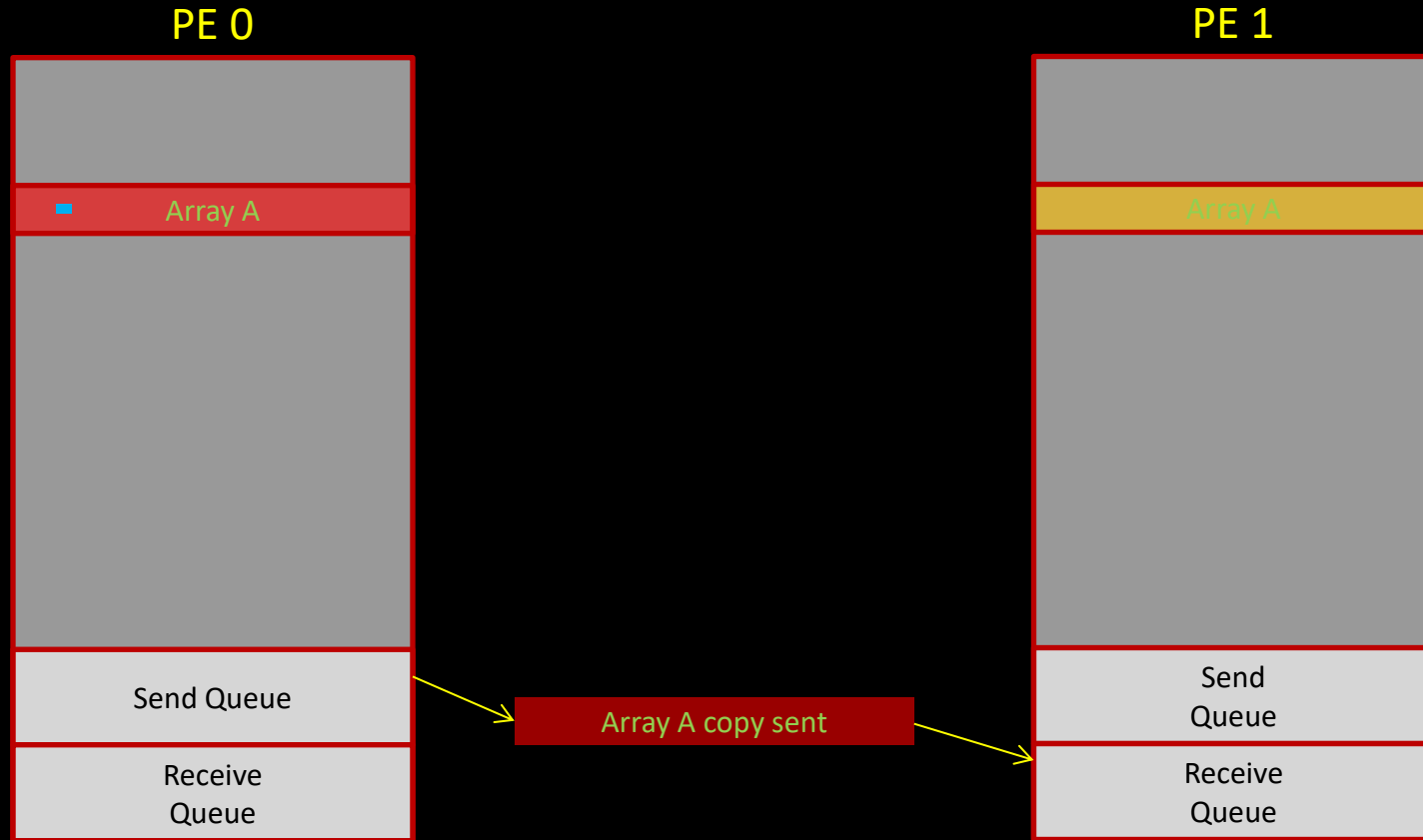


"A[12][32] = 18"

PE 1



Message in flight



“X = Y + Z * 4”

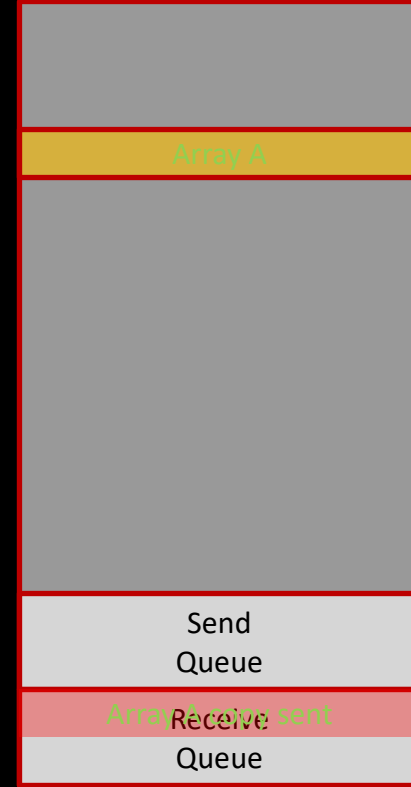
Sitting in receive queue

PE 0



"w = 3.14159 * 2"

PE 1



Array A received

Received

PE 0



“goto some_where”

PE 1



“MPI_Recv(A, PE 0)”

Receive Queue

The message queue contains all of the message that have already arrived. You can just grab the next one in the queue with MPI_ANY_SOURCE and MPI_ANY_TAG, or you can be selective.

Source	Tag	Results in Data
1	2	4444
3	2	1008*
2	1	43*
2	2	56
ANY_SOURCE	2	1008 or 4444 or 56*
1	ANY_TAG	4444*

* MPI's only ordering guarantee is that messages from the same source will stay in order. So, you will receive and retrieve the first sent message from any source first.

Note that communicators can also function as filters, but data types and data counts **do not**. Mismatched data types are an error, and may not even be detected at runtime!

Message Queue on PE 0

Data	Source	Tag
43	2	1
1008	3	2
4444	1	2
80999	2	1
5345	3	1
9044	1	1
5666	3	1
339	2	1
346	3	2
789879	2	1
78942	2	1
56	2	2
44509	1	1

Non-Blocking Sends and Receives

All of the receives that we are using here are blocking. This means that they will wait until a message matching their requirements for source and tag has been received into the queue.

The default Sends try not to block, but don't guarantee it. As your data structures grow large, this blocking behavior may well emerge.

Blocking MPI_Send

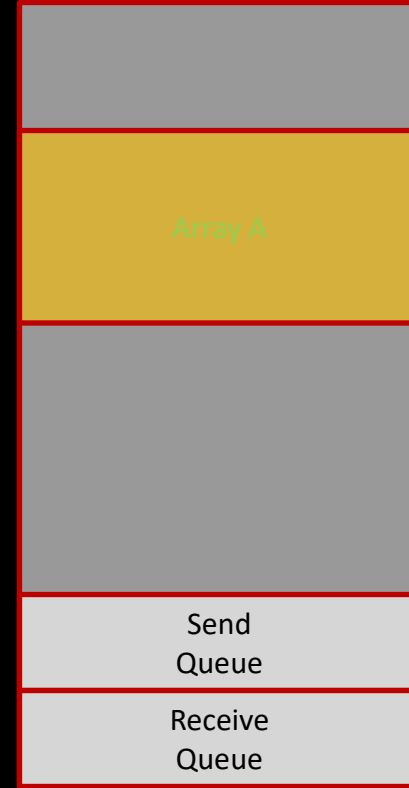
PE 0



`"MPI_Send(A, PE 1)"`

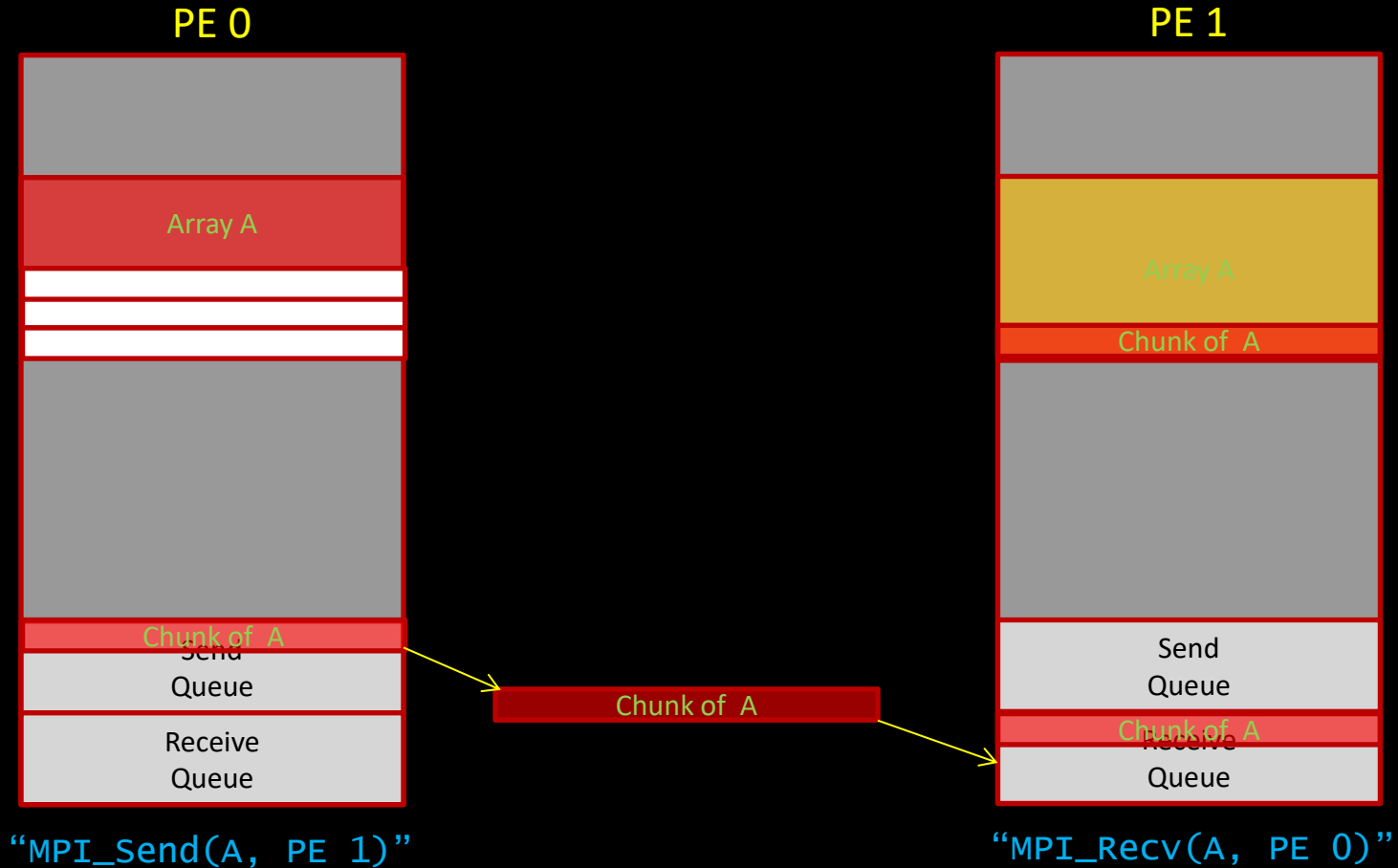
`"A[12][32] = 18"`

PE 1



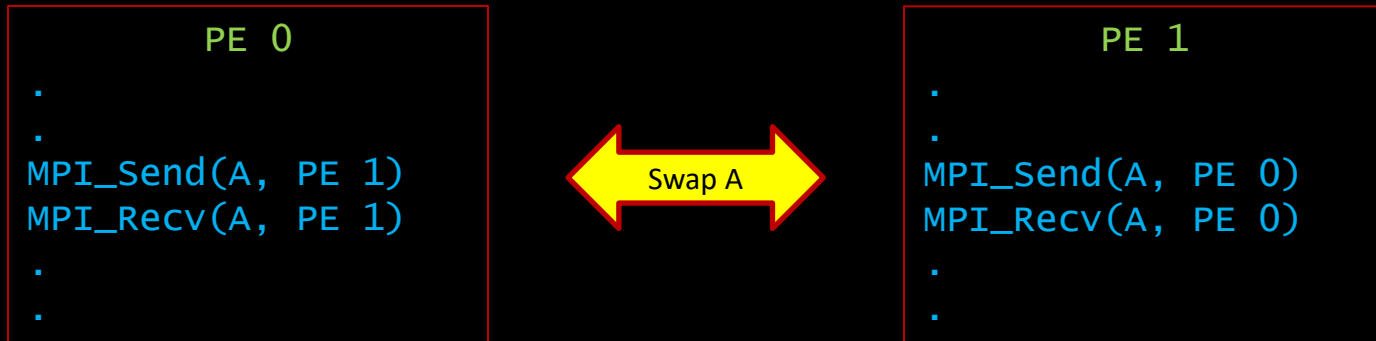
`"X = Y + 4"`

Receive Issued

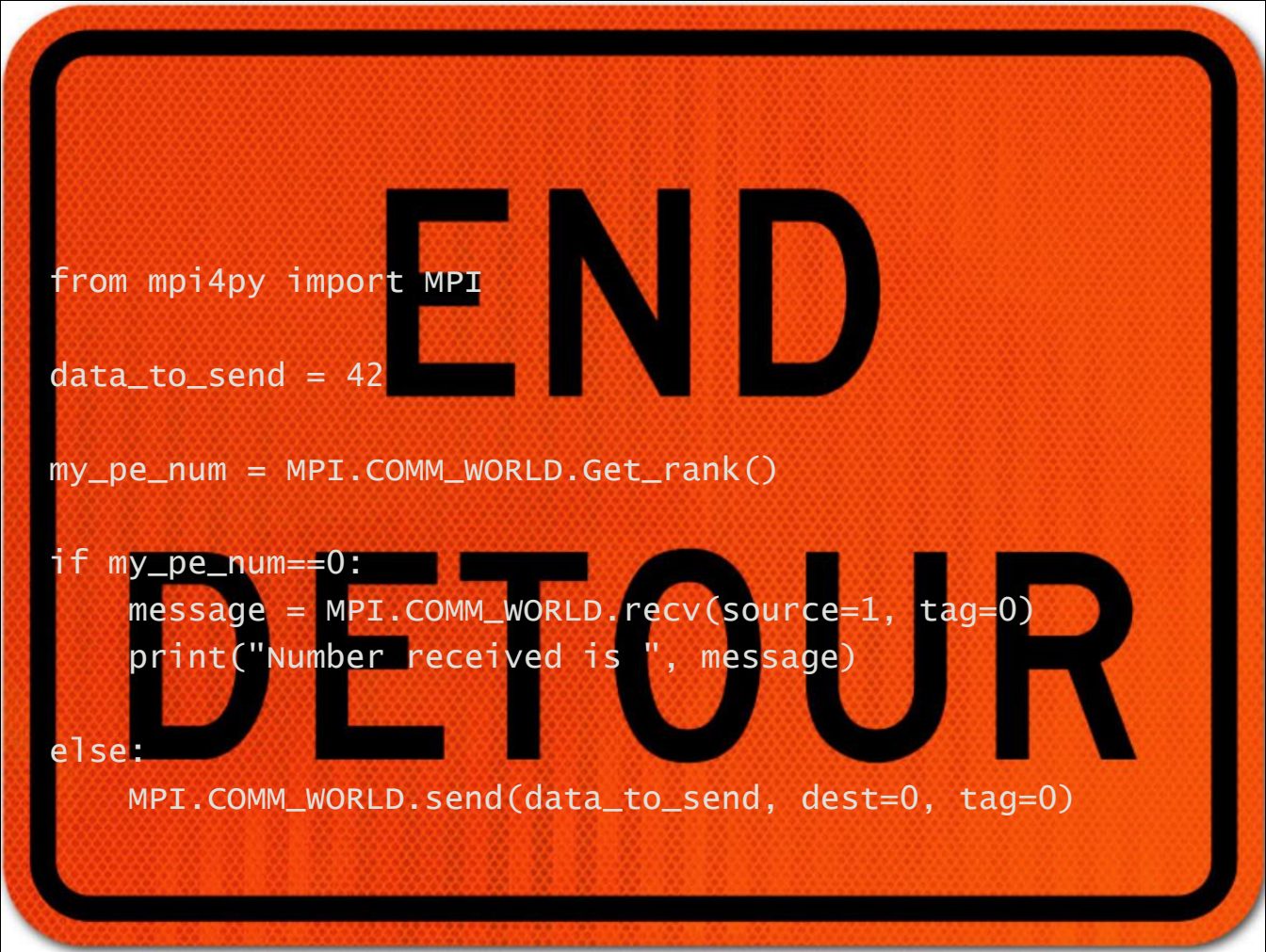


Non-Blocking Sends and Receives

This may seem like a strictly performance issue, but it is not. What about the very common case of:



If both PEs block on Send, we have a **deadlock**. The code will hang forever.



```
from mpi4py import MPI
```

```
data_to_send = 42
```

```
my_pe_num = MPI.COMM_WORLD.Get_rank()
```

```
if my_pe_num==0:
```

```
    message = MPI.COMM_WORLD.recv(source=1, tag=0)
```

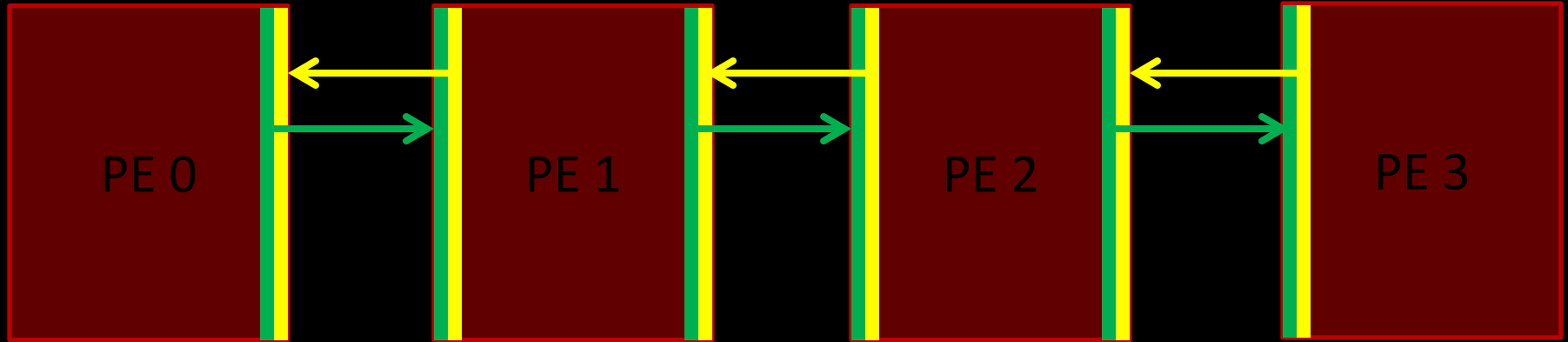
```
    print("Number received is ", message)
```

```
else:
```

```
    MPI.COMM_WORLD.send(data_to_send, dest=0, tag=0)
```

Not just a theoretical consideration

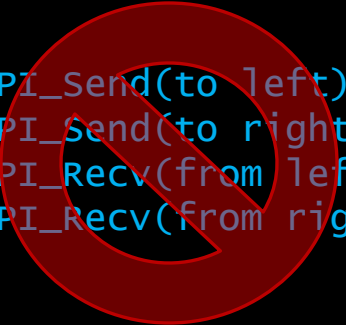
The Laplace solver is usually parallelized with a send/receive pattern like this:



The classic "ghost zone" data exchange.

Two Blocking Methods

There are two similar ways of coding this that we might try:



```
MPI_Send(to left)
MPI_Send(to right)
MPI_Recv(from left)
MPI_Recv(from right)
```

```
MPI_Send(to left)
MPI_Recv(from right)
MPI_Send(to right)
MPI_Recv(from left)
```

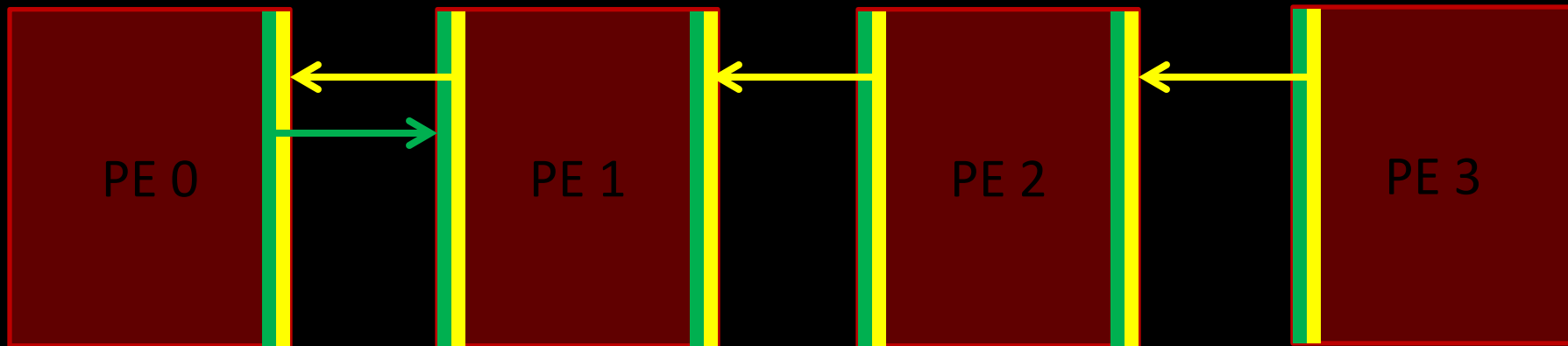
On Blue Waters they both work OK solving the beginning 1000x1000 problem. But when we scale up to the full competition size (10000x10000) one of them hangs. Where?

PE's 1-3 are blocking sending to the left, and PE 0 is blocking on the send to the right.

Hung...

PE's 1-3 are blocking sending to the left, and PE 0 is blocking on the send to the right.

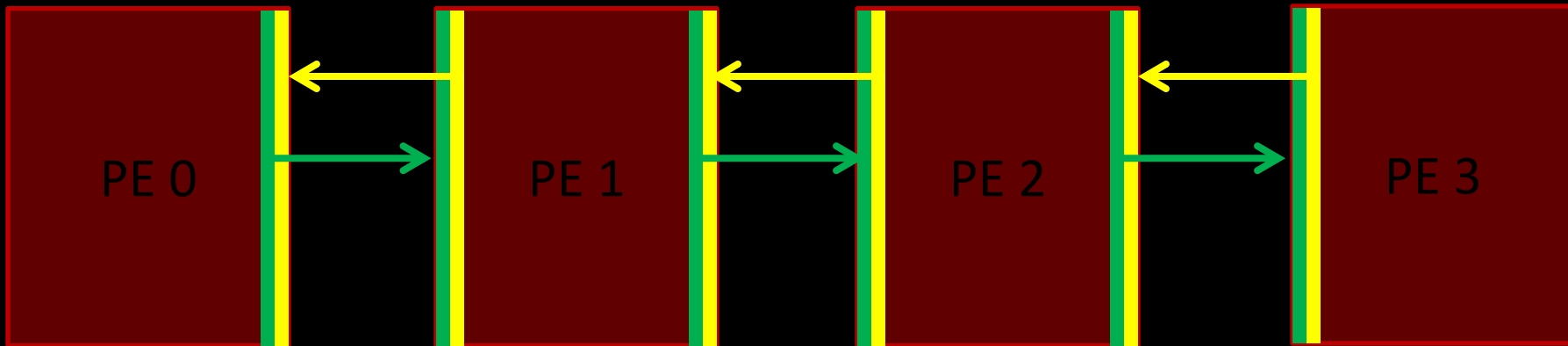
```
MPI_Send(to left)
MPI_Send(to right)
MPI_Recv(from left)
MPI_Recv(from right)
```



Is our other solution truly the answer? *Note that using `MPI_Ssend()` here would have caught this problem right away!*

Cascading Messages

At least the second solution doesn't hang. But it does results in a sequential process here that we don't really want.



We can do better, and without any complication.

```
MPI_Send(to left)
MPI_Recv(from right)
MPI_Send(to right)
MPI_Recv(from left)
```

Basic Non-Blocking Routines

As we said before, it is no big deal to modify your typical blocking algorithm to do this. Add an “l” to the routine name, a “request” flag parameter, and check to see when operations have completed before using the data.

```
MPI_Isend(void *buf, int count, MPI_Datatype datatype, int dest, int tag, MPI_Comm comm, MPI_Request *request)
```

```
MPI_Irecv(void *buf, int count, MPI_Datatype datatype, int source, int tag, MPI_Comm comm, MPI_Request *request)
```

```
MPI_Test(MPI_Request *request, int *flag, MPI_Status *status)
```

```
MPI_Wait(MPI_Request *request, MPI_Status *status)
```

Also:

```
MPI_Waitall()  
MPI_Waitany()  
MPI_Waitsome()  
MPI_Testall()  
MPI_Testany()  
MPI_Testsome()
```

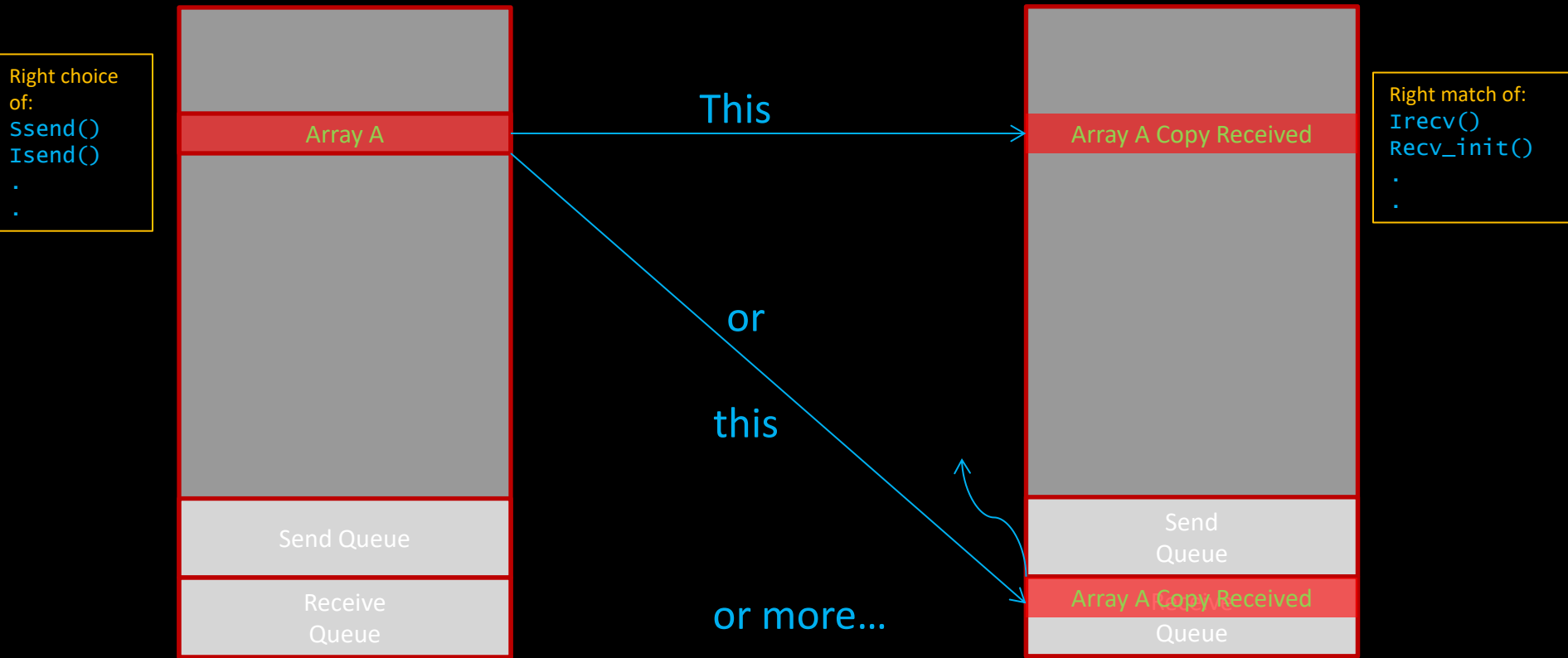
And the useful variations:

```
MPI_Send_Init()  
MPI_Recv_Init()  
MPI_Start()  
MPI_StartAll()
```

If you have unchanging communication patterns, you should really use these.

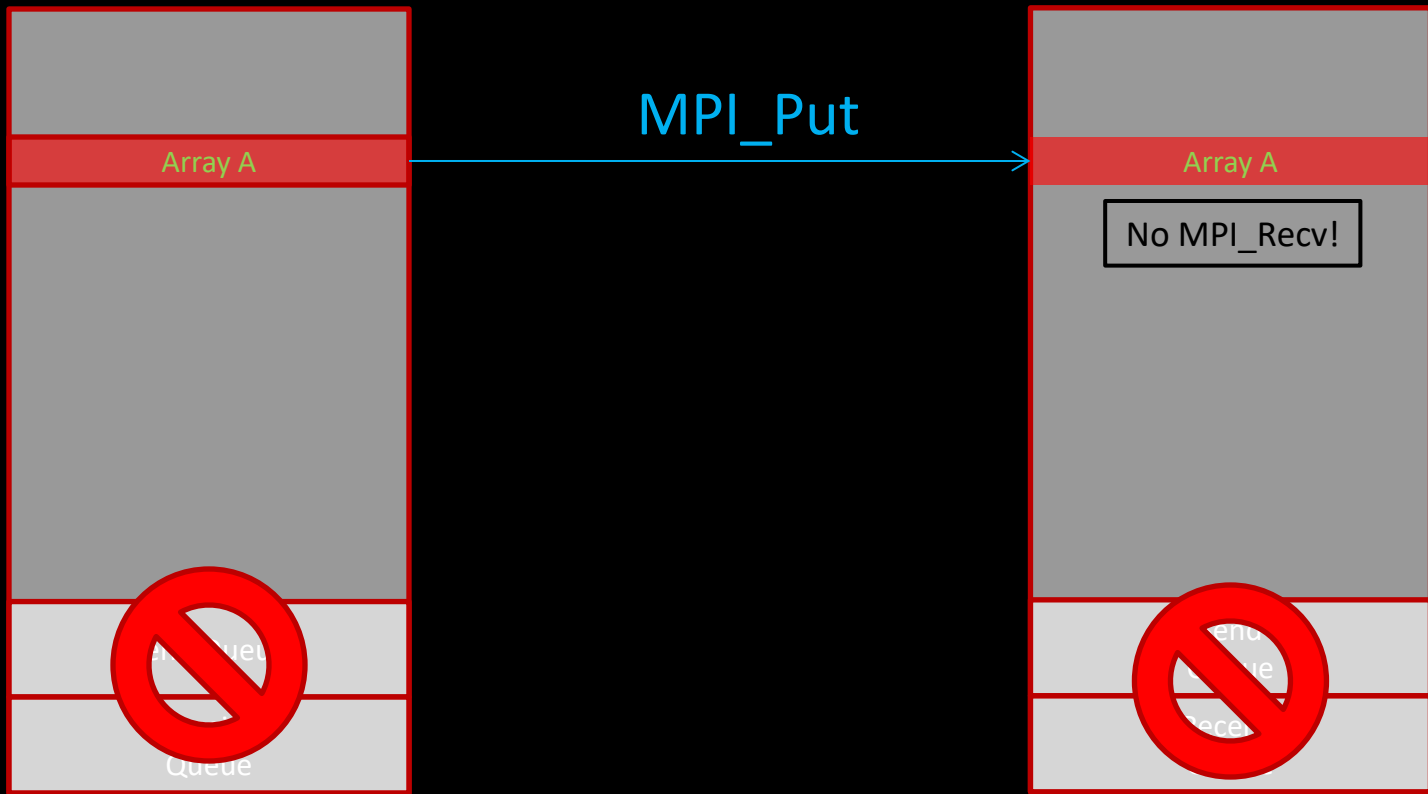
Optimized MPI Messaging

We now have the tools to control much of the buffering and synchronization of our cooperative message passing. If both ends do the right thing.



The directness of Put

Indeed, singled-sided communications has the ultimately direct capability of just cramming data into another PE at will. This has the previously mentioned advantages. Let's consider some of our obligations when using this method.



Moving Data

The routines used to move the data are pretty basic:

```
MPI_Put(void *origin_addr, int origin_count, MPI_Datatype origin_datatype, int target_rank, MPI_Aint target_disp,
        int target_count, MPI_Datatype target_datatype, MPI_Win win)
```

```
MPI_Get(void *origin_addr, int origin_count, MPI_Datatype origin_datatype, int target_rank, MPI_Aint target_disp,
        int target_count, MPI_Datatype target_datatype, MPI_Win win)
```

```
MPI_Accumulate(void *origin_addr, int origin_count, MPI_Datatype origin_datatype, int target_rank, MPI_Aint target_disp,
              int target_count, MPI_Datatype target_datatype, MPI_Op op, MPI_Win win)
```

For accumulate the operations can only be the predefined ones, and the basic *components* of the datatypes must be all the same predefined types (vector of MPI_INT fine).

There are versions of these routines that return request flags as well.

```
MPI_Rput(void *origin_addr, int origin_count, MPI_Datatype origin_datatype, int target_rank, MPI_Aint target_disp,
         int target_count, MPI_Datatype target_datatype, MPI_Win win, MPI_Request *request)
```

```
MPI_Rget(void *origin_addr, int origin_count, MPI_Datatype origin_datatype, int target_rank, MPI_Aint target_disp,
         int target_count, MPI_Datatype target_datatype, MPI_Win win, MPI_Request *request)
```

```
MPI_Accumulate(void *origin_addr, int origin_count, MPI_Datatype origin_datatype, int target_rank, MPI_Aint target_disp,
              int target_count, MPI_Datatype target_datatype, MPI_Op op, MPI_Win win, MPI_Request *request)
```

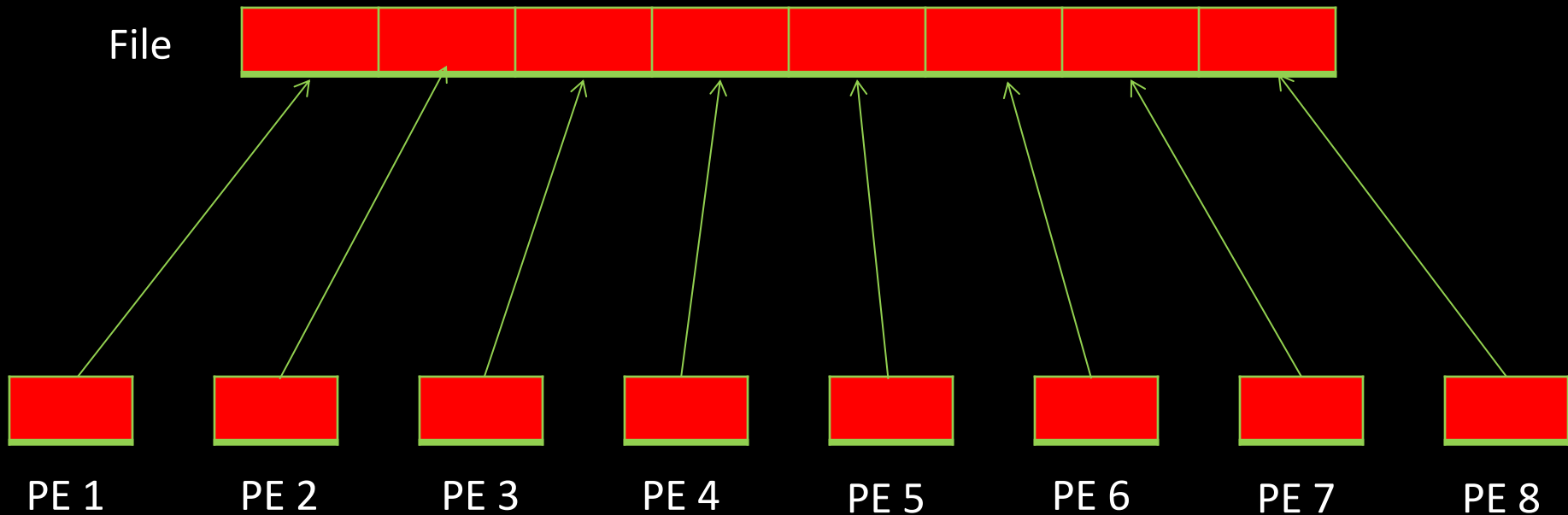
MPI Communication Hierarchy

Let's review all of the mechanisms available. Ranked by a general scalability and efficiency.

1. Blocking Message Passing
2. Non-blocking Message Passing
3. RMA Active Target ("fence")
4. RMA Passive Target ("Win_lock")
5. RMA Scalable Synchronization ("Win_post")

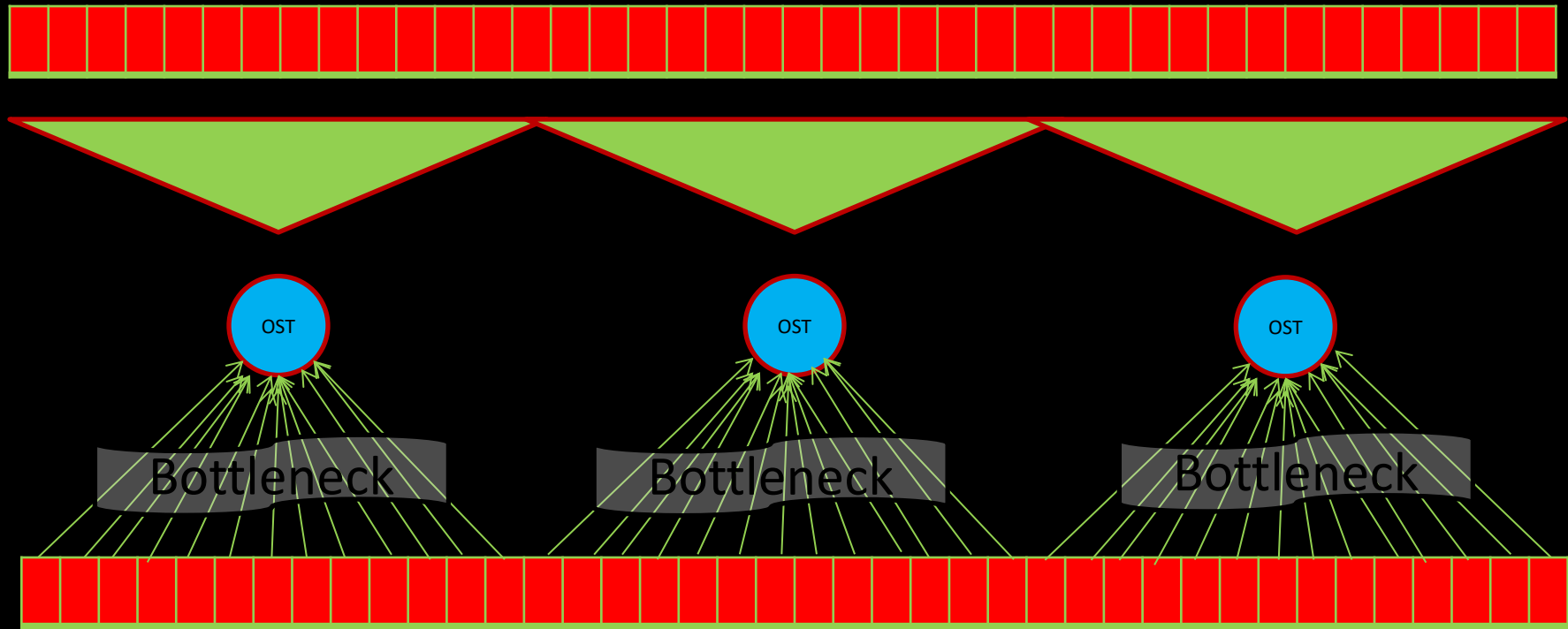
Parallel IO

A baseline IO operation for an parallel code might be something like the below for checkpointing a very regular data structure. A more difficult case might be with every PE doing irregular and random data reads and writes. But even this case is sufficient to demonstrate a typical IO bottleneck.



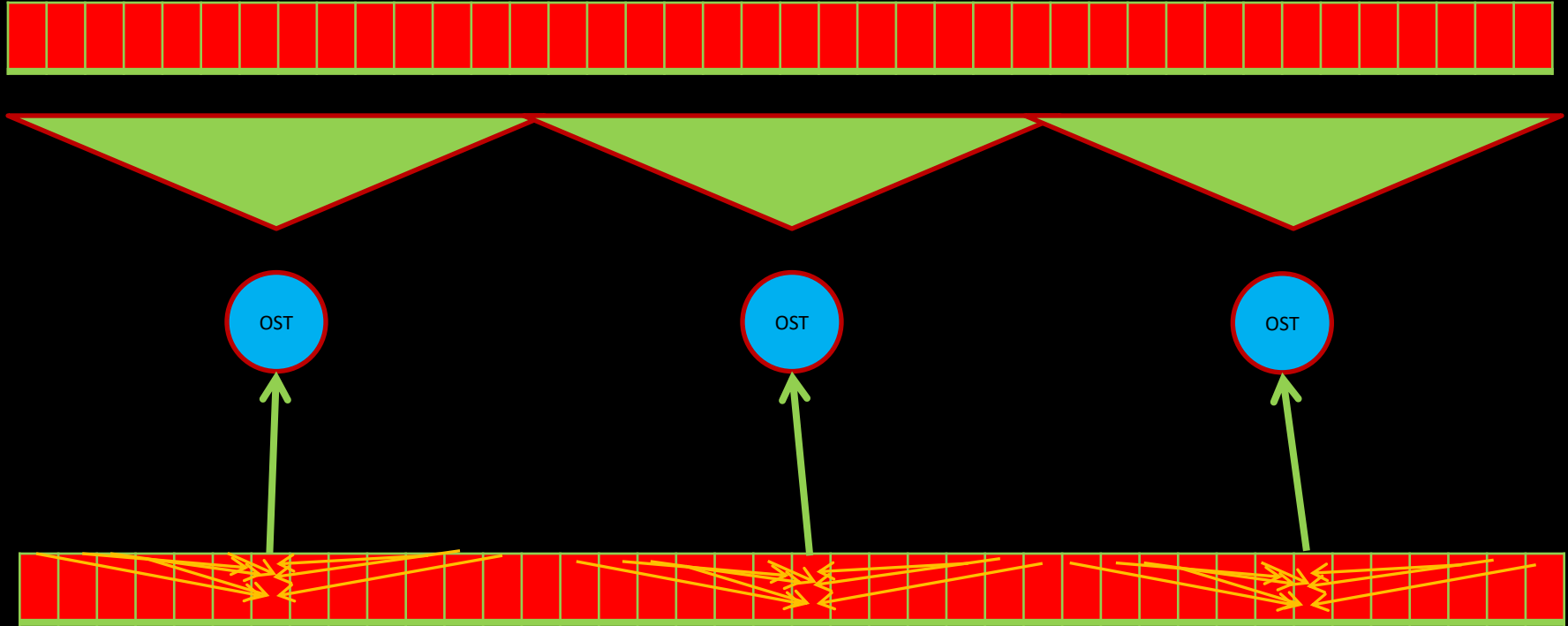
Parallel Disk System

A physical disk system looks something like the below (a simplified Lustre):

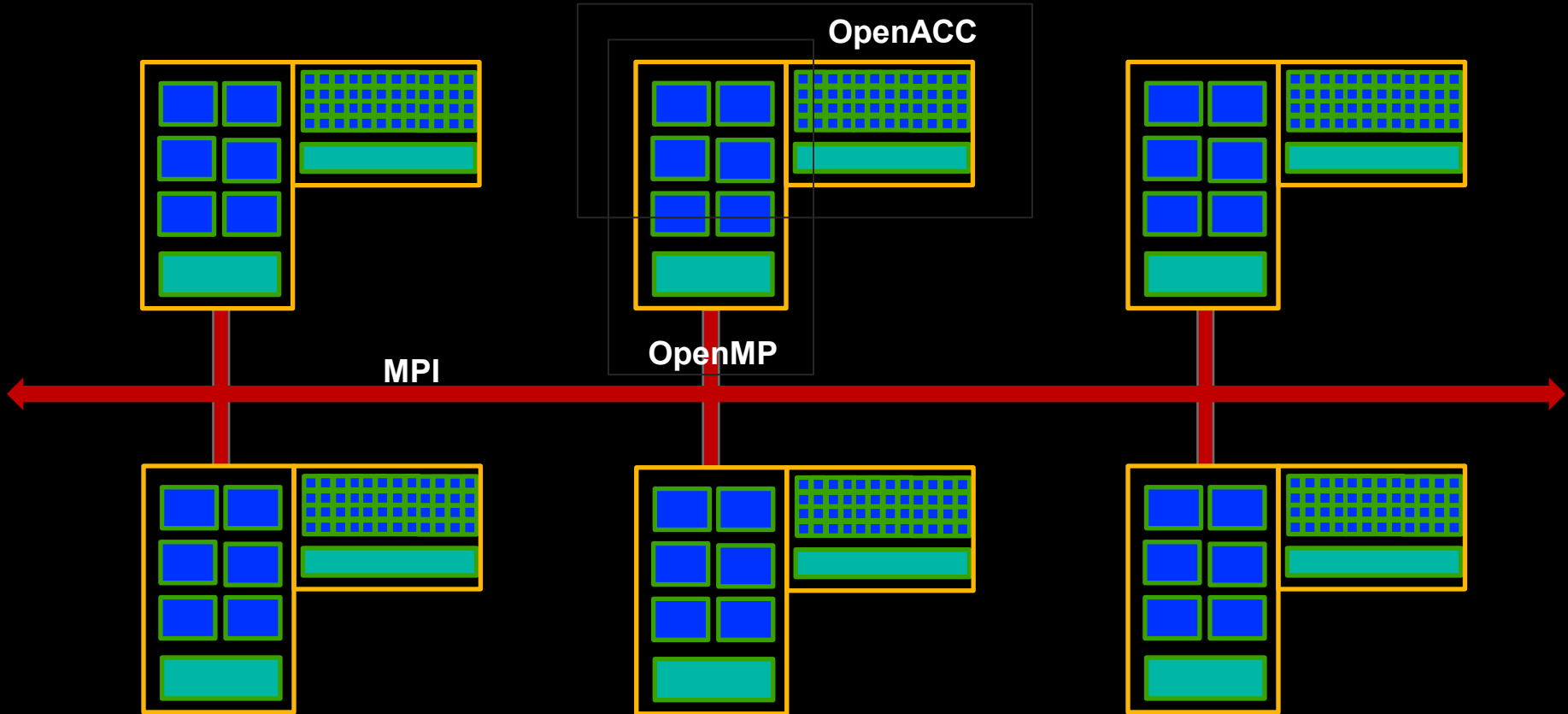


Optimized Parallel IO

Often, given the specific bandwidths and other limits, the optimal solution will look like:



The pieces fit like this...



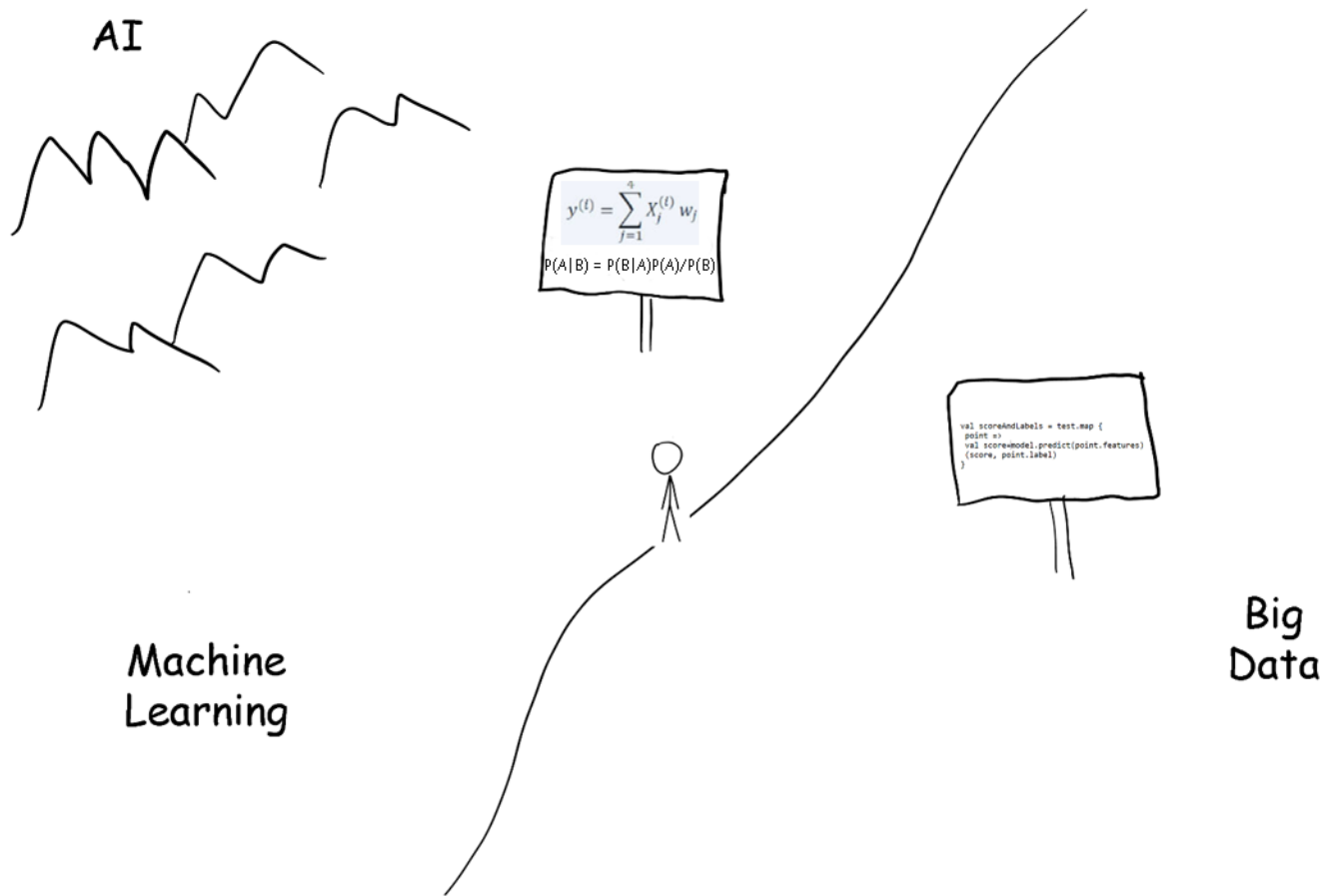
Pandas & SQL

John Urbanic

Parallel Computing Scientist
Pittsburgh Supercomputing Center

Distinguished Service Professor
Carnegie Mellon University

The landscape today.



As the Data Scientist wanders across the ill-defined boundary between Data Science and Machine Learning, in search of the fabled land of Artificial Intelligence, they find that the language changes from programming to a creole of linear algebra and probability and statistics.

Data Science Today

- Basic Data

- Pandas



- "Serious" Data Science

- SQL



- Big Data

- Spark



Machine Learning / AI

Pandas

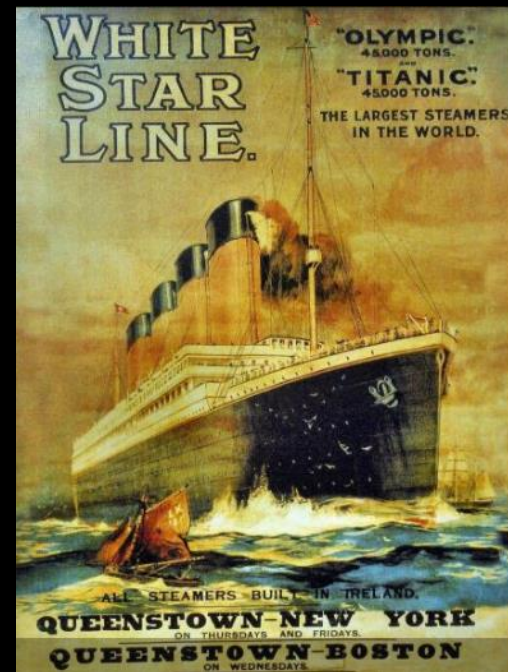


- Pandas has become the standard Python way to input, manipulate and write basic data.
- It also integrates well with other tools, like visualizing with Matplotlib.
- It has limitations, which is why SQL and big data techniques are essential for many tasks. But for quick-and-dirty, or limited applications it is very efficient.
- In many Python environments, it is there by default. If not, it is easy to add. In this course, if you start a python shell, it will be there.

Our First Dataset

We will begin our exploration of Pandas using a well known dataset drawn from the infamous Titanic disaster.

It has a variety of data on each of 891 passengers.



Amongst the typical demographic data is included their survival. It enables an interesting, if somewhat morbid, analysis to determine the foremost factors in survival. Women and children first? Or, save the rich?

Getting Started with Pandas

This "pd" is very standard

Smart, understands "csv"

```
import pandas as pd
```

```
titanic = pd.read_csv("titanic.csv")
```

```
titanic
```

```
   PassengerId  Survived  Pclass
0            1         0        3
4            5         0        3
5            6         0        3
6            7         0        1
7            8         0        3
..         ...         ...      ...
883         884         0        2
884         885         0        3
886         887         0        2
889         890         1        1
890         891         0        3
```

[577 rows x 12 columns]

Format Type	Data Description	Reader	Writer
text	CSV	<code>read_csv</code>	<code>to_csv</code>
text	Fixed-Width Text File	<code>read_fwf</code>	
text	JSON	<code>read_json</code>	<code>to_json</code>
text	HTML	<code>read_html</code>	<code>to_html</code>
text	LaTeX		<code>Styler.to_latex</code>
text	XML	<code>read_xml</code>	<code>to_xml</code>
text	Local clipboard	<code>read_clipboard</code>	<code>to_clipboard</code>
binary	MS Excel	<code>read_excel</code>	<code>to_excel</code>
binary	OpenDocument	<code>read_excel</code>	
binary	HDF5 Format	<code>read_hdf</code>	<code>to_hdf</code>
binary	Feather Format	<code>read_feather</code>	<code>to_feather</code>
binary	Parquet Format	<code>read_parquet</code>	<code>to_parquet</code>
binary	ORC Format	<code>read_orc</code>	
binary	Stata	<code>read_stata</code>	<code>to_stata</code>
binary	SAS	<code>read_sas</code>	
binary	SPSS	<code>read_spss</code>	
binary	Python Pickle Format	<code>read_pickle</code>	<code>to_pickle</code>
SQL	SQL	<code>read_sql</code>	<code>to_sql</code>
SQL	Google BigQuery	<code>read_gbq</code>	<code>to_gbq</code>

```
   Fare  Cabin Embarked
0  2500   NaN        S
4  9000   NaN        S
5  3538   NaN        Q
6  5380   E46        S
7  8050   NaN        S
..   ...   ...      ...
883  5000   NaN        S
884  5000   NaN        S
886  8000   NaN        S
889  8000  C148        C
890  7500   NaN        Q
```

Survived (0 = No; 1 = Yes)

Pclass Passenger Class (1 = 1st; 2 = 2nd; 3 = 3rd)

Name

Sex

Age

SibSp Number of Siblings/Spouses Aboard

Parch Number of Parents/Children Aboard

Ticket Ticket Number

Fare Fare (British pound)

Cabin Cabin number

Embarked Port of Embarkation (C = Cherbourg; Q = Queenstown; S = Southampton)

DataFrame Queries

```
titanic["Name"]
```

```
0          Braund, Mr. Owen Harris
1  Cumings, Mrs. John Bradley (Florence Briggs Th...
2          Heikkinen, Miss. Laina
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)
4          Allen, Mr. William Henry
...
886      Montvila, Rev. Juozas
887      Graham, Miss. Margaret Edith
888  Johnston, Miss. Catherine Helen "Carrie"
889      Behr, Mr. Karl Howell
890      Dooley, Mr. Patrick
```

DataFrame Queries

```
titanic[["Age","Sex"]]
```

	<i>Age</i>	<i>Sex</i>
<i>0</i>	<i>22.0</i>	<i>male</i>
<i>1</i>	<i>38.0</i>	<i>female</i>
<i>2</i>	<i>26.0</i>	<i>female</i>
<i>3</i>	<i>35.0</i>	<i>female</i>
<i>4</i>	<i>35.0</i>	<i>male</i>
...	...	...
<i>886</i>	<i>27.0</i>	<i>male</i>
<i>887</i>	<i>19.0</i>	<i>female</i>
<i>888</i>	<i>NaN</i>	<i>female</i>
<i>889</i>	<i>26.0</i>	<i>male</i>
<i>890</i>	<i>32.0</i>	<i>male</i>

DataFrame Conditional Queries

```
titanic[titanic["Age"]>30]
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	S
11	12	1	1	Bonnell, Miss. Elizabeth	female	58.0	0	0	113783	26.5500	C103	S
..	...	...	...	...	...	...	...	...	...	...	...	...
873	874	0	3	Vander Cruyssen, Mr. Victor	male	47.0	0	0	345765	9.0000	NaN	S
879	880	1	1	Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)	female	56.0	0	1	11767	83.1583	C50	C
881	882	0	3	Markun, Mr. Johann	male	33.0	0	0	349257	7.8958	NaN	S
885	886	0	3	Rice, Mrs. William (Margaret Norton)	female	39.0	0	5	382652	29.1250	NaN	Q
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	NaN	Q

DataFrame Sorting

```
titanic.sort_values(by="Age")
```

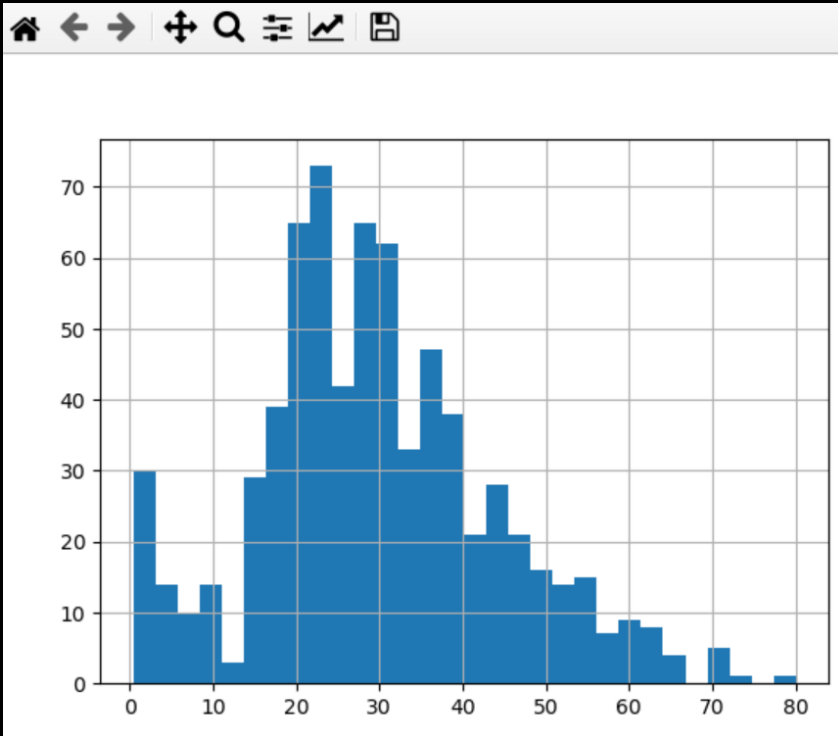
	Name	Age
803	Thomas, Master. Assad Alexander	0.42
755	Hamalainen, Master. Viljo	0.67
644	Baclini, Miss. Eugenie	0.75
469	Baclini, Miss. Helene Barbara	0.75
78	Caldwell, Master. Alden Gates	0.83
..	...	...
859	Razi, Mr. Raihed	NaN
863	Sage, Miss. Dorothy Edith "Dolly"	NaN
868	van Melkebeke, Mr. Philemon	NaN
878	Laleff, Mr. Kristo	NaN
888	Johnston, Miss. Catherine Helen "Carrie"	NaN

```
titanic.sort_values(by="Age")
```

	Name	Age
803	Thomas, Master. Assad Alexander	0.42
755	Hamalainen, Master. Viljo	0.67
644	Baclini, Miss. Eugenie	0.75
469	Baclini, Miss. Helene Barbara	0.75
78	Caldwell, Master. Alden Gates	0.83
..	...	...
859	Razi, Mr. Raihed	NaN
863	Sage, Miss. Dorothy Edith "Dolly"	NaN
868	van Melkebeke, Mr. Philemon	NaN
878	Laleff, Mr. Kristo	NaN
888	Johnston, Miss. Catherine Helen "Carrie"	NaN

If you like pictures (matplotlib)

```
import matplotlib.pyplot as plt  
titanic["Age"].hist(bins=30)  
plt.show()
```



This assumes you have an X server running on your laptop.

Which I do.

You may want to enable an X server on your laptop with something like MobaXterm in Windows or XQuartz on Mac.

Use -X with ssh if you don't have it set up to automatically do so.

Optional exercise: Can we find a significant survival variable?

Can you find a significant factor in the data which could be used to predict survival rates?

I will suggest you focus on one variable at a time.

Note that there are many possible answers. Going from a simple hypothesis ("Maybe people from Cherbourg are unlucky?") to a more complex formula incorporating multiple variables - with the goal of a more accurate prediction - is the path of data analysis. This is our first step on that journey.



- We are going to use a Virtual Machine for this optional exercise and our next Assignment. It is called msdas.psc.edu and you can `ssh` there like you would to Bridges.
- Copy the titanic dataset (using the `cp` command) from `~urbanic/LargeScaleComputing/Titanic/titanic.csv` to your own directory.
- Start a python shell (this assumes you have the anaconda module still loaded, otherwise do so).
- Find a meaningful factor and feel free to send me your script and results if you want my feedback.



SQL

What is a "Relational Database"?

An RDBMS (Relational DataBase Management System) organizes data into tables of columns (attributes, fields) and rows (records).

This concept has been developed and refined since 1970, and is a mature concept at this point.

Most RDBMSs use SQL as their query language. This has become an ISO standard (with many deviations).

What Is MySQL?

MySQL is an open source* RDBMS originating in 1995. It has spun off forks, and it has open source peers (most notably PostgreSQL) and commercial alternatives (Oracle and MS SQL Server). These each have their own deviations from the ISO standard, as well as significant performance differences.

MySQL operates as a server, with clients that connect from wherever, and in different languages: from JavaScript in some web page, or Java on the server side, or a program. We will be using a dedicated, if basic, MySQL client.



*MySQL was acquired by Oracle, so an alternative open source fork, MariaDB, was created.

In 2024, MariaDB was itself bought by the K1 private equity group.

Starting MySQL

We will use a MySQL client installed on the VM along with our database. To start it you need only log on to `msdas.psc.edu` and type:

```
[urbanic@msdas]$ mysql urbanic
mysql> SHOW DATABASES;
```

```
+-----+
| Database          |
+-----+
| performance_schema |
| urbanic            |
+-----+
2 rows in set (0.01 sec)
```

This shows us the available databases. By starting mysql with the command "mysql urbanic" I have loaded my own personal database already. Make sure to use your own username to start mysql, not "urbanic". We could also use the command `USE urbanic` to select this database at any time.

Note that all SQL commands end with a ";". Case matters, but SQL keywords can be upper or lower case. I will use upper case for them as that is a common convention and makes it clear what they are as you are learning.

Also note that SQL code formatting *varies wildly* and is inconsistent. This talk will adhere to that tradition. Although no one seems to care, there is a supposed standard, and the best summary of it I can find is here:

https://www.isbe.net/Documents/SQL_server_standards.pdf

Showing Our Tables

The structure, or schema, is the most important characteristic of any database. We can get a top level view by first listing the tables.

```
mysql> SHOW TABLES;
```

```
+-----+  
|Tables_in_urbanic|  
+-----+  
| Customer        |  
| Line            |  
| OrderDetail     |  
| Orders          |  
| Product         |  
| Vendor          |  
| Zips            |  
+-----+  
7 rows in set (0.00 sec)
```

From here on out I will drop the *mysql>* prompt from our examples. We are always working in the SQL client shell.

Showing Table Fields

And each table has fields, or columns. We can list them as so.

```
SHOW COLUMNS FROM Orders;
```

Field	Type	Null	Key	Default	Extra
orderId	int	NO		0	
Date	date	NO		NULL	
DateRequired	date	NO		NULL	
DateShipped	date	YES		NULL	
Status	varchar(15)	NO		NULL	
Comments	text	YES		NULL	
CustomerId	int	NO		NULL	

7 rows in set (0.00 sec)

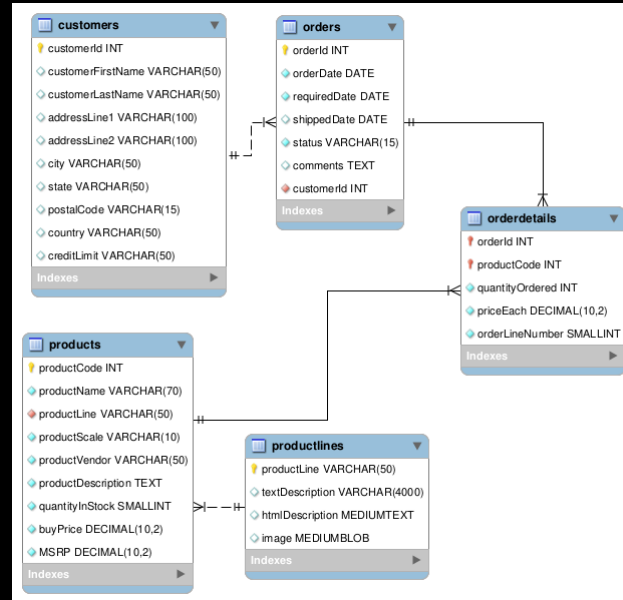
Each field has a type, and some have a size. There are 30+ types, but they are mostly obvious variations of strings, numbers and dates. There are some fancy Spatial, JSON and binary blob types as well. You can find a full list at:

<https://dev.mysql.com/doc/refman/8.0/en/data-types.html>

There are some other features attached to fields that we will get to later.

MySQL Workbench

MySQL Workbench is a wonderful tool for working with MySQL databases. If we were going to work deeply with SQL, we should surely involve this more in our work. However, in keeping with our theme of minimal distractions while we investigate the core concepts, we will only use it to create nice schematics.



These are most of our current tables. We will see what some of those little icons and arrows mean later.

SELECT

The SELECT command is our most useful command in manipulating data, and we will look at some of the common variations.

```
SELECT * FROM Customer;
```

CustomerId	FirstName	LastName	AddressLine1	AddressLine2	City	State	PostalCode	Country	CreditLimit
1	Mary	Yates	1414 East Anderson Street	#317	Savannah	GA	31404	USA	100
2	James	Parker	29 Lucian Street		Manchester	CT	06040	USA	100
3	Kim	Bond	1421 Floral Street Northwest		Washington	DC	20012	USA	100
4	Thomas	Broadnax	1915 Southeast 29th Street		Oklahoma City	OK	73129	USA	100
5	Stephen	Williams	9805 South Youngs Lane		Oklahoma City	OK	73159	USA	100
6	Mark	Hinton	8642 Yule Street		Arvada	CO	80007	USA	100
7	Deborah	Lloyd	5244 West Port Au Prince Lane		Glendale	AZ	85306	USA	100
8	Linda	Barnes	3377 Sandstone Court		Pleasanton	CA	94588	USA	100

...
whoops, too many!

```
SELECT * FROM Customer LIMIT 5;
```

CustomerId	FirstName	LastName	AddressLine1	AddressLine2	City	State	PostalCode	Country	CreditLimit
1	Mary	Yates	1414 East Anderson Street	#317	Savannah	GA	31404	USA	100
2	James	Parker	29 Lucian Street		Manchester	CT	06040	USA	100
3	Kim	Bond	1421 Floral Street Northwest		Washington	DC	20012	USA	100
4	Thomas	Broadnax	1915 Southeast 29th Street		Oklahoma City	OK	73129	USA	100
5	Stephen	Williams	9805 South Youngs Lane		Oklahoma City	OK	73159	USA	100

5 rows in set (0.00 sec)

Note how * is our "wildcard" for all the fields.

SELECTING Fields

We can select only the fields of interest

```
SELECT FirstName, LastName FROM Customer;
```

FirstName	LastName
Mary	Yates
James	Parker
Kim	Bond
Thomas	Broadnax
Stephen	Williams
Mark	Hinton
...	
...	

And we can sort them

```
SELECT FirstName, LastName FROM Customer ORDER BY LastName;
```

FirstName	LastName
Michael	Aaberg
Denver	Aaberg
Brenda	Aaberg
Mabel	Aaberg
Gary	Aaberg
Ellen	Aaberg
Frankie	Aaberg
Edward	Aaberg
...	
...	

SELECTING Rows

We can select specific rows with the WHERE command.

```
SELECT * FROM Customer WHERE CustomerId = 1;
```

CustomerId	FirstName	LastName	AddressLine1	AddressLine2	City	State	PostalCode	Country	CreditLimit
1	Mary	Yates	1414 East Anderson Street	#317	Savannah	GA	31404	USA	100

1 row in set (0.00 sec)

```
SELECT * FROM Orders WHERE CustomerId = 1;
```

OrderId	Date	DateRequired	DateShipped	Status	Comments	CustomerId
390977	2007-08-09	2007-08-29	2007-09-04	OK		1
396900	2007-10-15	2007-10-21	2007-10-29	OK		1
472220	2010-02-21	2010-03-02	2010-03-10	OK		1
486581	2010-08-04	2010-08-10	2010-08-10	OK		1
487083	2010-08-10	2010-08-12	2010-08-16	OK		1
513816	2011-06-14	2011-06-22	2011-06-14	OK		1
546268	2012-06-20	2012-06-23	2012-06-28	OK		1
585992	2013-09-16	2013-09-29	2013-10-12	OK		1

8 rows in set (0.01 sec)

AGGREGATE FUNCTIONS

There are aggregate functions that we can apply to a column of data. For example, we could find the average retail price of all of our products

```
SELECT AVG(RetailPrice) FROM Product;
```

```
+-----+
| avg(RetailPrice) |
+-----+
|          90.173951 |
+-----+
1 row in set (0.02 sec)
```

The most common aggregate function is `COUNT()`, frequently used to count the number of rows in a table. `MAX()`, `MIN()`, `SUM()`, `AVG()` are others that you will see.

```
SELECT COUNT(*) FROM Product;
```

```
+-----+
| count(*) |
+-----+
|      20592 |
+-----+
```

Given that `*` is used in `SELECT` statements to select all the columns, a normal person might think that `COUNT(*)` is asking to somehow count multiple columns, but it is really just allowing SQL to pick whatever column it thinks is quickest to use to count the total number of rows in the table. Get used to this common/weird idiom.

GROUPING

Grouping is a very useful tool in data analysis. And we have a particular meaning for the word "grouping" in data science. It means a rearrangement of a data table such that one of the columns becomes the rows.

After this rearrangement, we usually have to decide which of the other columns we want to keep or combine.

Original Table

A	B	C	D
A0	B0	C0	D0
A1	B1	C1	D1
A2	B0	C2	D1
A3	B3	C3	D3
A4	B0	C4	D0
A5	B2	C5	D1
A6	B3	C6	D3
A7	B1	C7	D0
A8	B0	C8	D1
A9	B3	C9	D0

Grouped on B, Average of C

B	C(avg)
B0	AVG(C0,C2,C4,C8)
B1	AVG(C1,C7)
B2	AVG(C5)
B3	AVG(C3,C6,C9)

An example might be where we are logging pollution alarms, and the table is

Time	Station	Level	Supervisor
2023-4-2-11:23	Hampton	11.3	Smith
2023-5-2-12:33	Landsdale	0.42	Li
...	...	...	...

If we want an quick insight into where any serious problems are, we might want to find the average at each station.

We don't care about the timestamps, and we probably don't want to bring the supervisor data along.

We do exactly what we just did at the left: GROUP BY Station and AVG(Level).

GROUP BY and AS

As we build more complex queries, we will find it very useful, and often necessary, to alias a column or table with *AS*. The alias will only exist for the duration of the query. GROUP BY will often require this.

```
SELECT CustomerId, COUNT(*) AS NumOrders
FROM Orders
GROUP BY CustomerId
ORDER BY NumOrders DESC
LIMIT 10;
```

CustomerId	NumOrders
25806	25
94364	25
96968	24
33646	24
57572	23
89204	23
27518	23
29451	23
26682	23
36709	22

10 rows in set (1.72 sec)

GROUP BY will group rows that have the same value (CustomerId here) into summary rows, which are then used with aggregate functions (*COUNT()*, *MAX()*, *MIN()*, *SUM()*, *AVG()*) to reduce the results.

These aggregate functions reduce the data on a selected column. Here, COUNT gives us the number in each group.

we need *NumOrders* to capture that value for subsequent use in the ORDER BY, hence our need for AS.

Note the grouping can be done hierarchically. You might group your data first by towns, and then zip codes within.

ORDER OF OPERATIONS

One counter intuitive, but central notion, to SQL is that the listed order of the specified commands does not correspond to the order in which they are executed. There is a mandated order to the evaluation of the clauses.

1. FROM and JOINS

The FROM clause, and subsequent JOINS are first executed to determine the set of data that is being queried. This includes subqueries in this clause, and can cause temporary tables to be created.

2. WHERE

Then any WHERE constraints are applied to the individual rows, and rows that do not satisfy the constraint are discarded. Each of the constraints can only access columns directly from the tables requested in the FROM clause. Aliases in the SELECT part of the query are not accessible since they may include expressions dependent on parts of the query that have not yet executed.

3. GROUP BY

Remaining rows after the WHERE constraints are applied are then grouped based on common values in the column specified in the GROUP BY clause. As a result of the grouping, there will only be as many rows as there are unique values in that column. This means that you should only use this when you have aggregate functions in your query.

4. HAVING

If the query has a GROUP BY clause, then any constraints of a HAVING clause are applied to the grouped rows. Like the WHERE clause, aliases may also not be accessible from this step.

5. SELECT

Any expressions in the SELECT part of the query are finally computed.

6. DISTINCT

Of the remaining rows, rows with duplicate values in the column marked as DISTINCT will be discarded.

7. ORDER BY

If an ORDER BY is specified, the rows are then sorted by the specified data in either ascending or descending order. Since all the expressions in the SELECT part of the query have been computed, you can reference aliases at this point.

8. LIMIT / OFFSET

Last, the rows that fall outside the range specified by the LIMIT and OFFSET are discarded.

ORDER OF OPERATIONS HERE

Here is the order of operations on our previous group example.

```
SELECT CustomerId, COUNT(*) AS NumOrders
FROM Orders
GROUP BY CustomerId
ORDER BY NumOrders DESC
LIMIT 10;
```

CustomerId	NumOrders
25806	25
94364	25
96968	24
33646	24
57572	23
89204	23
27518	23
29451	23
26682	23
36709	22

10 rows in set (1.72 sec)

First, we determine our data set. Here it is trivial as we have only Orders.

Then we GROUP the Orders on CustomerID.

We evaluate the SELECT. This means we need an aggregate function to apply to each sub-group, which is to COUNT the rows of each sub-group. We alias this count as Numorders, because we will need to use it in the ORDER clause later.

Last we use the SELECTED values as our output fields, ORDERED and up to a LIMIT of 10.

If this seems counter to how you have been conditioned to think from normal programming, you are not alone. All I can say is that you will have to get used to this "inside-out" thinking if you want to get truly comfortable with SQL. Fortunately there are a limited number of idioms (patterns) to deal with and you will soon get an intuitive understanding of the order of evaluation.

More SQL

John Urbanic

Parallel Computing Scientist
Pittsburgh Supercomputing Center

Distinguished Service Professor
Carnegie Mellon University

First:
Optional Titanic
Exercise Review



Optional exercise: Can we find a significant survival variable?

Can you find a significant factor in the data which could be used to predict survival rates?

I will suggest you focus on one variable at a time.

Note that there are many possible answers. Going from a simple hypothesis ("Maybe people from Cherbourg are unlucky?") to a more complex formula incorporating multiple variables - with the goal of a more accurate prediction - is the path of data analysis. This is our first step on that journey.



- We are going to use a Virtual Machine for this optional exercise and our next Assignment. It is called msdas.psc.edu and you can `ssh` there like you would to Bridges.
- Copy the titanic dataset (using the `cp` command) from `~urbanic/LargeScaleComputing/Titanic/titanic.csv` to your own directory.
- Start a python shell (this assumes you have the anaconda module still loaded, otherwise do so).
- Find a meaningful factor and feel free to send me your script and results if you want my feedback.

Getting Started with Titanic

```
import pandas as pd
titanic = pd.read_csv("titanic.csv")
```

```
males = titanic[titanic["Sex"]=="male"]
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
5	6	0	3	Moran, Mr. James	male	NaN	0	0	330877	8.4583	NaN	Q
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	S
7	8	0	3	Palsson, Master. Gosta Leonard	male	2.0	3	1	349909	21.0750	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...	...
883	884	0	2	Banfield, Mr. Frederick James	male	28.0	0	0	C.A./SOTON 34068	10.5000	NaN	S
884	885	0	3	Sutehall, Mr. Henry Jr	male	25.0	0	0	SOTON/OQ 392076	7.0500	NaN	S
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	NaN	S
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C148	C
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	NaN	Q

[577 rows x 12 columns]

```
males.shape
```

(577, 12)

```
males[males["Survived"]==1].shape
```

(109, 12)

109/577

0.18890814558058924

19% Survival Rate for Males

How did the women fare?

```
titanic[titanic["Sex"]=="female"].shape
```

(314, 12)

```
titanic[ (titanic["Sex"]=="female") & (titanic["Survived"]==1) ].shape
```

(233, 12)

233/314

0.7420382165605095

74% Survival Rate for Females

Hypothesis confirmed: chivalry not dead.

But Jack Dawson is.

Women and children first!?

```
men = titanic[ (titanic["Sex"]=="male") & (titanic["Age"]>15) ]
```

```
men.shape
```

```
(413, 12)
```

```
men[ men["Survived"]==1 ].shape
```

```
(72, 12)
```

```
72/413
```

```
0.17433414043583534233/314
```

17% Survival Rate for Men

NaNs are everywhere!

```
women_and_children = titanic[ (titanic["Sex"]=="female") | (titanic["Age"]<16) ]  
women_and_children.shape
```

```
(354, 12)
```

```
#Seems like some people are missing...
```

```
titanic[titanic["Age"].isna()]
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
5	6	0	3	Moran, Mr. James	male	NaN	0	0	330877	8.4583	NaN	Q
17	18	1	2	Williams, Mr. Charles Eugene	male	NaN	0	0	244373	13.0000	NaN	S
19	20	1	3	Masselmani, Mrs. Fatima	female	NaN	0	0	2649	7.2250	NaN	C
26	27	0	3	Emir, Mr. Farred Chehab	male	NaN	0	0	2631	7.2250	NaN	C
28	29	1	3	O'Dwyer, Miss. Ellen "Nellie"	female	NaN	0	0	330959	7.8792	NaN	Q
..	...	...	...	...	...	...	...	...	...	...	...	...
859	860	0	3	Razi, Mr. Raihed	male	NaN	0	0	2629	7.2292	NaN	C
863	864	0	3	Sage, Miss. Dorothy Edith "Dolly"	female	NaN	8	2	CA. 2343	69.5500	NaN	S
868	869	0	3	van Melkebeke, Mr. Philemon	male	NaN	0	0	345777	9.5000	NaN	S
878	879	0	3	Laleff, Mr. Kristo	male	NaN	0	0	349217	7.8958	NaN	S
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	w./C. 6607	23.4500	NaN	S

```
[177 rows x 12 columns]
```

```
413+354+177
```

```
944
```

This is bigger than the total passenger list (891).
But makes sense as we have double counted some
females with Age=NaN in our logic.

Women and children first!

```
women_and_children[ women_and_children["Survived"]==1 ].shape
```

```
(254, 12)
```

```
254/354
```

```
0.7175141242937854
```

72% Survival Rate for
Women & Children

How did Thurston Howell III make out?

Another obvious question we might ask is how did the wealthier, 1st class, passengers do versus the underclasses?

We could continue with our basic tools and separate out the various passenger classes, and perform some math to get at an answer.

However, we are now starting to ask questions that can utilize more sophisticated tools like:

- Joins (called Merges in Pandas)
- Grouping
- Pivot tables

Pandas has these capabilities. However, more complex data manipulation like this can often benefit from the more powerful capabilities of a Structured Query Language (SQL) database. Certainly at scale.

So we will preview the power of these operations with one last look at this problem, and then we will move on to SQL.

After you have learned SQL, you will easily be able to employ these operations in Pandas when you wish.



Grouping

Grouping typically performs 3 steps:

- Splits the data into groups base on some criteria:
- Applies an aggregate function to each group separately:
- Combines the results into a new table

Pclass
Survival Rate

That is one way to get directly at our answer. It becomes this simple:

```
titanic[['Pclass', 'Survived']].groupby('Pclass').mean()
```

Pclass	Survived
1	0.629630
2	0.472826
3	0.242363

That is a pretty brutal curve.
I believe it speaks for itself.

Bigger Titanic Lessons

Our predictive power here is going to be pretty limited. Titanic is an ongoing Kaggle challenge, and you will see that some pretty sophisticated techniques top out at about 80% there (you will see some higher claims, but they are all *overfitting*, a topic we will get very familiar with).

The reason for this is that the dataset is small. For accuracy we need *Big Data*. We will get there shortly.



This is our first foray into mortality prediction. It may seem a little sensationalistic here, but it is part of a very significant field. Think about how important it is to do accurate:

- **Medical Treatments:** Cancer prognosis for various chemotherapies or surgeries.
- **Nutritional Analysis:** Do Nitrates cause GI cancers and death?
- **Environmental Factors:** Microplastics, air pollution.

A particularly hilarious example is the 34th *Ig Noble* award to Justin Newman for debunking recent *Nature* and *Science* papers about Blue Zones, where people - supposedly - live very long. The dominant factor by far is simply pension fraud.

With the Greeks, by my estimates at least 72% of centenarians were dead, missing or essentially pension-fraud cases.

Back to SQL



ORDER OF OPERATIONS

One counter intuitive, but central notion, to SQL is that the listed order of the specified commands does not correspond to the order in which they are executed. There is a mandated order to the evaluation of the clauses.

1. FROM and JOINS

The FROM clause, and subsequent JOINS are first executed to determine the set of data that is being queried. This includes subqueries in this clause, and can cause temporary tables to be created.

2. WHERE

Then any WHERE constraints are applied to the individual rows, and rows that do not satisfy the constraint are discarded. Each of the constraints can only access columns directly from the tables requested in the FROM clause. Aliases in the SELECT part of the query are not accessible since they may include expressions dependent on parts of the query that have not yet executed.

3. GROUP BY

Remaining rows after the WHERE constraints are applied are then grouped based on common values in the column specified in the GROUP BY clause. As a result of the grouping, there will only be as many rows as there are unique values in that column. This means that you should only use this when you have aggregate functions in your query.

4. HAVING

If the query has a GROUP BY clause, then any constraints of a HAVING clause are applied to the grouped rows. Like the WHERE clause, aliases may also not be accessible from this step.

5. SELECT

Any expressions in the SELECT part of the query are finally computed.

6. DISTINCT

Of the remaining rows, rows with duplicate values in the column marked as DISTINCT will be discarded.

7. ORDER BY

If an ORDER BY is specified, the rows are then sorted by the specified data in either ascending or descending order. Since all the expressions in the SELECT part of the query have been computed, you can reference aliases at this point.

8. LIMIT / OFFSET

Last, the rows that fall outside the range specified by the LIMIT and OFFSET are discarded.

ORDER OF OPERATIONS HERE

Here is the order of operations on our previous group example.

```
SELECT CustomerId, COUNT(*) AS NumOrders
FROM Orders
GROUP BY CustomerId
ORDER BY NumOrders DESC
LIMIT 10;
```

CustomerId	NumOrders
25806	25
94364	25
96968	24
33646	24
57572	23
89204	23
27518	23
29451	23
26682	23
36709	22

10 rows in set (1.72 sec)

First, we determine our data set. Here it is trivial as we have only Orders.

Then we GROUP the Orders on CustomerID.

We evaluate the SELECT. This means we need an aggregate function to apply to each sub-group, which is to COUNT the rows of each sub-group. We alias this count as Numorders, because we will need to use it in the ORDER clause later.

Last we use the SELECTED values as our output fields, ORDERED and up to a LIMIT of 10.

If this seems counter to how you have been conditioned to think from normal programming, you are not alone. All I can say is that you will have to get used to this "inside-out" thinking if you want to get truly comfortable with SQL. Fortunately there are a limited number of idioms (patterns) to deal with and you will soon get an intuitive understanding of the order of evaluation.

ORDER OF OPERATIONS VS. ORDER OF STATEMENTS

This is probably a new and odd concept to many of you. We could say that SQL is a declarative language, as opposed to the more usual procedural languages.

Written Order	Order of Execution
SELECT	FROM and JOINS
FROM	WHERE
WHERE	GROUP BY
GROUP BY	HAVING
HAVING	SELECT
ORDER BY	DISTINCT
DISTINCT	ORDER BY
LIMIT/OFFSET	LIMIT/OFFSET

This means that the SQL optimizer determines what operations are actually run. These operations typically take the form of a directed acyclic graph. The database evaluates each Select query as a whole to determine the most efficient way to assemble the data set results. This is called the **Execution Plan**. The steps are performed on entire table sets and in parallel and usually do not have any of the names above. Instead a step might say "perform an index scan on table A", or "perform a nested loops join on the prior partial result set and table B". In some cases they will filter records before joining and in other cases they won't, for example. We shall wade into these details as we go.

SELECT SUBQUERIES

We will often wish to feed one result (in the form of a table) into another query. These subqueries are created by nesting selects within each other.

Let's say we wish to create a list of **all customers with more than 20 orders**.

```
SELECT CustomerId, COUNT(*) AS NumOrders
FROM Orders
GROUP BY CustomerId
HAVING NumOrders > 20;
```

```
+-----+-----+
| CustomerId | NumOrders |
+-----+-----+
|      1885 |         21 |
|      2311 |         21 |
|      2344 |         21 |
|      2364 |         21 |
|      ... |         ... |
|      ... |         ... |
|     88414 |         22 |
|     89204 |         23 |
|     91017 |         21 |
|     94364 |         25 |
|     96968 |         24 |
|     99255 |         22 |
|    100068 |         21 |
+-----+-----+
51 rows in set (1.03 sec)
```

Note that when filtering aggregated results we must use *HAVING* instead of *WHERE*.

SELECT SUBQUERIES

Now we can treat that query as a table itself. Here we just apply the count(*) to it. Next we will start connecting these together.

```
SELECT COUNT(*)
FROM (SELECT CustomerId, COUNT(*) AS NumOrders
      FROM Orders
      GROUP BY CustomerId
      HAVING NumOrders > 20) AS TopOrders;
```

```
+-----+
| COUNT(*) |
+-----+
|      51 |
+-----+
1 row in set (1.04 sec)
```

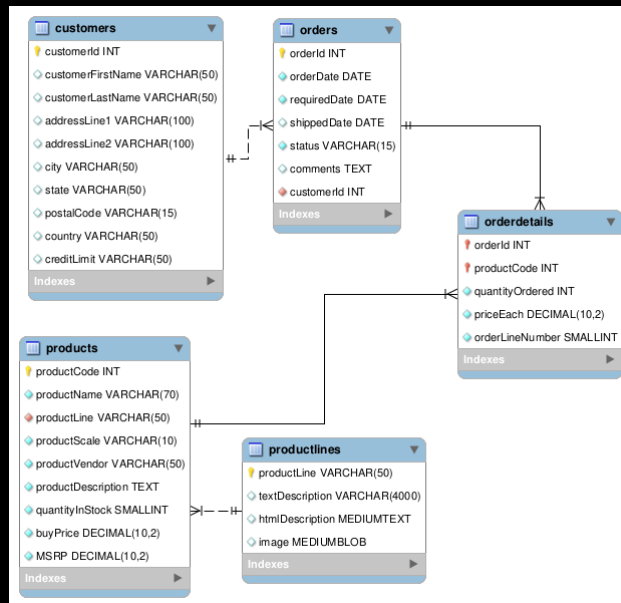
SQL insists that every derived table have a name (alias). So, we must name our subquery before we can use it, even for something as trivial as this.

Combining Table Data

This might be better with more detailed customer information included. But that isn't in our `Orders` table. Some obviously useful info can be found in our `Customers` table.

```
SHOW COLUMNS FROM Customer;
```

Field	Type	Null	Key	Default	Extra
CustomerId	int	NO	PRI	NULL	auto_increment
FirstName	varchar(50)	YES		NULL	
LastName	varchar(50)	YES		NULL	
AddressLine1	varchar(100)	YES		NULL	
AddressLine2	varchar(100)	YES		NULL	
City	varchar(50)	YES		NULL	
State	varchar(50)	YES		NULL	
PostalCode	varchar(15)	YES		NULL	
Country	varchar(50)	YES		NULL	
CreditLimit	varchar(50)	YES		NULL	



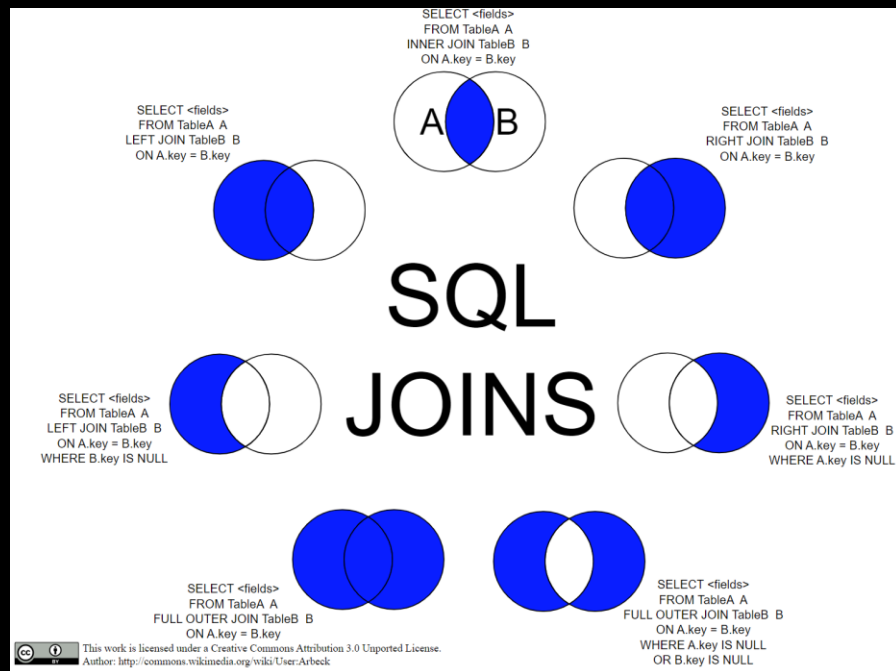
Combining Table Data With Joins

This is where the relational part of our RDBMS comes in. We want to combine data from different tables.

Our most powerful tool here will be *joins*.

There are a variety of these, and they have a logical relationship between them that is often summarized by their Venn Diagrams.

However, a few examples are generally enough to get the point across, and then this diagram will make total sense, and you won't have to memorize anything.



Inner Join

This is the default "join", and most common. It is used to collect only items with matching keys from both tables. The keys are specified with the `ON` clause and could be combinations of columns.

```
SELECT left_table.B, right_table.F
FROM left_table
JOIN right_table
  ON left_table.A = right_table.E;
```

Left Table

A	B	C	D
K0	B0	C0	D0
K1	B1	C1	D1
K2	B2	C2	D2
K3	B3	C3	D3

Right Table

E	F	G	H
K1	F0	G0	H0
K1	F1	G1	H1
K0	F2	G2	H2
K6	F3	G3	H3

Result

B	F
B0	F2
B1	F0
B1	F1

Notice how I am looping over the right table for each key in the left table.

Procedurally, this is a nested loop.

We will come back to this later.

The actual row order in the result could vary, unless we added an `ORDER BY` clause.

Implementation notes: In Pandas, these are referred to as merges.

Inner Join Example

Now we have **added customer details to our top orders**. Note that the table name is usually inferred from the FROM clause but in this JOIN columns must be disambiguated as there are multiple CustomerIDs, one in each table.

```
SELECT Customer.*, TopOrders.NumOrders FROM
(SELECT CustomerId, COUNT(*) AS NumOrders
  FROM Orders
 GROUP BY CustomerId
 HAVING NumOrders > 20) AS TopOrders
JOIN Customer
  ON TopOrders.CustomerId = Customer.CustomerId;
```

CustomerId	FirstName	LastName	AddressLine1	AddressLine2	City	State	PostalCode	Country	CreditLimit	NumOrders
1885	Karen	Roberge	44 Downey Drive		Manchester	CT	06040	USA	100	21
2311	Daniel	Jurado	709 Mildred Street		Montgomery	AL	36104	USA	100	21
2344	Lois	Hoskins	5631 West Colter Street	#2129	Glendale	AZ	85301	USA	100	21
2364	Arthur	Sosa	3145 19th Avenue Court		Greeley	CO	80631	USA	100	21
8049	Frances	Johnson	83 Oakdale Road		Newton	MA	02459	USA	100	22
9584	Claudia	Price	1452 55th Avenue	B	Oakland	CA	94621	USA	100	21
...										
...										
13803	Teresa	Pierce	154 Boca Lagoon Drive		Panama City Beach	FL	32408	USA	100	21
17740	Joseph	Grisson	8642 Yule Street		Arvada	CO	80007	USA	100	21
18420	Betty	Brown	1822 Pine Grove Court		Severn	MD	21144	USA	100	21
20647	Robert	Patton	1995 Nolensville Pike		Nashville	TN	37211	USA	100	21
25806	Kyle	Hosmer	95 Briarwood Drive		Manchester	CT	06040	USA	100	25
26682	Jana	Mapes	718 Newhall Drive		Nashville	TN	37206	USA	100	23
27518	Jason	Johnson	824 Main Street	D	Manchester	CT	06040	USA	100	23
27604	Matt	Mackey	1600 20th Street Northwest		washington	DC	20009	USA	100	22
27982	Linda	Parks	378 Bonny Street		Grand Junction	CO	81501	USA	100	22
96968	Ellis	Chaddock	509 Sea Breeze Drive		Panama City Beach	FL	32413	USA	100	24
99255	Elaine	Soto	721 Vermont 5A		Westmore	VT	05860	USA	100	22
100068	Glenna	Lloyd	6424 Simms Street	#71	Arvada	CO	80004	USA	100	21

51 rows in set (1.06 sec)

Inner Join Example

Let's break this down, one subquery at a time.

```
SELECT Customer.*, TopOrders.NumOrders FROM
(SELECT CustomerId, COUNT(*) AS NumOrders
  FROM Orders
  GROUP BY CustomerId
  HAVING NumOrders > 20) AS TopOrders
JOIN Customer ON TopOrders.CustomerId = Customer.CustomerId;
```

CustomerId	NumOrders
1885	21
2311	21
2344	21
2364	21
8049	22
9584	21
13803	21
...	
...	

Inner Join Example

Let's break this down, one subquery at a time.

```
SELECT Customer.*, TopOrders.NumOrders FROM
(SELECT CustomerId, COUNT(*) AS NumOrders
FROM Orders
GROUP BY CustomerId
HAVING NumOrders > 20) AS TopOrders
JOIN Customer ON TopOrders.CustomerId = Customer.CustomerId;
```

```
+-----+-----+
| CustomerId | NumOrders |
+-----+-----+
|      1885 |         21 |
|      2311 |         21 |
|      2344 |         21 |
|      2364 |         21 |
|      8049 |         22 |
|      9584 |         21 |
|     13803 |         21 |
...
...
```

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| CustomerId | FirstName | LastName | AddressLine1 | AddressLine2 | City | State | PostalCode | Country | CreditLimit |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|      1 | Mary | Yates | 1414 East Anderson Street | #317 | Savannah | GA | 31404 | USA | 100 |
|      2 | James | Parker | 29 Lucian Street | | Manchester | CT | 06040 | USA | 100 |
|      3 | Kim | Bond | 1421 Floral Street Northwest | | Washington | DC | 20012 | USA | 100 |
|      4 | Thomas | Broadnax | 1915 Southeast 29th Street | | Oklahoma City | OK | 73129 | USA | 100 |
|      5 | Stephen | Williams | 9805 South Youngs Lane | | Oklahoma City | OK | 73159 | USA | 100 |
|      6 | Mark | Hinton | 8642 Yule Street | | Arvada | CO | 80007 | USA | 100 |
|      7 | Deborah | Lloyd | 5244 West Port Au Prince Lane | | Glendale | AZ | 85306 | USA | 100 |
|      8 | Linda | Barnes | 3377 Sandstone Court | | Pleasanton | CA | 94588 | USA | 100 |
|      9 | Donald | Zawacki | 4709 North willow Avenue | | Bethany | OK | 73008 | USA | 100 |
|     10 | Rene | Spencer | 1797 Pasatiempo Drive | | Chico | CA | 95928 | USA | 100 |
...
...
```

Inner Join Example

Note that each subquery follows our official Order of Operations as we work our way to the topmost query.

```
SELECT Customer.*, TopOrders.NumOrders FROM
(SELECT CustomerId, COUNT(*) AS NumOrders
  FROM Orders
  GROUP BY CustomerId
  HAVING NumOrders > 20) AS TopOrders
JOIN Customer
  ON TopOrders.CustomerId = Customer.CustomerId;
```

CustomerId	FirstName	LastName	AddressLine1	AddressLine2	City	State	PostalCode	Country	CreditLimit	NumOrders
1885	Karen	Roberge	44 Downey Drive		Manchester	CT	06040	USA	100	21
2311	Daniel	Jurado	709 Mildred Street		Montgomery	AL	36104	USA	100	21
2344	Lois	Hoskins	5631 West Colter Street	#2129	Glendale	AZ	85301	USA	100	21
2364	Arthur	Sosa	3145 19th Avenue Court		Greeley	CO	80631	USA	100	21
8049	Frances	Johnson	83 Oakdale Road		Newton	MA	02459	USA	100	22
9584	Claudia	Price	1452 55th Avenue	B	Oakland	CA	94621	USA	100	21
...										
...										
13803	Teresa	Pierce	154 Boca Lagoon Drive		Panama City Beach	FL	32408	USA	100	21
17740	Joseph	Grisson	8642 Yule Street		Arvada	CO	80007	USA	100	21
18420	Betty	Brown	1822 Pine Grove Court		Severn	MD	21144	USA	100	21
20647	Robert	Patton	1995 Nolensville Pike		Nashville	TN	37211	USA	100	21
25806	Kyle	Hosmer	95 Briarwood Drive		Manchester	CT	06040	USA	100	25
26682	Jana	Mapes	718 Newhall Drive		Nashville	TN	37206	USA	100	23
27518	Jason	Johnson	824 Main Street	D	Manchester	CT	06040	USA	100	23
27604	Matt	Mackey	1600 20th Street Northwest		washington	DC	20009	USA	100	22
27982	Linda	Parks	378 Bonny Street		Grand Junction	CO	81501	USA	100	22
96968	Ellis	Chaddock	509 Sea Breeze Drive		Panama City Beach	FL	32413	USA	100	24
99255	Elaine	Soto	721 Vermont 5A		Westmore	VT	05860	USA	100	22
100068	Glenna	Lloyd	6424 Simms Street	#71	Arvada	CO	80004	USA	100	21

51 rows in set (1.06 sec)

ORDER OF OPERATIONS (Again)

1. FROM and JOINS

2. WHERE

3. GROUP BY

4. HAVING

5. SELECT

6. DISTINCT

7. ORDER BY

8. LIMIT / OFFSET

Views

As our queries, and subqueries, get more complex it becomes cumbersome and inefficient to keep recreating them. A VIEW will give us the ability to capture these as temporary tables.

```
CREATE VIEW TopCustomers AS
SELECT Customer.* FROM
  (SELECT CustomerId, COUNT(*) AS NumOrders
   FROM Orders
   GROUP BY CustomerId
   HAVING NumOrders > 20) AS TopOrders
JOIN Customer
  ON TopOrders.CustomerId = Customer.CustomerId;
```

```
SHOW TABLES;
```

```
+-----+
| Tables_in_urbanic |
+-----+
| Customer           |
| Line               |
| OrderDetail        |
| Orders             |
| Product            |
| TopCustomers       |
| Vendor             |
+-----+
7 rows in set (0.00 sec)
```

A Useful View

Marketing has asked us to identify **all the leather products customers have ordered**. That information is scattered across `Orders`, `OrderDetails` (which has the individual items) and `Products` (which has the item details).

```
CREATE VIEW LeatherOrders AS
SELECT Orders.*, orderprod.Fabric, orderprod.ProductId, orderprod.Item, orderprod.Color
FROM Orders
INNER JOIN (SELECT OrderDetail.OrderId, Product.Fabric, Product.ProductId, Product.Item, Product.Color
            FROM OrderDetail
            INNER JOIN Product
            ON OrderDetail.ProductId = Product.ProductId) AS orderprod
ON Orders.OrderId = orderprod.OrderId
WHERE orderprod.Fabric LIKE 'Leather%'
ORDER BY Orders.OrderId;
```

OrderId	Date	DateRequired	DateShipped	Status	Comments	CustomerId	Fabric	ProductId	Item	Color
14	1995-05-23	1995-06-11	1995-06-10	OK		28781	Leather	21876	Slacks	White
18	1995-05-23	1995-05-29	1995-05-29	OK		57443	Leather	27031	Blazer	Silver
20	1995-05-23	1995-05-23	1995-05-26	OK		73528	Leather	22949	Dungarees	Cyan
22	1995-05-23	1995-05-30	1995-06-10	OK		30334	Leather	39039	Fedora	Pink
34	1995-05-23	1995-06-04	1995-06-04	OK		13149	Leather	36187	Swimsuit	Magenta
40	1995-05-23	1995-06-07	1995-06-04	OK		66536	Leather	39541	Scarf	Orange
60	1995-05-23	1995-06-04	1995-06-04	OK		71618	Leather	22458	Pants	Plum
65	1995-05-23	1995-06-02	1995-06-03	OK		62043	Leather	40680	Jacket	Navy
69	1995-05-23	1995-06-02	1995-05-23	OK		46767	Leather	40748	Jacket	Bl
...										
...										
...										

We can match strings with the `LIKE` keyword and the `'%'` symbol works as a wildcard.

More Joins

We wish to include all of our top customers in a possible "Leather Sale" promotion even if they don't have a leather order.

LEFT JOIN will include all elements from the left table and matching ones from the right table. Unmatched values will be shown as NULL.

```
SELECT TopCustomers.customerId,LeatherOrders.OrderId,LeatherOrders.Fabric
FROM TopCustomers
LEFT JOIN LeatherOrders
    ON TopCustomers.CustomerId = LeatherOrders.CustomerId;
```

```
+-----+-----+-----+
| CustomerId | OrderId | Fabric |
+-----+-----+-----+
|      1885 |     81257 | Leather |
|      1885 |    238207 | Leather |
|      2364 |    359837 | Leather |
|      2364 |    488584 | Leather |
...
...
|      39578 |    414186 | Leather |
|      39578 |    650871 | Leather |
|      39578 |    705664 | Leather |
|      41072 |         NULL | NULL |
|      47511 |    529563 | Leather |
|      47511 |    676164 | Leather |
|      50446 |    598894 | Leather |
|      50446 |    821076 | Leather |
|      51859 |    660109 | Leather |
|      52328 |    160968 | Leather |
|      52328 |    750126 | Leather |
+-----+-----+-----+
```

We can match strings with the LIKE keyword and the '%' symbol works as a wildcard.

Left Join

LEFT JOIN will include all elements from the left table and matching ones from the right table. Unmatched values will be shown as NULL.

```
SELECT left_table.B, right_table.F
FROM left_table
LEFT JOIN right_table
  ON left_table.A = right_table.E;
```

Left Table

A	B	C	D
K0	B0	C0	D0
K1	B1	C1	D1
K2	B2	C2	D2
K3	B3	C3	D3

Right Table

E	F	G	H
K1	F0	G0	H0
K1	F1	G1	H1
K0	F2	G2	H2
K6	F3	G3	H3

Result

B	F
B0	F2
B1	F0
B1	F1
B2	NULL
B3	NULL

The actual row order in the result could vary, unless we added an ORDER BY clause.

Implementation notes: In Pandas, these are referred to as merges.

Right Join

As you might expect by now, RIGHT JOIN will include all elements from the right table and matching ones from the left table. Unmatched values will be shown as NULL.

```
SELECT left_table.B, right_table.F
FROM left_table
RIGHT JOIN right_table
ON left_table.A = right_table.E;
```

Left Table

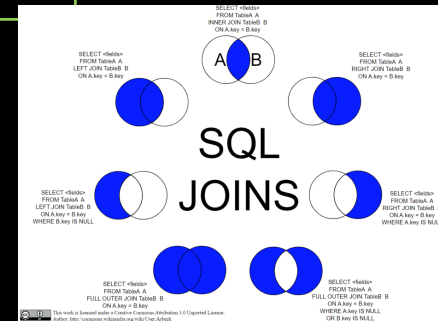
A	B	C	D
K0	B0	C0	D0
K1	B1	C1	D1
K2	B2	C2	D2
K3	B3	C3	D3

Right Table

E	F	G	H
K1	F0	G0	H0
K1	F1	G1	H1
K0	F2	G2	H2
K6	F3	G3	H3

Result

B	F
B1	F0
B1	F1
B0	F2
NULL	F3



The actual row order in the result could vary, unless we added an ORDER BY clause.

Implementation notes: In Pandas, these are referred to as merges.

Another Interesting Query

We'd also like to know **which leather products haven't been of interest to top customers**. We'll do a RIGHT JOIN to find all the leather products with no orders by this group. We also use the keyword *IN* in this query to filter down to only TopCustomer orders. Other comparison operators we will see are *NOT IN*, *BETWEEN*, *IS*, *IS NOT*, *IS NOT NULL*.

```
SELECT topLeatherOrders.OrderId, topLeatherOrders.CustomerId, LeatherProduct.Color, LeatherProduct.Fabric,
       LeatherProduct.Item, LeatherProduct.ProductId
FROM (SELECT *
      FROM LeatherOrders
      WHERE LeatherOrders.CustomerId IN (SELECT CustomerId FROM TopCustomers)) AS topLeatherOrders
RIGHT JOIN (SELECT ProductId, Fabric, Color, Item FROM Product WHERE Fabric LIKE 'Leather%') AS LeatherProduct
ON topLeatherOrders.ProductId = LeatherProduct.ProductId
LIMIT 10;
```

OrderId	CustomerId	Color	Fabric	Item	ProductId
NULL	NULL	Red	Leather	Skirt	20659
NULL	NULL	Navy	Leather	Skirt	20660
NULL	NULL	Cyan	Leather	Skirt	20661
NULL	NULL	Black	Leather	Skirt	20662
NULL	NULL	Brown	Leather	Skirt	20663
234297	17740	Ocher	Leather	Skirt	20664
NULL	NULL	Orange	Leather	Skirt	20665
NULL	NULL	White	Leather	Skirt	20666
NULL	NULL	Green	Leather	Skirt	20667
NULL	NULL	Puce	Leather	Skirt	20668

10 rows in set (1.14 sec)

Step by step

Get all the orders from our previously created `LeatherOrders` view, WHERE the customers are IN our `TopCustomers` view.

```
SELECT topLeatherOrders.OrderId, topLeatherOrders.CustomerId, LeatherProduct.Color, LeatherProduct.Fabric,
       LeatherProduct.Item, LeatherProduct.ProductId
FROM (SELECT *
      FROM LeatherOrders
      WHERE LeatherOrders.CustomerId IN (SELECT CustomerId FROM TopCustomers)) AS topLeatherOrders
RIGHT JOIN (SELECT ProductId, Fabric, Color, Item FROM Product WHERE Fabric LIKE 'Leather%') AS LeatherProduct
ON topLeatherOrders.ProductId = LeatherProduct.ProductId
LIMIT 10;
```

[illegible]

Step by step

Get all the leather products from Product.

```
SELECT topLeatherOrders.OrderId, topLeatherOrders.CustomerId, LeatherProduct.Color, LeatherProduct.Fabric,
       LeatherProduct.Item, LeatherProduct.ProductId
FROM (SELECT *
      FROM LeatherOrders
      WHERE LeatherOrders.CustomerId IN (SELECT CustomerId FROM TopCustomers)) AS topLeatherOrders
RIGHT JOIN (SELECT ProductId, Fabric, Color, Item FROM Product WHERE Fabric LIKE 'Leather%') AS LeatherProduct
ON topLeatherOrders.ProductId = LeatherProduct.ProductId
LIMIT 10;
```

```
+-----+-----+-----+-----+
| ProductId | Fabric | Color | Item |
+-----+-----+-----+-----+
| 20659 | Leather | Red | Skirt |
| 20660 | Leather | Navy | Skirt |
| 20661 | Leather | Cyan | Skirt |
| 20662 | Leather | Black | Skirt |
| 20663 | Leather | Brown | Skirt |
| 20664 | Leather | ocher | Skirt |
| 20665 | Leather | Orange | Skirt |
| 20666 | Leather | white | Skirt |
...
...
...
```


Then a JOIN

After the RIGHT JOIN we have all the leather products, with OderIDs for those that have been ordered by top customers, and NULLS for the rest. We keep only the fields we want with the SELECT.

```
SELECT topLeatherOrders.OrderId, topLeatherOrders.CustomerId, LeatherProduct.Color, LeatherProduct.Fabric,
       LeatherProduct.Item, LeatherProduct.ProductId
FROM (SELECT *
      FROM LeatherOrders
      WHERE LeatherOrders.CustomerId IN (SELECT CustomerId FROM TopCustomers)) AS topLeatherOrders
RIGHT JOIN (SELECT ProductId, Fabric, Color, Item FROM Product WHERE Fabric LIKE 'Leather%') AS LeatherProduct
ON topLeatherOrders.ProductId = LeatherProduct.ProductId
LIMIT 10;
```

OrderId	CustomerId	Color	Fabric	Item	ProductId
NULL	NULL	Red	Leather	Skirt	20659
NULL	NULL	Navy	Leather	Skirt	20660
NULL	NULL	Cyan	Leather	Skirt	20661
NULL	NULL	Black	Leather	Skirt	20662
NULL	NULL	Brown	Leather	Skirt	20663
234297	17740	Ocher	Leather	Skirt	20664
NULL	NULL	Orange	Leather	Skirt	20665
NULL	NULL	White	Leather	Skirt	20666
NULL	NULL	Green	Leather	Skirt	20667
NULL	NULL	Puce	Leather	Skirt	20668

10 rows in set (1.14 sec)

One last refinement

The NULL entries in the left table are what we're after here, so we can add one more clause. NULL values require the IS keyword since a value can not be equal to NULL. IS tests for values that are either TRUE, FALSE or NULL.

```
SELECT OrderId, CustomerId, LeatherProduct.Color, LeatherProduct.Fabric, LeatherProduct.Item,
       LeatherProduct.ProductId
FROM (SELECT *
      FROM LeatherOrders
      WHERE LeatherOrders.CustomerId IN (SELECT CustomerId FROM TopCustomers)) AS topLeatherOrders
RIGHT JOIN (SELECT ProductId, Fabric, Color, Item FROM Product WHERE Fabric LIKE 'Leather%') AS LeatherProduct
      ON topLeatherOrders.ProductId = LeatherProduct.ProductId
WHERE OrderId IS NULL
LIMIT 10;
```

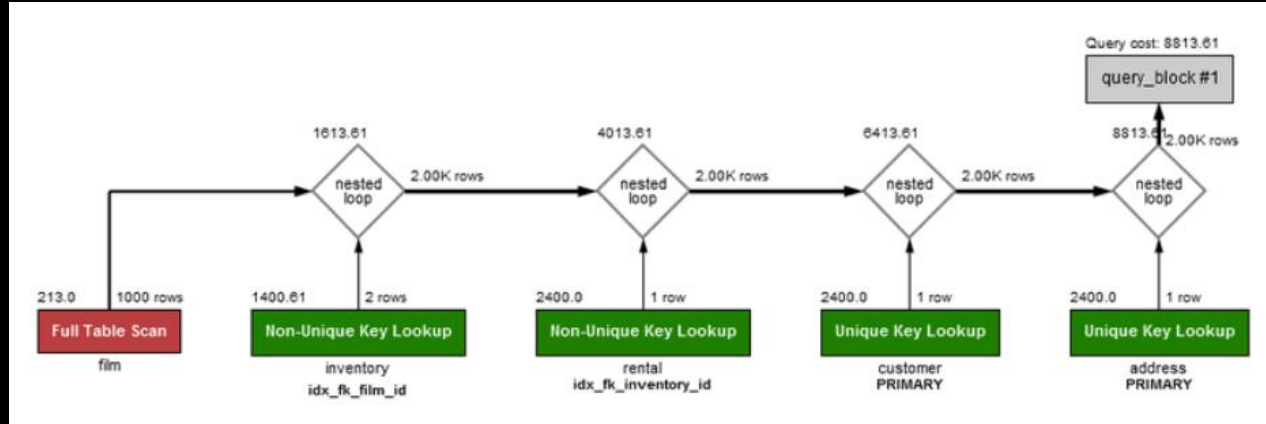
OrderId	CustomerId	Color	Fabric	Item	ProductId
NULL	NULL	Red	Leather	Skirt	20659
NULL	NULL	Navy	Leather	Skirt	20660
NULL	NULL	Cyan	Leather	Skirt	20661
NULL	NULL	Black	Leather	Skirt	20662
NULL	NULL	Brown	Leather	Skirt	20663
NULL	NULL	Orange	Leather	Skirt	20665
NULL	NULL	White	Leather	Skirt	20666
NULL	NULL	Green	Leather	Skirt	20667
NULL	NULL	Puce	Leather	Skirt	20668
NULL	NULL	Pink	Leather	Skirt	20669

Joins are Loops

You notice how we loop through the data as we manually create our joins. This is what our database must do as well. Nested joins turn into nested loops. Here is a typical query from a classic film rental database.

```
SELECT CONCAT(customer.last_name, ', ', customer.first_name)
  AS customer, address.phone, film.title FROM rental
INNER JOIN customer ON rental.customer_id = customer.customer_id
INNER JOIN address ON customer.address_id = address.address_id
INNER JOIN inventory ON rental.inventory_id = inventory.inventory_id
INNER JOIN film ON inventory.film_id = film.film_id
WHERE rental.return_date IS NULL
AND rental_date + INTERVAL film.rental_duration DAY < CURRENT_DATE()
LIMIT 5;
```

This is how MySQL workbench explains the operations.



Finding Stuff

This brings us to the central performance factor of an SQL database: how to organize our data.

A table is a big collection of records. We are repeatedly searching through all of them to find something in particular. Here perhaps a name, a customer ID, all the customers that live in a zip code or state.

11	Paul	Newlan	69 Washington Street		Manchester	CT	06042	USA	100
12	Donna	Lozier	19448 Currant Drive		Edmond	OK	73012	USA	100
13	Christi	Bailey	32532 Jean Drive		Union City	CA	94587	USA	100
14	Jo	Mahon	202 Deer Run Trail		Manchester	CT	06042	USA	100
15	Brian	Uber	11150 Sun Valley Drive		Oakland	CA	94605	USA	100
16	Derek	Auger	208 Solitude Circle		Goodlettsville	TN	37072	USA	100
17	Herbert	Hamilton	244 La Normandy Drive		Louisville	KY	40223	USA	100
18	James	Crawford	195 Kennedy Road		Manchester	CT	06042	USA	100
19	Jackie	Henson	6418 Iris Way		Arvada	CO	80004	USA	100
20	Brian	Looney	11724 Rushmore		Oklahoma City	OK	73162	USA	100
21	Alisha	Kim	6040 West Kings Avenue		Glendale	AZ	85306	USA	100
22	Michelle	Tutwiler	43 Howarth Avenue		Attleboro	MA	02703	USA	100
23	David	Fretwell	11 Meeting Place Circle		Boxford	MA	01921	USA	100
24	Cameron	Johnson	903 Burton Avenue		Louisville	KY	40208	USA	100
25	Earl	Nockels	7 Vantine Avenue		Alburgh	VT	05440	USA	100
26	Gloria	Piper	1215 Joseph Avenue		Nashville	TN	37207	USA	100
27	Ryan	Coleman	1402 North Capitol Street Northwest		Washington	DC	20002	USA	100
28	Joseph	Dugan	4 Elm Street		Windsor	VT	05089	USA	100
29	Brenda	Rose	2829 31st Street Northwest		Washington	DC	20008	USA	100
30	Dana	Pomeroy	601 Farragut Place Northeast		Washington	DC	20017	USA	100
31	Brandon	Waters	804 Dry Creek Road		Goodlettsville	TN	37072	USA	100
32	Janet	Dias	8526 Preston Highway		Louisville	KY	40219	USA	100
33	Doris	Evans	2236 Susitna Drive		Anchorage	AK	99517	USA	100
34	Robert	Katz	8300 Dover Way		Arvada	CO	80005	USA	100
35	Frank	Schmitt	3 Hawser Way		Port Wentworth	GA	31407	USA	100
36	Lashell	Roseboro	3311 Wiley Post Loop		Anchorage	AK	99517	USA	100
37	Doris	Jones	44 Colonial Road		Manchester	CT	06042	USA	100
38	Elmer	Lytle	3302 Woodland Park Drive	#APT 000001	Anchorage	AK	99517	USA	100
39	Marlene	Navor	9009 West Nicolet Avenue		Glendale	AZ	85305	USA	100
40	Brenda	Schultz	3605 Normandy Place South		Nashville	TN	37209	USA	100
41	Tammy	Mcfarland	1600 20th Street Northwest		Washington	DC	20009	USA	100
42	Terry	Averitt	8554 Twin Gable Drive		Montgomery	AL	36116	USA	100
43	Holly	Dommer	4136 East Virginia Lane		Montgomery	AL	36116	USA	100

Sorting?

We don't want to have to search linearly through each table every time. Think of how painful that would be with our JOINS! We would be doing an $O(N)$ operation for each iteration. This would be $O(M \times N)$ operations for joins between a M and N sized table.

So, how can we look up data faster? What about if we alphabetized our users? For name lookups we could certainly find them faster.

Let's overlook that fact that that doesn't help for looking up by ID or zipcode, for now.

The main problem with this is that we have to maintain that order. Every time we add or delete a customer we have to re-sort the whole table. For large tables this is painful.

11		Paul		Newlan		69 Washington Street				Manchester		CT		06042		USA		100	
12		Donna		Lozier		19448 Currant Drive				Edmond		OK		73012		USA		100	
13		Christi		Bailey		32532 Jean Drive				Union City		CA		94587		USA		100	
14		Jo		Mahon		202 Deer Run Trail				Manchester		CT		06042		USA		100	
15		Brian		Uber		11150 Sun Valley Drive				Oakland		CA		94605		USA		100	
16		Derek		Auger		208 Solitude Circle				Goodlettsville		TN		37072		USA		100	
17		Herbert		Hamilton		244 La Normandy Drive				Louisville		KY		40223		USA		100	
18		James		Crawford		195 Kennedy Road				Manchester		CT		06042		USA		100	
19		Jackie		Henson		6418 Iris Way				Arvada		CO		80004		USA		100	
20		Brian		Looney		11724 Rushmore				Oklahoma City		OK		73162		USA		100	
21		Alisha		Kim		6040 West Kings Avenue				Glendale		AZ		85306		USA		100	
22		Michelle		Tutwiler		43 Howarth Avenue				Attleboro		MA		02703		USA		100	

Indexing

The answer is to create an index for our records.

The index is some small part of the record (like just the last name, here) that points us to where the full record is stored.

Then, we can add, delete or move records as is most convenient or efficient for our data store (tack something on the end, or leave an empty space in the middle for now).

All we need to modify is an entry in the index table, which is much smaller than our full table data.

11		Paul		Newlan		69 Washington Street				Manchester		CT		06042		USA		100	
12		Donna		Lozier		19448 Currant Drive				Edmond		OK		73012		USA		100	
13		Christi		Bailey		32532 Jean Drive				Union City		CA		94587		USA		100	
14		Jo		Mahon		202 Deer Run Trail				Manchester		CT		06042		USA		100	
15		Brian		Uber		11150 Sun Valley Drive				Oakland		CA		94605		USA		100	
16		Derek		Auger		208 Solitude Circle				Goodlettsville		TN		37072		USA		100	
17		Herbert		Hamilton		244 La Normandy Drive				Louisville		KY		40223		USA		100	
18		James		Crawford		195 Kennedy Road				Manchester		CT		06042		USA		100	
19		Jackie		Henson		6418 Iris Way				Arvada		CO		80004		USA		100	
20		Brian		Looney		11724 Rushmore				Oklahoma City		OK		73162		USA		100	
21		Alisha		Kim		6040 West Kings Avenue				Glendale		AZ		85306		USA		100	
22		Michelle		Tutwiler		43 Howarth Avenue				Attleboro		MA		02703		USA		100	

Advanced SQL

John Urbanic

Parallel Computing Scientist
Pittsburgh Supercomputing Center

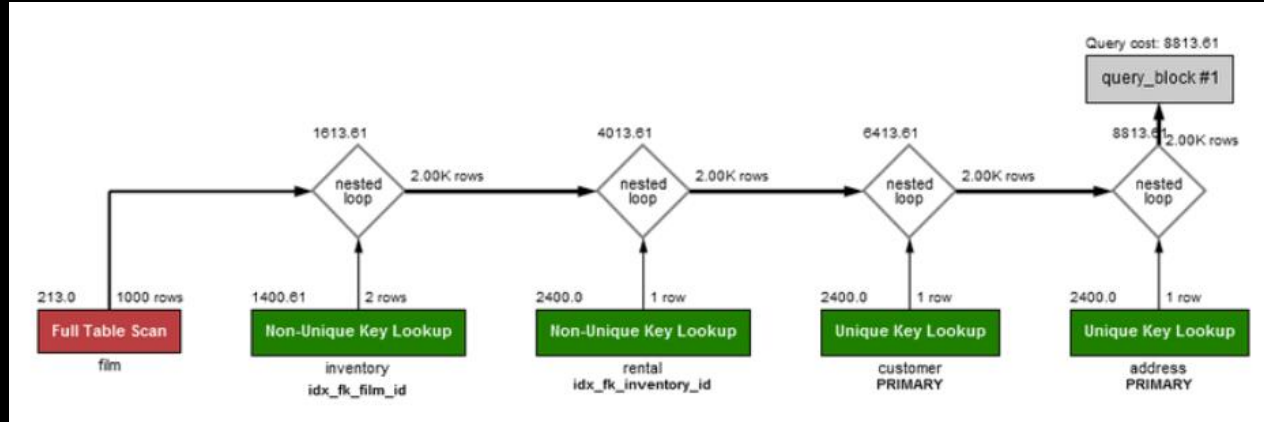
Distinguished Service Professor
Carnegie Mellon University

Joins are Loops

You notice how we loop through the data as we manually create our joins. This is what our database must do as well. Nested joins turn into nested loops. Here is a typical query from a classic film rental database.

```
SELECT CONCAT(customer.last_name, ', ', customer.first_name)
  AS customer, address.phone, film.title FROM rental
INNER JOIN customer ON rental.customer_id = customer.customer_id
INNER JOIN address ON customer.address_id = address.address_id
INNER JOIN inventory ON rental.inventory_id = inventory.inventory_id
INNER JOIN film ON inventory.film_id = film.film_id
WHERE rental.return_date IS NULL
AND rental_date + INTERVAL film.rental_duration DAY < CURRENT_DATE()
LIMIT 5;
```

This is how MySQL workbench explains the operations.



Finding Stuff

This brings us to the central performance factor of an SQL database: how to organize our data.

A table is a big collection of records. We are repeatedly searching through all of them to find something in particular. Here perhaps a name, a customer ID, all the customers that live in a zip code or state.

11	Paul	Newlan	69 Washington Street		Manchester	CT	06042	USA	100
12	Donna	Lozier	19448 Currant Drive		Edmond	OK	73012	USA	100
13	Christi	Bailey	32532 Jean Drive		Union City	CA	94587	USA	100
14	Jo	Mahon	202 Deer Run Trail		Manchester	CT	06042	USA	100
15	Brian	Uber	11150 Sun Valley Drive		Oakland	CA	94605	USA	100
16	Derek	Auger	208 Solitude Circle		Goodlettsville	TN	37072	USA	100
17	Herbert	Hamilton	244 La Normandy Drive		Louisville	KY	40223	USA	100
18	James	Crawford	195 Kennedy Road		Manchester	CT	06042	USA	100
19	Jackie	Henson	6418 Iris Way		Arvada	CO	80004	USA	100
20	Brian	Looney	11724 Rushmore		Oklahoma City	OK	73162	USA	100
21	Alisha	Kim	6040 West Kings Avenue		Glendale	AZ	85306	USA	100
22	Michelle	Tutwiler	43 Howarth Avenue		Attleboro	MA	02703	USA	100
23	David	Fretwell	11 Meeting Place Circle		Boxford	MA	01921	USA	100
24	Cameron	Johnson	903 Burton Avenue		Louisville	KY	40208	USA	100
25	Earl	Nockels	7 Vantine Avenue		Alburgh	VT	05440	USA	100
26	Gloria	Piper	1215 Joseph Avenue		Nashville	TN	37207	USA	100
27	Ryan	Coleman	1402 North Capitol Street Northwest		Washington	DC	20002	USA	100
28	Joseph	Dugan	4 Elm Street		Windsor	VT	05089	USA	100
29	Brenda	Rose	2829 31st Street Northwest		Washington	DC	20008	USA	100
30	Dana	Pomeroy	601 Farragut Place Northeast		Washington	DC	20017	USA	100
31	Brandon	Waters	804 Dry Creek Road		Goodlettsville	TN	37072	USA	100
32	Janet	Dias	8526 Preston Highway		Louisville	KY	40219	USA	100
33	Doris	Evans	2236 Susitna Drive		Anchorage	AK	99517	USA	100
34	Robert	Katz	8300 Dover Way		Arvada	CO	80005	USA	100
35	Frank	Schmitt	3 Hawser Way		Port Wentworth	GA	31407	USA	100
36	Lashell	Roseboro	3311 Wiley Post Loop		Anchorage	AK	99517	USA	100
37	Doris	Jones	44 Colonial Road		Manchester	CT	06042	USA	100
38	Elmer	Lytle	3302 Woodland Park Drive	#APT 000001	Anchorage	AK	99517	USA	100
39	Marlene	Navor	9009 West Nicolet Avenue		Glendale	AZ	85305	USA	100
40	Brenda	Schultz	3605 Normandy Place South		Nashville	TN	37209	USA	100
41	Tammy	Mcfarland	1600 20th Street Northwest		Washington	DC	20009	USA	100
42	Terry	Averitt	8554 Twin Gable Drive		Montgomery	AL	36116	USA	100
43	Holly	Dommer	4136 East Virginia Lane		Montgomery	AL	36116	USA	100

Sorting?

We don't want to have to search linearly through each table every time. Think of how painful that would be with our JOINS! We would be doing an $O(N)$ operation for each iteration. This would be $O(M \times N)$ operations for joins between a M and N sized table.

So, how can we look up data faster? What about if we alphabetized our users? For name lookups we could certainly find them faster.

Let's overlook that fact that that doesn't help for looking up by ID or zipcode, for now.

The main problem with this is that we have to maintain that order. Every time we add or delete a customer we have to re-sort the whole table. For large tables this is painful.

11		Paul		Newlan		69 Washington Street				Manchester		CT		06042		USA		100	
12		Donna		Lozier		19448 Currant Drive				Edmond		OK		73012		USA		100	
13		Christi		Bailey		32532 Jean Drive				Union City		CA		94587		USA		100	
14		Jo		Mahon		202 Deer Run Trail				Manchester		CT		06042		USA		100	
15		Brian		Uber		11150 Sun Valley Drive				Oakland		CA		94605		USA		100	
16		Derek		Auger		208 Solitude Circle				Goodlettsville		TN		37072		USA		100	
17		Herbert		Hamilton		244 La Normandy Drive				Louisville		KY		40223		USA		100	
18		James		Crawford		195 Kennedy Road				Manchester		CT		06042		USA		100	
19		Jackie		Henson		6418 Iris Way				Arvada		CO		80004		USA		100	
20		Brian		Looney		11724 Rushmore				Oklahoma City		OK		73162		USA		100	
21		Alisha		Kim		6040 West Kings Avenue				Glendale		AZ		85306		USA		100	
22		Michelle		Tutwiler		43 Howarth Avenue				Attleboro		MA		02703		USA		100	

Indexing

The answer is to create an index for our records.

The index is some small part of the record (like just the last name, here) that points us to where the full record is stored.

Then, we can add, delete or move records as is most convenient or efficient for our data store (tack something on the end, or leave an empty space in the middle for now).

All we need to modify is an entry in the index table, which is much smaller than our full table data.

11		Paul		Newlan		69 Washington Street				Manchester		CT		06042		USA		100	
12		Donna		Lozier		19448 Currant Drive				Edmond		OK		73012		USA		100	
13		Christi		Bailey		32532 Jean Drive				Union City		CA		94587		USA		100	
14		Jo		Mahon		202 Deer Run Trail				Manchester		CT		06042		USA		100	
15		Brian		Uber		11150 Sun Valley Drive				Oakland		CA		94605		USA		100	
16		Derek		Auger		208 Solitude Circle				Goodlettsville		TN		37072		USA		100	
17		Herbert		Hamilton		244 La Normandy Drive				Louisville		KY		40223		USA		100	
18		James		Crawford		195 Kennedy Road				Manchester		CT		06042		USA		100	
19		Jackie		Henson		6418 Iris Way				Arvada		CO		80004		USA		100	
20		Brian		Looney		11724 Rushmore				Oklahoma City		OK		73162		USA		100	
21		Alisha		Kim		6040 West Kings Avenue				Glendale		AZ		85306		USA		100	
22		Michelle		Tutwiler		43 Howarth Avenue				Attleboro		MA		02703		USA		100	

Keys

We will call these indexes keys. The correct selection of keys for each table is the most important consideration for performance.

There are a variety of key types. Two are very important.

Primary Key

A column (*or possibly combination of columns!*) with unique values.

Foreign Key

A column whose values point to a Primary Key in a different table.

There are other terms for keys that are less important to know. *Candidate Keys* are any keys that could be a Primary Key. A *Unique Key* could have a single NULL value (which is not allowed for a Primary Key). A *Composite Key* is a key created from multiple columns, etc.

To answer the question of how we efficiently search for data that isn't the primary key, we could create a *Secondary Key*. However, that isn't free as we also have to maintain that index as our data is modified.

Keys

Primary Keys are very important as the database will use that as an index to allow us to quickly find a record. This is usually via a good *hashing algorithm* or *B-tree*.

When we are doing a join, this allows us to quickly find any two items we are wishing to compare.

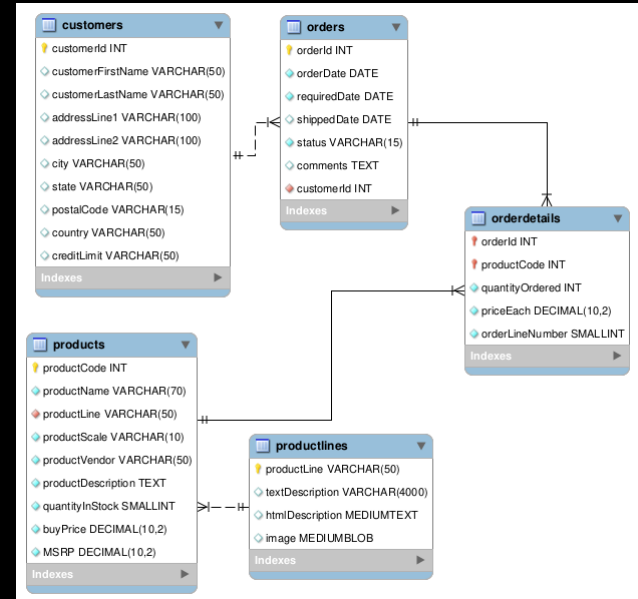
This is why we really prefer our joins to use the assigned table keys if possible.

Keys can also aid greatly in ensuring data integrity.

If it is the case that every record should be unique (order #s, for example), then using that as the primary key will enforce that condition.

A necessary relationship between data in different tables can be enforced with foreign keys. If an Order table uses a customer ID as a foreign key, they will ensure that a matching customer exists in a Customer data table.

A common default Primary Key is simply an integer that might be auto-incremented as each new record is added. In *Pandas* we always have a row number.



Hashing

You won't get very far in data science without hearing about how hashing is used to organize important data. It is by far the most common way to index any substantial RDBMS table.

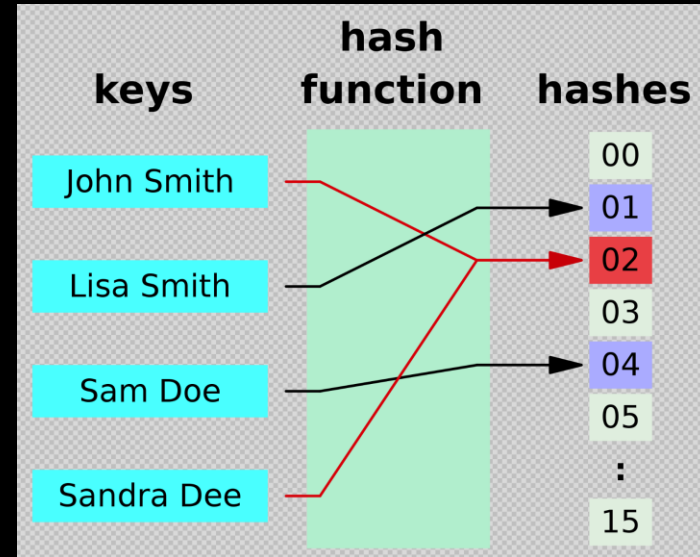
In this context, a hashing algorithm's job is to take a key and use it to generate an index into the data storage.

From the mathematical perspective, it takes some string - of possibly arbitrary length - and generates a fixed size number. In general this means that it can't guarantee the uniqueness of that number, but you hope it does a good job of distributing the indices around. And, you hope it is fast.

A *collision* can occur, and we have various schemes to cope with that.

Hashes are very fast. $O(1)$!

But, they aren't great at finding a range of values. Think dates or timestamps. This also means they aren't helpful in ordering data.



B-tree

This is the other common RDBMS indexing scheme.

A B-tree is a self-balancing tree data structure that keeps data sorted and allows searches, sequential access, insertions, and deletions in logarithmic time.

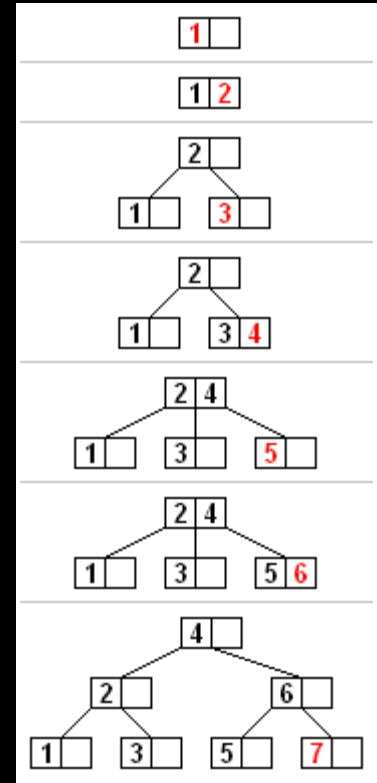
We walk the tree until we find our index and its pointer to the rest of our data.

While the B-tree is potentially slower, it does allow us to efficiently find a range of keys. So it is better if you care about comparisons or sorts

There is a more elaborate variation of this called a B+ Tree.

An awesome place to play with these algorithms, and many others, is:

<https://www.cs.usfca.edu/~galles/visualization/Algorithms.html>



B-tree insertion example from wikipedia.

Keys In Action

We wish to find the shipping addresses for our pending orders. This is a straightforward join.

```
SELECT Orders.Number, Customers.Name, Customers.Address
FROM Orders
JOIN Customers
  ON Orders.CustomerID = Customer.CustomerID;
```

Orders

Number	Customer ID
12	55
16	21
34	36
17	55
21	49
18	21
24	34
25	23
26	18
29	23
36	55

Customers

Customer ID	Name	Address
4	Joe	Cherry Lane
36	Sally	Walnut Street
23	Anand	Elm Alley
49	Jack	Rock Avenue
34	Fred	Electric Avenue
18	Tracy	Easy Street
21	Susan	Wrong Way
42	John	Garrick Drive
55	Alex	Fifth Avenue
13	Bob	Forbes Avenue
23	John	Shady Lane

Shipping Info

Order	Name	Address
12	Alex	Fifth Avenue
16	Susan	Wrong Way
34	Sally	Walnut Street
17	Alex	Fifth Avenue
21	Jack	Rock Avenue
18	Susan	Wrong Way
24	Fred	Electric Avenue
25	John	Shady Lane
26	Tracy	Easy Street
29	Anand	Elm Alley
36	Alex	Fifth Avenue

with the right key, this becomes much easier.

```
SELECT Orders.Number, Customers.Name, Customers.Address
FROM Orders
JOIN Customers
  ON  Orders.CustomerID = Customer.CustomerID;
```

HASH(55) = 9
HASH(21) = 7
HASH(36) = 2
...

Orders

Number	Customer ID
12	55
16	21
34	36
17	55
21	49
18	21
24	34
25	23
26	18
29	23
36	55

Customers

Customer ID	Name	Address
4	Joe	Cherry Lane
36	Sally	Walnut Street
23	Anand	Elm Alley
49	Jack	Rock Avenue
34	Fred	Electric Avenue
18	Tracy	Easy Street
21	Susan	Wrong Way
42	John	Garrick Drive
55	Alex	Fifth Avenue
13	Bob	Forbes Avenue
23	John	Shady Lane

Shipping Info

Order	Name	Address
12	Alex	Fifth Avenue
16	Susan	Wrong Way
34	Sally	Walnut Street
17	Alex	Fifth Avenue
21	Jack	Rock Avenue
18	Susan	Wrong Way
24	Fred	Electric Avenue
25	John	Shady Lane
26	Tracy	Easy Street
29	Anand	Elm Alley
36	Alex	Fifth Avenue

Advanced SQL



Let's get creative.

So far we have just been analyzing our data. We haven't been creating, or even modifying it. Let do that.

```
CREATE DATABASE clothing;
```

```
CREATE TABLE vendors (  
  vendorId int NOT NULL AUTO_INCREMENT, vendorName varchar(100) DEFAULT NULL, addressLine1 varchar(100) DEFAULT NULL,  
  addressLine2 varchar(100) DEFAULT NULL, city varchar(50) DEFAULT NULL, state varchar(50) DEFAULT NULL,  
  postalCode varchar(15) DEFAULT NULL, country varchar(50) DEFAULT NULL,  
  PRIMARY KEY (vendorId)  
);
```

```
show columns from vendors;
```

Field	Type	Null	Key	Default	Extra
vendorId	int	NO	PRI	NULL	auto_increment
vendorName	varchar(100)	YES		NULL	
addressLine1	varchar(100)	YES		NULL	
addressLine2	varchar(100)	YES		NULL	
city	varchar(50)	YES		NULL	
state	varchar(50)	YES		NULL	
postalCode	varchar(15)	YES		NULL	
country	varchar(50)	YES		NULL	

Altering Existing Tables

```
ALTER TABLE vendors ADD COLUMN comment VARCHAR(200);
```

```
show columns from vendors;
```

Field	Type	Null	Key	Default	Extra
vendorId	int	NO	PRI	NULL	auto_increment
vendorName	varchar(100)	YES		NULL	
addressLine1	varchar(100)	YES		NULL	
addressLine2	varchar(100)	YES		NULL	
city	varchar(50)	YES		NULL	
state	varchar(50)	YES		NULL	
postalCode	varchar(15)	YES		NULL	
country	varchar(50)	YES		NULL	
comment	varchar(200)	YES		NULL	

Inserting Data

```
INSERT INTO vendors(vendorName,addressLine1,addressLine2,city,state,postalCode,country,comment) VALUES ('ThreadBlasters','123  
Imaginary Place',NULL,'Sometown','PA','15217','USA',NULL);
```

```
SELECT * FROM vendors;
```

vendorId	vendorName	addressLine1	addressLine2	city	state	postalCode	country	comment
1	ThreadBlasters	123 Imaginary Place	NULL	Sometown	PA	15217	USA	NULL

Updating Data

```
UPDATE vendors SET vendorName = 'ThreadBlazers' WHERE vendorId = 1;
```

```
Query OK, 1 row affected (0.00 sec)
```

```
Rows matched: 1  Changed: 1  Warnings: 0
```

```
SELECT * FROM vendors;
```

vendorId	vendorName	addressLine1	addressLine2	city	state	postalCode	country	comment
1	ThreadBlazers	123 Imaginary Place	NULL	Sampletown	PA	15217	USA	NULL

```
1 row in set (0.00 sec)
```

Deleting Data

```
DELETE FROM vendors WHERE city = 'Sampletown';  
Query OK, 1 row affected (0.00 sec)
```

```
SELECT * FROM vendors;  
Empty set (0.00 sec)
```

Deleting Tables

```
DROP TABLE vendors;  
Query OK, 0 rows affected (0.02 sec)
```

```
SELECT * FROM vendors;  
ERROR 1146 (42S02): Table 'clothing.vendors' doesn't exist
```

Undo?

There is no simple "undo" in SQL. However, there is a very powerful feature that we can introduce for this function: transactions.

SQL gives us the ability to lump a coherent set of interactions into a transaction so that we are not left in an illegal state if something goes wrong. We can simply roll back to the state before our transaction started.

Let's demonstrate this as a simple undo. You may want to use this if you are aggressively experimenting with the database.

```
select * from Customer limit 5;
```

CustomerId	FirstName	LastName	AddressLine1	AddressLine2	City	State	PostalCode	Country	CreditLimit
1	Mary	Yates	1414 East Anderson Street	#317	Savannah	GA	31404	USA	100
2	James	Parker	29 Lucian Street		Manchester	CT	06040	USA	100
3	Kim	Bond	1421 Floral Street Northwest		Washington	DC	20012	USA	100
4	Thomas	Broadnax	1915 Southeast 29th Street		Oklahoma City	OK	73129	USA	100
5	Stephen	Williams	9805 South Youngs Lane		Oklahoma City	OK	73159	USA	100

Transaction

I want to increase Mary's credit limit to 200.

```
start transaction;
```

```
update Customer set CreditLimit = "200";
```

```
select * from Customer limit 5;
```

CustomerId	FirstName	LastName	AddressLine1	AddressLine2	City	State	PostalCode	Country	CreditLimit
1	Mary	Yates	1414 East Anderson Street	#317	Savannah	GA	31404	USA	200
2	James	Parker	29 Lucian Street		Manchester	CT	06040	USA	200
3	Kim	Bond	1421 Floral Street Northwest		Washington	DC	20012	USA	200
4	Thomas	Broadnax	1915 Southeast 29th Street		Oklahoma City	OK	73129	USA	200
5	Stephen	Williams	9805 South Youngs Lane		Oklahoma City	OK	73159	USA	200

Uh, oh, I forget to limit the change to WHERE CustomerID = 1. No problem. Just roll it back.

```
rollback;
```

```
select * from Customer limit 5;
```

CustomerId	FirstName	LastName	AddressLine1	AddressLine2	City	State	PostalCode	Country	CreditLimit
1	Mary	Yates	1414 East Anderson Street	#317	Savannah	GA	31404	USA	100
2	James	Parker	29 Lucian Street		Manchester	CT	06040	USA	100
3	Kim	Bond	1421 Floral Street Northwest		Washington	DC	20012	USA	100
4	Thomas	Broadnax	1915 Southeast 29th Street		Oklahoma City	OK	73129	USA	100
5	Stephen	Williams	9805 South Youngs Lane		Oklahoma City	OK	73159	USA	100

Commit

Let's try that one more time.

```
start transaction;
```

```
update Customer set CreditLimit = "200" WHERE CustomerID = 1;
```

```
select * from Customer limit 5;
```

CustomerId	FirstName	LastName	AddressLine1	AddressLine2	City	State	PostalCode	Country	CreditLimit
1	Mary	Yates	1414 East Anderson Street	#317	Savannah	GA	31404	USA	200
2	James	Parker	29 Lucian Street		Manchester	CT	06040	USA	100
3	Kim	Bond	1421 Floral Street Northwest		Washington	DC	20012	USA	100
4	Thomas	Broadnax	1915 Southeast 29th Street		Oklahoma City	OK	73129	USA	100
5	Stephen	Williams	9805 South Youngs Lane		Oklahoma City	OK	73159	USA	100

```
commit;
```

Now we are done, but can't undo this. You can see how useful this would be if there are a lot of conditions and limits on a transaction that could cause a problem. This way you are never left in a partially valid state.

This notion extends into a set of very formal guarantees...

ACID

ACID is a set of properties of database transactions intended to guarantee data validity despite errors, power failures, and other mishaps. In the context of databases, a sequence of database operations that satisfies the ACID properties (which can be perceived as a single logical operation on the data) is called a transaction. For example, a transfer of funds from one bank account to another, even involving multiple changes such as debiting one account and crediting another, is a single transaction.

Atomicity

An atomic system must guarantee atomicity in each and every situation, including power failures, errors, and crashes. A guarantee of atomicity prevents updates to the database from occurring only partially, which can cause greater problems than rejecting the whole series outright.

Consistency

Consistency ensures that a transaction can only bring the database from one consistent state to another, preserving database invariants: any data written to the database must be valid according to all defined rules, including constraints, cascades, triggers, and any combination thereof. This prevents database corruption by an illegal transaction.

Isolation

Isolation ensures that concurrent execution of transactions leaves the database in the same state that would have been obtained if the transactions were executed sequentially.

Durability

Durability guarantees that once a transaction has been committed, it will remain committed even in the case of a system failure (e.g., power outage or crash). This usually means that completed transactions (or their effects) are recorded in non-volatile memory.

SQL Injection Attacks

Consider a typical website, which asks the user to enter their username. It then constructs a string to use in querying the database for that user's info:

```
var statement = "SELECT * FROM users WHERE name = '" + userName + "'";
```

This seems reasonable. However, what if a nefarious user enters this as their username:

```
a';DROP TABLE users; SELECT * FROM userinfo WHERE 't' = 't
```

Then the SQL command that gets constructed is

```
SELECT * FROM users WHERE name = 'a';DROP TABLE users; SELECT * FROM userinfo WHERE 't' = 't';
```

And we have not only exposed all user data, but also deleted our users table.

Good practices can help to mitigate this and sanitize the inputs. Be aware.

Normalization

Any serious, semester-long, SQL course will spend some time talking about Normalization. This is the very formal process of eliminating redundant data and ensuring consistency. If you are a DB admin working for a bank, you should probably know what this is.

Otherwise, common sense and an understanding of what you are asking of the database is usually sufficient, especially for scientific data bases.

And it is important to note that databases are frequently *denormalized* to help with performance. A redundant field may avoid an expensive join.

[illegible]

Triggers

Triggers allow us to dynamically enforce conditions and check integrity whenever certain actions occur. We'll just discuss this simple example, but they can involve some complex behavior and take advantage of some of the more dynamic programming capabilities of SQL like variables and control flow.

```
CREATE TRIGGER upd_check BEFORE UPDATE ON account
FOR EACH ROW
BEGIN
    IF NEW.amount < 0 THEN
        SET NEW.amount = 0;
    ELSEIF NEW.amount > 100 THEN
        SET NEW.amount = 100;
    END IF;
END;
```

Procedures

Since we have strayed into the programmatic capabilities of SQL, I must at least mention stored procedures.

The simplest ones look like

```
CREATE PROCEDURE SelectAllCustomers
AS
SELECT * FROM Customers
GO;

EXEC SelectAllCustomers;
```

But, what is the point of that? In reality, these are used to capture serious business logic and have features like:

- Parameters
- Variables
- Conditional Statements: `IF`, `CASE`
- Loops: `LOOP`, `WHILE`, `REPEAT`

None of this should intimidate any basic programmer, but we aren't going to dive into the details here.

Connectors

MySQL is a server, and Connectors allow clients using other languages to connect to the databases. This is extremely common in web applications, which are written in their own native languages (Javascript or maybe a Java backend). Here is a query to our database from Python:

```
import datetime
import mysql.connector

cnx = mysql.connector.connect(user='adaptuser', database='clothing')
cursor = cnx.cursor()

query = ("SELECT orderId, orderDate FROM orders "
        "WHERE orderDate BETWEEN %s AND %s")

sale_start = datetime.date(1999, 1, 1)
sale_end = datetime.date(1999, 12, 31)

cursor.execute(query, (sale_start, sale_end))

for (orderId, orderDate) in cursor:
    print("Order {} placed on {:%d %b %Y}".format(
        orderId, orderDate))

cursor.close()
cnx.close()
```


Input and Output

There are many ways to get data into and out of a database. The most direct is from a text file, although formats such as XML are supported too. *LOAD DATA* will do this and has many options. The most basic looks like this

```
LOAD DATA LOCAL INFILE '/path/product.txt' INTO TABLE product;
```

Likewise, output can be done with a *SELECT* statement using *INTO OUTFILE*

```
SELECT * FROM orders WHERE orderDate < '1997-01-01' LIMIT 100 INTO OUTFILE 'MySQLHomework.txt';
```

Documentation

There is of course much we haven't covered and, even considering the topics we have, you want to have some documentation as a guide. The best place to go is

<https://dev.mysql.com/doc/refman/8.1/en/>



And, I like to have a "cheat sheet" by my side every time I have to revisit this subject. There are some really nice ones:

SQL CHEAT SHEET <http://www.sqltutorial.org>

QUERYING DATA FROM A TABLE

SELECT c1, c2 FROM t;
Query data in columns c1, c2 from a table

SELECT * FROM t;
Query all rows and columns from a table

SELECT c1, c2 FROM t
WHERE condition;
Query data and filter rows with a condition

SELECT DISTINCT c1 FROM t
WHERE condition;
Query distinct rows from a table

SELECT c1, c2 FROM t
ORDER BY c1 ASC (DESC);
Sort the result set in ascending or descending order

SELECT c1, c2 FROM t
ORDER BY c1
LIMIT n OFFSET offset;
Skip offset of rows and return the next n rows

SELECT c1 aggregate(c2)
FROM t
GROUP BY c1;
Group rows using an aggregate function

SELECT c1 aggregate(c2)
FROM t
GROUP BY c1
HAVING condition;
Filter groups using HAVING clause

QUERYING FROM MULTIPLE TABLES

SELECT c1, c2
FROM t1
INNER JOIN t2 ON condition;
Inner join t1 and t2

SELECT c1, c2
FROM t1
LEFT JOIN t2 ON condition;
Left join t1 and t2

SELECT c1, c2
FROM t1
RIGHT JOIN t2 ON condition;
Right join t1 and t2

SELECT c1, c2
FROM t1
FULL OUTER JOIN t2 ON condition;
Perform full outer join

SELECT c1, c2
FROM t1
CROSS JOIN t2;
Produce a Cartesian product

SELECT c1, c2
FROM t1
INNER JOIN t2 ON condition;
Another way to perform

SELECT c1, c2
FROM t1
INNER JOIN t2 ON condition;
Join t1 to itself using INNER

SQL Basics Cheat Sheet

SQL, or Structured Query Language, is a language for talking to databases. It allows you to query specific data and to build complex reports. Today, SQL is a universal language of data. It is mostly primarily at technology that process data.

SAMPLE DATA

SELECT * FROM cities
WHERE name IN ('Berlin');
Fetch names of cities that are neither Berlin nor Madrid.

SELECT * FROM cities
WHERE name LIKE 'M%';
Fetch names of cities that start with a 'M' or end with an 'M'.

SELECT * FROM cities
WHERE name LIKE 'M%';
Fetch names of cities that start with any letter followed by 'M' (like Madrid in Ireland or London in Poland).

SELECT * FROM cities
WHERE name LIKE 'M%';
Fetch names of cities that start with any letter followed by 'M' (like Madrid in Ireland or London in Poland).

SELECT * FROM cities
WHERE name LIKE 'M%';
Fetch names of cities that start with any letter followed by 'M' (like Madrid in Ireland or London in Poland).

SELECT * FROM cities
WHERE name LIKE 'M%';
Fetch names of cities that start with any letter followed by 'M' (like Madrid in Ireland or London in Poland).

SELECT * FROM cities
WHERE name LIKE 'M%';
Fetch names of cities that start with any letter followed by 'M' (like Madrid in Ireland or London in Poland).

USING SQL OPERATORS

COMPARISON OPERATORS

SELECT * FROM cities
WHERE rating > 3;
Fetch names of cities that have a rating above 3.

TEXT OPERATORS

SELECT * FROM cities
WHERE name LIKE 'M%';
Fetch names of cities that start with a 'M' or end with an 'M'.

SELECT * FROM cities
WHERE name LIKE 'M%';
Fetch names of cities that start with any letter followed by 'M' (like Madrid in Ireland or London in Poland).

SELECT * FROM cities
WHERE name LIKE 'M%';
Fetch names of cities that start with any letter followed by 'M' (like Madrid in Ireland or London in Poland).

SELECT * FROM cities
WHERE name LIKE 'M%';
Fetch names of cities that start with any letter followed by 'M' (like Madrid in Ireland or London in Poland).

SELECT * FROM cities
WHERE name LIKE 'M%';
Fetch names of cities that start with any letter followed by 'M' (like Madrid in Ireland or London in Poland).

SELECT * FROM cities
WHERE name LIKE 'M%';
Fetch names of cities that start with any letter followed by 'M' (like Madrid in Ireland or London in Poland).

SELECT * FROM cities
WHERE name LIKE 'M%';
Fetch names of cities that start with any letter followed by 'M' (like Madrid in Ireland or London in Poland).

QUERYING MULTIPLE TABLES

SELECT * FROM cities
WHERE name IN ('Berlin');
Fetch names of cities that are neither Berlin nor Madrid.

LEFT JOIN

SELECT * FROM cities
WHERE name IN ('Berlin');
Fetch names of cities that are neither Berlin nor Madrid.

SELECT * FROM cities
WHERE name IN ('Berlin');
Fetch names of cities that are neither Berlin nor Madrid.

SELECT * FROM cities
WHERE name IN ('Berlin');
Fetch names of cities that are neither Berlin nor Madrid.

SELECT * FROM cities
WHERE name IN ('Berlin');
Fetch names of cities that are neither Berlin nor Madrid.

SELECT * FROM cities
WHERE name IN ('Berlin');
Fetch names of cities that are neither Berlin nor Madrid.

SELECT * FROM cities
WHERE name IN ('Berlin');
Fetch names of cities that are neither Berlin nor Madrid.

SELECT * FROM cities
WHERE name IN ('Berlin');
Fetch names of cities that are neither Berlin nor Madrid.

RIGHT JOIN

SELECT * FROM cities
WHERE name IN ('Berlin');
Fetch names of cities that are neither Berlin nor Madrid.

NATURAL JOIN

SELECT * FROM cities
WHERE name IN ('Berlin');
Fetch names of cities that are neither Berlin nor Madrid.

SELECT * FROM cities
WHERE name IN ('Berlin');
Fetch names of cities that are neither Berlin nor Madrid.

SELECT * FROM cities
WHERE name IN ('Berlin');
Fetch names of cities that are neither Berlin nor Madrid.

SELECT * FROM cities
WHERE name IN ('Berlin');
Fetch names of cities that are neither Berlin nor Madrid.

SELECT * FROM cities
WHERE name IN ('Berlin');
Fetch names of cities that are neither Berlin nor Madrid.

SELECT * FROM cities
WHERE name IN ('Berlin');
Fetch names of cities that are neither Berlin nor Madrid.

SELECT * FROM cities
WHERE name IN ('Berlin');
Fetch names of cities that are neither Berlin nor Madrid.

datacamp
SQL for Data Science
SQL Basics Cheat Sheet
Learn SQL online at www.datacamp.com

What is SQL?

SQL stands for "Structured Query Language". It is a language used to query, update, and manipulate data from databases. Today, SQL is one of the most widely used languages in the world.

The different dialects of SQL

Although SQL languages all share a basic structure, some of the specific commands and rules can differ slightly. Popular dialects include MySQL, PostgreSQL, Oracle SQL, and more. PostgreSQL is a good place to start learning SQL as it is widely used and is easily adapted to other dialects.

Sample Data

Throughout this cheat sheet, we'll use the columns listed in this sample table of cities, 1000 rows.

id	city	country	number_of_cities	year_founded
1	Paris	France	5	2000
2	Madrid	Spain	3	2001
3	New York	USA	2	2002

Querying tables

1. Get all the cities from a table.
SELECT * FROM cities;

2. Select only the cities from the table.
SELECT city, country FROM cities;

3. Select only the cities from the table.
SELECT city, country FROM cities WHERE rating > 3;

4. Select only the cities from the table.
SELECT city, country FROM cities WHERE name LIKE 'M%';

datacamp
SQL for Data Science
SQL Basics Cheat Sheet
Learn SQL online at www.datacamp.com

Filtering Data

Filtering on numeric columns

1. Get all the cities from a table.
SELECT * FROM cities;

2. Select only the cities from the table.
SELECT city, country FROM cities WHERE rating > 3;

3. Select only the cities from the table.
SELECT city, country FROM cities WHERE name LIKE 'M%';

4. Select only the cities from the table.
SELECT city, country FROM cities WHERE year_founded > 2000;

5. Select only the cities from the table.
SELECT city, country FROM cities WHERE rating > 3 AND year_founded > 2000;

6. Select only the cities from the table.
SELECT city, country FROM cities WHERE rating > 3 AND year_founded > 2000;

7. Select only the cities from the table.
SELECT city, country FROM cities WHERE rating > 3 AND year_founded > 2000;

8. Select only the cities from the table.
SELECT city, country FROM cities WHERE rating > 3 AND year_founded > 2000;

9. Select only the cities from the table.
SELECT city, country FROM cities WHERE rating > 3 AND year_founded > 2000;

10. Select only the cities from the table.
SELECT city, country FROM cities WHERE rating > 3 AND year_founded > 2000;

11. Select only the cities from the table.
SELECT city, country FROM cities WHERE rating > 3 AND year_founded > 2000;

datacamp
SQL for Data Science
SQL Basics Cheat Sheet
Learn SQL online at www.datacamp.com

Aggregating Data

Simple aggregations

1. Get the number of cities from the table.
SELECT COUNT(*) FROM cities;

2. Get the average number of cities per country.
SELECT AVG(number_of_cities) FROM cities;

3. Get the average number of cities per country.
SELECT AVG(number_of_cities) FROM cities;

4. Get the average number of cities per country.
SELECT AVG(number_of_cities) FROM cities;

5. Get the average number of cities per country.
SELECT AVG(number_of_cities) FROM cities;

6. Get the average number of cities per country.
SELECT AVG(number_of_cities) FROM cities;

7. Get the average number of cities per country.
SELECT AVG(number_of_cities) FROM cities;

8. Get the average number of cities per country.
SELECT AVG(number_of_cities) FROM cities;

9. Get the average number of cities per country.
SELECT AVG(number_of_cities) FROM cities;

10. Get the average number of cities per country.
SELECT AVG(number_of_cities) FROM cities;

11. Get the average number of cities per country.
SELECT AVG(number_of_cities) FROM cities;

Just the first pages shown. Go to the URLs to get the full ones.

A little GIS

MySQL includes functions that allow us to measure geographic distance. This is a lightweight introduction to the important data science domain of Geographic Information Systems (GIS).

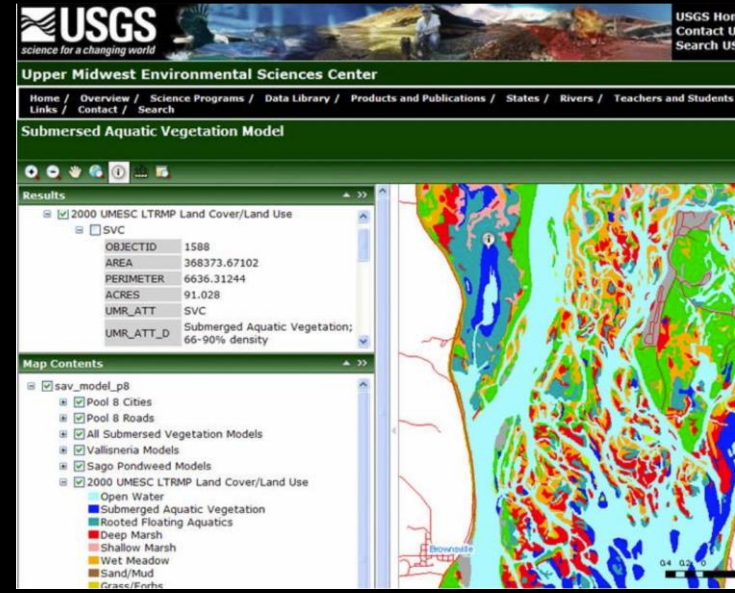
We are going to draw upon a zips table that you have already have in your database. Amongst the other fields, you can find the geographic center of each zip code (in the US these are the same as Postal Codes). That will be sufficient location resolution for our next task.

We can find our Customer's (rough) locations from this:

```
SELECT CustomerID, Lng, Lat, PostalCode
FROM Customer
JOIN Zips
  ON Customer.PostalCode = Zips.zip;
```

Or we can find the coordinates for one particular Cambridge, MA zip code, 02139:

```
SELECT Lng, Lat FROM Zips WHERE Zip = 02139;
```



ST_DISTANCE_SPHERE

Let's find out how far away from Cambridge customer number 1 is. *ST_DISTANCE_SPHERE()* takes coordinates in the order longitude then latitude and returns a distance in meters:

```
SELECT CustomerId, Lng, Lat, Zip,
       ST_DISTANCE_SPHERE(POINT(Lng,Lat), POINT(-71.10253,42.36224)) AS distance
FROM Customer JOIN Zips on PostalCode = Zip
WHERE CustomerId = 1;
```

CustomerId	Lng	Lat	Zip	distance
1	-81.05367	32.05064	31404	1443923.3563356877

1443 kilometers. Pretty far!

Assignment #2:

Our company is going to test a drone delivery service, and we wish to find the three best launch sites within 100 km of our Cambridge, MA headquarters (in zip code 02139). The drones can only fly a few km from their launch site. The 100 km limit is so our service techs don't have to commute too far.

Our initial search will consider which zip codes, within that eligible radius, have the most orders. We will locate our three sites within those three zip codes.

We could probably do a more in-depth analysis and consider the relative locations of the sites, or clusters of zip codes that might be within flight range of our drones. But we will start with this basic approach.

Your final email submission must include:

- Your SQL script
- Your answer

This assignment is due [October 7th](#). We will review that day, so no late credit is possible. This is 5 points of your grade, because it shouldn't take much effort. Nevertheless, I suggest you don't wait until the last minute.



A Brief History of Big Data

John Urbanic

Parallel Computing Scientist
Pittsburgh Supercomputing Center

Distinguished Service Professor
Carnegie Mellon University

Big data is a broad term for data sets so large or complex that traditional data processing applications are inadequate.

—*Wikipedia*

Once there was only small data...



A classic amount of “small” data

Find a tasty appetizer – Easy!

Find something to use up these oranges – grumble...

What if....



Big Data

John Urbanic

Parallel Computing Scientist
Pittsburgh Supercomputing Center

Distinguished Service Professor
Carnegie Mellon University

Once there was only small data...



A classic amount of “small” data

Find a tasty appetizer – Easy!

Find something to use up these oranges – grumble...

What if....



Less sophisticated is sometimes better...



“Chronologically” or “geologically” organized.
Familiar to some of you at tax time.

Get all articles from 2015.

Get all papers on “fault tolerance”
– grumble and cough

Indexing will determine your individual performance.
Teamwork can scale that up.



The culmination of centuries...



Find books on Modern Physics (DD# 539)

Find books by Wheeler

where he isn't the first author – grumble...



Your only hope...



Then data started to grow.

1956 IBM Model 350



5 MB of data!

But still pricey. \$

Better think about what
you want to save.

And finally got BIG.

10TB for \$150

Whys:

Storage got cheap

So why not keep it all?

Today data is a hot commodity \$

And we got better at generating it

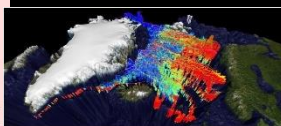
= Facebook
Deep Learning
IoT
Science... **10 TB** *



Pan-STARRS



Genome sequencers
(Wikipedia Commons)



Horniman museum:
<http://www.horniman.ac.uk>
[get_involved/blog/bioblitz-insects-reviewed](http://get_involved.blog/bioblitz-insects-reviewed)

Wikipedia
Commons

http://www.arctic.noaa.gov/report1/biodiv_whales_walrus.html

**Actually, a silly estimate. The original reference in 2013 the digital collection alone was 3PB.*

mentions a more accurate 208TB, and in

A better sense of biggish

Size

- 1000 Genomes Project
 - AWS hosted
 - 260TB
- Common Crawl
 - Hosted on Bridges
 - 300-800TB+

Throughput

- Square Kilometer Array
 - Building now
 - Exabyte of raw data/day – compressed to 10PB
- Internet of Things (IoT) / motes
 - Endless streaming

Records

- GDELT (Global Database of Events, Language, and Tone) (also soon to be hosted on Bridges)
 - Only about 2.5TB per year, but...
 - 250M rows and 58 fields (BigTable)
 - *“during periods with relatively little content, maximal translation accuracy can be achieved, with accuracy linearly degraded as needed to cope with increases in volume in order to ensure that translation always finishes within the 15 minute window.... and prioritizes the highest quality material, accepting that lower-quality material may have a lower-quality translation to stay within the available time window.”*

3 V's of Big Data

- Volume
- Velocity
- Variety

Good Ol' SQL couldn't keep up.



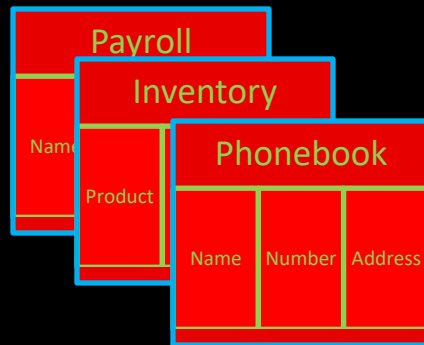
Oracle



```
SELECT  NAME, NUMBER, FROM PHONEBOOK
```

Why it *wasn't* fashionable:

- Schemas set in stone:
 - Need to define before we can add data
 - Not a fit for *agile development*
"What do you mean we didn't plan to keep logs of everyone's heartbeat?"
- Queries often require accessing multiple indexes and joining and sorting multiple tables
- Sharding isn't trivial
- Caching is tough
 - ACID (Atomicity, Consistency, Isolation, Durability) in a transaction is costly.



So we gave up: Key-Value

Redis, Memcached, Amazon DynamoDB, Riak, Ehcache

```
GET foo
```

- Certainly agile (no schema)
- Certainly scalable (linear in most ways: hardware, storage, cost)
- Good hash might deliver fast lookup
- Sharding, backup, etc. could be simple
- Often used for “session” information: online games, shopping carts

```
GET cart:joe:15~4~7~0723
```

foo	bar
2	fast
6	0
9	0
0	9
text	pic
1055	stuff
bar	foo

How does a pile of unorganized data solve our problems?

Sure, giving up ACID buys us a lot performance, but doesn't our crude organization cost us something? Yes, but remember these guys?



This is what they look like today.



Document



GET foo

- Value must be an object the DB can understand
- Common are: XML, JSON, Binary JSON and nested thereof
- This allows server side operations on the data

GET plant=daisy

- Can be quite complex: Linq query, JavaScript function
- Different DB's have different update/staleness paradigms

foo	
2	<pre><CATALOG> <PLANT> <COMMON>Bloodroot</COMMON> <BOTANICAL>Sanguinaria canadensis</BOTANICAL> <ZONE>4</ZONE> <LIGHT>Mostly Shady</LIGHT> <PRICE>\$2.44</PRICE> <AVAILABILITY>031599</AVAILABILITY> </PLANT> <PLANT> <COMMON>Columbine</COMMON> <BOTANICAL>Aquilegia canadensis</BOTANICAL> <ZONE>3</ZONE> <LIGHT>Mostly Shady</LIGHT> <PRICE>\$9.37</PRICE> <AVAILABILITY>030699</AVAILABILITY> </PLANT> </pre>
6	JSON
9	XML
0	Binary JSON
bar	JSON XML
12	XML XML

Wide Column Stores



Google BigTable

APACHE
HBASE

```
SELECT Name, Occupation FROM People WHERE key IN (199, 200, 207);
```

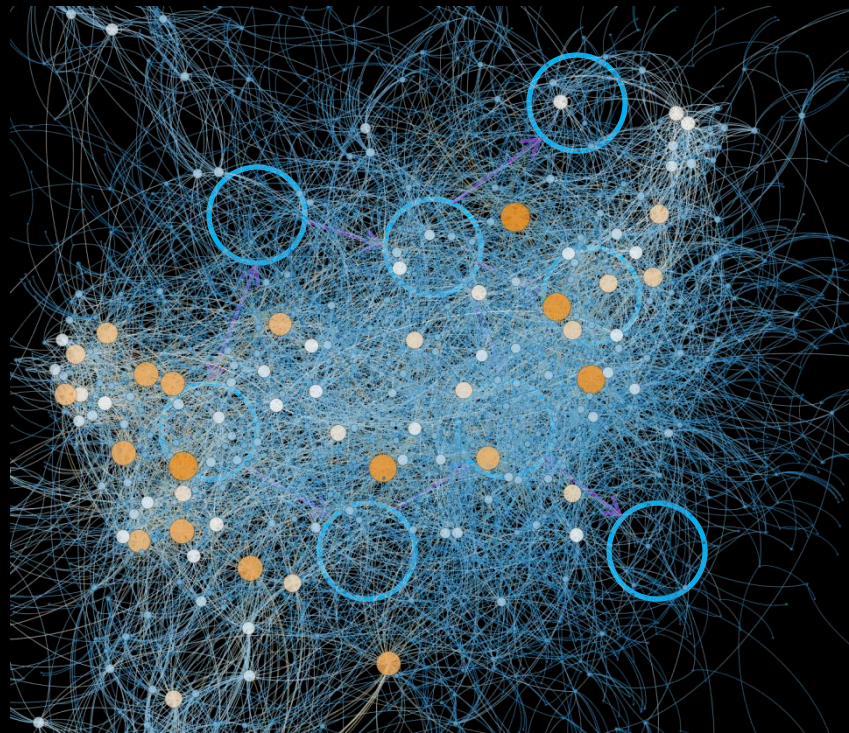
- No predefined schema
- Can think of this as a 2-D key-value store: the value may be a key-value store itself
- Different databases aggregate data differently on disk with different optimizations

Key			
Joe	Email: joe@gmail	Web: www.joe.com	
Fred	Phone: 412-555-3412	Email: fred@yahoo.com	Address: 200 S. Main Street
Julia	Email: julia@apple.com		
Mac	Phone: 214-555-5847		

Graph

Neo4j Titan, GEMS

- Great for semantic web
- Great for graphs 😊
- Can be hard to visualize
- Serialization can be difficult
- Queries more complicated



From [PDX Graph Meetup](#)

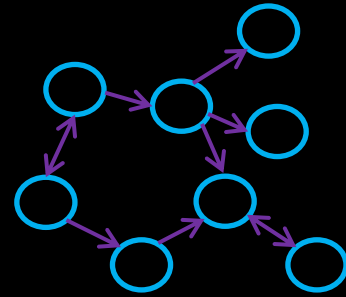
Queries

SPARQL, Cypher

SPARQL (W3C Standard)

- Uses Resource Description Framework format
 - triple store
- RDF Limitations
 - No named graphs
 - No quantifiers or general statements
 - “Every page was created by some author”
 - “Cats meow”
- Requires a schema or *ontology* (RDFS) to define rules
 - "The object of 'homepage' must be a Document."
 - "Link from an actor to a movie must connect an object of type Person to an object of type Movie."

```
SELECT ?name ?email
WHERE {
    ?person a foaf:Person.
    ?person foaf:name ?name.
    ?person foaf:mbox ?email. }
```



Cypher (Neo4J only)

- No longer proprietary
- Stores whole graph, not just triples
- Allows for named graphs
- ...and general Property Graphs (edges and nodes may have values)

```
SMATCH (Jack:Person
        { name:'Jack Nicolson' })-[:ACTED_IN]-(movie:Movie)
RETURN movie
```

Graph Databases

- These are not curiosities, but are behind some of the most high-profile pieces of Web infrastructure.
- They are definitely big data.

Microsoft Bing Knowledge Graph	Search and conversations.	~2 billion primary entries ~55 billion facts
Facebook		~50 million primary entries ~500 million assertions
Google Knowledge Graph	Search and conversations.	~1 billion entries ~55 billion facts
LinkedIn graph		590 million members 30 million companies

Hadoop & Spark



What kind
of databases
are they?

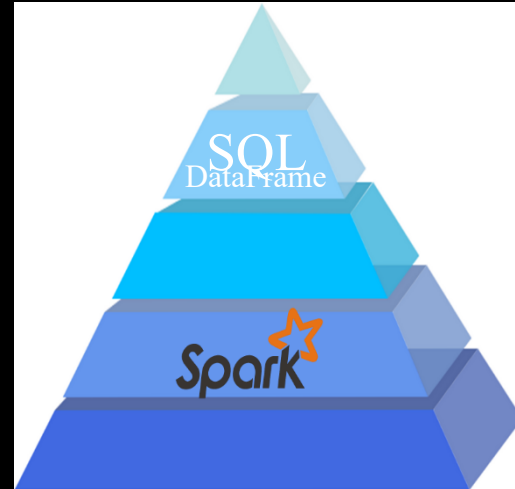
Frameworks for Data

These are both frameworks for distributing and retrieving data. Hadoop is focused on disk based data and a basic map-reduce scheme, and Spark evolves that in several directions that we will get in to. Both can accommodate multiple types of databases and *achieve their performance gains by using parallel workers.*



The mother of Hadoop was necessity. It is trendy to ridicule its primitive design, but it was the first step.

We have repurposed many of these blocks to build a better framework.

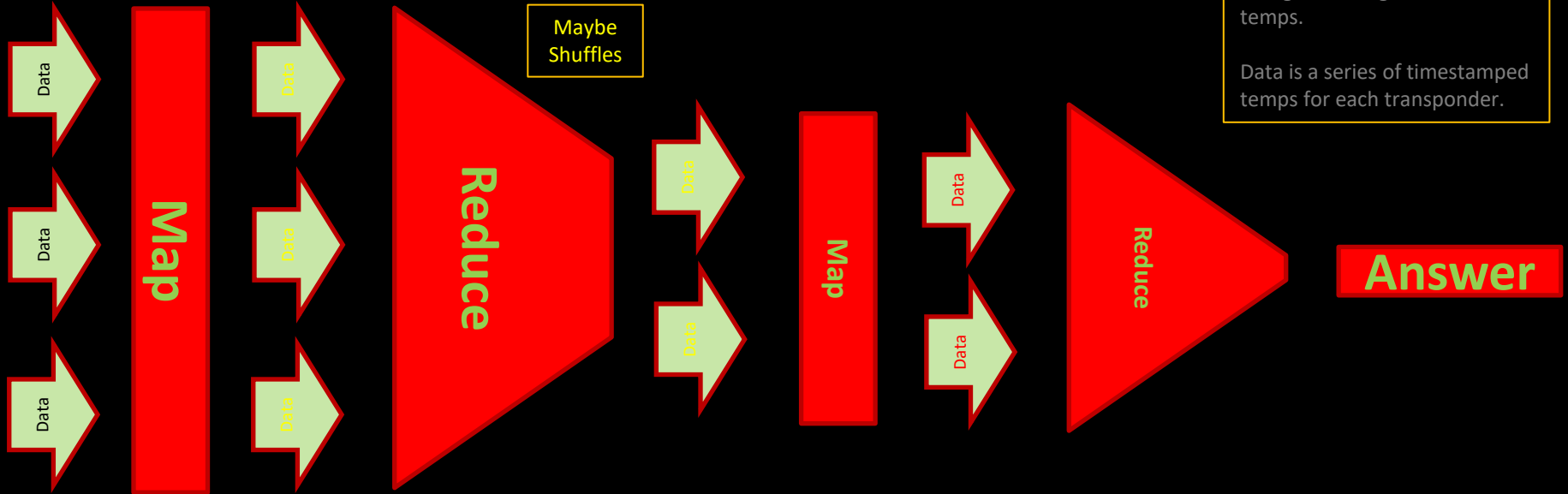


What exactly is this Hadoop "framework"?

- Programming platform
- Distributed filesystem
- Parallel execution environment
- Software ecosystem

Programming = MapReduce

In Place



Transponder ID -> Geo Coordinates
00154301 -> 59.33, 177.60
04435354 -> 56.71, 171.73
04539340 -> 25.18, -118.89

Only keep data for Bering Sea
00154301 -> 59.33, 177.60
04435354 -> 56.71, 171.73

Find biggest change
at each transponder
in last 24h

Keep any over 20
degrees

00154301 -> 30

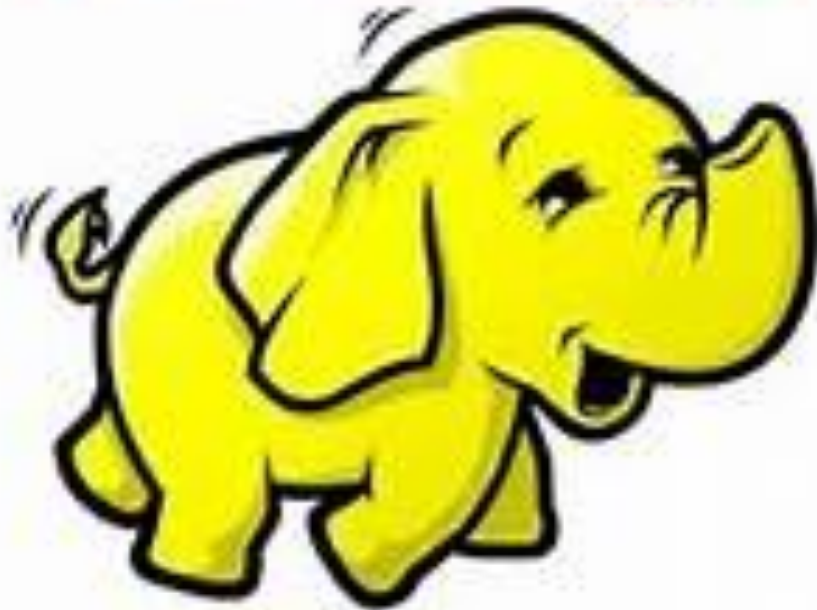
00154301 -> 30
04435354 -> 5

HDFS: Hadoop Distributed File System

- Replication
- Write Once Read Many (WORM)
- Requires underlying filesystem
- But very friendly to set up

Hadoop Ecosystem Lives On

hadoop



And lots
more...

Intro to Spark

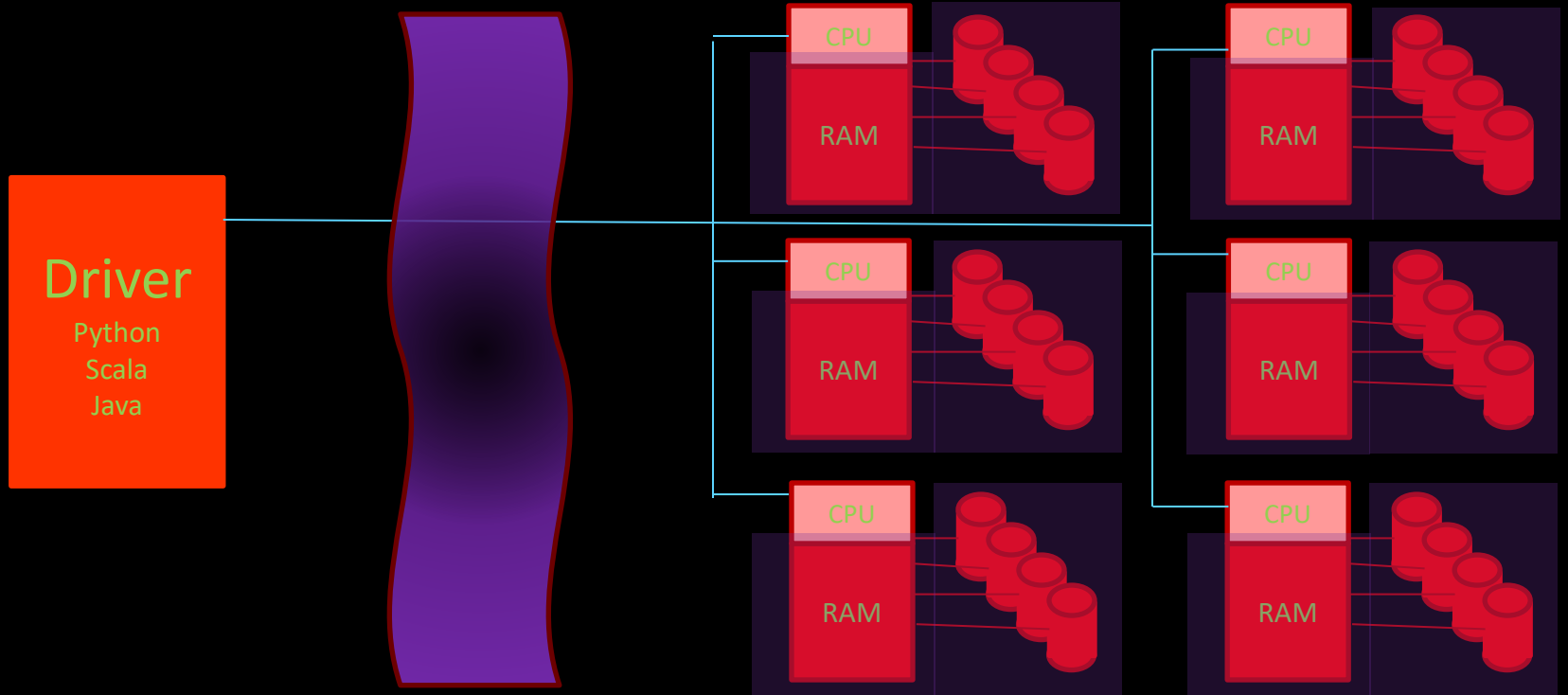
Spark Capabilities

(i.e. Hadoop shortcomings)

- Performance
 - First, use RAM
 - Also, be smarter
- Ease of Use
 - Python, Scala, Java first class citizens
- New Paradigms
 - SparkSQL
 - Streaming
 - MLib
 - GraphX
 - ...more

But using HDFS as the backing store is a common and sensible option.

Same Idea (improved)



RDD

Resilient Distributed Dataset

Spark Formula

1. Create/Load RDD

Webpage visitor IP address log

2. Transform RDD

"Filter out all non-U.S. IPs"

3. But don't do anything yet!

Wait until data is actually needed

Maybe apply more transforms ("Distinct IPs")

4. Perform *Actions* that return data

Count "How many unique U.S. visitors?"

Let's invite forum visitors to a local conference. How many might there be?

Simple Example

```
>>> lines_rdd = sc.textFile("nasa_serverlog_202204.csv")
```

NASA asks: are
people viewing our
Hubble Space
Telescope content?

Simple Example

```
>>> lines_rdd = sc.textFile("nasa_serverlog_202204.csv")  
>>> HubbleLines_rdd = lines_rdd.filter(lambda line: "Hubble" in line)
```

Lambdas

We'll see a lot of these. A lambda is simply a function that is too simple to deserve its own subroutine. Anywhere we have a lambda we could also just name a real subroutine that could go off and do anything.

Most modern languages have adopted this nicety.

The Python syntax is simply *lambda input_parameter: code*

Simple Example

```
>>> lines_rdd = sc.textFile("nasa_serverlog_202204.csv")  
>>> HubbleLines_rdd = lines_rdd.filter(lambda line: "Hubble" in line)
```

Lambdas

If we look at the spark documentation for filter, we will see that it asks us for a True/False function to determine if it should retain any element in the RDD. In this case, we could write the above line as:

```
HubbleLines_rdd = lines_rdd.filter(HubbleCheckFunction)
```

But then we have to write a separate function

```
def HubbleCheckFunction(element):  
    return ("Hubble" in element)
```

Lambdas allow us to be much more concise.

Simple Example

```
>>> lines_rdd = sc.textFile("nasa_serverlog_202204.csv")
```

```
>>> HubbleLines_rdd = lines_rdd.filter(lambda line: "Hubble" in line)
```

```
>>> HubbleLines_rdd.count()
```

```
4788
```



Transform



Action

Simple Example

```
>>> lines_rdd = sc.textFile("nasa_serverlog_202204.csv")
```

```
>>> HubbleLines_rdd = lines_rdd.filter(lambda line: "Hubble" in line)
```

```
>>> HubbleLines_rdd.count()  
4788
```

```
>>> HubbleLines_rdd.first()  
'www.nasa.gov\shuttle/missions/Hubble.gif'
```



Transform



Actions

Transformations vs. Actions

Transformations go from one RDD to another¹.

Actions bring some data back from the RDD.

Transformations are where the Spark machinery can do its magic with lazy evaluation and clever algorithms to minimize communication and parallelize the processing. You want to keep your data in the RDDs as much as possible.

Actions are mostly used either at the end of the analysis when the data has been distilled down (*collect*), or along the way to "peek" at the process (*count*, *take*).

¹ Yes, some of them also create an RDD (parallelize), but you get the idea.

More Spark

John Urbanic

Parallel Computing Scientist
Pittsburgh Supercomputing Center

Distinguished Service Professor
Carnegie Mellon University

Solution Review:

Drone Delivery with SQL



My Plan

There is no one correct answer, and there are several reasonable paths to the solution (as well as countless awkward - but acceptable - ones).

My initial thought was to do the obvious, in which case I would:

1. Find the eligible zip codes

- a) Get the coordinates for all the zip codes
- b) Get the distances from Cambridge
- c) Keep only those <100km

2. Get the orders for each zipcode

Maybe I should have tried to process only those from step 1...
but this approach was stupidly obvious and lazy and we aren't
grading on performance, today. You may have been smarter.

3. Use the first step to filter the results of the second step

In other words, inner join the first table with the second

4. Now just sort the results and take the top 3

Find The Eligible Zip Codes

This is the obvious thing to do with the GIS distance function:

```
SELECT Zip
FROM Zips
WHERE ST_Distance_Sphere(point(Lng,Lat),point(-71.10253,42.36224)) < 100000
```

But, we really shouldn't hard code this number in there, so I computed it properly

```
SELECT Zip
FROM Zips
WHERE ST_Distance_Sphere( point(Lng,Lat),point((SELECT Lng FROM Zips WHERE Zip=02139),(SELECT Lat FROM Zips WHERE Zip=02139))) < 100000;
```

```
+-----+
| Zip   |
+-----+
| 2133  |
| 2203  |
| 2205  |
| 2872  |
| 2542  |
...
...
| 2115  |
| 2113  |
+-----+
552 rows in set (0.21 sec)
```

Looks reasonable. We found 552 zip codes.

Get The Order Counts For Each Zip Code

This looks a lot like what we did in our GROUP BY example earlier. Let's do the analog here:

```
SELECT PostalCode, count(*) AS OrdersInPostalCode
FROM Orders
JOIN
Customer
  ON Orders.CustomerID=Customer.CustomerID
GROUP BY PostalCode;
```

```
+-----+-----+
| PostalCode | OrdersInPostalCode |
+-----+-----+
| 01001      | 819                |
| 01007      | 812                |
| 01010      | 868                |
| 01020      | 751                |
| 01027      | 1649               |
| ...       | ...                |
| ...       | ...                |
| ...       | ...                |
| 99518      | 7361               |
| 99567      | 4878               |
| 99577      | 14453              |
| 99603      | 809                |
| 99611      | 1530               |
| 99669      | 763                |
+-----+-----+
684 rows in set (19.83 sec)
```

Note that this step does take some **time**. We might want to consider implementing our logic so that we only perform this on the eligible zip codes.

Combine The First Two Steps

Now I have a list of eligible zip codes, and I have all the zip code order counts. Sounds like I just want to find the common items: an inner join. Something like

```
SELECT Results_I_Want_Displayed
FROM Eligible_Zips
JOIN
Orders_by_Zip
  ON Zips_as_key
```

Now we just cut and paste the right stuff from our first two queries. However, after we do that we will find that we need to alias those tables/queries so that we can properly specify the keys and results.

Building Our Query

Here is what I ended up with.

```
SELECT AllowedZips.Zip, OrdersByPostalCode.OrdersInThisPostalCode
FROM
  (SELECT Zip
   FROM Zips
    WHERE ST_Distance_Sphere( point(Lng,Lat),point((SELECT Lng FROM Zips WHERE Zip=02139),(SELECT Lat FROM Zips WHERE Zip=02139))) < 100000
   ) AS AllowedZips
JOIN
  (SELECT PostalCode, count(*) AS OrdersInThisPostalCode
   FROM Orders
    JOIN
      Customer
      ON Orders.CustomerID=Customer.CustomerID
    GROUP BY PostalCode) AS OrdersByPostalCode
ON
OrdersByPostalCode.PostalCode=AllowedZips.Zip;
```

```
+-----+-----+
| Zip | OrdersInThisPostalCode |
+-----+-----+
| 1010 | 868 |
| 1031 | 676 |
| 1331 | 2538 |
| 1420 | 737 |
| 1440 | 1529 |
...
...
...
| 2745 | 1670 |
| 2747 | 781 |
| 2748 | 815 |
| 2766 | 711 |
| 2790 | 899 |
+-----+-----+
108 rows in set (21.89 sec)
```

The Two Subqueries

Let's make sure you understand the pieces. Here are our two original queries.

```
SELECT AllowedZips.Zip, OrdersByPostalCode.OrdersInThisPostalCode
FROM
(SELECT Zip
 FROM Zips
  WHERE ST_Distance_Sphere( point(Lng,Lat),point((SELECT Lng FROM Zips WHERE Zip=02139),(SELECT Lat FROM Zips WHERE..< 100000
 ) AS AllowedZips
JOIN
(SELECT PostalCode, count(*) AS OrdersInThisPostalCode
 FROM Orders
 JOIN
 Customer
  ON Orders.CustomerID=Customer.CustomerID
 GROUP BY PostalCode) AS OrdersByPostalCode
ON
OrdersByPostalCode.PostalCode=AllowedZips.Zip;
```

Results

These are our results. Also make sure you see how the aliases are used here.

```
SELECT AllowedZips.Zip, OrdersByPostalCode.OrdersInThisPostalCode
FROM
  (SELECT Zip
   FROM Zips
    WHERE ST_Distance_Sphere( point(Lng,Lat),point((SELECT Lng FROM Zips WHERE Zip=02139),(SELECT Lat FROM Zips WHERE Zip=02139))) < 100000
   ) AS AllowedZips
JOIN
  (SELECT PostalCode, count(*) AS OrdersInThisPostalCode
   FROM Orders
    JOIN
      Customer
      ON Orders.CustomerID=Customer.CustomerID
    GROUP BY PostalCode) AS OrdersByPostalCode
ON
OrdersByPostalCode.PostalCode=AllowedZips.Zip;
```

Zip	OrdersInThisPostalCode
1010	868
1031	676
1331	2538
1420	737
1440	1529
...	
...	
...	
2745	1670
2747	781
2748	815
2766	711
2790	899

Final Answer

All that remains is to sort and select the top 3.

```
SELECT AllowedZips.Zip, OrdersByPostalCode.OrdersInThisPostalCode
FROM
(SELECT Zip
  FROM Zips
   WHERE ST_Distance_Sphere( point(Lng,Lat),point((SELECT Lng FROM Zips WHERE Zip=02139),(SELECT Lat FROM Zips WHERE Zip=02139))) < 100000
 ) AS AllowedZips
JOIN
(SELECT PostalCode, count(*) AS OrdersInThisPostalCode
  FROM Orders
   JOIN
     Customer
   ON Orders.CustomerID=Customer.CustomerID
  GROUP BY PostalCode) AS OrdersByPostalCode
ON
OrdersByPostalCode.PostalCode=AllowedZips.Zip
ORDER BY OrdersByPostalCode.OrdersInThisPostalCode DESC
LIMIT 3;
```

```
+-----+-----+
| Zip | OrdersInThisPostalCode |
+-----+-----+
| 2169 | 4688 |
| 2155 | 4127 |
| 2446 | 2622 |
+-----+-----+
3 rows in set (18.13 sec)
```

Actual Variables

One thing that seems especially awkward is the way we had to refer to our fixed Cambridge Lng and Lat values. This begged for a "do it once" solution. Indeed we do have regular variables we can use for these tasks.

```
SET @HeadquartersLng = (SELECT Lng FROM Zips WHERE Zip=02139);
SET @HeadquartersLat = (SELECT Lat FROM Zips WHERE Zip=02139);
```

And our solution cleans up a bit.

```
SELECT AllowedZips.Zip, OrdersByPostalCode.OrdersInThisPostalCode
FROM
  (SELECT Zip
   FROM Zips
   WHERE ST_Distance_Sphere( point(Lng,Lat),point(@HeadquartersLng,@HeadquartersLat) ) < 100000 ) AS AllowedZips
JOIN
  (SELECT PostalCode, count(*) AS OrdersInThisPostalCode
   FROM Orders
   JOIN
     Customer
     ON Orders.CustomerID=Customer.CustomerID
   GROUP BY PostalCode) AS OrdersByPostalCode
ON
OrdersByPostalCode.PostalCode=AllowedZips.Zip
ORDER BY OrdersByPostalCode.OrdersInThisPostalCode DESC
LIMIT 3;
```

Why didn't I mention this capability earlier? Mostly because you might tend to start thinking like a sequential programming, using variables to move from one state to the next. The standard SQL paradigm is to build these "inside out" queries instead, and you have to understand that if you want to swim in those waters. But in this case, this is perfectly acceptable.

A Little More Efficient

As mentioned earlier, we could also be a little more thoughtful about minimizing the larger joins or groupings by eliminating ineligible zip codes in an earlier subquery. Here is an approach using that philosophy.

```
SELECT COUNT(*) as ordersbyzip, PostalCode
FROM Orders
JOIN Customer
  ON Orders.CustomerId = Customer.CustomerId
 WHERE Customer.PostalCode IN (SELECT DISTINCT distances.PostalCode
                               FROM ( SELECT * FROM
                                     (SELECT Customer.PostalCode,ST_Distance_Sphere( point(Lng,Lat),
                                                                 point((SELECT Lng FROM Zips WHERE Zip = 02139),
                                                                 (SELECT Lat FROM Zips WHERE Zip = 02139 ))) AS distance
                                     FROM Customer
                                     JOIN Zips
                                       ON PostalCode = Zip) AS alldistances
                                     WHERE alldistances.distance < 100000)
                               AS distances)

GROUP BY PostalCode
ORDER BY ordersbyzip DESC
LIMIT 3;
```

```
+-----+-----+
| ordersbyzip | PostalCode |
+-----+-----+
|          4688 | 02169      |
|          4127 | 02155      |
|          2622 | 02446      |
+-----+-----+
3 rows in set (7.23 sec)
```

This is well over twice as fast. These kind of optimizations abound in the SQL database world.

Faster Still

Here is an even simpler looking approach that involves multiple joins at once.

```
SELECT Zips.Zip, COUNT(Orders.OrderId) as num_orders
FROM Orders
JOIN Customer
      ON Orders.CustomerId = Customer.CustomerId
JOIN Zips
      ON Customer.PostalCode = Zips.Zip
WHERE ST_DISTANCE_SPHERE(POINT(Zips.Lng, Zips.Lat), POINT(-71.10253,42.36224)) <= 100000
GROUP BY Zips.Zip
ORDER BY num_orders
DESC LIMIT 3;
```

```
+-----+-----+
| Zip | num_orders |
+-----+-----+
| 2169 |         4688 |
| 2155 |         4127 |
| 2446 |         2622 |
+-----+-----+
3 rows in set (1.98 sec)
```

At first glance this would seem less efficient as it is joining a lot of data (Orders, Customers and Zips) before it finally pares it down with the WHERE. But it is our fastest approach yet.

This is because the SQL engine is clever and can optimize joins well (and "looks ahead" at the WHERE). We could do a simple comparison by using the `EXPLAIN` command in front of our queries, but we will leave further SQL optimization for your independent exploration.

An Interesting Submission

A submission from an earlier student had an interesting approach.

```
SELECT Zip, COUNT(OrderId) AS NumOrders
FROM
(SELECT CustomerId, Lng, Lat, Zip, ST_DISTANCE_SPHERE(POINT(Lng,Lat), POINT(-71.10253,42.36224)) AS distance
  FROM Customer JOIN Zips on PostalCode = Zip HAVING distance <100000) AS withinRadius
JOIN
Orders
  ON withinRadius.CustomerId = Orders.CustomerId
GROUP BY Zip
ORDER BY NumOrders DESC
LIMIT 3;
```

The peculiar thing here is this bit:

```
SELECT CustomerId, Lng, Lat, Zip, ST_DISTANCE_SPHERE(POINT(Lng,Lat), POINT(-71.10253,42.36224)) AS distance
FROM
Customer
JOIN Zips
  ON PostalCode = Zip
  HAVING distance <100000
```

HAVING should only be used with GROUP BY. The answer here can be found in the MySQL Reference Manual:

The SQL standard requires that HAVING must reference only columns in the GROUP BY clause or columns used in aggregate functions. However, MySQL supports an extension to this behavior...

It then goes on to explain the complications involved with this extension. Don't do this.

And now:

Back To Spark



Common Transformations

Transformation	Result	
map(func)	Return a new RDD by passing each element through <i>func</i> .	Same Size
filter(func)	Return a new RDD by selecting the elements for which <i>func</i> returns true.	Fewer Elements
flatMap(func)	<i>func</i> can return multiple items, and generate a sequence, allowing us to “flatten” nested entries (JSON) into a list.	More Elements
distinct()	Return an RDD with only distinct entries.	
sample(...)	Various options to create a subset of the RDD.	
union(RDD)	Return a union of the RDDs.	
intersection(RDD)	Return an intersection of the RDDs.	
subtract(RDD)	Remove argument RDD from other.	
cartesian(RDD)	Cartesian product of the RDDs.	
parallelize(list)	Create an RDD from this (Python) list (using a spark context).	

Full list at <http://spark.apache.org/docs/latest/api/python/reference/api/pyspark.RDD.html#pyspark.RDD>

Some Common Actions

Action	Result
<code>collect()</code>	Return all the elements from the RDD.
<code>count()</code>	Number of elements in RDD.
<code>countByValue()</code>	List of times each value occurs in the RDD.
<code>reduce(func)</code>	Aggregate the elements of the RDD by providing a function which combines any two into one (sum, min, max, ...).
<code>first()</code> , <code>take(n)</code>	Return the first, or first n elements.
<code>top(n)</code>	Return the n highest valued elements of the RDDs.
<code>takeSample(...)</code>	Various options to return a subset of the RDD..
<code>saveAsTextFile(path)</code>	Write the elements as a text file.
<code>foreach(func)</code>	Run the <i>func</i> on each element. Used for side-effects (updating accumulator variables) or interacting with external systems.

Full list at <http://spark.apache.org/docs/latest/api/python/reference/api/pyspark.RDD.html#pyspark.RDD>

Reduce!

Action	Result
<code>reduce(func)</code>	Aggregate the elements of the RDD by providing a function which combines any two into one (sum, min, max, ...).

This may seem like it is an odd way of getting a sum, or a minimum, but this is a very key concept in data science, and throughout computing in general.

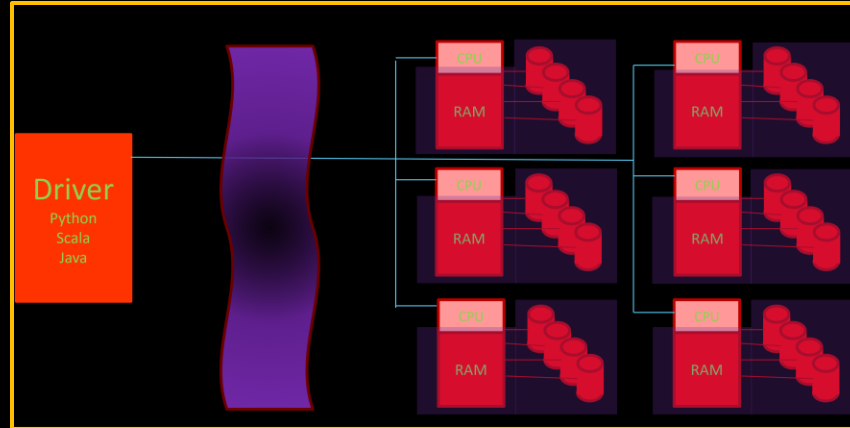
You might think of it as an elegant way to specify many small tasks, each specifying how any two elements combine.

But, the advantage it offers as data scales up is why it is so important. Let's get familiar with the concept.



Spark Context

```
>>> lines_rdd = sc.textFile("nasa_serverlog_202204.csv")
```



Spark Sessions

Later on we will look at more complex APIs built on top of RDDs, the most important being DataFrames. To simplify access to those features Spark allows us to load them initially with the context as a SparkSession.

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("Spark SQL example") \
    .config("some.config", "some-value").getOrCreate()

df = spark.read.json("demographics.json")
```

We are lucky that our Spark environment is configured by the PySpark shell, and we won't have to bother with much of this, but you will see this a lot in other Spark codes.

It is also useful to know how to create separate stand-alone spark scripts. They may need to setup a Spark Context manually

```
from pyspark import SparkConf, SparkContext

conf = SparkConf().setMaster("local").setAppName("Test_App")

sc = SparkContext(conf = conf)
```

You would typically run these scripts like so:

```
spark-submit Test_App.py
```

But it is all quite simple.

```
>>> lines_rdd = sc.textFile("nasa_serverlog_202204.csv")
>>> HubbleLines_rdd = lines_rdd.filter(lambda line: "Hubble" in line)
>>> HubbleLines_rdd.count()
4788
>>> HubbleLines_rdd.first()
'www.nasa.gov\shuttle/missions/Hubble.gif'
```

Pair RDDs

- Key/Value organization is a simple, but often very efficient schema, as we mentioned in our NoSQL discussion.
- Spark provides special operations on RDDs that contain key/value pairs. They are similar to the general ones that we have seen.
- Keys do not need to be unique. Unlike a Python dictionary or a primary key in SQL.
- On the language (*Python, Scala, Java*) side key/values are simply tuples. If you have an RDD all of whose elements happen to be tuples of two items, it is a Pair RDD and you can use the key/value operations that follow.

Pair RDD Transformations

Transformation	Result
<code>reduceByKey(func)</code>	Reduce values using <i>func</i> , but on a key by key basis. That is, combine values with the same key.
<code>groupByKey()</code>	Combine values with same key. Each key ends up with a list.
<code>sortByKey()</code>	Return an RDD sorted by key.
<code>mapValues(func)</code>	Use <i>func</i> to change values, but not key.
<code>keys()</code>	Return an RDD of only keys.
<code>values()</code>	Return an RDD of only values.

Note that all of the regular transformations are available as well.

Pair RDD Actions

As with transformations, all of the regular actions are available to Pair RDDs, and there are some additional ones that can take advantage of key/value structure.

Action	Result
<code>countByKey()</code>	Count the number of elements for each key.
<code>lookup(key)</code>	Return all the values for this key.

Two Pair RDD Transformations

Transformation	Result
<code>subtractByKey(otherRDD)</code>	Remove elements with a key present in other RDD.
<code>join(otherRDD)</code>	Inner join: Return an RDD containing all pairs of elements with matching keys in self and other. Each pair of elements will be returned as a $(k, (v1, v2))$ tuple, where $(k, v1)$ is in self and $(k, v2)$ is in other.
<code>leftOuterJoin(otherRDD)</code>	For each element (k, v) in self, the resulting RDD will either contain all pairs $(k, (v, w))$ for w in other, or the pair $(k, (v, None))$ if no elements in other have key k .
<code>rightOuterJoin(otherRDD)</code>	For each element (k, w) in other, the resulting RDD will either contain all pairs $(k, (v, w))$ for v in this, or the pair $(k, (None, w))$ if no elements in self have key k .
<code>cogroup(otherRDD)</code>	Group data from both RDDs by key.

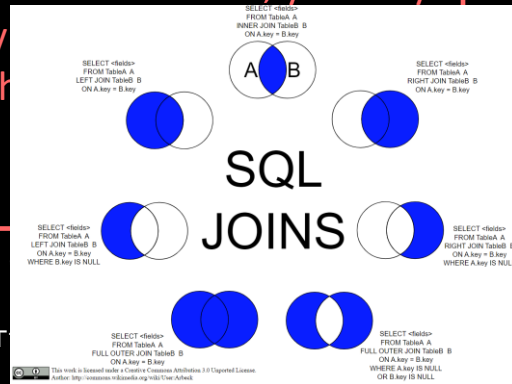
Joins Are Essential

Any database designer can tell you how common joins are. They are how we combine information from different ~~tables~~ RDDs. Let's look at a simple example. We make an RDD of our top purchasing customers.

And an RDD with all of our customers' addresses.

To create a mailing list of special coupons for join on the two datasets.

If you are coming from a Pandas DataFrame background, *joins* are congruent with the *Merge* functions. If you've used them, you may have noticed that they even small datasets. The



```
>>> best_customers_rdd = sc.parallelize([("Joe", "$103"), ("Alice", "$2000"), ("Bob",
```

```
>>> customer_addresses_rdd = sc.parallelize([("Joe", "23 State St."), ("Frank", "555 T  
Forest Rd."), ("Alice", "3 Elm Road"), ("Bob", "88 west Oak")])
```

```
>>> promotion_mail_rdd = best_customers_rdd.join(customer_addresses_rdd)
```

```
>>> promotion_mail_rdd.collect()  
[('Bob', ('$1200', '88 west Oak')), ('Joe', ('$103', '23 State St.')), ('Alice', ('$2000', '3 Elm Road'))]
```

Shakespeare, a Data Analytics Favorite

Applying data analytics to the works of Shakespeare has become all the rage. Whether determining the legitimacy of his authorship (it wasn't Marlowe) or if Othello is actually a comedy (perhaps), or which word makes Macbeth so creepy ("the", yes) it is amazing how much publishable research has sprung from the recent analysis of 400 year old text.



We're going to do some exercises here using a text file containing all of his works.

Some Simple Problems

We have an input file, Complete_Shakespeare.txt, that you can also find at <http://www.gutenberg.org/ebooks/100>.

You might find it useful to have <http://spark.apache.org/docs/latest/api/python/reference/api/pyspark.RDD.html#pyspark.RDD> in a browser window.

If you are starting from scratch on the login node:

1) interact 2) cd BigData/Shakespeare 3) module load spark 4) pyspark

```
...  
>>> rdd = sc.textFile("Complete_Shakespeare.txt")
```

Let's try a few simple exercises.

- 1) Count the number of lines
- 2) Count the number of words (hint: Python "split" is a workhorse)
- 3) Count unique words
- 4) Count the occurrence of each word
- 5) Show the top 5 most frequent words

These last two are a bit more challenging. One approach is to think "key/value". If you go that way, think about which data should be the key and don't be afraid to swap it about with value. This is a very common manipulation when dealing with key/value organized data.

Some Simple Answers

```
>>> lines_rdd = sc.textFile("Complete_Shakespeare.txt")
>>>
>>> lines_rdd.count()
124787
>>>
>>> words_rdd = lines_rdd.flatMap(lambda x: x.split())
>>> words_rdd.count()
904061
>>>
>>> words_rdd.distinct().count()
67779
>>>
```

Next, I know I'd like to end up with a pair RDD of sorted word/count pairs:

```
(23407, 'the'), (19540, 'I'), (15682, 'to'), (15649, 'of') ...
```

Why not just `words_rdd.countByValue()`? It is an *action* that gives us a massive Python unsorted dictionary of results:

```
... 1, 'precious-princely': 1, 'christenings?': 1, 'empire': 11, 'vaunts': 2, 'Lubber's': 1,
'poet.': 2, 'Toad!': 1, 'leaden': 15, 'captains': 1, 'leaf': 9, 'Barnes.': 1, 'lead': 101, 'Hell':
1, 'wheat.': 3, 'lean': 28, 'Toad.': 1, 'trencher!': 2, '1.F.2.': 1, 'leas': 2, 'leap': 17, ...
```

Where to go next? Sort this in Python or try to get back into an RDD? If this is truly *BIG* data, we want to remain as an RDD until we reach our final results. So, no.

Some Harder Answers

Things data
scientists do.

} Turn these into k/v pairs

} Reduce to get words counts

} Flip keys and values
so we can sort on
wordcount instead of
words. Could trigger
repartition.

```
>>> lines_rdd = sc.textFile("Complete_Shakespeare.txt")
>>>
>>> lines_rdd.count()
124787
>>>
>>> words_rdd = lines_rdd.flatMap(lambda x: x.split())
>>> words_rdd.count()
904061
>>>
>>> words_rdd.distinct().count()
67779
>>>
>>> key_value_rdd = words_rdd.map(lambda x: (x,1))
>>>
>>> key_value_rdd.take(5)
[('The', 1), ('Project', 1), ('Gutenberg', 1), ('EBook', 1), ('of', 1)]
>>>
>>> word_counts_rdd = key_value_rdd.reduceByKey(lambda x,y: x+y)
>>> word_counts_rdd.take(5)
[('fawn', 11), ('considered-', 1), ('Fame,', 3), ('mustachio', 1), ('protested,', 1)]
>>>
>>> flipped_rdd = word_counts_rdd.map(lambda x: (x[1],x[0]))
>>> flipped_rdd.take(5)
[(11, 'fawn'), (1, 'considered-'), (3, 'Fame, '), (1, 'mustachio'), (1, 'protested,')]
>>>
>>> results_rdd = flipped_rdd.sortByKey(False)
>>> results_rdd.take(5)
[(23407, 'the'), (19540, 'I'), (18358, 'and'), (15682, 'to'), (15649, 'of')]
>>>
```

```
results_rdd = lines_rdd.flatMap(lambda x: x.split()).map(lambda x: (x,1)).reduceByKey(lambda x,y: x+y).map(lambda x: (x[1],x[0])).sortByKey(False)
```

Spark Anti-Patterns

Here are a couple code clues that you are not working with Spark, but probably against it.

```
for loops, collect in middle of analysis, large data structures

...
intermediate_results = data_rdd.collect()
python_data = []
for datapoint in intermediate_results:
    python_data.append(modify_datapoint(datapoint))
next_rdd = sc.parallelize(python_data)
...
```

Ask yourself, "would this work with billions of elements?". And likely anything you are doing with a for is something that Spark will gladly parallelize for you, if you let it.

Some Homework Problems

To do research-level text analysis, we generally want to clean up our input. Here are some of the kinds of things you could do to get a more meaningful distinct word count.

1) **Remove punctuation.** Often punctuation is just noise, and it is here. Do a Map and/or Filter (some punctuation is attached to words, and some is not) to eliminate all punctuation from our Shakespeare data. Note that if you are familiar with regular expressions, Python has a ready method to use those.

2) **Remove stop words.** Stop words are common words that are also often removed. You can remove many obvious stop words with a list of your own, and the *ML* has a convenient *StopWordsRemover()* method with default lists for various languages.

3) **Stemming.** Recognizing that various different words share the same root is often easy to do simply. Once again, Spark brings powerful libraries into the ecosystem via the Language Tool Kit. You should look at the docs, but you can give it a quick try:

```
import nltk
from nltk.stem.porter import PorterStemmer
stemmer = PorterStemmer()
stems_rdd = words_rdd.map(lambda x: stemmer.stem(x))
```

Regular Expressions

This may not be a "Big Data" topic, but this is an incredibly useful capability to have in this field.

These are useful in navigating the command line, building filtering scripts and as integral parts of many programming languages, such as Python, which makes them immediately useful here.

You probably already know some of them, like the `*` wildcard and can pick up much of the rest in a 10 minute tutorial:

`.at` matches any three-character string ending with "at", including "hat", "cat", "bat", "4at", "#at" and " at" (starting with a space).

`[hc]?at` matches "at", "hat", and "cat".

Who needs this Spark stuff?

As we do our first Spark exercises, you might think of several ways to accomplish these tasks that you already know. For example, Python *Pandas* is a fine way to do our following problem, and it will probably work on your laptop reasonably well. But they do not scale well*.

However we are learning how to leverage scalable techniques that work on very big data. Shortly, we will encounter problems that are considerable in size, and you will leave this workshop knowing how to harness very large resources.

Searching the *Complete Works of William Shakespeare* for patterns is a lot different from searching the entire Web (perhaps as the 800TB *Common Crawl* dataset).

So everywhere you see an RDD, realize that it is actually a parallel databank that could scale to PBs.



* See Panda's creator Wes McKinney's "10 Things I Hate About Pandas" at <https://wesmckinney.com/blog/apache-arrow-pandas-internals/>

DataFrames: Spark's SQL Side

As mentioned earlier, an appreciation for having some defined structure to your data has come back into vogue. SQL has returned! Or never left!

It simply makes sense and naturally emerges in many applications. It documents your data and helps to organize your code and thinking.

Also very importantly, it can greatly aid optimization, certainly with the Java VM that Spark uses, but also in general. SQL optimization engines are an enormous part of the world of data.

For these reasons, you will see that the newest set of APIs to Spark are DataFrame based. This is simply SQL type columns. Similar to Python pandas DataFrames, but based on RDDs, so much more scalable and flexible.

Creating DataFrames

It is very pretty intuitive to utilize DataFrames. Your elements just have labeled columns.

A row RDD is the basic way to go from RDD to DataFrame, and back, if necessary. A "row" is just a tuple.

```
>>> row_rdd = sc.parallelize([ ("Joe","Pine St.,"PA",12543), ("Sally","Fir Dr.,"WA",78456),  
                               ("Jose","Elm Pl.,"ND",45698) ])
```

```
>>>
```

```
>>> aDataFrameFromRDD = spark.createDataFrame( row_rdd, ["name", "street", "state", "zip"] )
```

```
>>> aDataFrameFromRDD.show()
```

```
+-----+-----+-----+-----+  
| name| street|state| zip|  
+-----+-----+-----+-----+  
|  Joe|Pine St.|  PA|12543|  
|Sally| Fir Dr.|  WA|78456|  
| Jose| Elm Pl.|  ND|45698|  
+-----+-----+-----+-----+
```

Creating DataFrames

You will come across DataFrames created without a schema. They get default column names.

```
>>> noSchemaDataFrame = spark.createDataFrame( row_rdd )
>>> noSchemaDataFrame.show()
+-----+-----+-----+-----+
|   _1|      _2|   _3|   _4|
+-----+-----+-----+-----+
|  Joe|Pine St.|  PA|12543|
|Sally|  Fir Dr.|  WA|78456|
| Jose|  Elm Pl.|  ND|45698|
+-----+-----+-----+-----+
```

Datasets

Spark has added a variation (technically a superset) of *DataFrames* called *Datasets*. For compiled languages with strong typing (Java and Scala) these provide static typing and can detect some errors at compile time.

This is not relevant to Python or R.

And you can create them inline as well.

[illegible]

Spark DataFrames making life easier...

Data from <https://github.com/spark-examples/pyspark-examples/raw/master/resources/zipcodes.json>

```
{"RecordNumber":1,"Zipcode":704,"ZipCodeType":"STANDARD","City":"PARC PARQUE","State":"PR","LocationType":"NOT ACCEPTABLE","Lat":17.96,"Long":-66.22,"Xaxis":0.38,"Yaxis":-0.87,"Zaxis":0.3,"WorldRegion":1}
{"RecordNumber":2,"Zipcode":704,"ZipCodeType":"STANDARD","City":"PASEO COSTA DEL SUR","State":"PR","LocationType":"NOT ACCEPTABLE","Lat":17.96,"Long":-66.22,"Xaxis":0.38,"Yaxis":-0.87,"Zaxis":0.3,"WorldRegion":1}
{"RecordNumber":10,"Zipcode":709,"ZipCodeType":"STANDARD","City":"BDA SAN LUIS","State":"PR","LocationType":"NOT ACCEPTABLE","Lat":18.14,"Long":-66.26,"Xaxis":0.38,"Yaxis":-0.86,"Zaxis":0.31,"WorldRegion":1}
```

```
>>> df = spark.read.json("zipcodes.json")
>>> df.printSchema()
root
  |-- City: string (nullable = true)
  |-- Country: string (nullable = true)
  |-- Decommissioned: boolean (nullable = true)
  |-- EstimatedPopulation: long (nullable = true)
  |-- Lat: double (nullable = true)
  |-- Location: string (nullable = true)
  |-- LocationText: string (nullable = true)
  |-- LocationType: string (nullable = true)
  |-- Long: double (nullable = true)
  |-- Notes: string (nullable = true)
  |-- RecordNumber: long (nullable = true)
  |-- State: string (nullable = true)
  |-- TaxReturnsFiled: long (nullable = true)
  |-- TotalWages: long (nullable = true)
  |-- worldRegion: string (nullable = true)
  |-- Xaxis: double (nullable = true)
  |-- Yaxis: double (nullable = true)
  |-- Zaxis: double (nullable = true)
  |-- ZipCodeType: string (nullable = true)
  |-- Zipcode: long (nullable = true)
```

```
>>> df.show()
```

City	Country	Decommissioned	EstimatedPopulation	Lat	Location
PARC PARQUE	US	false	null	17.96	NA-US-PR-PARC PARQUE
PASEO COSTA DEL SUR	US	false	null	17.96	NA-US-PR-PASEO CO...
BDA SAN LUIS	US	false	null	18.14	NA-US-PR-BDA SAN ...
CINGULAR WIRELESS	US	false	null	32.72	NA-US-TX-CINGULAR...
FORT WORTH	US	false	4053	32.75	NA-US-TX-FORT WORTH
FT WORTH	US	false	4053	32.75	NA-US-TX-FT WORTH
URB EUGENE RICE	US	false	null	17.96	NA-US-PR-URB EUGE...
MESA	US	false	26883	33.37	NA-US-AZ-MESA
MESA	US	false	25446	33.38	NA-US-AZ-MESA
HILLIARD	US	false	7443	30.69	NA-US-FL-HILLIARD
HOLDER	US	false	null	28.96	NA-US-FL-HOLDER
HOLT	US	false	2190	30.72	NA-US-FL-HOLT
HOMOSASSA	US	false	null	28.78	NA-US-FL-HOMOSASSA
BDA SAN LUIS	US	false	null	18.14	NA-US-PR-BDA SAN ...
SECT LANAUSSÉ	US	false	null	17.96	NA-US-PR-SECT LAN...
SPRING GARDEN	US	false	null	33.97	NA-US-AL-SPRING G...
SPRINGVILLE	US	false	7845	33.77	NA-US-AL-SPRINGVILLE
SPRUCE PINE	US	false	1209	34.37	NA-US-AL-SPRUCE PINE
ASH HILL	US	false	1666	36.4	NA-US-NC-ASH HILL
ASHEBORO	US	false	15228	35.71	NA-US-NC-ASHEBORO

And Sometime DataFrames Are Limiting

DataFrames are not as flexible as plain RDDs, and it isn't uncommon to find yourself fighting to do something that would be simple with a map, for example. In that case, don't hesitate to flip back into a plain RDD.

```
>>> row_rdd = sc.parallelize([ ("Joe","Pine St.,""PA",12543), ("Sally","Fir Dr.,""WA",78456),  
                             ("Jose","Elm Pl.,""ND",45698) ])

>>> aDataFrameFromRDD = spark.createDataFrame( row_rdd, ["name", "street", "state", "zip"] )

>>> another_row_rdd = aDataFrameFromRDD.rdd
```

Notice that this is not even a method, it is just a property (probably just pointing at the data attribute). This is a clue that behind the scenes we are always working with RDDs.

A minor technicality here is that the returned object is actually a "Row" type. You may not care. If you want it be the original tuple type then

```
>>> tuple_rdd = aDataFrameFromRDD.rdd.map(tuple)
```

Note that when our map function is a function that already exists, there is no need for a lambda.

Speaking of types...

DataFrames require much more attention to types. When a schema (column names and types) is missing, it will try to infer the schema from the data, which should be an RDD of either Row, namedtuple, or dict.

So, not strings. If we don't want to fill in the schema argument, the simplest answer is to give it tuples. So you can simply make everything a tuple:

```
tdata = sc.parallelize([("a",),("b",),("c",)])  
df = spark.createDataFrame(tdata)
```

```
tdata = sc.parallelize( ["a","b","c"] )  
Error when creating DF.
```

A slightly nicer way to do this is to make a genuine Spark "Row" type.

```
from pyspark.sql import Row  
rdd = sc.parallelize(['a','b','c'])  
df = rdd.map(Row).toDF()  
df.show()
```

```
+---+  
| _1 |  
+---+  
| a  |  
| b  |  
| c  |  
+---+
```

Why aren't we using DataFrames much today?

For one, this aforementioned attention to types would clutter up our examples. But, that is not the important reason.

It is important to realize that RDDs are not just Pandas DataFrames or SQL tables. Many of my graduate students, with previous SQL experience, attempt to use Spark by immediately creating DataFrames and just doing what they would do with any relational database. But, Spark has

- Transforms and Actions
- Explicit and efficient mapping and reduction operations

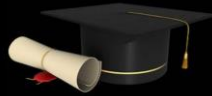
which enable great scalability and should not be overlooked.

But, DataFrames are very sensible and efficient and a core part of Spark. So don't shy away from using them. They may well become your default API for most tasks.

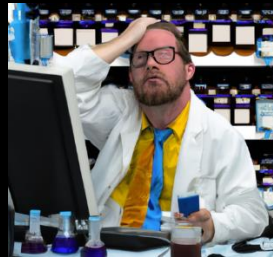
Data Scientist vs. Data-Proficient Scientist

This is an appropriate place to define our data science goals, now that we have some context.

If you are calling yourself a *DATA SCIENTIST* you might want to be fluent with all of these terms and their specific implementations in these three dialects (Pandas, SQL, Spark).



If you are a data-proficient scientist, you may well get by just being familiar with the options and then asking yourself “how to I get this data into that form?” If it takes you a few minutes and a quick google (or ChatGPT), there is no shame. At least I hope not!



BTW, in Pandas I often stumble there in the less direct way (maybe a few groupbys) which is acceptable with small data. With large data you often can't afford this cavalier attitude and want to leverage the Spark SQL optimizer.

Speaking of pandas, or SciPy, or...

Some of you may have experience with the many Python libraries that accomplish some of these tasks. Immediately relevant to today, *pandas* allows us to sort and query data, and *SciPy* provides some nice clustering algorithms. So why not just use them?

The answer is that Spark does these things in the context of having potentially huge, parallel resources at hand. We don't notice it as Spark is also convenient, but behind every Spark call:

- every RDD could be many TB in size
- every transform could use many thousands of cores and TB of memory
- every algorithm could also use those thousands of cores

So don't think of Spark as just a data analytics library because our exercises are modest. You are learning how to cope with **Big Data**.

Optimizations

We said one of the advantages of Spark is that we can control things for better performance. There are a multitude of optimization, performance, tuning and programmatic features to enable better control. We quickly look at a few of the most important.

- Persistence
- Partitioning
- Parallel Programming Capabilities
- Performance and Debugging Tools

Persistence

- Lazy evaluation implies by default that all the RDD dependencies will be computed when we call an action on that RDD.
- If we intend to use that data multiple times (say we are filtering some log, then dumping the results, but we will analyze it further) we can tell Spark to persist the data.
- We can specify different levels of persistence: *MEMORY_ONLY*, *MEMORY_ONLY_SER*, *MEMORY_AND_DISK*, *MEMORY_AND_DISK_SER*, *DISK_ONLY*

```
>>> lines_rdd = sc.textFile("nasa_19950801.tsv")
>>> stanfordLines_rdd = lines.filter(lambda line: "stanford" in line)
>>> stanfordLines_rdd.persist(StorageLevel.MEMORY_AND_DISK)
>>> stanfordLines_rdd.count()
47
```

```
>>> stanfordLines_rdd.first(1)
['glim.stanford.edu\t-\t807258394\tGET\t/shuttle/.../orbiters-logo.gif\t200\t1932\t\t']
.
.
.
>>> stanfordLines.unpersist()
```

**Do before
first action.**

Actions

**Otherwise will just
get evicted when
out of memory
(which is fine).**

Partitions

- Spark distributes the data of your RDDs across its resources. It tries to do some obvious things.
- With key/value pairs we can help keep that data grouped efficiently.
- We can create custom partitioners that beat the default (which is probably a hash or maybe range).
- Use `persist()` if you have partitioned your data in some smart way. Otherwise it will keep getting re-partitioned.

Parallel Programming Features

Spark has several parallel programming features that make it easier and more efficient to do operations in parallel in a more explicit way.

Accumulators are variables that allow many copies of a variable to exist on the separate worker nodes.

It is also possible to have replicated data that we would like all the workers to have access to. Perhaps a lookup table of IP addresses to country codes so that each worker can transform or filter on such information. Maybe we want to exclude all non-US IP entries in our logs. You might think of ways you could do this just by passing variables, but they would likely be expensive in actual operation (usually requiring multiple sends). The solution in Spark is to send an (immutable, read only) broadcast variable

Accumulators

```
log = sc.textFile("logs")
blanks = sc.accumulator(0)

def tokenizeLog(line)
    global blanks          # write-only variable
    if (line == "")
        blanks += 1
    return line.split(" ")

entries = log.flatMap(tokenizeLog)
entries.saveAsTextFile("parsedlogs.txt")
print "Blank entries: %d" blanks.value
```

Broadcast Variables

```
log = sc.textFile("log.txt")

IPTable = sc.broadcast(loadIPTable())

def countryFilter(IPentry, IPTable)
    return (IPentry.prefix() in IPTable)

USentries = log.filter(countryFilter)
```

Performance & Debugging

We will give unfortunately short shrift to performance and debugging, which are both important. Mostly, this is because they are very configuration and application dependent.

Here are a few things to at least be aware of:

- `SparkConf()` class. A lot of options can be tweaked here.
- Spark Web UI. A very friendly way to explore all of these issues.

IO Formats

Spark has an impressive, and growing, list of input/output formats it supports. Some important ones:

- Text
- CSV
- SQL type Query/Load
 - JSON (can infer schema)
 - Parquet
 - Hive
 - XML
 - Sequence (Hadoop key/value)
 - Databases: JDBC, Cassandra, HBase, MongoDB, etc.
- Compression (gzip...)

And it can interface directly with a variety of filesystems: local, HDFS, Lustre, Amazon S3,...

Spark Streaming

Spark addresses the need for streaming processing of data with a API that divides the data into batches, which are then processed as RDDs.

There are features to enable:

- Fast recovery from t
- Load balancing
- Integration with sta
- Integration with oth

15% of the "global datasphere" (quantification of the amount of data created, captured, and replicated across the world) is currently real-time. That number is growing quickly both in absolute terms and as a percentage.

Other Scalable Alternatives: Dask



Of the many alternatives to play with data on your laptop, there are only a few that aspire to scale up to big data. The only one, besides Spark, that seems to have any traction is Dask.

It attempts to retain more of the "laptop feel" of your toy codes, making for an easier port. The tradeoff is that the scalability is a lot more mysterious. If it doesn't work - or someone hasn't scaled the piece you need - your options are limited.

At this time, I'd say it is riskier, but academic projects can often entertain more risk than industry.

Numpy like operations

```
import dask.array as da
a = da.random.random(size=(10000, 10000),
                      chunks=(1000, 1000))
a + a.T - a.mean(axis=0)
```

Dataframes implement Pandas

```
import dask.dataframe as dd
df = dd.read_csv('/.../2020-*-.csv')
df.groupby(df.account_id).balance.sum()
```

Pieces of Scikit-Learn

```
from dask_ml.linear_model import \
LogisticRegression
lr = LogisticRegression()
lr.fit(train, test)
```

Drill Down?

Other Scalable Alternatives: Ray



Ray tries to do it all. Take a look at *docs.ray.io* to get some idea. *Dask* on Ray, *Spark* on Ray, *PyTorch* on Ray, and on the left an example of *Spark* with *TensorFlow* on Ray.

However, it seems the core philosophy is to run Python in parallel. A much better version of the ubiquitous Python Multiprocessing. Often used to run something like a parameter search.

Python is awesome for gluing together calls to higher performance languages. Like the Scala JVM in *Spark*. And we will soon see how well it does using the (hidden) capabilities of C++ in *TensorFlow*.

Trying to get scalable efficiency from Python itself has not been a successful path to high performance. I am skeptical.

But, people do seem to be using it as framework to combine other frameworks to do things like hyperparameter searches in *TensorFlow*. There are simpler ways to do that in general.

Your mileage may vary.

Training a Spark DataFrame with TensorFlow

`raydp.tf.TFEstimator` provides an API for training with TensorFlow.

```
from pyspark.sql.functions import col
df = spark.range(1, 1000)
# calculate z = x + 2y + 1000
df = df.withColumn("x", col("id")*2)\
        .withColumn("y", col("id") + 200)\
        .withColumn("z", col("x") + 2*col("y") + 1000)

from raydp.utils import random_split
train_df, test_df = random_split(df, [0.7, 0.3])

# TensorFlow code
from tensorflow import keras
input_1 = keras.Input(shape=(1,))
input_2 = keras.Input(shape=(1,))

concatenated = keras.layers.concatenate([input_1, input_2])
output = keras.layers.Dense(1, activation='sigmoid')(concatenated)
model = keras.Model(inputs=[input_1, input_2],
                    outputs=output)

optimizer = keras.optimizers.Adam(0.01)
loss = keras.losses.MeanSquaredError()

from raydp.tf import TFEstimator
estimator = TFEstimator(
    num_workers=2,
    model=model,
    optimizer=optimizer,
    loss=loss,
    metrics=["accuracy", "mse"],
    feature_columns=["x", "y"],
    label_column="z",
    batch_size=1000,
    num_epochs=2,
    use_gpu=False,
    config={"fit_config": {"steps_per_epoch": 2}})

estimator.fit_on_spark(train_df, test_df)

tensorflow_model = estimator.get_model()

estimator.shutdown()
```

Intro to Machine Learning with Spark

John Urbanic

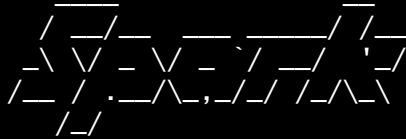
Parallel Computing Scientist
Pittsburgh Supercomputing Center

Distinguished Service Professor
Carnegie Mellon University

Run My Programs Or Yours

`exec()`

```
[urbanic@r001 ~]$ pyspark
Python 3.7.4 (default, Aug 13 2019, 20:35:49)
Type 'copyright', 'credits' or 'license' for more information
IPython 7.8.0 -- An enhanced Interactive Python. Type '?' for help.
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use
setLogLevel(newLevel).
Welcome to
```



version 3.0.0-preview2

```
Using Python version 3.7.4 (default, Aug 13 2019 20:35:49)
SparkSession available as 'spark'
In [1]: exec(open("./clustering.py").read())
1 5.76807041184e+14
2 3.73234816206e+14
3 2.13508993715e+14
4 1.38250712993e+14
5 1.2632806251e+14
6 7.97690150116e+13
7 7.14156965883e+13
8 5.7815194802e+13
...
...
...
```

If you have another session window open on bridge's login node, you can edit this file, save it while you remain in the editor, and then run it again in the python shell window with `exec(...)`.

You do not need this second session to be on a compute node. Do not start another interactive session.

Using MLlib

One of the reasons we use spark is for easy access to powerful data analysis tools. The MLlib library gives us a machine learning library that is easy to use and utilizes the scalability of the Spark system.

It has supported APIs for Python (with NumPy), R, Java and Scala.

We will use the Python version in a generic manner that looks very similar to any of the above implementations.

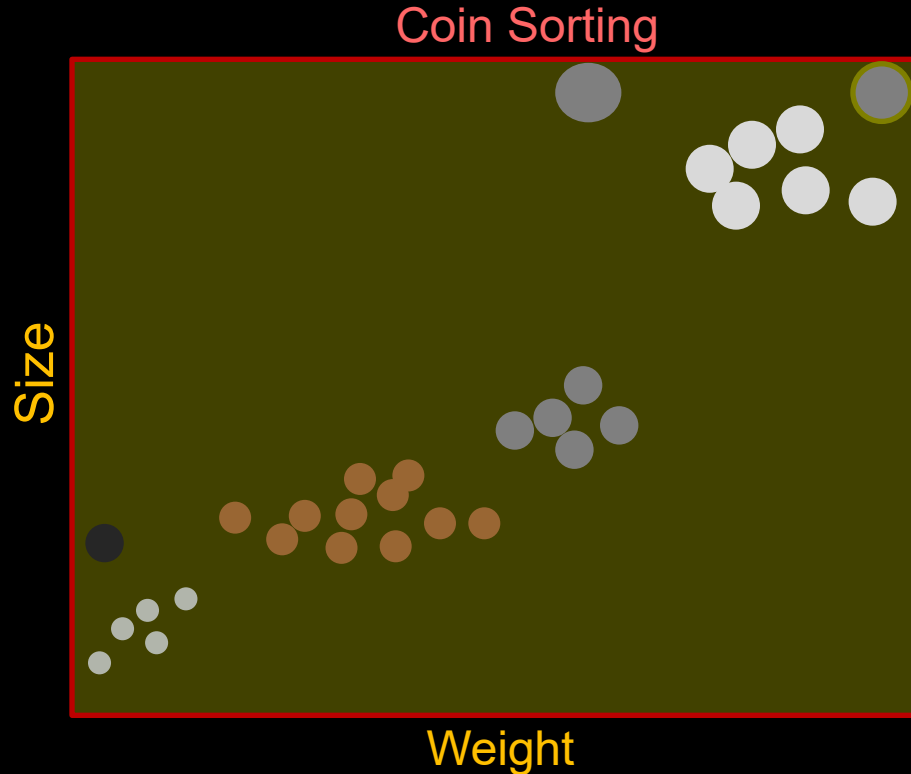
There are good example documents for the clustering routine we are using, as well as alternative clustering algorithms, here:

<http://spark.apache.org/docs/latest/mllib-clustering.html>

I suggest you use these pages for your Spark work.

Clustering

Clustering is a very common operation for finding grouping in data and has countless applications. This is a very simple example, but you will find yourself reaching for a clustering algorithm frequently in pursuing many diverse machine learning objectives, sometimes as one part of a pipeline.



Clustering

As intuitive as clustering is, it presents challenges to implement in an efficient and robust manner.

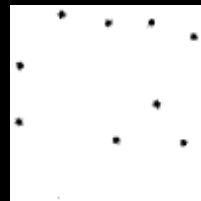
You might think

But it can get

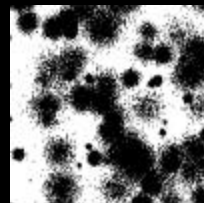
Sometimes you

How hard can

dimensional spaces.



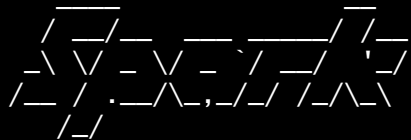
start with. Often you don't.
ere?



From 1900 until 1956 humans were considered to have 48 chromosomes, instead of 46, based upon the interpretation of this camera lucida image.

We will start with 5000 2D points. We want to figure out how many clusters there are, and their centers. Let's fire up pyspark and get to it...

Finding Clusters



version 1.6.0

Using Python version
SparkContext available

>>>

>>> rdd1 = sc.textFile

>>>

>>> rdd2 = rdd1.map()

>>> rdd3 = rdd2.map()

>>>

br06% interact

...

r288%

r288% module load spark

r288% pyspark

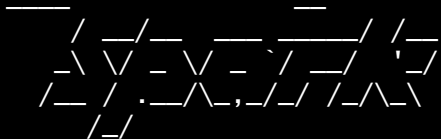
Make sure you are in the directory with the data file. Otherwise, Spark is dangerously quiet when you `textFile()` a file that does not exist. It is "lazy" and you won't find out that you have missing data until a later error.

and integers

Finding Our Way

```
>>> rdd1 = sc.textFile("5000_points.txt")
>>> rdd1.count()
5000
>>> rdd1.take(4)
['    664159    550946', '    665845    557965', '    597173    575538', '    618600    551446']
>>> rdd2 = rdd1.map(lambda x:x.split())
>>> rdd2.take(4)
[['664159', '550946'], ['665845', '557965'], ['597173', '575538'], ['618600', '551446']]
>>> rdd3 = rdd2.map(lambda x: [int(x[0]),int(x[1])])
>>> rdd3.take(4)
[[664159, 550946], [665845, 557965], [597173, 575538], [618600, 551446]]
>>>
```

Finding Clusters



version 1.6.0

Using Python version 2.7.5 (default, Nov 20 2015 02:00:19)
SparkContext available as sc, HiveContext available as sqlContext.

```
>>>
>>> rdd1 = sc.textFile("5000_points.txt")
>>>
>>> rdd2 = rdd1.map(lambda x:x.split())
>>> rdd3 = rdd2.map(lambda x: [int(x[0]),int(x[1])])
>>>
>>>
>>> from pyspark.mllib.clustering import KMeans
```



Read into RDD



Transform



Import Kmeans

`class pyspark.mllib.clustering.KMeans`

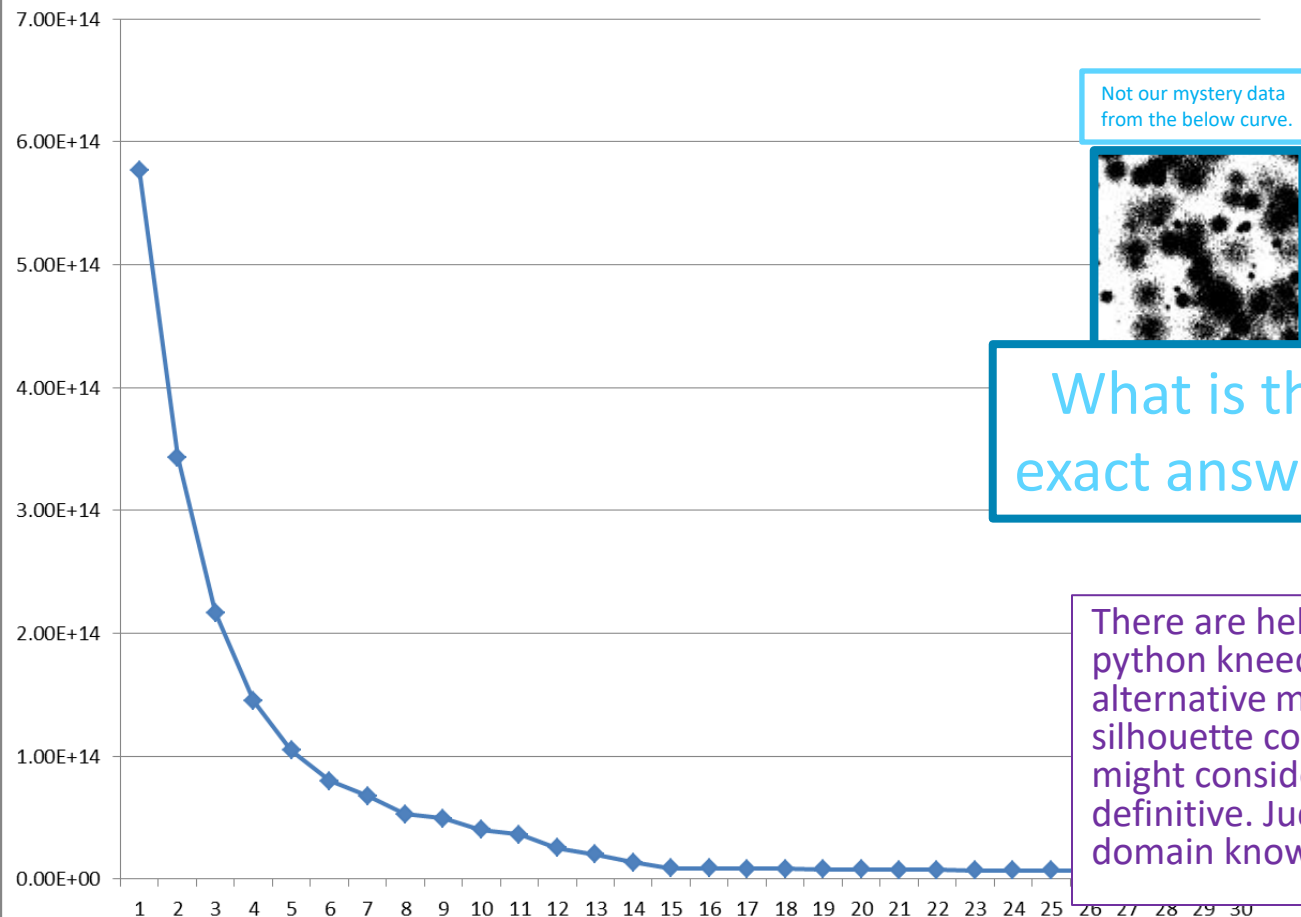
New in version 0.9.0.

`classmethod train(rdd, k, maxIterations=100, runs=1, initializationMode='k-means||', seed=None, initializationSteps=5, epsilon=0.0001, initialModel=None)` ¶

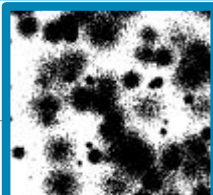
Train a k-means clustering model.

- Parameters:**
- **rdd** – Training points as an *RDD* of *Vector* or convertible sequence types.
 - **k** – Number of clusters to create.
 - **maxIterations** – Maximum number of iterations allowed. (default: 100)
 - **runs** – This param has no effect since Spark 2.0.0.
 - **initializationMode** – The initialization algorithm. This can be either "random" or "k-means||". (default: "k-means||")
 - **seed** – Random seed value for cluster initialization. Set as None to generate seed based on system time. (default: None)
 - **initializationSteps** – Number of steps for the k-means|| initialization mode. This is an advanced setting – the default of 5 is almost always enough. (default: 5)
 - **epsilon** – Distance threshold within which a center will be considered to have converged. If all centers move less than this Euclidean distance, iterations are stopped. (default: 1e-4)
 - **initialModel** – Initial cluster centers can be provided as a *KMeansModel* object rather than using the random or k-means|| initializationModel. (default: None)

Finding Clusters



Not our mystery data
from the below curve.



What is the
exact answer?

There are helper algorithms (the python kneed package) or alternative metrics, such as the silhouette coefficient, that you might consider. None are definitive. Judgement and domain knowledge are critical.

Finding Clusters



Using Python version 2.7.5 (default, Nov 20 2015 02:00:19)
SparkContext available as sc, HiveContext available as sqlContext.

```
>>>
>>> rdd1 = sc.textFile("5000_points.txt")
>>>
>>> rdd2 = rdd1.map(lambda x:x.split())
>>> rdd3 = rdd2.map(lambda x: [int(x[0]),int(x[1])])
>>>
>>> from pyspark.mllib.clustering import KMeans
>>>
>>> for clusters in range(1,30):
...     model = KMeans.train(rdd3, clusters)
...     print (clusters, model.computeCost(rdd3))
... 
```



Let's see results for 1-30 cluster tries

```
1 5.76807041184e+14
2 3.43183673951e+14
3 2.23097486536e+14
4 1.64792608443e+14
5 1.19410028576e+14
6 7.97690150116e+13
7 7.16451594344e+13
8 4.81469246295e+13
9 4.23762700793e+13
10 3.65230706654e+13
11 3.16991867996e+13
12 2.94369408304e+13
13 2.04031903147e+13
14 1.37018893034e+13
15 8.91761561687e+12
16 1.31833652006e+13
17 1.39010717893e+13
18 8.22806178508e+12
19 8.22513516563e+12
20 7.79359299283e+12
21 7.79615059172e+12
22 7.70001662709e+12
23 7.24231610447e+12
24 7.21990743993e+12
25 7.09395133944e+12
26 6.92577789424e+12
27 6.53939015776e+12
28 6.57782690833e+12
29 6.37192522244e+12
```

Right Answer?

```
>>> for trials in range(10):  
...     print  
...     for clusters in range(12,18):  
...         model = KMeans.train(rdd3,clusters)  
...         print (clusters, model.computeCost(rdd3))
```

```
12 2.45472346524e+13  
13 2.00175423869e+13  
14 1.90313863726e+13  
15 1.52746006962e+13  
16 8.67526114029e+12  
17 8.49571894386e+12
```

```
12 2.62619056924e+13  
13 2.90031673822e+13  
14 1.52308079405e+13  
15 8.91765957989e+12  
16 8.70736515113e+12  
17 8.49616440477e+12
```

```
12 2.5524719797e+13  
13 2.14332949698e+13  
14 2.11070395905e+13  
15 1.47792736325e+13  
16 1.85736955725e+13  
17 8.42795740134e+12
```

```
12 2.31466242693e+13  
13 2.10129797745e+13  
14 1.45400177021e+13  
15 1.52115329071e+13  
16 1.41347332901e+13  
17 1.31314086577e+13
```

```
12 2.47927778784e+13  
13 2.43404436887e+13  
14 2.1522702068e+13  
15 8.91765000665e+12  
16 1.4580927737e+13  
17 8.57823507015e+12
```

```
12 2.31466520037e+13  
13 1.91856542103e+13  
14 1.49332023312e+13  
15 1.3506302755e+13  
16 8.7757678836e+12  
17 1.60075548613e+13
```

```
12 2.5187054064e+13  
13 1.83498739266e+13  
14 1.96076943156e+13  
15 1.41725666214e+13  
16 1.41986217172e+13  
17 8.46755159547e+12
```

```
12 2.38234539188e+13  
13 1.85101922046e+13  
14 1.91732620477e+13  
15 8.91769396968e+12  
16 8.64876051004e+12  
17 8.54677681587e+12
```

```
12 2.5187054064e+13  
13 2.04031903147e+13  
14 1.95213876047e+13  
15 1.93000628589e+13  
16 2.07670831868e+13  
17 8.47797102908e+12
```

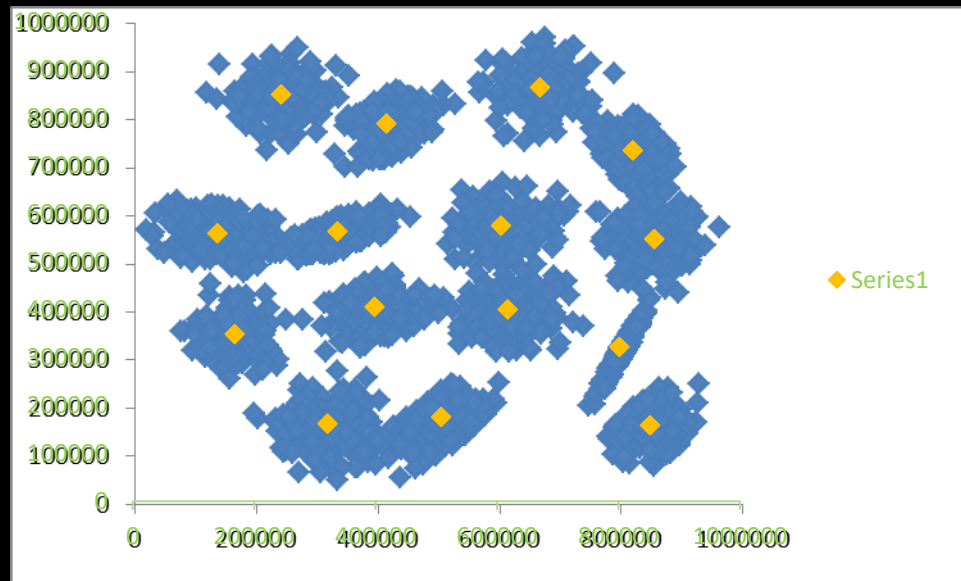
```
12 2.39830397362e+13  
13 2.00248378195e+13  
14 1.34867337672e+13  
15 2.09299321238e+13  
16 1.32266735736e+13  
17 8.50857884943e+12
```

Find the Centers

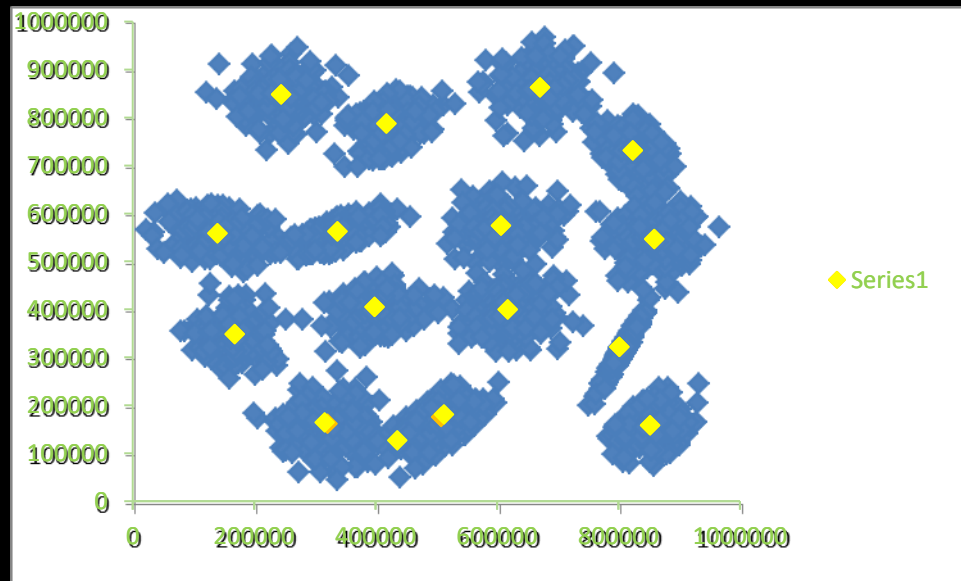
```
>>> for trials in range(10):                                #Try ten times to find best result
...     for clusters in range(12, 16):                     #Only look in interesting range
...         model = KMeans.train(rdd3, clusters)
...         cost = model.computeCost(rdd3)
...         centers = model.clusterCenters                 #Let's grab cluster centers
...         if cost<1e+13:                                  #If result is good, print it out
...             print (clusters, cost)
...             for coords in centers:
...                 print (int(coords[0]), int(coords[1]))
...             break
...
...
```

```
15 8.91761561687e+12
852058 157685
606574 574455
320602 161521
139395 558143
858947 546259
337264 562123
244654 847642
398870 404924
670929 862765
823421 731145
507818 175610
801616 321123
617926 399415
417799 787001
167856 347812
15 8.91765957989e+12
670929 862765
139395 558143
244654 847642
852058 157685
617601 399504
801616 321123
507818 175610
337264 562123
858947 546259
823421 731145
606574 574455
167856 347812
398555 404855
417799 787001
320602 161521
```

Fit?



16 Clusters



Assignment #3: Using Spark to mine Astro signals

Q: Can you find the repeating cosmological signal (pulsar?) in the captured data?

In `~urbanic/LargeScaleComputing/Spark/pulsar.dat` on Bridges you will find the data with a series of signals stored as:

```
ascension (degrees), declination (degrees), time (seconds), frequency (MHz)
```

These are radiofrequency signals captured by an array of instruments scanning a solid angle of the sky.

The data is, like almost all real data, a little noisy and has sources of errors. In our case the angular coordinates have 0.1 degrees error, the signal frequency has 0.1 MHz error and the timestamp error is <0.01 s (that is one STD or standard deviation).

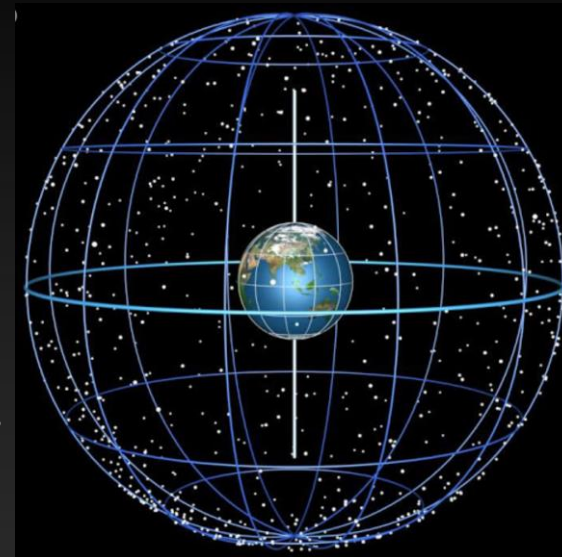
Your job is to find the source with the most signals (that is "blips", or entries in the datalog).

Your target will be found in the same location (within error) of the sky, on the same frequency (within error) chirping for the most blips, regularly spaced in time during that active period. So...

.....blip...blip...blip...blip...blip...blip.....

Again, you are looking for the signal with the most blips, and your answer (not including your Spark code) might be:

There are 40 blips at location 50,60 degrees with a frequency of 7MHZ and a period of 8 seconds.



Assignment #3: Using Spark to mine Astro signals

There are multiple approaches to this problem that will work. I can think of several that would be painful (hints in lecture).

Biggest Hint: I am a nice guy! While the data could have nasty surprises, it does not. Use common sense and experiment around and you will find an clear answer. But maybe not the first answer that comes to mind.

This is doable interactively. You can explore the data within PySpark using operations that are reasonably quick. You do not need to create long running scripts to get to the answer.

To recap:

1. log on
2. cp the datafile to where you want to work with your pyspark session
3. get an interactive node
4. load the spark module
5. start a pyspark session
6. load the datafile
7. and use transforms to wring out the answer: **the coordinates, the frequency and the period** (rounded to integers is fine).

Use Spark commands (RDDs) to derive your answer. Yes, with a dataset this size you could use python directly, but a real dataset would be far too large. Use Spark transforms to boil down your data until you have a modest amount (screenful) of data to inspect. Use python at that point, if you wish, or just observe your answer. Ask if you don't understand this point. You could lose credit otherwise.

Submit your answer along with the exact sequence of spark commands that got you there. *This means a single email.* **A text attachment of the spark commands, with your answer in the body of the mail, and any explanation you care to provide, is the required format.**

This assignment is due before we review in class on October 23rd. It is worth 10 points.

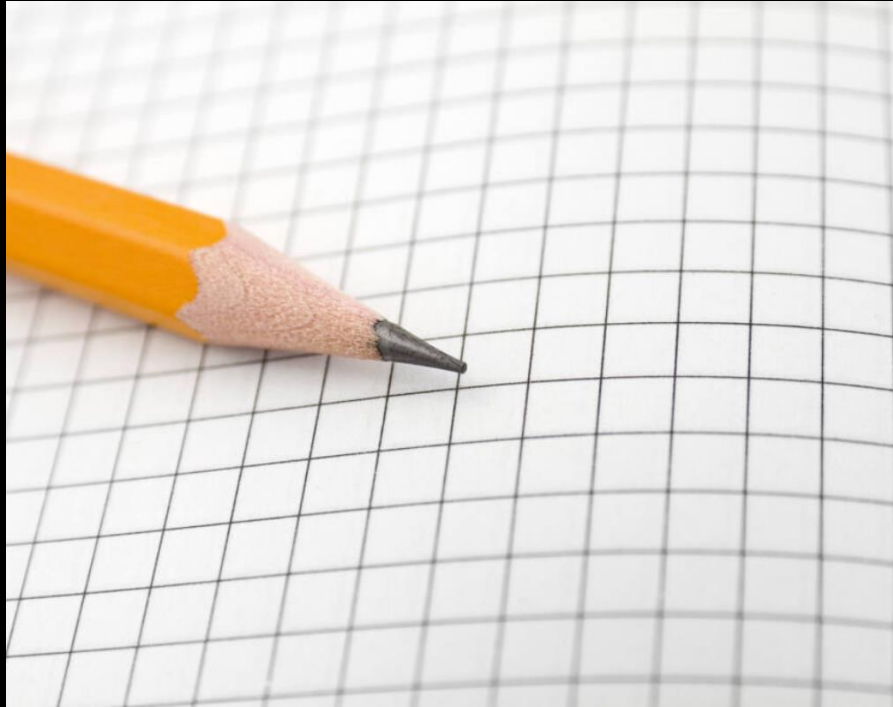
Assignment #3: Approach Hints

Don't overthink this problem, particularly the errors. While it is good to be diligent about these types of issues, it can also lead you into the weeds. Here are a few ways you might "reset" and approach the problem:

- One of the great things about having Spark at your disposal is that you can interactively explore large data. Because of the speed of the parallel system, you can ask questions in real time. With a dataset like this you can just start poking around and looking for large signals and interesting points. Then you can ask yourself how to find the "biggest" signal.
- But, if you are more comfortable starting algorithmically, it is often best to think of a tiny version of the problem that you have to solve on paper with only a pencil - no computer. Ask yourself how you would solve that. Then turn that into an algorithm.
- Or, start by ignoring the errors. Just find the answer as if the errors didn't matter. Then, since there are errors, what data might you be overlooking? And how might you find it?

These are three different approaches, but none of them should take more than a few steps to get to the answer. And none of them involve complex type conversion issues or routines from MLLIB, although you are welcome to use those too.

How to approach an algorithmic problem...

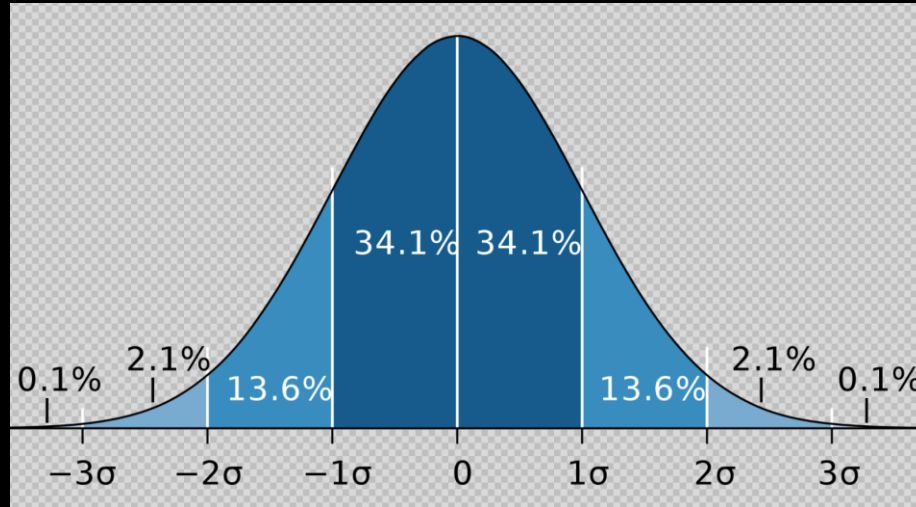


Think about how you, without a computer, would attempt a tiny version of the problem. Once you think about how you would do it, an algorithm often becomes clear. If you can't think about how you would do it, writing lines of code isn't going to make a solution materialize.

Assignment #3: More Hints

- Comments in your code are helpful to me - and to you too. If you make a mistake it is easier for me to give you credit if I know your intent. And of course comments are just good professional practice.
- We are exploring clustering just now. *Do not assume that that approach is necessary for Assignment #3*, although you could maybe use it. It will be essential for Assignment #4.
- If you see an occasional "Warning" from Spark (or really any of these machine learning packages) you may want to ignore it. That is terrible general programming advice, but a fact of life in this domain.
- I don't want anyone to struggle with Python accuracy issues, so you can treat the signal repeating time accuracy to be ~ 0.1 s if you want. You can't ignore the precision issues, but you should be able to see by inspection the signal in rounded off data as you home in on the answer. I don't want you to have to squint and compare 5th decimal places.
- The source *frequency* is like the *color* to a radiotelescope. It is completely independent of the *period*, which we can find from the timestamps.

A brief word about errors...



For a **normal** or **gaussian distribution**, the standard deviation indicates the region where about 2/3 of the points will fall. And for small sample sizes, this may be noisy.

We are closer to leading edge science than you might think.



The LIGO gravitational wave detector was able to confirm the collision of two neutron stars with both a gamma ray satellite and optical and other electromagnetic spectrum telescopes. For these transient events, it requires rapid real-time signal analysis to steer other instruments to the proper celestial coordinates. The 2 second gamma-ray burst was detected 1.7 seconds after the GW merger signal. 70 observatories were able to mine signatures in the following days. Even so, the refined location alert took a long time, and much improvement lies ahead.

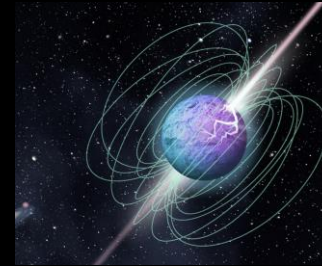


This rapid processing requirement will only become more extreme as the Square Kilometer Array comes fully on-line. It will generate over an Exabyte of data a day. It will require extreme real-time processing to classify and compress this data down to an archivable size.

Strange, repeating radio signal near the center of the Milky Way has scientists stumped



This article (www.livescience.com/strange-radio-source-milky-way-center) is the summary of the paper (arxiv.org/pdf/2109.00652.pdf) that looks an awful lot like what we are doing.



In April 2020, astronomers picked up some bursts of activity, in the X-ray band of the spectrum, a “run-of-the-mill” magnetar. But the team found that, shortly after the magnetar burst in the X-ray band, CHIME picked up two sharp staccato peaks in the radio band, within several milliseconds of each other, signaling a fast radio burst. The researchers were able to track the radio bursts to a point in the sky that was within a fraction of a degree of SGR 1935+2154 — the same magnetar that was blasting out X-rays around the same time. The team used calibration data from other astrophysical sources to estimate the magnetar’s brightness. They calculated that the magnetar, in the fraction of a second that the FRB flashed, was 3,000 times brighter than any other magnetar radio signal that has yet been observed. Happening in our own galaxy, thousands of times brighter than any other pulse we’ve ever seen.

Dimensionality Reduction



Dimensionality Reduction

We are going to find a recurring theme throughout machine learning:

- Our data naturally resides in higher dimensions
- Reducing the dimensionality makes the problem more tractable
- And simultaneously provides us with insight

This last two bullets highlight the principle that "learning" is often finding an effective compressed representation.

As we return to this theme, we will highlight these slides with our Dimensionality Reduction badge so that you can follow this thread and appreciate how fundamental it is.



Why all these dimensions?



The problems we are going to address, as well as the ones you are likely to encounter, are naturally highly dimensional. If you are new to this concept, let's look at an intuitive example to make it less abstract.

Category	Purchase Total (\$)
Children's Clothing	\$800
Pet Supplies	\$0
Cameras (Dash, Security, Baby)	\$450
Containers (Storage)	\$350
Romance Book	\$0
Remodeling Books	\$80
Sporting Goods	\$25
Children's Toys	\$378
Power Tools	\$0
Computers	\$0
Garden	\$0
Children's Books	\$180

< 2900 Categories >

This is a 2900 dimensional vector.

Why all these dimensions?



If we apply our newfound clustering expertise, we might find we have 80 clusters (with an acceptable error).

People spending on “child’s toys “ and “children’s clothing” might cluster with “child’s books” and, less obvious, "cameras (Dashcams, baby monitors and security cams)", because they buy new cars and are safety conscious. We might label this cluster "Young Parents". We also might not feel obligated to label the clusters at all. We can now represent any customer by their distance from these 80 clusters.

Customer Representation									80 dimensional vector.
Cluster	Young Parents	College Athlete	Auto Enthusiast	Knitter	Steelers Fan	Shakespeare Reader	Sci-Fi Fan	Plumber	...
Distance	0.02	2.3	1.4	8.4	2.2	14.9	3.3	0.8	...

We have now accomplished two things:

- we have compressed our data
- learned something about our customers (who to send a dashcam promo to).

More Machine Learning with Spark

John Urbanic

Parallel Computing Scientist
Pittsburgh Supercomputing Center

Distinguished Service Professor
Carnegie Mellon University

Assignment #3: Using Spark to mine Astro signals

Q: Can you find the repeating cosmological signal (pulsar?) in the captured data?

In `~urbanic/LargeScaleComputing/Spark/pulsar.dat` on Bridges you will find the data with a series of signals stored as:

```
ascension (degrees), declination (degrees), time (seconds), frequency (MHz)
```

These are radiofrequency signals captured by an array of instruments scanning a solid angle of the sky.

The data is, like almost all real data, a little noisy and has sources of errors. In our case the angular coordinates have 0.1 degrees error, the signal frequency has 0.1 MHz error and the timestamp error is <0.01 s (that is one STD or standard deviation).

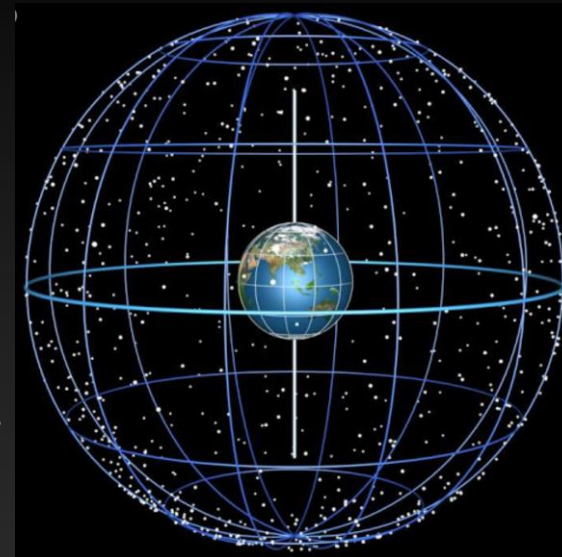
Your job is to find the source with the most signals (that is "blips", or entries in the datalog).

Your target will be found in the same location (within error) of the sky, on the same frequency (within error) chirping for the most blips, regularly spaced in time during that active period. So...

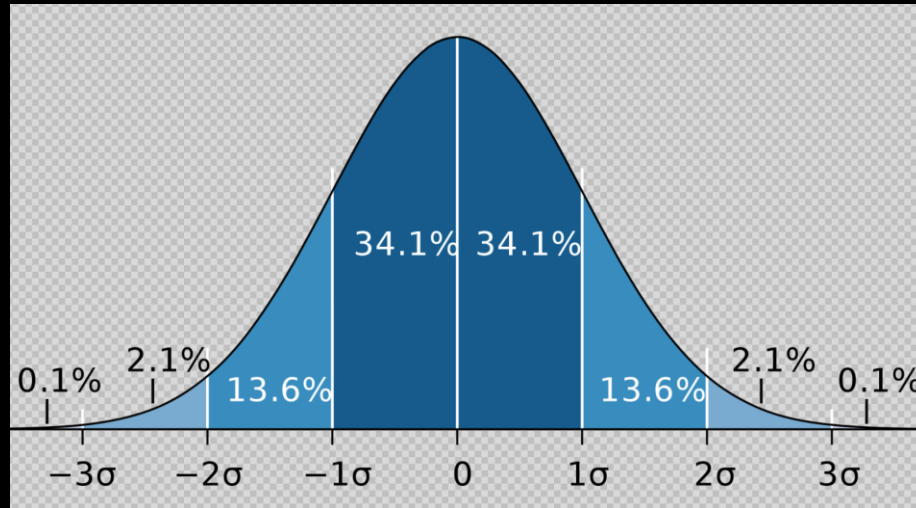
.....blip...blip...blip...blip...blip...blip.....

Again, you are looking for the signal with the most blips, and your answer (not including your Spark code) might be:

There are 40 blips at location 50,60 degrees with a frequency of 7MHZ and a period of 8 seconds.



A brief word about errors...

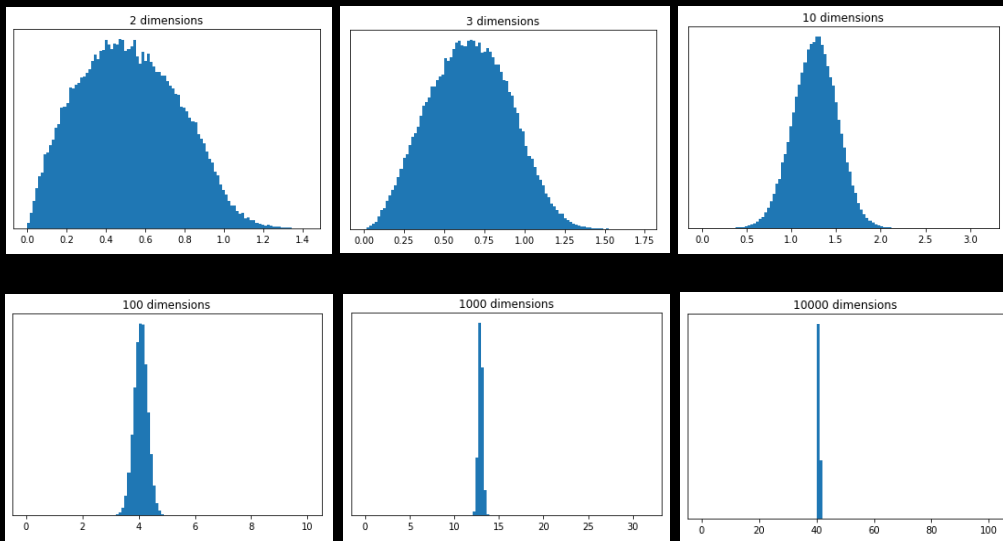


For a **normal** or **gaussian distribution**, the standard deviation indicates the region where about 2/3 of the points will fall. And for small sample sizes, this may be noisy.

Curse of Dimensionality



This is a good time to point out how our intuition can lead us astray as we increase the dimensionality of our problems - which we will certainly be doing - and to a great degree. There are several related aspects to this phenomenon, often referred to as the *Curse of Dimensionality*. One root cause of confusion is that our notion of Euclidian distance starts to fail in higher dimensions.



These plots show the distributions of pairwise distances between randomly distributed points within differently dimensioned unit hypercubes. Notice how all the points start to be about the same distance apart.

One can imagine this makes life harder on a clustering algorithm!

There are other surprising effects: random vectors are almost all orthogonal; the unit sphere takes almost no volume in the unit square. These cause all kinds of problems when generalizing algorithms from our lowly 3D world.

Most common misuse of this term is to note that points are sparser in a high number of features. This is true, but not this effect.

Metrics



Even the definition of distance (the *metric*) can vary based upon application. If you are solving chess problems, you might find the Manhattan distance (or taxicab metric) to be most useful.

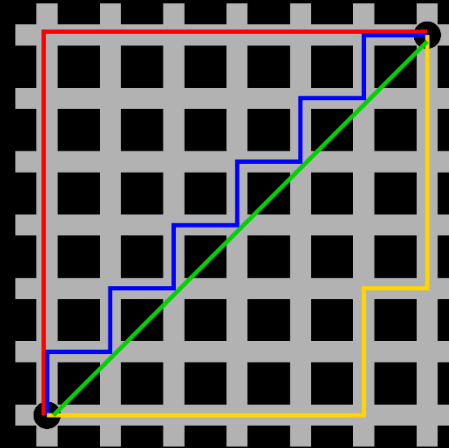


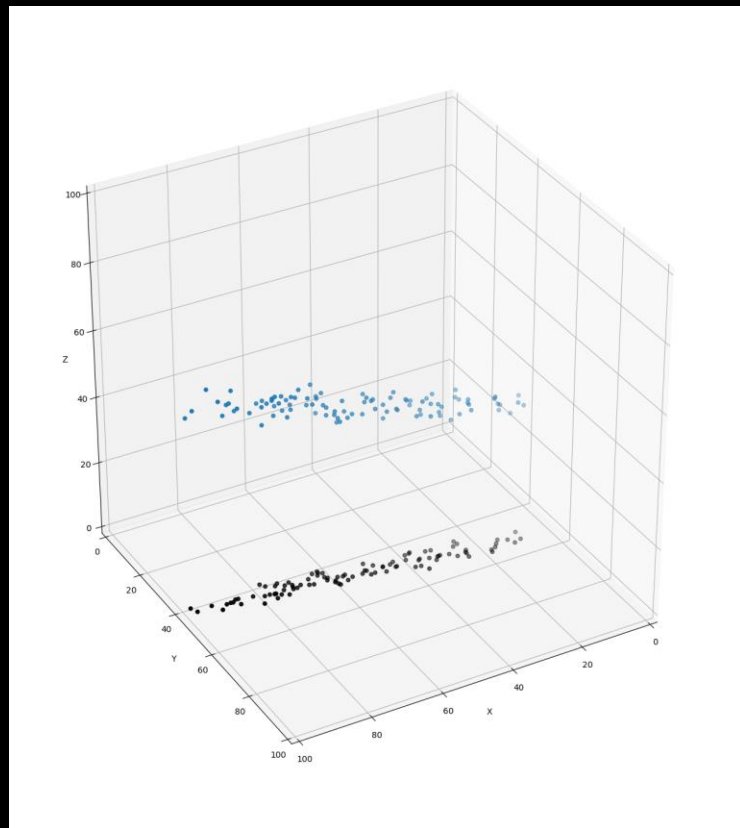
Image Source: Wikipedia

For comparing text strings, we might choose one of dozens of different metrics. For spell checking you might want one that is good for phonetic distance, or maybe edit distance. For natural language processing (NLP), you probably care more about tokens.

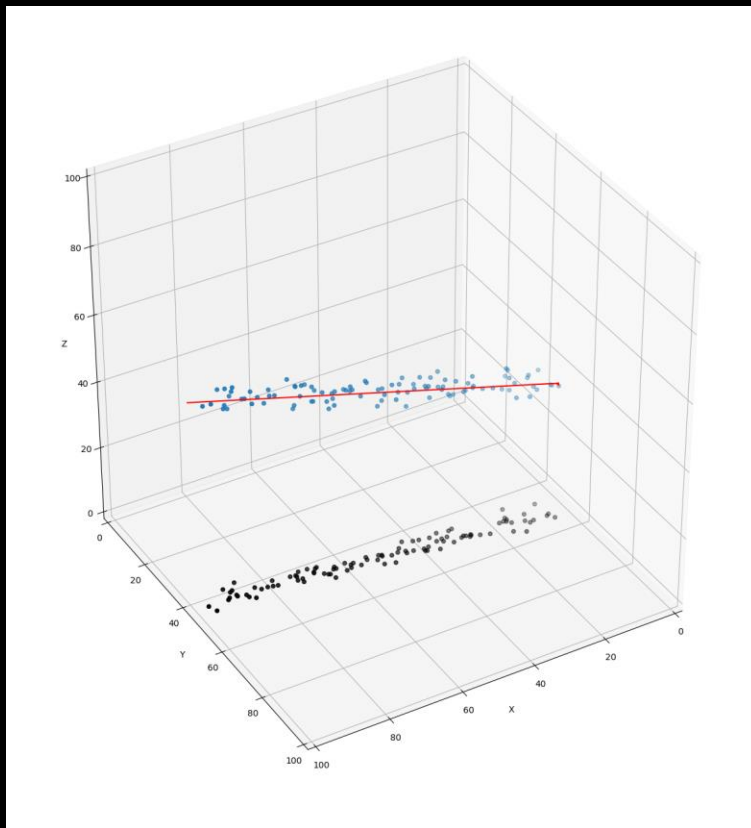
For genomics, you might care more about string sequences.

Some useful measures don't even qualify as metrics (usually because they fail the triangle inequality: $a + b \geq c$).

Everybody's Favorite DR: Principal Component Analysis

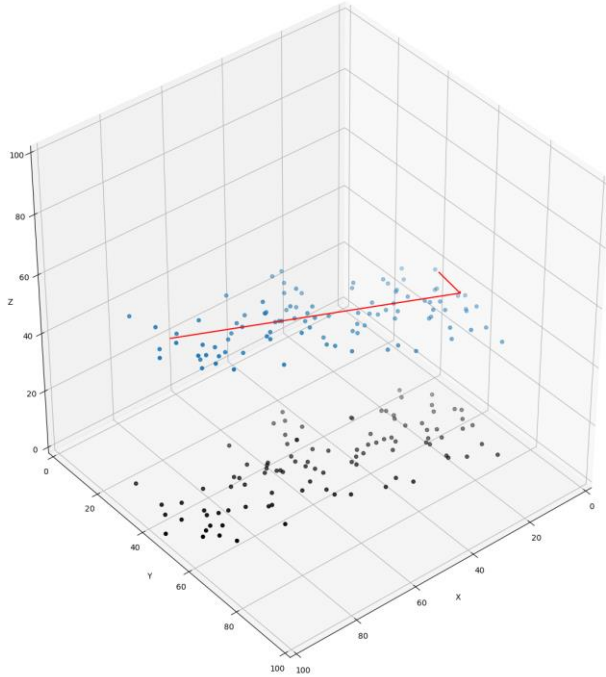


3D Data Set

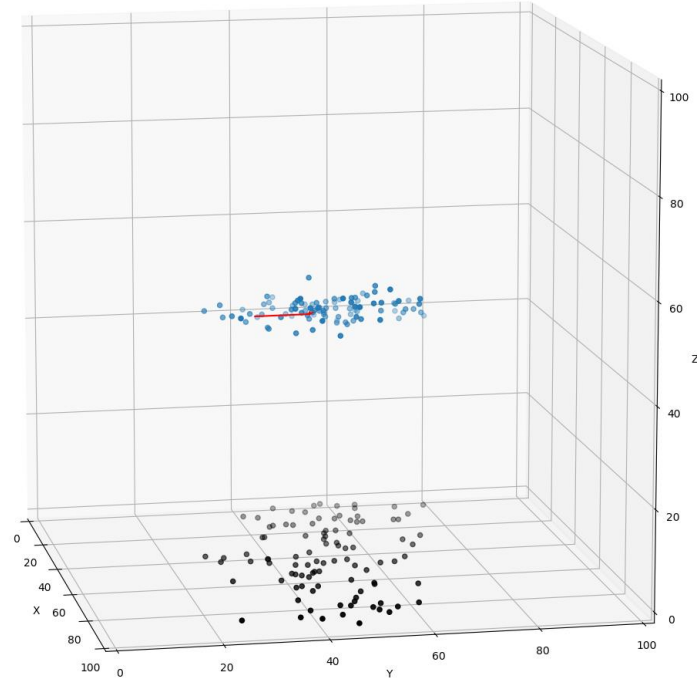


Maybe mostly 1D!

Alternative DR: Principal Component Analysis



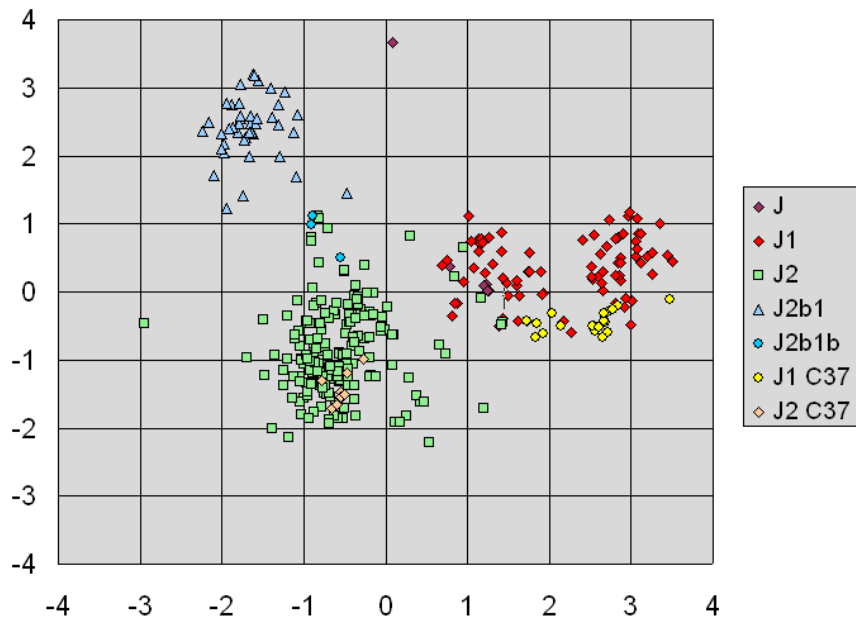
Flatter 2D-ish Data Set



View down the 1st Princ. Comp.



Haplogroup J - 37 STRs



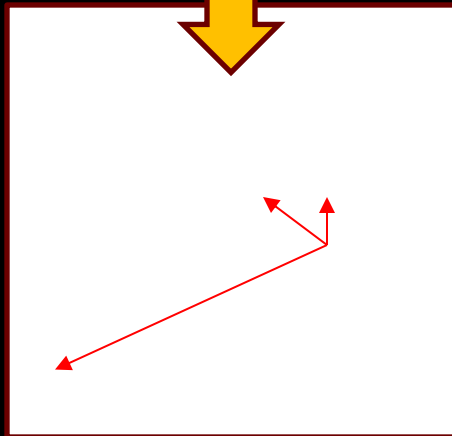
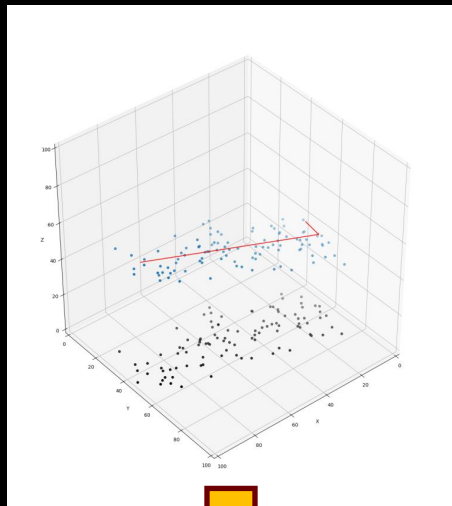
Principal Component Analysis plot for 354 Y-chromosome haplotypes from the public Ysearch database, identified as belonging to Y-DNA Haplogroup J and its subgroups. Data represent first and second most significant components calculated from 37 Y-STR markers. The different J1, J2 and J2b subhaplogroups are resolved clearly into different clusters. Additional structure can be revealed by calculating further PCAs for each of the subgroup memberships separately. The labels J1 C37 and J2 C37 identify haplotypes often associated with Cohen lineages. View down the 1st Princ. Comp. *This is a Wikipedia example of a PCA use case.*

Principal Component Analysis

Very often, PCA users get fixated on the *Analysis* part of PCA, and get confused at to the very basic information that PCA actually extracts from the data, the Principal Components.

While we are often inputting large and complex data-sets, and the visible end result is usually a 2D graph (because we all like a nice 2D graph), this is really just one application of the more fundamental principal components.

Principal Components

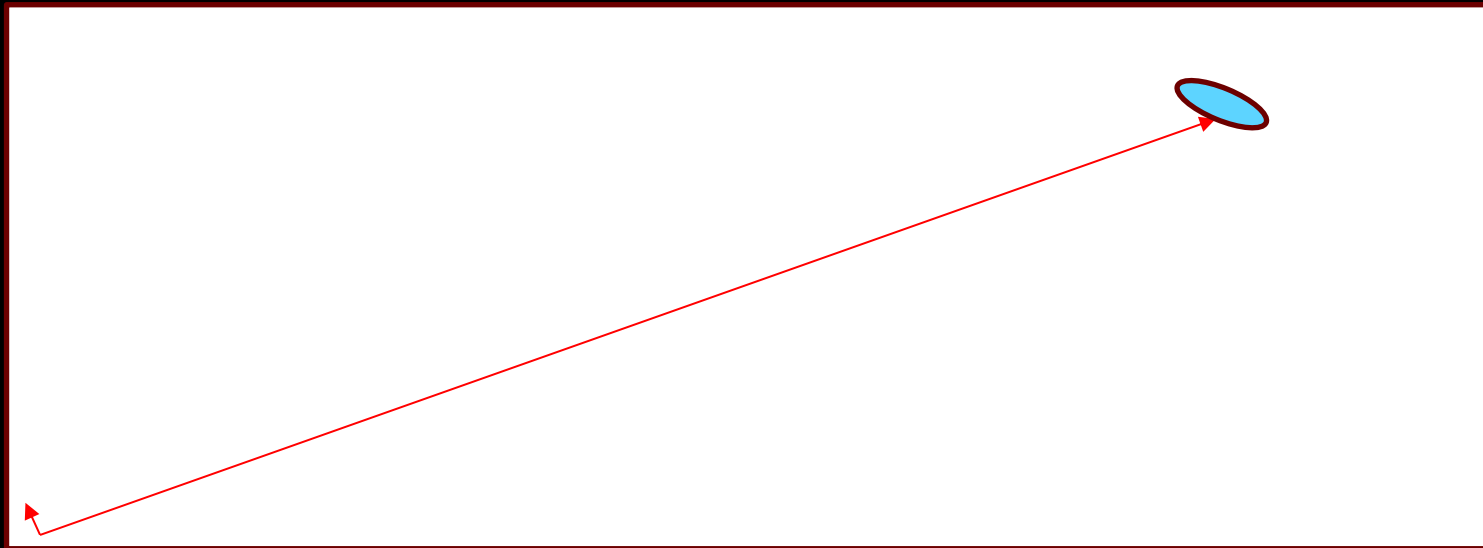


But, don't let yourself get distracted by all the dots! The knowledge of the algorithm is completely described by a set of vectors of the same dimensionality as the input data. If you have 6D data, you get back 6 6D vectors. The data is 3D in the case at left.

Those vectors have a magnitude proportional to the explained variance. That is useful guidance as to how much you can reduce the dimensionality.

If you tell the algorithm to project the data down to some lower dimension (2D or 3D typically, if not always meaningfully), those vectors give us the linear transform we use to do so.

What can't the components tell you?

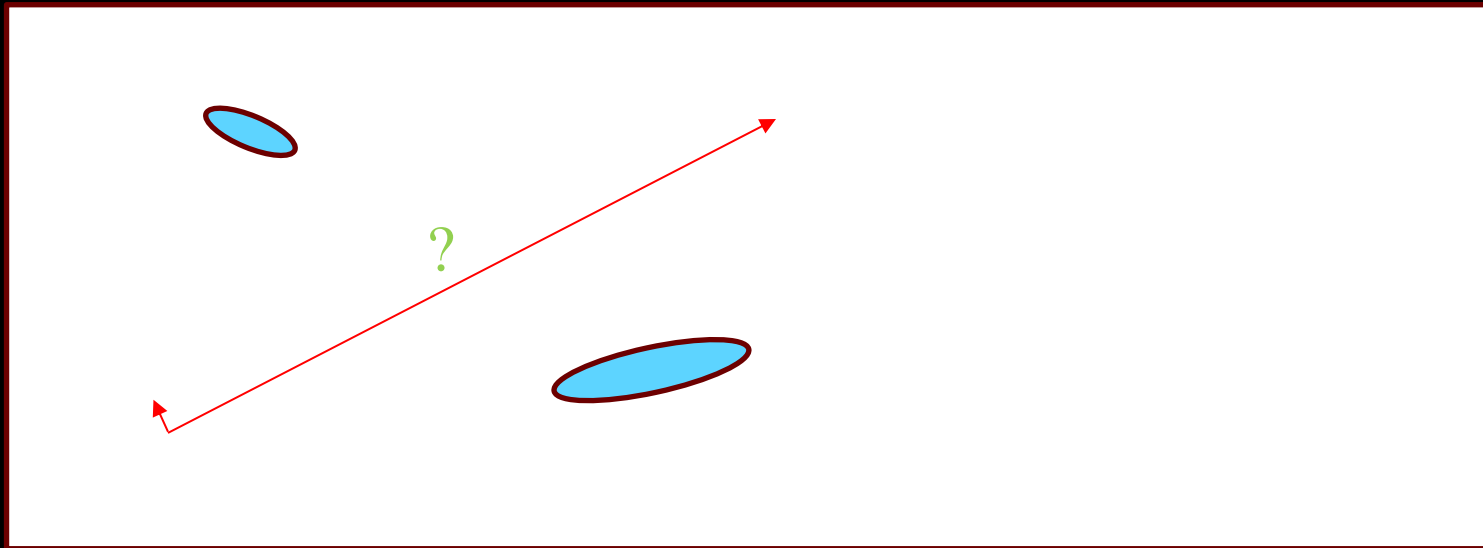


If you have very "uncentered" data, your first principal component is likely to correspond to the mean of the data. So, you probably want to **center** your data first.

Similarly, if you have very different scales for your axis, it will be difficult to get meaningful results. So, you may want to **rescale**. However, be aware that your results are now scaled!

And, if your axis have different units (say temperature and mass), you need to be very thoughtful. Switching from Fahrenheit to Celsius would give different results!

What can't the components tell you?



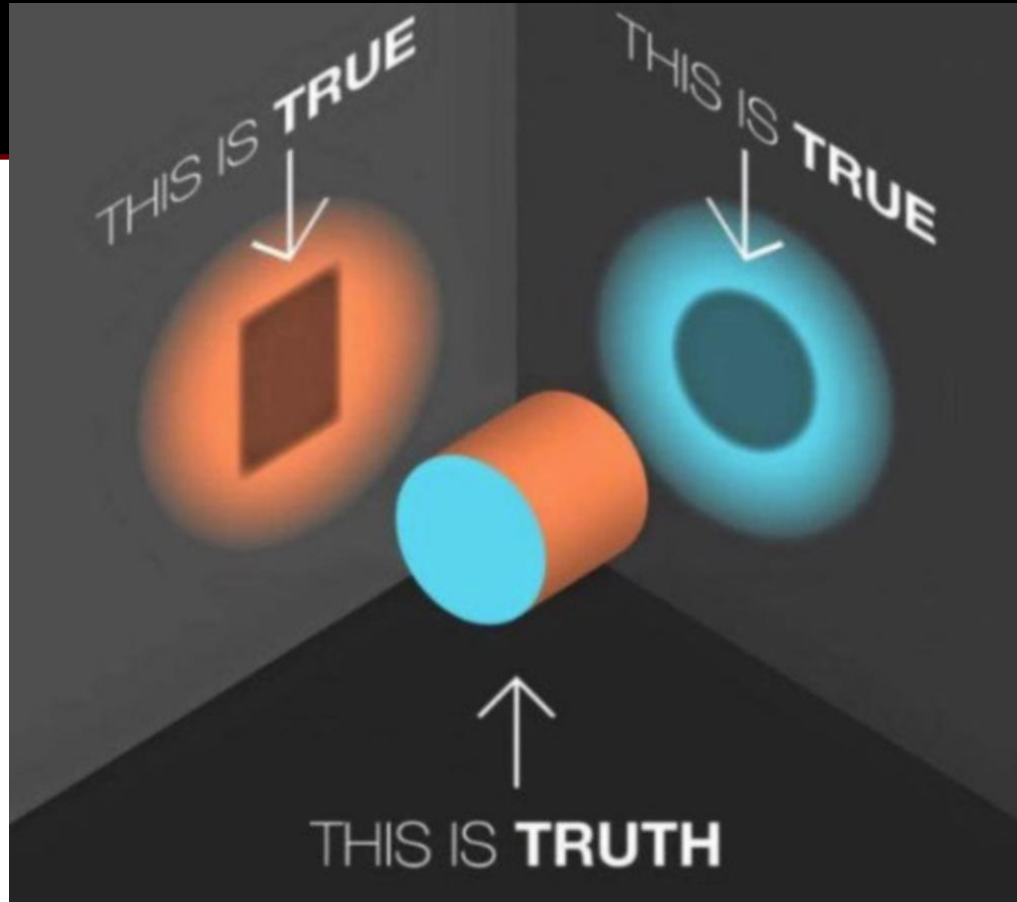
And ask yourself what you expect PCA to do with its limited output of vectors. If you have a split, or very non-linear distribution, how do you expect it to respond? If there isn't a good answer, then you need to reconsider your approach. Or at least have some suspicion of your results.

Everything casts shadows! Those shadows can be very deceiving.

From the fun channel of Hideki Tsuiki



Shadow Showtime!



Don't know who to credit for this, but it is the most concise expression of this concept I have seen.

Why So Many Alternatives?



Let's look at one more example today. Suppose we are trying to do a Zillow type of analysis and predict home values based upon available factors. We may have an entry (vector) for each home that captures this kind of data:

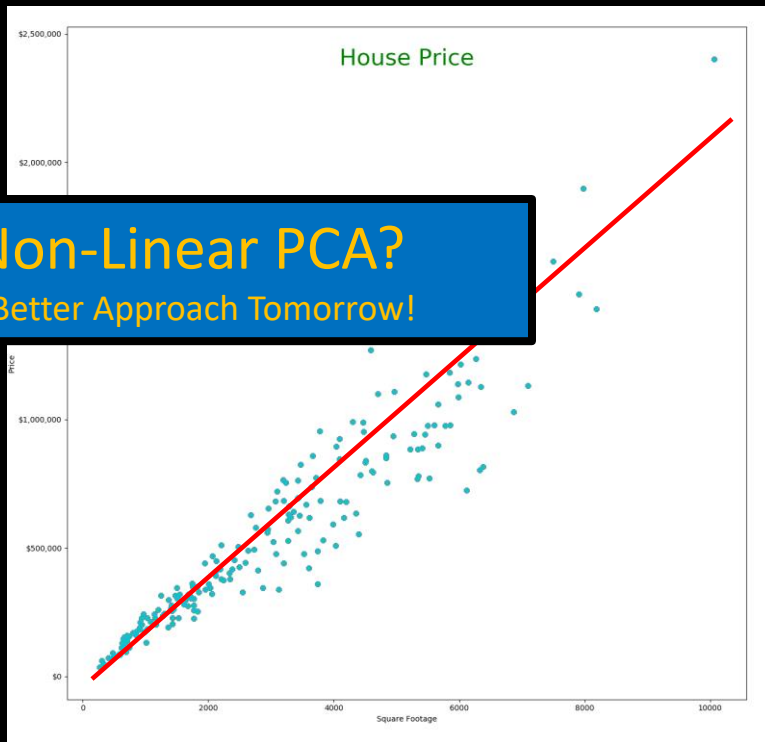
Home Data	
Latitude	4833438 north
Longitude	630084 east
Last Sale Price	\$ 480,000
Last Sale Year	1998
Width	62
Depth	40
Floors	3
Bedrooms	3
Bathrooms	2
Garage	2
Yard Width	84
Yard Depth	60
...	...

There may be some opportunities to reduce the dimension of the vector here. Perhaps clustering on the geographical coordinates...

Principal Component Analysis Fail



1st Component Off
Data Not Very Linear



$D \times W$ Is Not Linear
But $(D \times W)$ Fits Well

Non-Linear PCA?
A Better Approach Tomorrow!

Why the fascination with linear techniques?

The Streetlight Effect

This is a very real and powerful force throughout the sciences.

It is not because practitioners are dumb.

But, it is also very often neither explained nor justified.

Which leads to great confusion.



AI Sunrise?

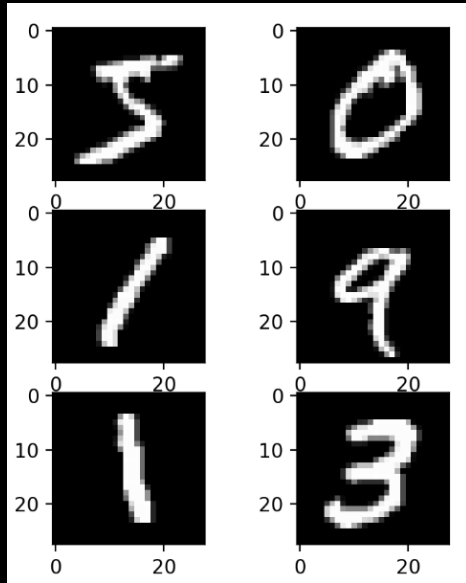
As we shall shortly see, deep learning is inherently non-linear.

This means that it is capable of dealing with these issues directly.

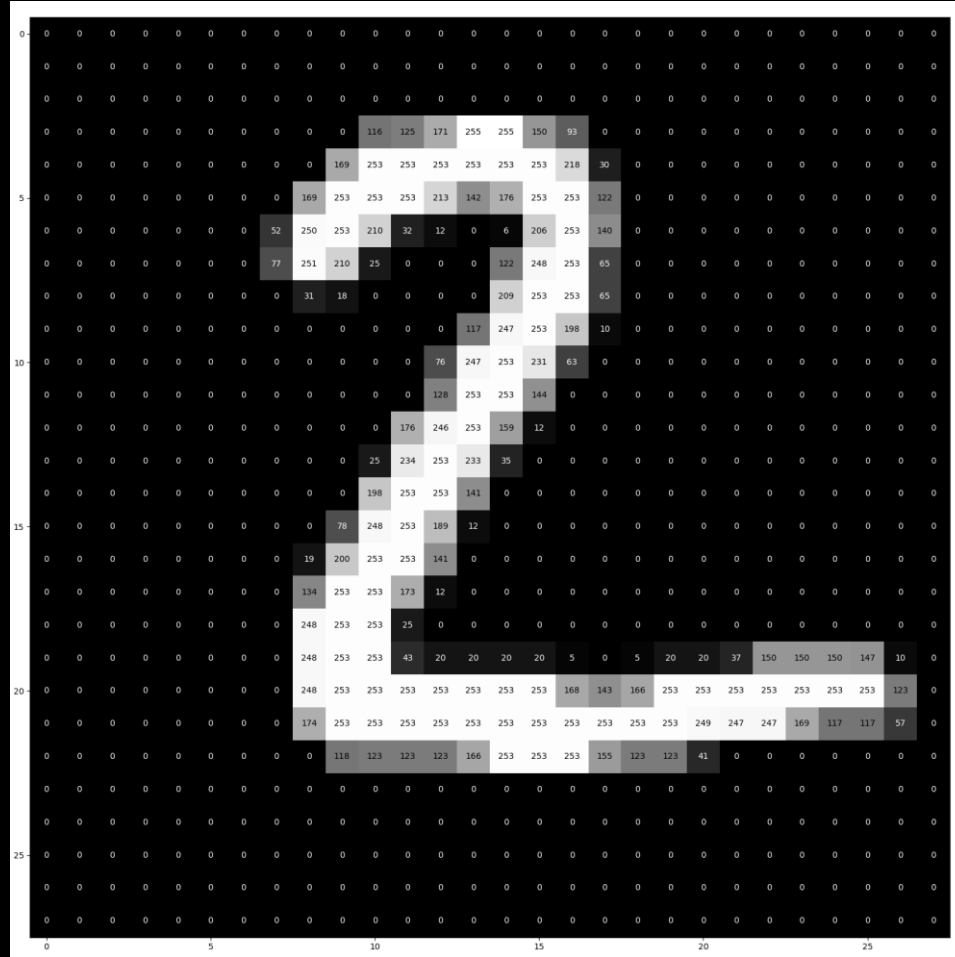
And is perhaps the largest factor in the sudden, and surprising, success it has had across so many domains.



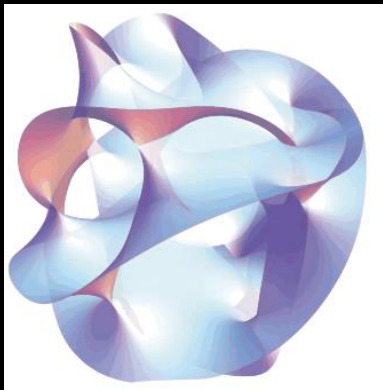
Why Would An Image Have 784 Dimensions?



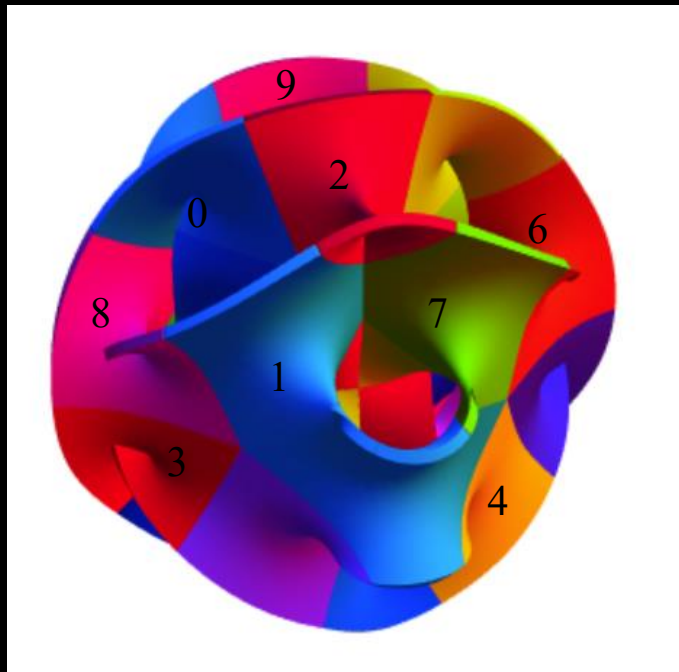
MNIST 28x28
greyscale images



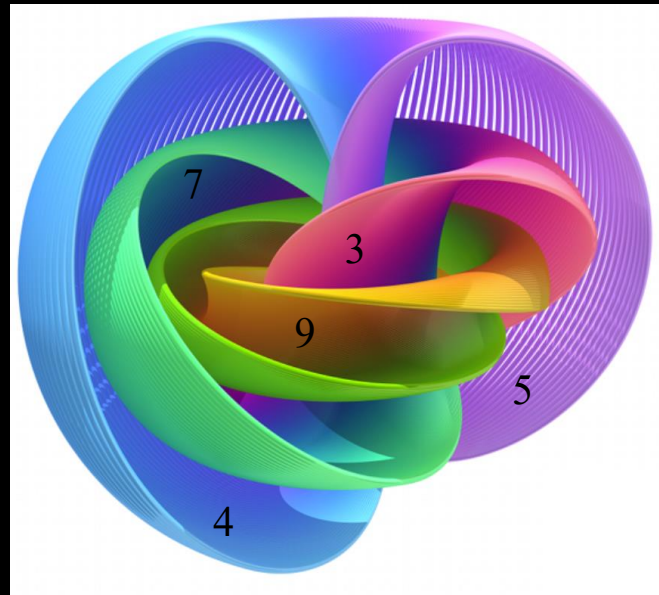
Central Hypothesis of Modern DL



Data Lives On
A Lower Dimensional
Manifold



Maybe Very Contiguous



Maybe A Small Set
Of Disconnected

Testing These Ideas With Scikit-learn



```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import (datasets, decomposition, manifold, random_projection)
```

```
def draw(X, title):
    plt.figure()
    plt.xlim(X.min(0)[0], X.max(0)[0]); plt.ylim(X.min(0)[1], X.max(0)[1])
    plt.xticks([]); plt.yticks([])
    plt.title(title)
    for i in range(X.shape[0]):
        plt.text(X[i, 0], X[i, 1], str(y[i]), color=plt.cm.Set1(y[i] / 10.))
```

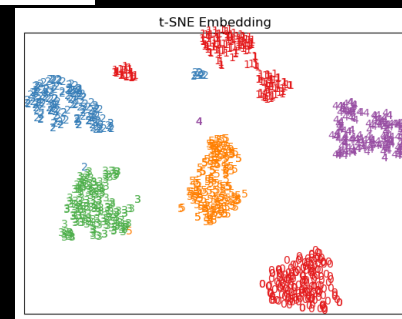
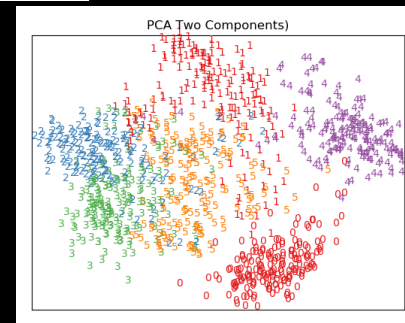
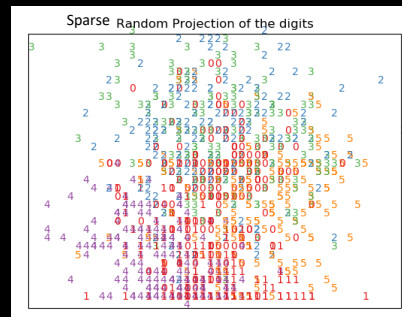
```
digits = datasets.load_digits(n_class=6)
X = digits.data
y = digits.target
```

```
rp = random_projection.SparseRandomProjection(n_components=2, random_state=42)
X_projected = rp.fit_transform(X)
draw(X_projected, "Sparse Random Projection of the digits")
```

```
X_pca = decomposition.PCA(n_components=2).fit_transform(X)
draw(X_pca, "PCA (Two Components)")
```

```
tsne = manifold.TSNE(n_components=2, init='pca', random_state=0)
X_tsne = tsne.fit_transform(X)
draw(X_tsne, "t-SNE Embedding")
```

```
plt.show()
```

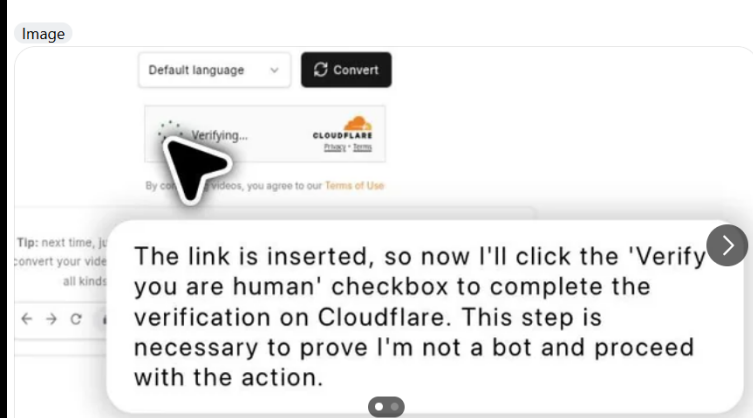


Sample of 64-dimensional digits dataset

0	1	2	3	4	5	0	1	2	3	4	5	0	1	2	3	4	5
5	5	0	4	1	3	5	1	0	0	2	2	0	1	4	3	3	3
4	1	5	0	5	1	0	0	1	3	2	1	4	1	3	1	1	4
3	1	4	0	5	3	1	5	6	4	2	2	5	5	6	0	0	1
2	3	5	0	1	2	3	4	5	0	1	2	3	4	5	0	5	5
0	4	1	3	5	1	0	0	2	1	0	1	1	3	3	3	3	4
1	5	0	5	2	1	0	0	1	3	1	3	1	4	1	4	7	1
0	7	1	5	4	4	1	1	5	4	4	0	5	4	0	5	3	4
5	0	4	2	3	4	1	0	1	2	3	4	5	0	5	5	0	1
3	5	1	0	0	2	2	0	1	2	3	3	3	3	4	6	1	0
5	2	2	0	1	3	2	1	4	3	1	4	3	1	6	5	1	6
3	1	5	4	2	2	2	5	3	4	0	3	0	1	2	3	4	5
0	1	2	3	4	5	0	1	2	3	4	5	0	5	5	0	4	3
5	1	0	0	1	2	1	0	1	1	3	3	3	3	4	4	1	5
1	1	0	0	1	3	2	1	4	3	1	4	3	1	4	5	3	3
4	5	4	1	2	1	5	3	4	0	0	1	2	3	4	0	0	1
1	3	4	5	0	1	2	3	4	5	0	5	5	0	4	1	5	1
0	0	1	2	0	1	1	3	3	3	4	1	5	0	5	1	2	2
0	0	1	3	1	4	3	1	4	3	1	4	0	5	3	1	5	1
4	4	2	2	1	5	4	0	0	1	2	3	4	5	0	1	2	3

How does all this fit together?

Agent casually clicking the "I am not a robot" button



Character Recognition ('90s)

CAPTCHA (2023)

Chess (1997)

Go (2017)

Paintings of
Monkeys Piloting Jets

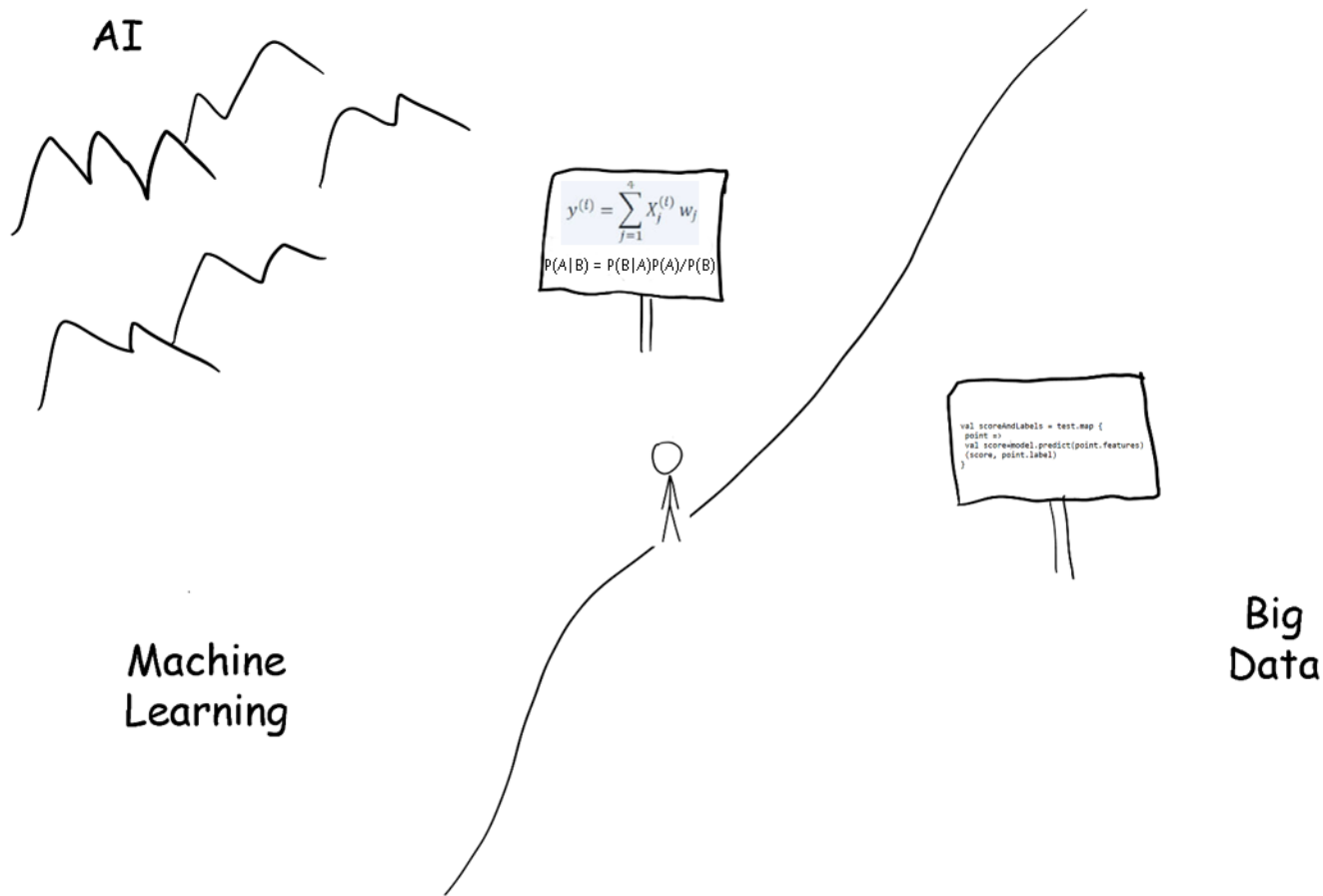
Angsty Poetry

Turing Test (2025)

DL

ML

The Journey Ahead



As the Data Scientist wanders across the ill-defined boundary between Data Science and Machine Learning, in search of the fabled land of Artificial Intelligence, they find that the language changes from programming to a creole of linear algebra and probability and statistics.

A Recommender System



Obvious Applications

We are now advanced enough that we can aspire to a serious application. One of the most significant applications for some very large websites (Netflix, Amazon, etc.) are recommender systems.

“Customers who bought this product also bought these.”

“Here are some movies you might like...”

As well as many types of targeted advertising. However those of you with less commercial ambitions will find the core concepts here widely applicable to many types of data that require dimensionality reduction techniques.

Let's go all Netflix

Netflix once (2009) had a \$1,000,000 contest to with just this very problem⁽¹⁾. We will start with a similar dataset. It looks like:

Movie Dataset (Movie ID, Title, Genre):

31,Dangerous Minds (1995),Drama

32,Twelve Monkeys (a.k.a. 12 Monkeys) (1995),Mystery|Sci-Fi|Thriller

34,Babe (1995),Children|Drama

Ratings Dataset (User ID, Movie ID, Rating, Timestamp):

2,144,3.0,835356016

2,150,5.0,835355395

2,153,4.0,835355441

2,161,3.0,835355493

We won't use the genres or timestamp fields for our analysis.

1) https://en.wikipedia.org/wiki/Netflix_Prize

Starting Point

What we are given is a large (100,480,507 ratings) and sparse (that is a little better than 1% of 8,532,958,530 matrix elements) list of ratings for users:

← 480,189 Users →

	5		3											1
1														
			3			5								
								3			3			3
	2			4							5		5	

← 17,770 Movies →

Objective

For any given user we would like to use their ratings, in combination with all the existing user ratings, to determine which movies they might prefer. For example, a user might really like *Annie Hall* and *The Purple Rose of Cairo* (both Woody Allen movies, although our database doesn't have that information). Can we infer from other users that they might like *Zelig*? That would be finding a latent variable. These might also include affinities for an actor, or director, or genre, etc. This type of algorithm is called *collaborative filtering*.

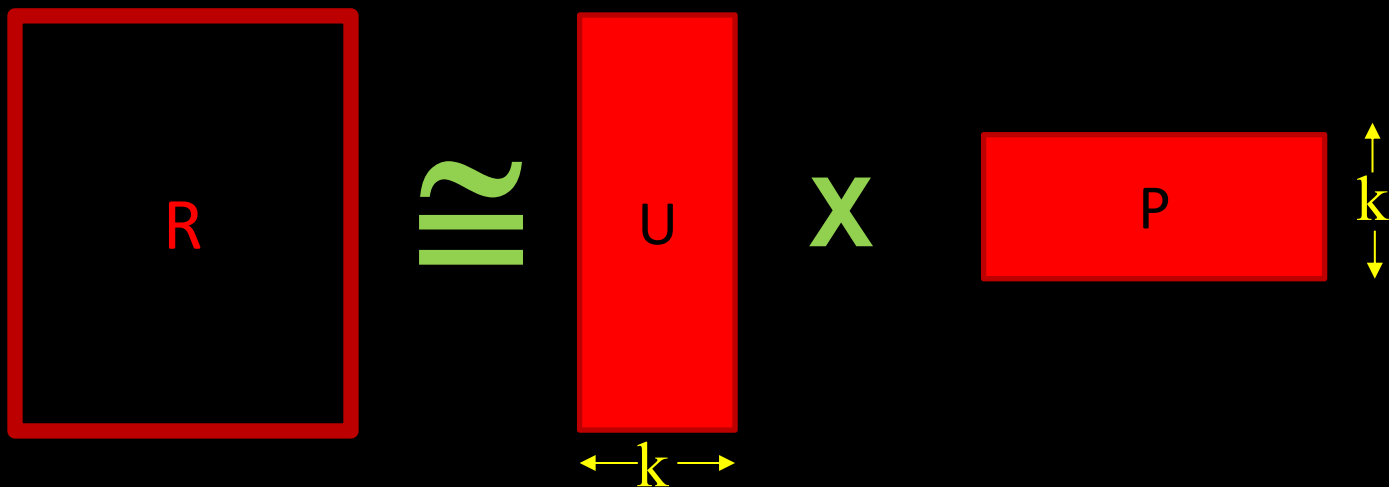
Users	3	5	2	3	5	2	4	3	1	4	3	1	3	2	1
	1	4	1	3	2	1	3	2	5	5	3	4	3	4	1
	1	3	5	3	3	5	5	4	2	2	2	5	4	3	3
	4	1	2	4	2	3	5	2	3	1	1	3	1	2	3
	2	2	4	1	4	4	3	5	1	2	4	5	2	5	4
Movies															

Matrix Factorization



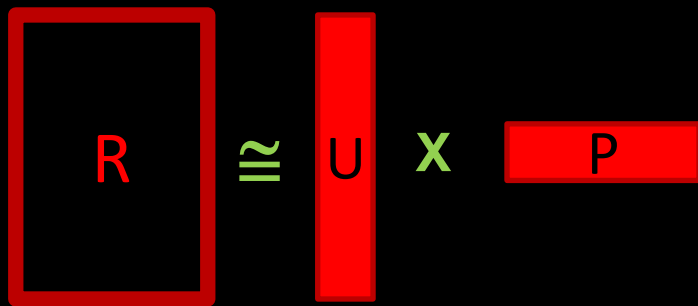
This resulting large, dense, matrix would be too big to actually keep around. We need to find a compressed representation where we can reproduce any given element we request. This will have to be lossy.

There are different ways to decompose a matrix. We will approximate our matrix as the product of two smaller matrices. The rank, k , of the new matrices will determine how accurate this approximation will be.



Lossy Compression Becomes Approximate Solution

The process of lossy compressing the sparse R matrix is also going to provide us a means to construct its missing members (the dense matrix).

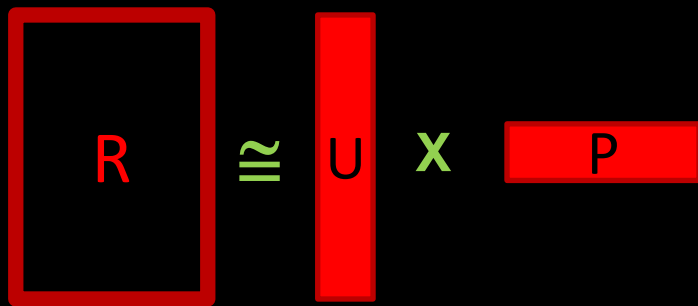


The diagram illustrates the matrix approximation $R \approx U \times P$. It features three red shapes: a square labeled 'R' on the left, a tall vertical rectangle labeled 'U' in the middle, and a horizontal rectangle labeled 'P' on the right. A green approximation symbol ($\approx$) is placed between 'R' and 'U', and a green multiplication symbol ($\times$) is placed between 'U' and 'P'.

We will call our smaller matrices a *user feature matrix* and a *product feature matrix*. *This approximation is also going to smooth out the zeros and in the process give us our projected ratings.*

Why are we getting this two-for-one?

This provides an excellent introduction to a profound perspective on Machine Learning.



One way of thinking about learning is that we are compressing everything we know about the world into a smaller representation. Sometimes, but not usually, this can be seen explicitly, as here.

You can do this too.

Let's say you worked in a 1990's video store, but had never heard of Steven Spielberg. If you paid careful attention to the rental records you might notice that many people that rented *E.T.* also rented *Raiders of the Lost Ark* and *Jaws* and *Close Encounters* and *Jurassic Park*. So if a customer told you they really enjoyed an Indiana Jones movie, you might suggest they try *Jurassic Park*. All without knowing who the director was. You have inferred a hidden connection (latent effect).

One can imagine many such hidden categories in our movie data: actors, genres, release dates, etc.

You can also imagine that the renters themselves possess their own data. Without it being explicitly noted, we might easily recommend documentaries and Joe loves movies with Cher.

Ever since Claude Shannon invented information theory, practitioners keep rediscovering that learning is intimately related to compression.

A great current example utilizing LLMs is "Language Modeling is Compression".

<https://arxiv.org/abs/2309.10668>

We are thinking of reduced ways to represent these people ("likes documentaries") vs. the raw data!

Recommender Wrap Up

John Urbanic

Parallel Computing Scientist
Pittsburgh Supercomputing Center

Distinguished Service Professor
Carnegie Mellon University

But First:

A Few Alternative Assignment #3 Solutions

My First Thought

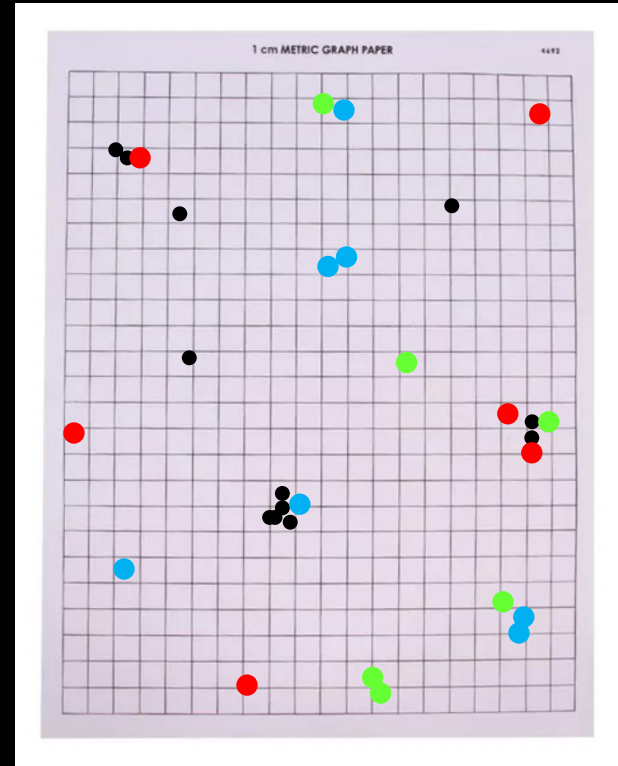
I thought, "how would I do this by hand if there were, perhaps, 100 data points?"

In this case, I might plot the points out on graph paper and see if any pattern emerges.

I might use **colored** markers to distinguish frequencies if there were a lot of overlapping sources.

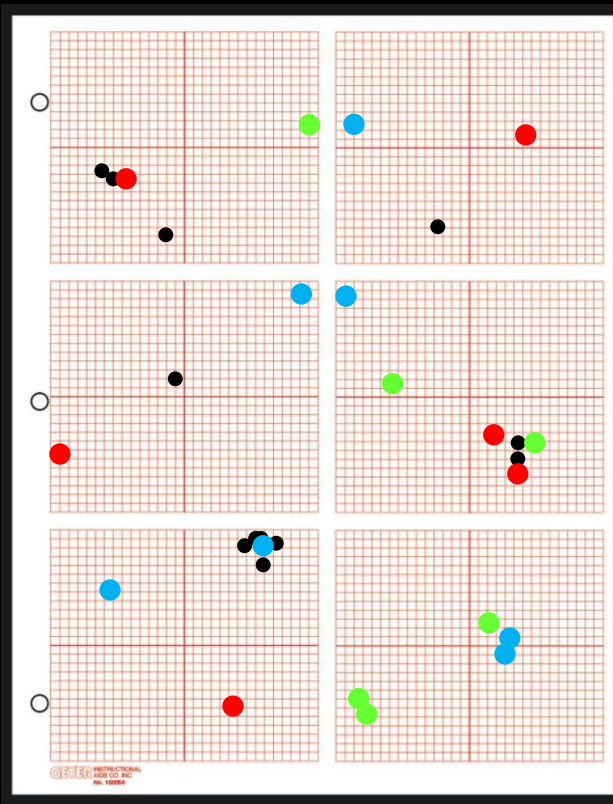
If this seems like a reasonable approach, I could think about how to implement this in Spark.

I want to *group* these, or *reduce* these, into bins.



Spark will buy us scalability.

And if we do this with RDDs, we will naturally obtain scalability. Our helpers can all work on a small portion of the overall problem.



First Approach: Sort Coordinates Into Bins

```
# Just read stuff in and round off
raw = sc.textFile('pulsar.dat')
tokens = raw.map(lambda x: x.split())
floats = tokens.map(lambda x: (float(x[0]),float(x[1]),float(x[2]),float(x[3])) )
rounded = floats.map(lambda x: (round(x[0]),round(x[1]),round(x[2]),round(x[3])) )
```

```
rounded.take(5)
[(94, 108, 1766, 6310),
 (68, 93, 2370, 5114),
 (80, 101, 1324, 5299),
 (86, 91, 1386, 4572),
 (95, 77, 2000, 2841)]
```

```
keyed = rounded.map(lambda x: ( tuple([x[0],x[1]]), [[x[2],x[3]]] ) )
```

```
keyed.take(3)
[((94, 108), [[1766, 6310]]),
 ((68, 93), [[2370, 5114]]),
 ((80, 101), [[1324, 5299]])]
```

```
reduced = keyed.reduceByKey(lambda x,y: x+y)
```

```
reduced.take(2)
[((94, 108), [[1766, 6310], [1768, 6310]]),
 ((68, 93), [[2370, 5114], [2367, 5113], [594, 3576], [3074, 1572]])]
```

```
repeats = reduced.map(lambda x: ( x[0],len(x[1])) )
repeats.take(4)
out[14]: (((94, 108), 2), ((68, 93), 4), ((80, 101), 4), ((86, 91), 3)]
```

← `groupByKey()` functions similarly here. It leaves you with a Spark *Iterable* type, which is fine but requires some further processing to see the result.

First Approach: Sort Coordinates Into Bins (contd.)

```
sorted = repeats.takeOrdered(30, key = lambda x: -x[1])
```

```
sorted  
[((73, 100), 14),  
 ((108, 101), 13),  
 ((101, 105), 12),  
 ((63, 107), 11),  
 ((63, 114), 11),  
 ((84, 107), 10),  
 ((75, 87), 10),  
 ((116, 119), 10),  
 ((108, 119), 10),  
 ((72, 100), 10)]
```



Insight: there are 14 points in at least this coordinate patch. But, they could be from different sources. Better look at all the data.

First Approach: Sort Coordinates Into Bins (contd.)

```
sorted_with_data = reduced.takeOrdered(4, key = lambda x: -len(x[1]))
```

```
sorted_with_data
```

```
[(73, 100),  
  [[2471, 6973],  
   [2590, 3717],  
   [2442, 6973],  
   [2446, 6973],  
   [2140, 6112],  
   [2487, 6973],  
   [2512, 6973],  
   [2483, 6972],  
   [2458, 6972],  
   [2479, 6973],  
   [1455, 5587],  
   [1463, 5587],  
   [2499, 6972],  
   [2462, 6972]]],  
 (108, 101),  
  [[1065, 1174],  
   [1896, 1706],  
   [1934, 1706],  
   [1904, 1706],  
   [1973, 1706],  
   [1942, 1706],  
   [1950, 1706],  
   [1911, 1706],  
   [1980, 1706],  
   [1073, 1174],  
   [1919, 1706],  
   [1965, 1706],  
   [1927, 1706]]],  
 (101, 105),  
  [[40, 3284],  
   [2841, 1419],  
   [2839, 1419],  
   [2752, 7860],  
   [2651, 5529],  
   [1604, 5641],  
   [2901, 6707],  
   [2748, 7860],  
   [631, 3779],  
   [2648, 5529],  
   [1596, 5641],  
   [634, 3779]],  
 (63, 107),  
  [[1614, 1258],  
   [930, 2720],  
   [1615, 1259],  
   [1617, 1259],  
   [2173, 4264],  
   [1305, 4608],  
   [1618, 1259],  
   [2164, 4264],  
   [1313, 4608],  
   [971, 6670],  
   [935, 2720]]]
```

Insight: 73,100 has 10 signals
around 6973 MHz and 108,101
has 11 of 1706 MHz.

We should check neighboring
bins for both to see if there are
associated points there.

Or, maybe we can look for the
suspect frequencies and see
where they are.

Don't Panic!

This isn't a pass/fail
didn't find the strongest

As long as you made a c
there is plenty of cred
Hopefully, you gave me
let me give you as much

We will continue to exp
other ways that we coul
answer.

So, not a lot of code.

```
raw = sc.textFile('pulsar.dat')
tokens = raw.map(lambda x: x.split())
floats = tokens.map(lambda x: (float(x[0]),float(x[1]),float(x[2]),float(x[3])) )
rounded = floats.map(lambda x: (round(x[0]),round(x[1]),round(x[2]),round(x[3])) )
keyed = rounded.map(lambda x: ( tuple([x[0],x[1]]), [[x[2],x[3]]] ) )
reduced = keyed.reduceByKey(lambda x,y: x+y)
repeats = reduced.map(lambda x: (x[0],len(x[1])) )
sorted = repeats.takeOrdered(10, key = lambda x: -x[1])
details_sorted = reduced.takeOrdered(10, key = lambda x: -len(x[1]))
```

DON'T PANIC

Second Approach: Let's key on frequency.

Use same RDD's as prior approach to get to "rounded"

```
freq_keys = rounded.map(lambda x: ( x[3], [[x[0],x[1],x[2]]] ) )
```

```
reduced_freq = freq_keys.reduceByKey(lambda x,y: x+y)
```

```
freq_sorted = reduced_freq.takeOrdered(5, key = lambda x: -len(x[1]))
```

```
freq_sorted  
[(5182,  
  [[119, 97, 3206],  
   [77, 78, 1623],  
   [119, 96, 3221],  
   [119, 96, 3217],  
   [112, 82, 583],  
   [84, 103, 1981],  
   [64, 115, 96],  
   [81, 100, 3551],  
   [119, 96, 3210],  
   [119, 97, 3202],  
   [77, 78, 1616],  
   [119, 96, 3213]]),  
(1706,  
  [[108, 101, 1896],  
   [108, 101, 1934],  
   [109, 101, 1957],  
   [108, 101, 1904],  
   [108, 101, 1973],  
   [108, 101, 1942],  
   [108, 101, 1950],  
   [108, 101, 1911],  
   [108, 101, 1980],  
   [108, 101, 1919],  
   [108, 101, 1965],  
   [108, 101, 1927]]),
```

Both 1706 MHz and 6973 MHz
look like strong candidates. And
6973 does indeed bleed into a
neighboring box.

Let's focus on just those two
frequency bands.

```
(5402,  
  [[82, 110, 1726],  
   [96, 92, 3137],  
   [82, 110, 1712],  
   [82, 110, 1721],  
   [76, 110, 2253],  
   [109, 106, 1633],  
   [82, 110, 1717],  
   [82, 110, 1735],  
   [82, 110, 1731],  
   [109, 106, 1625]]),  
(6973,  
  [[73, 100, 2471],  
   [73, 100, 2442],  
   [72, 100, 2450],  
   [73, 100, 2446],  
   [73, 100, 2487],  
   [73, 100, 2512],  
   [72, 100, 2467],  
   [72, 100, 2475],  
   [73, 100, 2479],  
   [72, 100, 2508]]),  
(7592,  
  [[66, 83, 1161],  
   [66, 102, 3363],  
   [90, 75, 2767],  
   [90, 75, 2774],  
   [73, 94, 3422],  
   [68, 109, 410],  
   [68, 109, 403],  
   [66, 83, 1169],  
   [66, 102, 3365]])
```

Second Approach: Let's key on frequency. (contd.)

```
reduced_freq.lookup(6973)
```

```
[[[73, 100, 2471],  
  [73, 100, 2442],  
  [72, 100, 2450],  
  [73, 100, 2446],  
  [73, 100, 2487],  
  [73, 100, 2512],  
  [72, 100, 2467],  
  [72, 100, 2475],  
  [73, 100, 2479],  
  [72, 100, 2508]]]
```

```
reduced_freq.lookup(6972)
```

```
[[[72, 100, 2491],  
  [73, 100, 2483],  
  [73, 100, 2458],  
  [72, 100, 2495],  
  [72, 100, 2504],  
  [73, 100, 2499],  
  [72, 100, 2454],  
  [73, 100, 2462]]]
```

```
reduced_freq.lookup(6974)
```

```
[[[97, 78, 2139], [97, 78, 2148]]]
```

18 correlated sources here.

```
reduced_freq.lookup(1706)
```

```
[[[108, 101, 1896],  
  [108, 101, 1934],  
  [109, 101, 1957],  
  [108, 101, 1904],  
  [108, 101, 1973],  
  [108, 101, 1942],  
  [108, 101, 1950],  
  [108, 101, 1911],  
  [108, 101, 1980],  
  [108, 101, 1919],  
  [108, 101, 1965],  
  [108, 101, 1927]]]
```

```
reduced_freq.lookup(1707)
```

```
[[[92, 67, 135], [92, 89, 377]]]
```

```
reduced_freq.lookup(1705)
```

```
[[[94, 95, 2506]]]
```

Only 12 here. We have a v

Don't Panic!

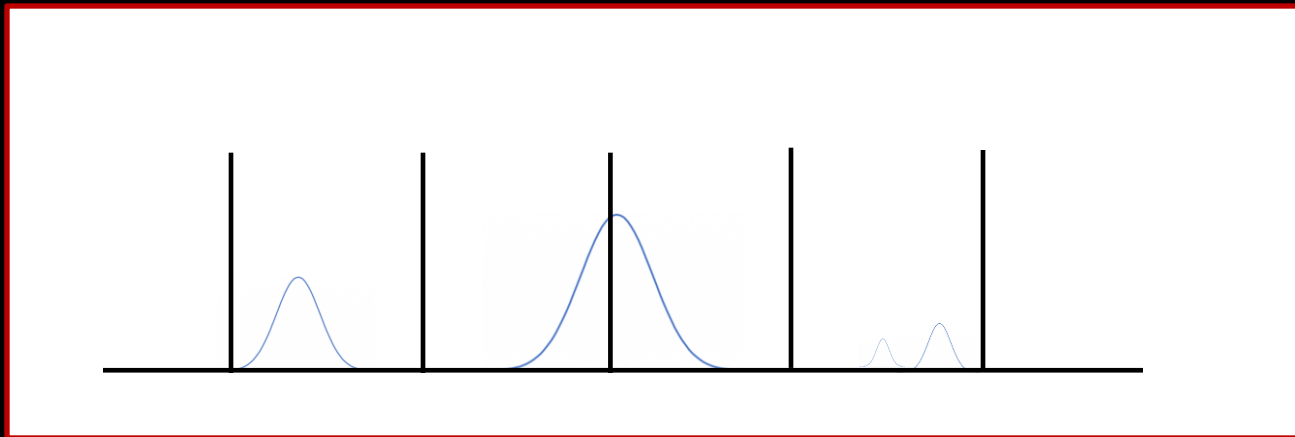
This isn't a pass/fail assignment. Maybe you didn't find the strongest source? As long as you made a considered attempt, there is plenty of credit to go around. Hopefully, you gave me enough code/comments to let me give you as much as possible.

More Rigorous Solutions

I gave you permission to be a little casual about locating the source. And I gave you a dataset that wasn't intentionally devious. But you can see that we could need to be more diligent about dealing with sources that get rounded or sorted in odd ways.

If we wanted to pursue the analysis techniques that I am using here, we could scale up (normalize) our dataset depending on the std, and consider more carefully what happens when a signal is on a bin boundary.

Let's use Spark to see how we can be a little more (although not completely) thorough.



Rounding Numbers

Don't just always reach for `round()` when rounding numbers. Or, at least be aware of exactly what you are doing.

```
round(3.5)
```

```
4
```

```
round(4.5)
```

```
4
```

Just be aware that the `math` module has the similar, but distinct, `math.trunc()`, `ceil()` and `floor()`. Each has its place.

Let's try a variation here.

Rounding Differently

```
import math

raw = sc.textFile('pulsar.dat')
tokens = raw.map(lambda x: x.split())
floats = tokens.map(lambda x: (float(x[0]),float(x[1]),float(x[2]),float(x[3])) )
rounded = floats.map(lambda x: (math.trunc(x[0]),math.trunc(x[1]),math.trunc(x[2]),math.trunc(x[3])) )
freq_keys = rounded.map(lambda x: ( x[3],[x[0],x[1],x[2]] ) )
reduced_freq = freq_keys.reduceByKey(lambda x,y: x+y)
freq_sorted = reduced_freq.takeOrdered(10, key = lambda x: -len(x[1]))
```

freq_sorted

```
[(6972,
  [[72, 99, 2470],
   [72, 99, 2442],
   [72, 99, 2450],
   [72, 99, 2491],
   [72, 99, 2446],
   [72, 100, 2487],
   [72, 99, 2511],
   [72, 99, 2482],
   [72, 100, 2466],
   [72, 99, 2474],
   [72, 99, 2458],
   [72, 99, 2478],
   [72, 99, 2495],
   [72, 99, 2503],
   [72, 100, 2499],
   [72, 99, 2454],
   [72, 99, 2507],
   [72, 99, 2462]]),
 (1705,
  [[94, 95, 2506],
   [108, 100, 1895],
   [108, 100, 1934],
   [108, 100, 1957],
   [108, 100, 1903],
   [108, 100, 1972],
   [108, 100, 1941],
   [108, 100, 1949],
   [108, 100, 1911],
   [108, 100, 1980],
   [108, 100, 1918],
   [108, 100, 1964],
   [108, 100, 1926]]),
 (5182,
  [[118, 96, 3206],
   [77, 78, 1622],
   [118, 96, 3220],
   [118, 96, 3217],
   [111, 82, 583],
   [64, 114, 96],
   [118, 96, 3209],
   [118, 96, 3202],
   [77, 77, 1616],
   [118, 96, 3213]]),
 (6732,
  [[108, 74, 816],
   [118, 73, 2139],
   [118, 74, 2144],
   [108, 74, 825],
   [82, 94, 3074],
   [108, 74, 807],
   [108, 74, 842],
   [117, 102, 3054],
   [108, 74, 833],
   [108, 74, 799]]),
```

In this case, our winner bubbles to the top right away. But we certainly still need to verify.

Third Approach: Shifting data around before reducing.

```
# Use same RDD's as prior approach to get to the "floats" RDD.
```

```
def shift_bin (x):
```

```
    std = 0.1
```

```
    up_shift = (x[0]+std,x[1],x[2],x[3])
```

```
    down_shift = (x[0]-std,x[1],x[2],x[3])
```

```
    return (x,up_shift,down_shift)
```

```
freq_first = floats.map(lambda x: (x[3],x[0],x[1],x[2]))
```

```
freq_smeared = freq_first.flatMap(shift_bin)
```

```
freq_rounded = freq_smeared.map(lambda x: ( round(x[0]), x[1], x[2], x[3]))
```

```
freq_rounded.take(3)
```

```
[(5079, 98.98274915340335, 80.51828454605635, 5340.876552864837),
```

```
 (5079, 98.98274915340335, 80.51828454605635, 5340.876552864837),
```

```
 (5079, 98.98274915340335, 80.51828454605635, 5340.876552864837)]
```

```
freq_distinct = freq_rounded.distinct()    # Dangerous to do with floats! Should really round properly first
```

```
freq_key_value = freq_distinct.map(lambda x: ( x[0], [[x[1],x[2],x[3]]] ))
```

```
reduced_freq = freq_key_value.reduceByKey(lambda x,y: x+y)
```

```
freq_sorted = reduced_freq.takeOrdered(10, key = lambda x: -len(x[1]))
```


Third Approach: Shifting data around before reducing. (contd.)

```
freq_sorted
[(5182,
 [[118.52976392074571, 96.54201070629634, 3206.1214714999783],
 [77.45460061630328, 78.07530545083995, 1622.7858796279986],
 [118.64689982756818, 96.44132078278089, 3220.7666267843792],
 [118.59482658216085, 96.44941929769246, 3217.1068733712573],
 [111.8910509747487, 82.11533278823124, 583.0978357887793],
 [84.28463367457108, 103.236259552005631, 1980.5177436710767],
 [64.22176401688995, 114.73331621175502, 96.17267725692972],
 [81.26205876660073, 100.49647521225083, 3550.781584237549],
 [118.64551263852323, 96.49666570792286, 3209.784326499607],
 [118.70400080192714, 96.55672285267048, 3202.462710244625],
 [77.34895154230571, 77.89713080431697, 1616.348257398497],
 [118.56628035253055, 96.40234392252746, 3213.445682114889]]),
 (6973,
 [[72.50191364575532, 99.92189255143647, 2470.699753412948],
 [72.73381667361762, 99.83371460919332, 2442.003948596213],
 [72.39253672571188, 99.81561711859166, 2450.194875456176],
 [72.58351485796673, 99.75978902115273, 2446.099585222095],
 [72.55759922126946, 100.0294214129675, 2487.100038667084],
 [72.57307064374842, 99.85451048898713, 2511.701509335189],
 [72.51428256839976, 99.72397545952776, 2482.998216754375],
 [72.34625610801322, 100.0444800094792, 2466.598744544187],
 [72.13653068287205, 99.87661917278416, 2474.799710524056],
 [72.64006627072443, 99.78935257630796, 2478.899786929185],
 [72.40932961147622, 99.94220119655255, 2454.295945248991],
 [72.48122227881026, 99.84273302955303, 2507.600661605101]]),
 (1706,
 [[108.3443282919675, 100.84273369075228, 1895.8400028984754],
 [108.36077607279039, 100.8823430099223, 1934.2533854297046],
 [108.50283974674959, 100.79203616247979, 1957.3002332427275],
 [108.32695404645516, 100.92893729274489, 1903.5232083185826],
 [108.33282596888398, 100.68171905697531, 1972.6684081408775],
 [108.27555113633017, 100.84239054725518, 1941.9362817726542],
 [108.32603028445537, 100.98607041292506, 1949.6191849036916],
 [108.43945026646126, 100.80429909066194, 1911.20545694204],
 [108.41938821659426, 100.786398653622, 1980.3506962812596],
 [108.33984731223752, 100.88363740996039, 1918.888494813312],
 [108.32525890645205, 100.68360289978423, 1964.984398622469],
 [108.31696912684708, 100.69812697897636, 1926.571082107931]]),
 ...
 ...]
```

It's a little messy, but you can see that we have an interesting signal in the 6973 MHz band.

Working With Floating Point Numbers

Floating point numbers have many subtleties, and are the source of countless bugs. Some quite painful to discover.

There are many great discussions about this, and the classic paper *What Every Computer Scientist Should Know About Floating-Point Arithmetic* never goes out of style. It is worth a read and still totally relevant.

One of the more common traps that users of floating point fall into is forgetting that equality isn't so simple when one considers all the mechanisms that make digital math possible.

For example, is

$$9/4 == 18/8$$

or is

$$7*3/3 == 7/3*3 ?$$

The answer is a firm "sometimes". Your logic should never rely upon this.

Many *Linters* (programs to detect possible issues in your code, such as Pylint or Pyflakes in the Python world) will catch these. You should already be using these tools.

What Every Computer Scientist Should Know About Floating-Point Arithmetic

DAVID GOLDBERG

Xerox Palo Alto Research Center, 3333 Coyote Hill Road, Palo Alto, California 94304

Floating-point arithmetic is considered an esoteric subject by many people. This is rather surprising, because floating-point is ubiquitous in computer systems: Almost every language has a floating-point datatype; computers from PCs to supercomputers have floating-point accelerators; most compilers will be called upon to compile floating-point algorithms from time to time; and virtually every operating system must respond to floating-point exceptions such as overflow. This paper presents a tutorial on the aspects of floating-point that have a direct impact on designers of computer systems. It begins with background on floating-point representation and rounding error, continues with a discussion of the IEEE floating-point standard, and concludes with examples of how computer system builders can better support floating point.

Categories and Subject Descriptors: (Primary) C.0 [Computer Systems Organization]: General—instruction set design; D.3.4 [Programming Languages]: Processors—compilers, optimization; G.1.0 [Numerical Analysis]: General—computer arithmetic, error analysis, numerical algorithms (Secondary) D.2.1 [Software Engineering]: Requirements/Specifications—languages; D.3.1 [Programming Languages]: Formal Definitions and Theory—semantics D.4.1 [Operating Systems]: Process Management—synchronization

General Terms: Algorithms, Design, Languages

Additional Key Words and Phrases: denormalized number, exception, floating-point, floating-point standard, gradual underflow, guard digit, NaN, overflow, relative error, rounding error, rounding mode, ulp, underflow

INTRODUCTION

Builders of computer systems often need information about floating-point arithmetic. There are however, remarkably few sources of detailed information about it. One of the few books on the subject, *Floating-Point Computation* by Pat Sterbenz, is long out of print. This paper is a tutorial on those aspects of floating-point arithmetic (*floating-point* hereafter) that have a direct connection to systems building. It consists of three loosely connected parts. The first (Section 1) discusses the implications of using different rounding strategies for the basic opera-

tions of addition, subtraction, multiplication, and division. It also contains background information on the two methods of measuring rounding error, *ulps* and *relative error*. The second part discusses the IEEE floating-point standard, which is becoming rapidly accepted by commercial hardware manufacturers. Included in the IEEE standard is the rounding method for basic operations; therefore, the discussion of the standard draws on the material in Section 1. The third part discusses the connections between floating point and the design of various aspects of computer systems. Topics include instruction set design,

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its data appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.
© 1991 ACM 0360-0300/91/0300-0005 \$01.50

Comparing Floats

So, how should one compare floats? The traditional approach for most cases is to create a function that checks for some maximum absolute difference.

As of Python 3.5 there is a nice standard implementation of this that should be the obvious choice in most cases, *math.isclose*.

Whether or not two values are considered close could be determined by *relative tolerance*, which the maximum allowed difference between arguments, relative to the larger absolute value:

```
import math
a = 5.0
b = 4.99998
math.isclose(a, b, rel_tol=1e-5)
True
math.isclose(a, b, rel_tol=1e-6)
False
```

You might instead require *absolute tolerance*,

```
import math
a = 5.0
b = 4.99998
math.isclose(a, b, abs_tol=0.00003)
True
math.isclose(a, b, abs_tol=0.00001)
False
```

Third Approach: Shifting data around before reducing. (contd.)

#take a step back and redo a few steps with rounded values. Just to make it easier to see.

```
freq_key_value_rounded = freq_distinct.map(lambda x: ( x[0], [[round(x[1]),round(x[2]),round(x[3]])] ] ) )
reduced_freq_key_value_rounded = freq_key_value_rounded.reduceByKey(lambda x,y: x+y)
freq_rounded_sorted = reduced_freq_key_value_rounded.takeOrdered(4, key = lambda x: -len(x[1]))
```

```
freq_rounded_sorted
[(5182,
 [[119, 97, 3206],
  [77, 78, 1623],
  [119, 96, 3221],
  [119, 96, 3217],
  [112, 82, 583],
  [84, 103, 1981],
  [64, 115, 96],
  [81, 100, 3551],
  [119, 96, 3210],
  [119, 97, 3202],
  [77, 78, 1616],
  [119, 96, 3213]]],
 (6973,
 [[73, 100, 2471],
  [73, 100, 2442],
  [72, 100, 2450],
  [73, 100, 2446],
  [73, 100, 2487],
  [73, 100, 2512],
  [73, 100, 2483],
  [72, 100, 2467],
  [72, 100, 2475],
  [73, 100, 2479],
  [72, 100, 2454],
  [72, 100, 2508]]],
 (1706,
 [[108, 101, 1896],
  [108, 101, 1934],
  [109, 101, 1957],
  [108, 101, 1904],
  [108, 101, 1973],
  [108, 101, 1942],
  [108, 101, 1950],
  [108, 101, 1911],
  [108, 101, 1980],
  [108, 101, 1919],
  [108, 101, 1965],
  [108, 101, 1927]]],
 (6972,
 [[73, 100, 2442],
  [72, 100, 2491],
  [73, 100, 2483],
  [73, 100, 2458],
  [73, 100, 2479],
  [72, 100, 2495],
  [72, 100, 2504],
  [73, 100, 2499],
  [72, 100, 2454],
  [72, 100, 2508],
  [73, 100, 2462]]])
```

Third Approach: Shifting data around before reducing. (contd.)

#Now that I have the answer, I'll just do a quick python `sort on the timebase` to clean it up.

```
data = freq_rounded_sorted[1][1]
```

```
data.sort(key = lambda x: x[2])
```

```
data  
[[73, 100, 2442],  
 [73, 100, 2446],  
 [72, 100, 2450],  
 [72, 100, 2454],  
 [72, 100, 2467],  
 [73, 100, 2471],  
 [72, 100, 2475],  
 [73, 100, 2479],  
 [73, 100, 2483],  
 [73, 100, 2487],  
 [72, 100, 2508],  
 [73, 100, 2512]]
```



Once again we find that coords 73,100 really do have a single source of frequency 6792 MHz. With a cycle time of ~4s.

Notice we are missing a few points from our other analysis. If we go back and look and the floating point values, they fell just outside our 1 STD. We could relax that in our shift function.

Creating a Reusable Solution

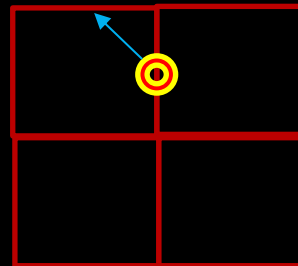
We did this in an exploratory mode, which is a wonderful capability of Spark. You can play around with real datasets, not just toy problems.

However we might want to pull these ideas together and create a rigorous code. For our spatial approach, we can use the same shift technique, but now in two dimensions. If we think about the worst case scenario in which our target cluster is centered on a border, we would realize that no single shift is assured to not land on another border. We would actually need two slides just to be sure.

We could also make the (rounded) frequency part of the key to deal with distinguishing frequencies at the same time. We would also need to do a similar bin shift trick there as well.

If you are worried about how well this scales, especially as we consider we may want more boxes as our resolution goes up, realize that this technique scales proportionate to the number of sources, not the number of bins of the size of the sky.

And of course there are alternate techniques as well.



Clustering

```
import pyspark
from pyspark.sql import SparkSession
from pyspark.ml.clustering import KMeans
from pyspark.ml.feature import StandardScaler
from pyspark.ml.linalg import Vectors
from pyspark.ml.feature import VectorAssembler
from pyspark.sql import functions as F

spark = pyspark.sql.SparkSession.builder.appName('clustering').getOrCreate()

df = spark.read.csv('hw3.dat', sep=' ', header=False, inferSchema=True)
df = df.withColumnRenamed('_c0', 'ascension')\
      .withColumnRenamed('_c1', 'declination')\
      .withColumnRenamed('_c2', 'time')\
      .withColumnRenamed('_c3', 'frequency')

vec_assembler = VectorAssembler(inputCols=['ascension', 'declination', 'frequency'], outputCol='features')
df_feature = vec_assembler.transform(df)

scaler = StandardScaler(inputCol="features", outputCol="scaledFeatures")
df_feature = scaler.fit(df_feature).transform(df_feature)

"""
cluster_num = [i for i in range(1000, 5000, 500)]
all_std = []

for n_clusters in cluster_num:
    kmeans = KMeans().setK(n_clusters).setFeaturesCol("scaledFeatures")
    model = kmeans.fit(df_feature)
    transformed = model.transform(df_feature)
    cluster_std = transformed.groupBy('prediction').agg(
        F.stddev('ascension').alias('asc_std'),
        F.stddev('declination').alias('dec_std'),
        F.stddev('frequency').alias('fre_std')
    )

    avg_std = cluster_std.agg(
        F.mean('asc_std').alias('mean_asc_std'),
        F.mean('dec_std').alias('mean_dec_std'),
        F.mean('fre_std').alias('mean_fre_std')
    ).collect()[0]

    overall_avg_std = (avg_std['mean_asc_std'] + avg_std['mean_dec_std'] + avg_std['mean_fre_std']) / 3
    all_std.append(overall_avg_std)

"""

mychoice = 4000
kmeans = KMeans().setK(mychoice).setFeaturesCol("scaledFeatures")
model = kmeans.fit(df_feature)
transformed = model.transform(df_feature)
mode_cluster = transformed.groupBy('prediction').count().orderBy('count', ascending=False).first()['prediction']
df_mode = transformed.filter(transformed.prediction == mode_cluster).orderBy('time')
df_mode.show()
```

I really applaud the initiative.

If you run this you will notice that it takes some time.

There are some ideas here you may find useful in the next exercise.

Introduces ideas of which features to include (cluster on time?), and how to scale or normalize them (spatial vs. frequency).

Back to our Recommender System...



Starting Point

What we are given is a large (100,480,507 ratings) and sparse (that is a little better than 1% of 8,532,958,530 matrix elements) list of ratings for users:

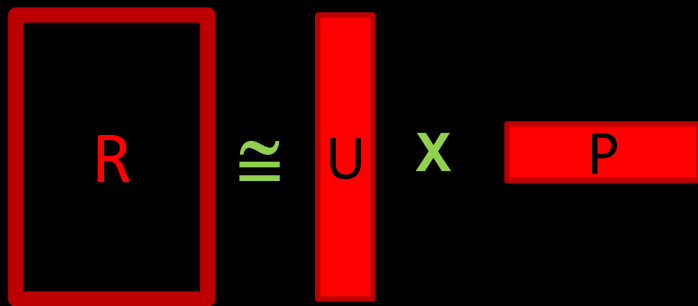
← 480,189 Users →

	5		3											1
1														
			3			5								
								3			3			3
	2			4							5		5	

← 17,770 Movies →

Lossy Compression Becomes Approximate Solution

The process of lossy compressing the sparse R matrix is also going to provide us a means to construct its missing members (the dense matrix).



The diagram illustrates the matrix approximation $R \approx U \times P$. It features three red shapes: a square labeled 'R' on the left, a tall vertical rectangle labeled 'U' in the middle, and a horizontal rectangle labeled 'P' on the right. A green approximation symbol ($\approx$) is placed between 'R' and 'U', and a green multiplication symbol ($\times$) is placed between 'U' and 'P'.

We will call our smaller matrices a *user feature matrix* and a *product feature matrix*. *This approximation is also going to smooth out the zeros and in the process give us our projected ratings.*

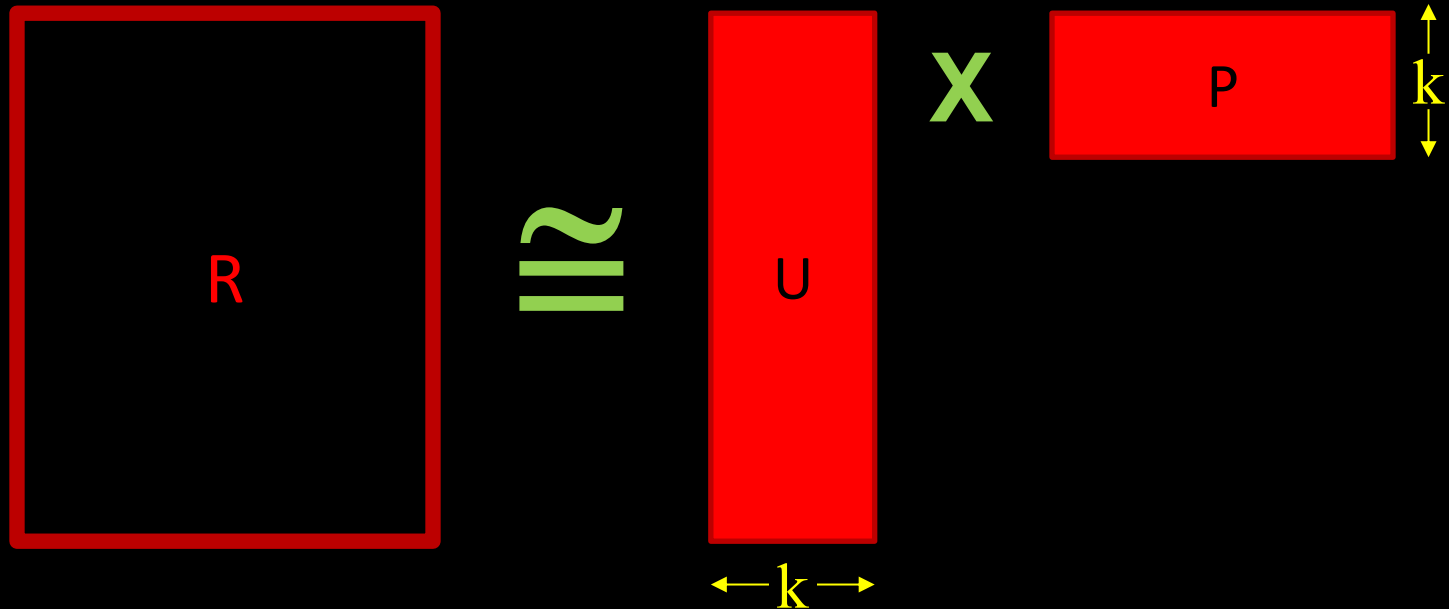
Objective

For any given user we would like to use their ratings, in combination with all the existing user ratings, to determine which movies they might prefer. For example, a user might really like *Annie Hall* and *The Purple Rose of Cairo* (both Woody Allen movies, although our database doesn't have that information). Can we infer from other users that they might like *Zelig*? That would be finding a latent variable. These might also include affinities for an actor, or director, or genre, etc. This type of algorithm is called *collaborative filtering*.

Users	3	5	2	3	5	2	4	3	1	4	3	1	3	2	1
	1	4	1	3	2	1	3	2	5	5	3	4	3	4	1
	1	3	5	3	3	5	5	4	2	2	2	5	4	3	3
	4	1	2	4	2	3	5	2	3	1	1	3	1	2	3
	2	2	4	1	4	4	3	5	1	2	4	5	2	5	4
Movies															

Matrix Factorization

The rank k can now also be thought of as the number of *latent effects* we are incorporating. But it will not be as intuitively explicit as a simple category, and we will have to investigate an optimal size for this parameter.



Defining our error

In ML, defining the *error* (or *loss*, or *cost*) is often the core of defining the objective solution. Once we define the error, we can usually plug it into a canned solver which can minimize it. Defining the error can be obvious, or very subtle, or have multiple acceptable methods.

Clustering: For k-means we simply used the geometrical distance. It was actually the sum of the squared distances, but you get the idea.

Image Recognition: If our algorithm tags a picture of a cat as a dog, is that a larger error than if it tags it as a horse? Or a car? How would you quantify these?

How about if our self driving car mistakes a crosswalk for an on-ramp?!

Regression: Do you want to penalize a lot of medium errors more than an occasional large error? If you are predicting stock prices, you most likely care more about the average, and an occasional bad call is OK. If you are projecting drug doses, that large error could kill!

Rare events: Want to make a 99% accurate tornado warning algorithm? Just put a piece of paper up saying "No tornado today." How do you weigh your error to deal with the significance of false negatives or positives?

Large Language Models: What is the next word in the sentence "My cat is ____ ". Black? White? Fat? These are all acceptable predictions. And if no one choice can be 100% right, then our error can also never be zero.

GPT-4

Many emerging applications can be challenging to quantify. Some of the things GPT-4 does are quite easy to grade. Literally.

Exam	GPT-4	GPT-4 (no vision)	GPT-3.5
Uniform Bar Exam (MBE+MEE+MPT)	298 / 400 (~90th)	298 / 400 (~90th)	213 / 400 (~10th)
LSAT	163 (~88th)	161 (~83rd)	149 (~40th)
SAT Evidence-Based Reading & Writing	710 / 800 (~93rd)	710 / 800 (~93rd)	670 / 800 (~87th)
SAT Math	700 / 800 (~89th)	690 / 800 (~89th)	590 / 800 (~70th)
Graduate Record Examination (GRE) Quantitative	163 / 170 (~80th)	157 / 170 (~62nd)	147 / 170 (~25th)
Graduate Record Examination (GRE) Verbal	169 / 170 (~99th)	165 / 170 (~96th)	154 / 170 (~63rd)
Graduate Record Examination (GRE) Writing	4 / 6 (~54th)	4 / 6 (~54th)	4 / 6 (~54th)
USABO Semifinal Exam 2020	87 / 150 (99th - 100th)	87 / 150 (99th - 100th)	43 / 150 (31st - 33rd)
USNCO Local Section Exam 2022	36 / 60	38 / 60	24 / 60
Medical Knowledge Self-Assessment Program	75 %	75 %	53 %
Codeforces Rating	392 (below 5th)	392 (below 5th)	260 (below 5th)
AP Art History	5 (86th - 100th)	5 (86th - 100th)	5 (86th - 100th)
AP Biology	5 (85th - 100th)	5 (85th - 100th)	4 (62nd - 85th)
AP Calculus BC	4 (43rd - 59th)	4 (43rd - 59th)	1 (0th - 7th)
AP Chemistry	4 (71st - 88th)	4 (71st - 88th)	2 (22nd - 46th)
AP English Language and Composition	2 (14th - 44th)	2 (14th - 44th)	2 (14th - 44th)
AP English Literature and Composition	2 (8th - 22nd)	2 (8th - 22nd)	2 (8th - 22nd)
AP Environmental Science	5 (91st - 100th)	5 (91st - 100th)	5 (91st - 100th)
AP Macroeconomics	5 (84th - 100th)	5 (84th - 100th)	2 (33rd - 48th)
AP Microeconomics	5 (82nd - 100th)	4 (60th - 82nd)	4 (60th - 82nd)
AP Physics 2	4 (66th - 84th)	4 (66th - 84th)	3 (30th - 66th)
AP Psychology	5 (83rd - 100th)	5 (83rd - 100th)	5 (83rd - 100th)
AP Statistics	5 (85th - 100th)	5 (85th - 100th)	3 (40th - 63rd)
AP US Government	5 (88th - 100th)	5 (88th - 100th)	4 (77th - 88th)
AP US History	5 (89th - 100th)	4 (74th - 89th)	4 (74th - 89th)
AP World History	4 (65th - 87th)	4 (65th - 87th)	4 (65th - 87th)
AMC 10 ³	30 / 150 (6th - 12th)	36 / 150 (10th - 19th)	36 / 150 (10th - 19th)
AMC 12 ³	60 / 150 (45th - 66th)	48 / 150 (19th - 40th)	30 / 150 (4th - 8th)
Introductory Sommelier (theory knowledge)	92 %	92 %	80 %
Certified Sommelier (theory knowledge)	86 %	86 %	58 %
Advanced Sommelier (theory knowledge)	77 %	77 %	46 %
Leetcode (easy)	31 / 41	31 / 41	12 / 41
Leetcode (medium)	21 / 80	21 / 80	8 / 80
Leetcode (hard)	3 / 45	3 / 45	0 / 45

Table 1. GPT performance on academic and professional exams. In each case, we simulate the conditions and scoring of the real exam. We report GPT-4's final score graded according to exam-specific rubrics, as well as the percentile of test-takers achieving GPT-4's score.

How do we even?

But some of the capabilities, especially the *generative* ones, defy simple quantification.

User

What is funny about this image? Describe it panel by panel.



Source: <https://www.reddit.com/r/hmmm/comments/ubab5v/hmmm/>

GPT-4

The image shows a package for a "Lightning Cable" adapter with three panels.

Panel 1: A smartphone with a VGA connector (a large, blue, 15-pin connector typically used for computer monitors) plugged into its charging port.

Panel 2: The package for the "Lightning Cable" adapter with a picture of a VGA connector on it.

Panel 3: A close-up of the VGA connector with a small Lightning connector (used for charging iPhones and other Apple devices) at the end.

The humor in this image comes from the absurdity of plugging a large, outdated VGA connector into a small, modern smartphone charging port.



write a poem about traffic lights



In the heart of a bustling city's night,
A sentinel stands, glowing with light,
With colors of red, yellow, and green,
A triad of hues, a silent machine.

A beacon of order midst chaos and speed,
Guiding the masses, an unspoken creed,
Whispers of safety, a language of hue,
The traffic lights speak, their purpose imbued.

Red, a guardian with a stern, watchful eye,

Perhaps most interesting,
some of these capabilities
weren't entirely anticipated,
and were *emergent*!

In unity they stand, those traffic light sentries,
A dance of transitions, a cycle of entry,
And so, in the chaos, a pattern ignites,
The rhythm of life, in colors of traffic lights.

Paint a photo of Disneyland in the style of Vincent Van Gogh



Do we need a comedian with a calculator?

Or a poetry reviewer?

Or an art critic?

Mean Square Error plus Regularization

Here, we will take the Mean Square Error distance between our given matrix and our approximation as a starting point.

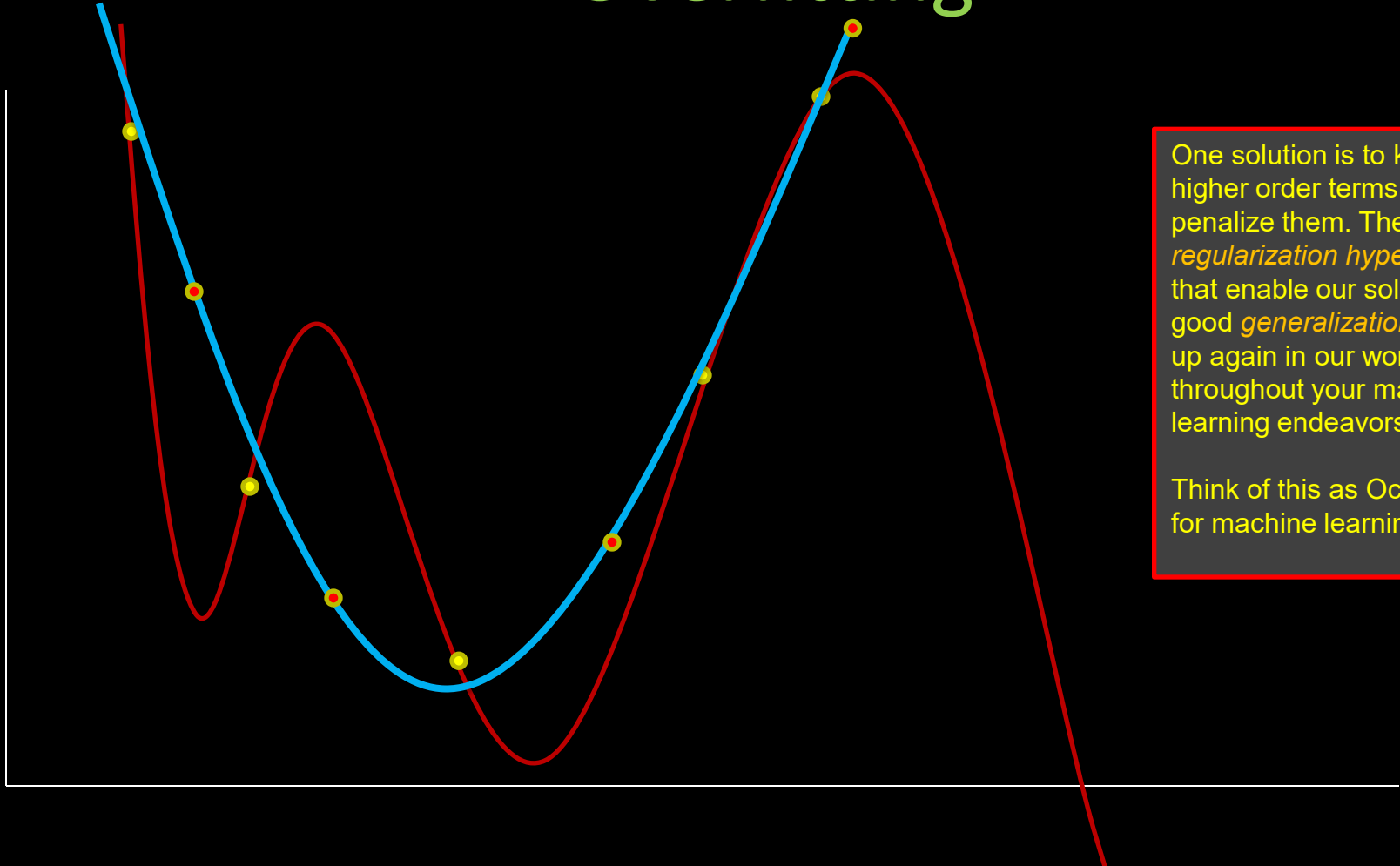
We will add a term to discourage *overfitting* by damping large, spikey elements of U or P. This is called *regularization* and versions appear frequently in error functions.

$$\text{Error} = \| R - U \times P \|^2 + \lambda (\text{Penalty for large elements})$$

The $\|$ notation means “sum the squares of all the elements and then take the square root”.

You may wonder how we can have “too little” error – the pursuit of which leads to overfitting. Think back to our clustering problem. We could drive the error as low as we wanted by adding more clusters (up to 5000!). But we weren’t really finding new clusters. Variations of this phenomena occur throughout machine learning.

Overfitting



One solution is to keep using higher order terms, but to penalize them. These *regularization hyperparameters* that enable our solution to have good *generalization* will show up again in our workshop, and throughout your machine learning endeavors.

Think of this as Occam's Razor for machine learning.

Different Schemes For Regularization

There are many options for regularization. In classic machine learning you will often see:

- L1 or LASSO regression, where we penalize the absolute value of the terms.
- L2 or "L squared" or Ridge regression, where we penalize the squared value of the terms. In deep learning this is often called Weight Decay.

The hyperparameter to control the strength of this effect is usually called lambda.

We will explore some other regularization options later.

Mean Square Error plus Regularization

Here is exactly our error term with regularization. MLLIB can scale the lambda factor for us based on the number of ratings (this tweak is called ALS-WR).

$$\text{Error} = \|R - UxP\|^2 + \lambda(\|U\|^2 + \|P\|^2)$$

The $\|\cdot\|$ notation means “sum the squares of all the elements and then take the square root”.

Additionally, we need to account for our missing (unrated) values. We just zero out those terms. Here it is term-by-term:

$$\text{Error} = \sum_{i,j} w_{i,j} (R_{i,j} - (UxP)_{ij})^2 + \lambda(\|U\|^2 + \|P\|^2) \quad w_{i,j} = 0 \text{ if } R_{i,j} \text{ is unknown}$$

Note that we now have two hyperparameters, k and λ , that we need to select intelligently.

Alternating Least Squares

To actually find the U and P that minimize this error we need a solving algorithm.

SGD, a go-to for many ML problems and one we will use later, is not quite as efficient for billions of sparse parameters, which we can easily reach with these types of problems. We are dealing with *Users X Items* elements here.

Instead we use Alternating Least Squares (ALS), also built into MLLIB.

Alternating least squares cheats by holding one of the arrays constant and then doing a classic least squares fit on the other array parameters. Then it does this for the other array.

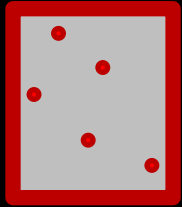
This is easily parallelized.

It works well with sparse inputs. The algorithm scales linearly with observed entries.

$$R \approx U X P$$

Here Is Our Plan

Given
Ratings



$\approx$

Use
ALS

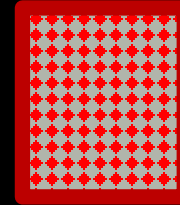
U

$\times$

P

$=$

Predicted
Ratings



Sparse

	5	3																	1
1																			
		3		5															
						3				3									3
2			4									5		5					

Movies

Dense

	3	5	2	3	5	2	4	3	1	4	3	1	3	2	1				
1	4	1	3	2	1	3	2	5	5	3	4	3	4	1					
	3	5	3	3	5	5	4	2	2	2	5	4	3	3					
4	1	2	4	2	3	5	2	3	1	1	3	1	2	3					
2	2	4	1	4	4	3	5	1	2	4	5	2	5	4					

Movies

Training, Validation and Test Data

We use the training data to create our solution, the UxP matrix here.

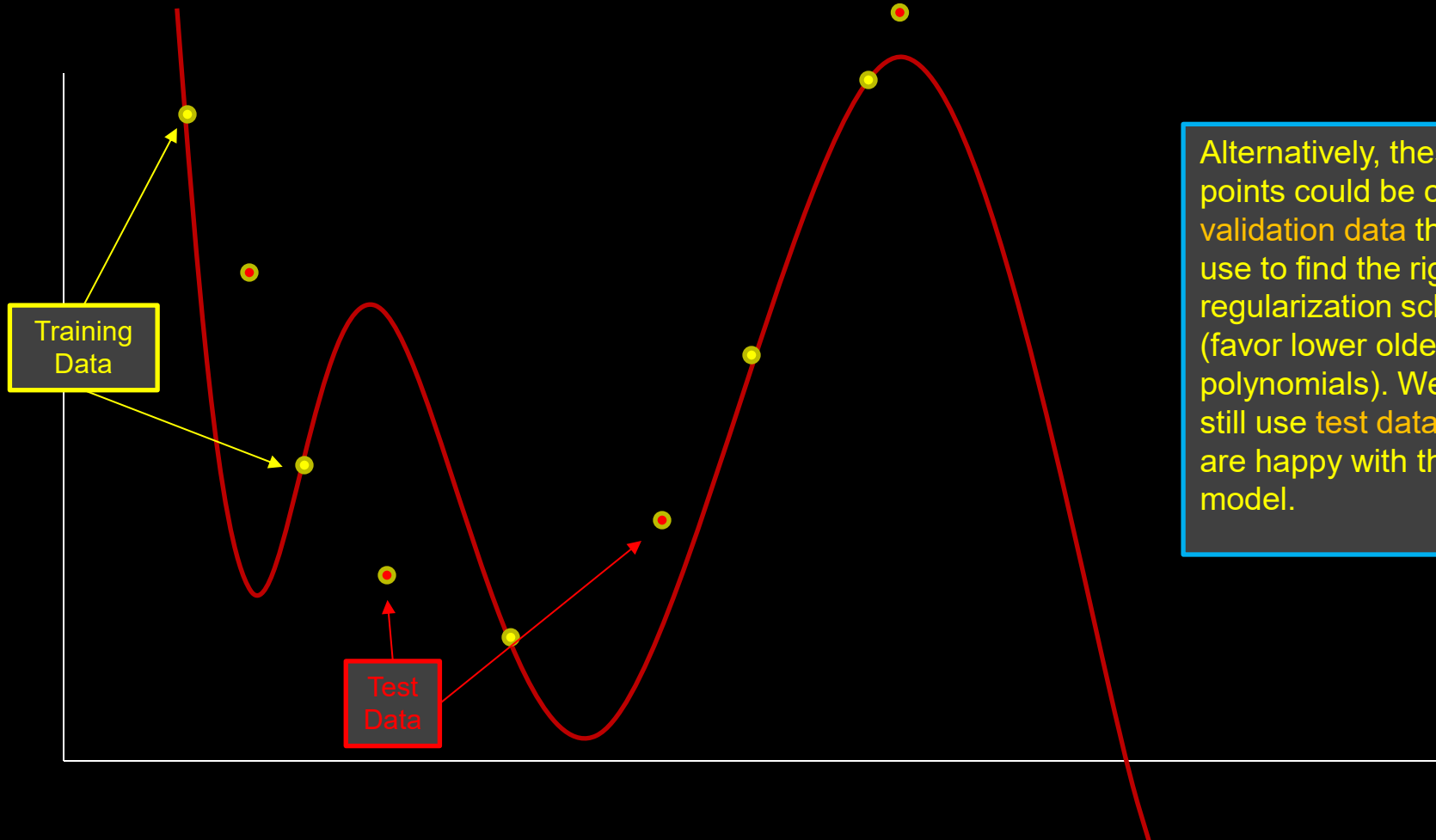
The validation data is used to verify we are not overfitting: to stop training after enough iterations, to adjust λ or k here, or to optimize the many other *hyperparameters* you will encounter in ML.

The test data must be saved to judge our final solution.

Reusing, or subtly mixing, the training, validation and test data is a recipe for confusion.

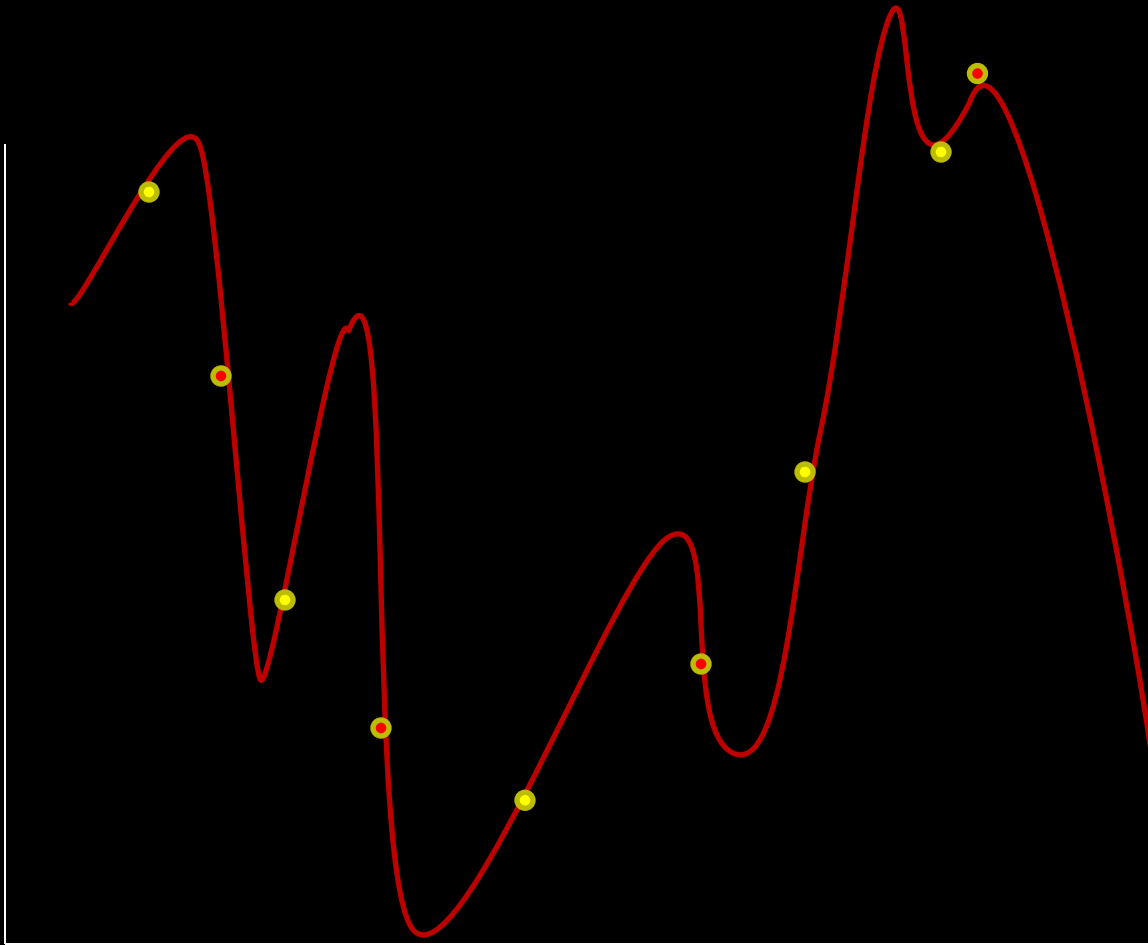
What proportions of your data to use for each of these is a question that you might want to start by copying from similar work or examining the literature. There are techniques to slice-and-cycle share the training and validation data, called *cross-validation*. Don't try this with the test data!

Reality Check By Test Data



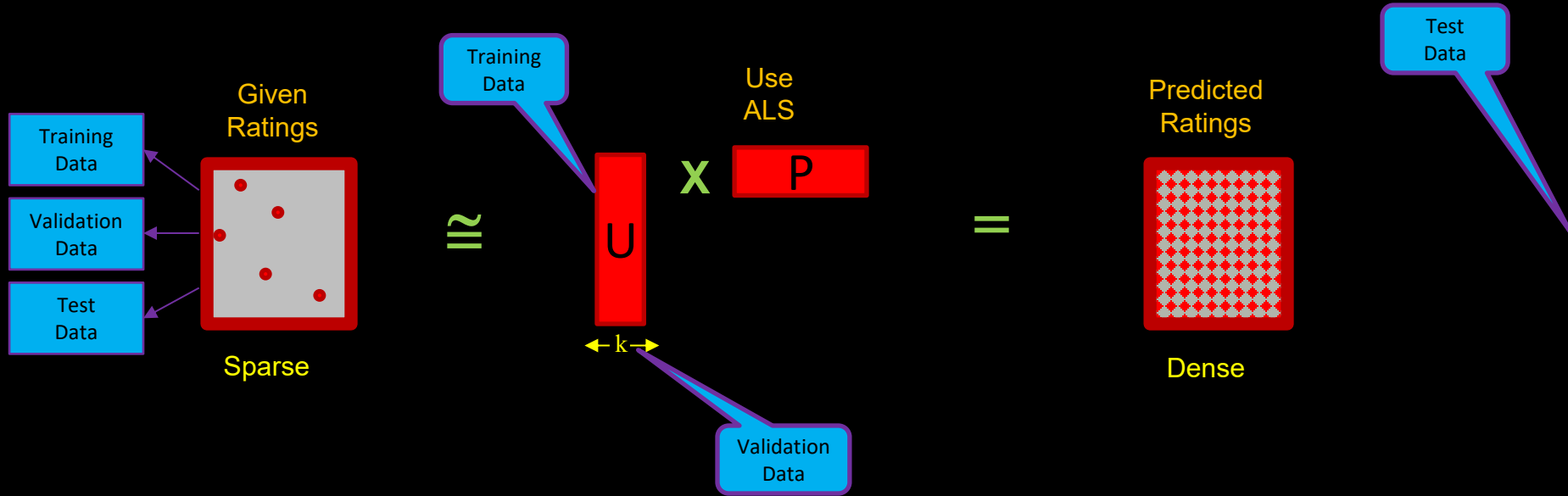
Alternatively, these new points could be our validation data that we use to find the right regularization scheme (favor lower order polynomials). We would still use test data after we are happy with that model.

Used The
Test Data
For
Training



We can also say
that this model has
low bias and high
variance.

Where does our data come into play?

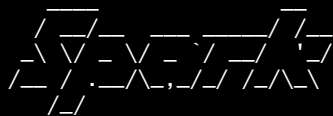


Let's Build A Recommender

We have all the tools we need, so let's fire up PySpark and create a scalable recommender.
Our plan is:

1. Load and parse data files
2. Create ALS model
3. Train it with varying ranks (k) to find reasonable hyperparameters
4. Add a new user
5. Get top recommendations for new user

Building a Recommender



Using Python version 3.7.4 (default) (d...
SparkSession available as 'spark'.

```
>>>  
>>> ratings_raw_RDD = sc.textFile('ratings.txt')  
>>> ratings_RDD = ratings_raw_RDD.map(lambda line: line.split(','))  
>>> training_RDD, validation_RDD, test_RDD = ratings_RDD.partition(3)  
>>> predict_validation_RDD = validation_RDD.map(lambda (movie, rating): (movie, float(tokens[2])))  
>>> predict_test_RDD = test_RDD.map(lambda (movie, rating): (movie, float(tokens[2])))
```

```
login06% interact
```

```
...  
r288%
```

```
r288%
```

```
r288% module load spark
```

```
r288% pyspark
```

```
) , float(tokens[2]))
```

```
>>> training_RDD.take(4)  
[(1, 1029, 3.0), (1, 1061, 3.0), (1, 1263, 2.0), (1, 1371, 2.5)]  
>>> predict_validation_RDD.take(4)  
[(1, 1129), (1, 1172), (1, 1405), (1, 2105)]  
>>>
```

ovie,rating) data.

Ds.

data for our prediction RDDs.



Using Python version 2.7.5 (default, Nov 20 2015 02:00:19)
SparkContext available as sc, HiveContext available as sqlContext.

```
>>> ratings_raw_RDD = sc.textFile('ratings.csv')
>>> ratings_RDD = ratings_raw_RDD.map(lambda line: line.split(",")).map(lambda tokens: (int(tokens[0]),int(tokens[1]),float(tokens[2])))
>>>
>>> training_RDD, validation_RDD, test_RDD = ratings_RDD.randomSplit([3, 1, 1], 0)
>>>
>>> predict_validation_RDD = validation_RDD.map(lambda x: (x[0], x[1]))
>>> predict_test_RDD = test_RDD.map(lambda x: (x[0], x[1]))
>>>
>>>
>>> from pyspark.mllib.recommendation import ALS
>>> import math
>>>
>>> seed = 5
>>> iterations = 10
>>> regularization = 0.1
>>> trial_ranks = [4, 8, 12]
>>> lowest_error = float('inf')
```

Import mllib and set some variables we are about to use.

```

>>> ratings_raw_RDD = sc.textFile('ratings.csv')
>>> ratings_RDD = ratings_raw_RDD.map(lambda line: line.split(",")).map(lambda tokens: (int(tokens[0]),int(tokens[1]),float(tokens[2])))
>>>
>>> training_RDD, validation_RDD, test_RDD = ratings_RDD.randomSplit([3, 1, 1], 0)
>>>
>>> predict_validation_RDD = validation_RDD.map(lambda x: (x[0], x[1]))
>>> predict_test_RDD = test_RDD.map(lambda x: (x[0], x[1]))
>>>
>>>
>>> from pyspark.mllib.recommendation import ALS
>>> import math
>>>
>>> seed = 5
>>> iterations = 10
>>> regularization = 0.1
>>> trial_ranks = [4, 8, 12]
>>> lowest_error = float('inf')
>>>
>>> for k in trial_ranks:
>>>     model = ALS.train(training_RDD, k, seed=seed, iterations=iterations, lambda_=regularization)
>>>     #Coercing ((u,p),r) tuple format to accomodate join
>>>     predictions_RDD = model.predictAll(predict_validation_RDD).map(lambda r: ((r[0], r[1]), r[2]))
>>>     ratings_and_preds_RDD = validation_RDD.map(lambda r: ((r[0], r[1]), r[2])).join(predictions_RDD)
>>>     error = math.sqrt(ratings_and_preds_RDD.map(lambda r: (r[1][0] - r[1][1])**2).mean())
>>>     print('For k=',k,'the RMSE is', error)
>>>     if error < lowest_error:
>>>         best_k = k
>>>         lowest_error = error
>>>
>>>
For k= 4 the RMSE is 0.9357038861004305
For k= 8 the RMSE is 0.9438612625240242
For k= 12 the RMSE is 0.9390638322819614
>>>
>>> print('The best rank is size', best_k)
The best rank is size 4

```

Run our ALS model on various ranks to see which is best.

```

>>> for k in trial_ranks:
>>>     model = ALS.train(training_RDD, k, seed=seed, iterations=iterations, lambda_=regularization)
>>>     #Coercing ((u,p),r) tuple format to accomodate join
>>>     predictions_RDD = model.predictAll(predict_validation_RDD).map(lambda r: ((r[0], r[1]), r[2]))
>>>     ratings_and_preds_RDD = validation_RDD.map(lambda r: ((r[0], r[1]), r[2])).join(predictions_RDD)
>>>     error = math.sqrt(ratings_and_preds_RDD.map(lambda r: (r[1][0] - r[1][1])**2).mean())
>>>     print ('For k=',k,'the RMSE is', error)
>>>     if error < lowest_error:
>>>         best_k = k
>>>         lowest_error = error
>>>
For k= 4 the RMSE is 0.9357038861004305
For k= 8 the RMSE is 0.9438612625240242
For k= 12 the RMSE is 0.9390638322819614

```

The ALS.train() routines gives us:

```

>>> model.predictAll(predict_validation_RDD).take(2)
[Rating(user=463, product=4844, rating=2.7640960482284322), Rating(user=380, product=4844, rating=2.399938320644199)]

```

To do the "RMS error" math, we want elements with a *(Given,Predicted)* value for each *(User,Movie)* key:

```

>>> ratings_and_preds_RDD.take(2)
[((119, 145), (4.0, 2.903215714486778)), ((407, 5995), (4.5, 4.604779028840272))]

```

So the next two lines get us from *here to there*.

```

>>> for k in trial_ranks:
>>>     model = ALS.train(training_RDD, k, seed=seed, iterations=iterations, lambda_=regularization)
>>>     #Coercing ((u,p),r) tuple format to accomodate join
>>>     predictions_RDD = model.predictAll(predict_validation_RDD).map(lambda r: ((r[0], r[1]), r[2]))
>>>     ratings_and_preds_RDD = validation_RDD.map(lambda r: ((r[0], r[1]), r[2])).join(predictions_RDD)
>>>     error = math.sqrt(ratings_and_preds_RDD.map(lambda r: (r[1][0] - r[1][1])**2).mean())
>>>     print ('For k=',k,'the RMSE is', error)
>>>     if error < lowest_error:
>>>         best_k = k
>>>         lowest_error = error
>>>
For k= 4 the RMSE is 0.9357038861004305
For k= 8 the RMSE is 0.9438612625240242
For k= 12 the RMSE is 0.9390638322819614

```

```

>>> model.predictAll(predict_validation_RDD).map(lambda r: ((r[0], r[1]), r[2])).take(2)
[((463, 4844), 2.7640960482284322), ((380, 4844), 2.399938320644199)]

```

That map gets us to a pair RDD with *[(User,Movie), rating]* format.

Now do this with the validation RDD:

```

>>> validation_RDD.take(2)
[(1, 1129, 2.0), (1, 1172, 4.0)]
>>>
>>> validation_RDD.map(lambda r: ((r[0], r[1]), r[2])).take(2)
[((1, 1129), 2.0), ((1, 1172), 4.0)]

```

```

>>> for k in trial_ranks:
>>>     model = ALS.train(training_RDD, k, seed=seed, iterations=iterations, lambda_=regularization)
>>>     #Coercing ((u,p),r) tuple format to accomodate join
>>>     predictions_RDD = model.predictAll(predict_validation_RDD).map(lambda r: ((r[0], r[1]), r[2]))
>>>     ratings_and_preds_RDD = validation_RDD.map(lambda r: ((r[0], r[1]), r[2])).join(predictions_RDD)
>>>     error = math.sqrt(ratings_and_preds_RDD.map(lambda r: (r[1][0] - r[1][1])**2).mean())
>>>     print ('For k=',k,'the RMSE is', error)
>>>     if error < lowest_error:
>>>         best_k = k
>>>         lowest_error = error
>>>
For k= 4 the RMSE is 0.9357038861004305
For k= 8 the RMSE is 0.9438612625240242
For k= 12 the RMSE is 0.9390638322819614

```

To collect rating values for common (User,Movie) keys calls for a join()

Data before join:

```

>>> predictions_RDD.take(2)
[((463, 4844), 2.7640960482284322), ((380, 4844), 2.399938320644199)]
>>>
>>> validation_RDD.map(lambda r: ((r[0], r[1]), r[2])).take(2)
[((1, 1129), 2.0), ((1, 1172), 4.0)]

```

Results of join:

```

>>> ratings_and_preds_RDD.take(2)
[((119, 145), (4.0, 2.903215714486778)), ((407, 5995), (4.5, 4.604779028840272))]

```



```

>>> for k in trial_ranks:
>>>     model = ALS.train(training_RDD, k, seed=seed, iterations=iterations, lambda_=regularization)
>>>     #Coercing ((u,p),r) tuple format to accomodate join
>>>     predictions_RDD = model.predictAll(predict_validation_RDD).map(lambda r: ((r[0], r[1]), r[2]))
>>>     ratings_and_preds_RDD = validation_RDD.map(lambda r: ((r[0], r[1]), r[2])).join(predictions_RDD)
>>>     error = math.sqrt(ratings_and_preds_RDD.map(lambda r: (r[1][0] - r[1][1])**2).mean())
>>>     print ('For k=',k,'the RMSE is', error)
>>>     if error < lowest_error:
>>>         best_k = k
>>>         lowest_error = error
>>>
For k= 4 the RMSE is 0.9357038861004305
For k= 8 the RMSE is 0.9438612625240242
For k= 12 the RMSE is 0.9390638322819614
>>>
>>> print('The best rank is size', best_k)
The best rank is size 4
>>>
>>> model = ALS.train(training_RDD, best_k, seed=seed, iterations=iterations, lambda_=regularization)
>>> predictions_RDD = model.predictAll(predict_test_RDD).map(lambda r: ((r[0], r[1]), r[2]))
>>> ratings_and_preds_RDD = test_RDD.map(lambda r: ((r[0], r[1]), r[2])).join(predictions_RDD)
>>> error = math.sqrt(ratings_and_preds_RDD.map(lambda r: (r[1][0] - r[1][1])**2).mean())
>>> print ('For testing data the RMSE is %s' % (error))
For testing data the RMSE is 0.9406803213698973

```

This is our fully tested model (smallest dataset).
These results were reported against the test_RDD.

6D Data Hunt

John Urbanic

Parallel Computing Scientist
Pittsburgh Supercomputing Center

Distinguished Service Professor
Carnegie Mellon University

```

>>> for k in trial_ranks:
>>>     model = ALS.train(training_RDD, k, seed=seed, iterations=iterations, lambda_=regularization)
>>>     #Coercing ((u,p),r) tuple format to accomodate join
>>>     predictions_RDD = model.predictAll(predict_validation_RDD).map(lambda r: ((r[0], r[1]), r[2]))
>>>     ratings_and_preds_RDD = validation_RDD.map(lambda r: ((r[0], r[1]), r[2])).join(predictions_RDD)
>>>     error = math.sqrt(ratings_and_preds_RDD.map(lambda r: (r[1][0] - r[1][1])**2).mean())
>>>     print ('For k=',k,'the RMSE is', error)
>>>     if error < lowest_error:
>>>         best_k = k
>>>         lowest_error = error
>>>
For k= 4 the RMSE is 0.9357038861004305
For k= 8 the RMSE is 0.9438612625240242
For k= 12 the RMSE is 0.9390638322819614
>>>
>>> print('The best rank is size', best_k)
The best rank is size 4
>>>
>>> model = ALS.train(training_RDD, best_k, seed=seed, iterations=iterations, lambda_=regularization)
>>> predictions_RDD = model.predictAll(predict_test_RDD).map(lambda r: ((r[0], r[1]), r[2]))
>>> ratings_and_preds_RDD = test_RDD.map(lambda r: ((r[0], r[1]), r[2])).join(predictions_RDD)
>>> error = math.sqrt(ratings_and_preds_RDD.map(lambda r: (r[1][0] - r[1][1])**2).mean())
>>> print ('For testing data the RMSE is %s' % (error))
For testing data the RMSE is 0.9406803213698973

```

This is our fully tested model (smallest dataset).
These results were reported against the test_RDD.

Adding a User

```
>>>
>>> new_user_ID = 0
>>> new_user = [
    (0,100,4), # City Hall (1996)
    (0,237,1), # Forget Paris (1995)
    (0,44,4), # Mortal Kombat (1995)
    (0,25,5), # etc....
    (0,456,3),
    (0,849,3),
    (0,778,2),
    (0,909,3),
    (0,478,5),
    (0,248,4)
]
```

I checked that ID 0 is unused with a quick
`ratings_RDD.filter(lambda x: x[0]!='0').count()`

Note that we are joining, and then training, with ALL data
now - the ratings RDD. We are confident we know what
we are doing and are done testing.

```
>>>
>>> new_user_RDD = sc.parallelize(new_user)
>>>
>>> updated_ratings_RDD = ratings_RDD.union(new_user_RDD)
>>>
>>> updated_model = ALS.train(updated_ratings_RDD, best_rank, seed=seed, iterations=iterations,
lambda=regularization)
>>>
```

This need to retrain on all the data even though we just added a single user is call a *cold-start* problem. We might patch over that with a different technique like a Deep Learning *content based recommender* system. Most serious recommenders use an *ensemble* of algorithms.

Let's get some predictions...

```
.  
. .  
>>>  
>>> movies_raw_RDD = sc.textFile('movies.csv')  
>>> movies_RDD = movies_raw_RDD.map(lambda line: line.split(",")).map(lambda tokens: (int(tokens[0]),tokens[1]))  
>>>  
>>> new_user Rated_movie_ids = map(lambda x: x[1], new_user)  
>>> new_user_unrated_movies_RDD = movies_RDD.filter(lambda x: x[0] not in new_user Rated_movie_ids).map(lambda x: (new_user_ID, x[0]))  
>>> new_user_recommendations_RDD = updated_model.predictAll(new_user_unrated_movies_RDD)
```

```
>>> new_user_unrated_movies_RDD.take(3)  
[(0, 1), (0, 2), (0, 3)]  
>>> new_user_recommendations_RDD.take(2)  
[Rating(user=0, product=4704, rating=3.606560950463134), Rating(user=0, product=4844, rating=2.1368358868224036)]
```

Let's see some titles

```
>>> product_rating_RDD = new_user_recommendations_RDD.map(lambda x: (x.product, x.rating))
>>> new_user_recommendations_titled_RDD = product_rating_RDD.join(movies_RDD)
>>> new_user_recommendations_formatted_RDD = new_user_recommendations_titled_RDD.map(lambda x: (x[1][1],x[1][0]))
>>>
>>> top_recomends = new_user_recommendations_formatted_RDD.takeOrdered(10, key=lambda x: -x[1])
>>> for line in top_recomends: print (line)
```

```
...
('Wicker Man', 6.541041538414092)
('Dylan Moran: Monster (2004)', 6.485367991621634)
('Excision (2012)', 6.270479381570646)
('Dead Man's Shoes (2004)', 6.230858975698519)
('The Lair of the White Worm (1988)', 5.958594728894122)
('Hiroshima Mon Amour (1959)', 5.835915047073178)
('King Is Alive', 5.808692546936052)
('Maelström (2000)', 5.808692546936052)
('Innocence (2000)', 5.808692546936052)
('Let It Be (1970)', 5.798298934269299)
```

```
>>>
>>> one_movie_RDD = sc.parallelize([(0, 800)]) # Lone Star (1996)
>>> rating_RDD = updated_model.predictAll(one_movie_RDD)
>>> rating_RDD.take(1)
[Rating(user=0, product=800, rating=4.100848893773136)]
```

Looks like we can sort by value after all!

Behind the scenes takeOrdered() just does the key/value swap and SortByKey that we previously did ourselves.

```
>>> new_user_recommendations_titled_RDD.take(2)
[(111360, (1.0666741148393921, 'Lucy (2014)')), (49530, (1.8020006042285814, 'Blood Diamond (2006)'))]
>>> new_user_recommendations_formatted_RDD.take(2)
[('Lucy (2014)', 1.0666741148393921), ('Blood Diamond (2006)', 1.8020006042285814)]
```

Optional Exercises (No credit!)

1) We noticed that our top ranked movies have ratings higher than 5. This makes perfect sense as there is no ceiling implied in our algorithm and one can imagine that certain combinations of factors would combine to create “better than anything you’ve seen yet” ratings.

Maybe you have a friend that really likes Anime. Many of her ratings for Anime are 5. And she really likes Scarlett Johansson and gives her movies lots of 5s. Wouldn’t it be fair to consider her rating for *Ghost in the Shell* to be a 7/5?

Nevertheless, we may have to constrain our ratings to a 1-5 range. Can you normalize the output from our recommender such that our new users only sees ratings in that range?

2) We haven’t really investigated our convergence rate. We specify 10 iterations, but is that reasonable? Graph your error against iterations and see if that is a good number.

3) I mentioned that our larger dataset does benefit from a rank of 12 instead of 4 (as one might expect). The larger datasets (`ratings-large.csv` and `movies-large.csv`) are available to you in `~training/LargeMovies`. Prove that the error is less with a larger rank. How does this dataset benefit from more iterations? Is it more effective to spend the computation cycles on more iterations or larger ranks?

4) We could have used the very similar `pyspark.ml.recommendation` API, which uses dataframes. It requires a little more type checking, so we used the classic RDD API `pyspark.mllib.recommendation` instead - for conciseness. Try porting this example to that API. Is this a better way to work?

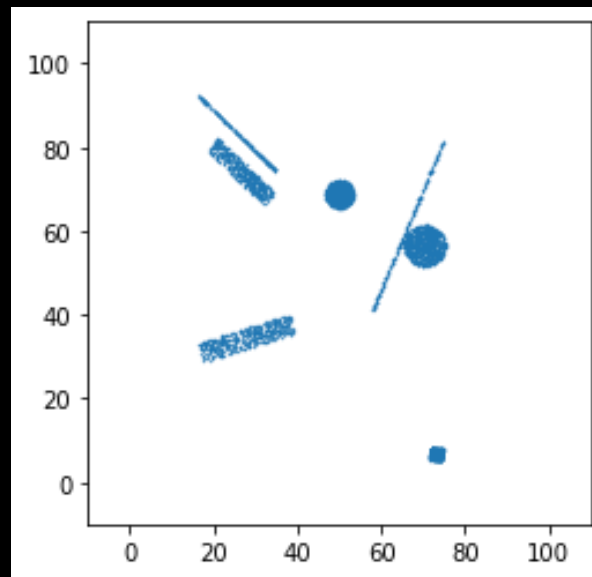
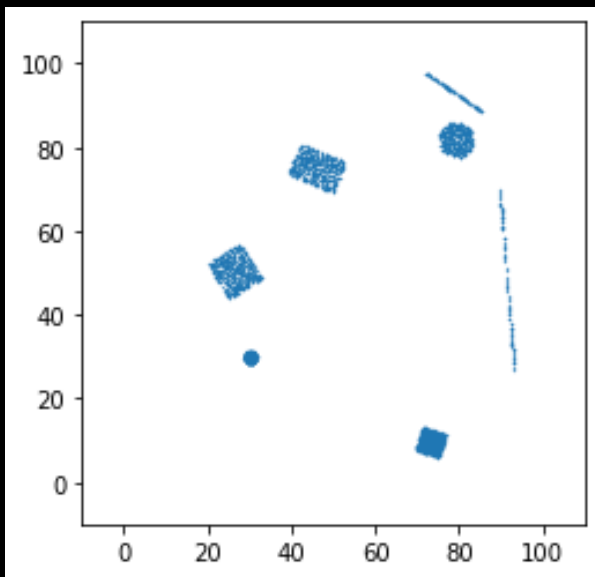
5) (Advanced) When we added our new user, we also retrained with the full ratings RDD. We no longer excluded the test and validation data. This seems reasonable, but are we still sure that our hyperparameters remain valid? And this will remain an issue as we continue to add new users and movies. You should investigate the sensitivity of our various hyperparameters to Ratings size, and consider how to deal with this problem.

Assignment #4

Q: Using Spark, what features can we discover in a 6 dimensional data set?

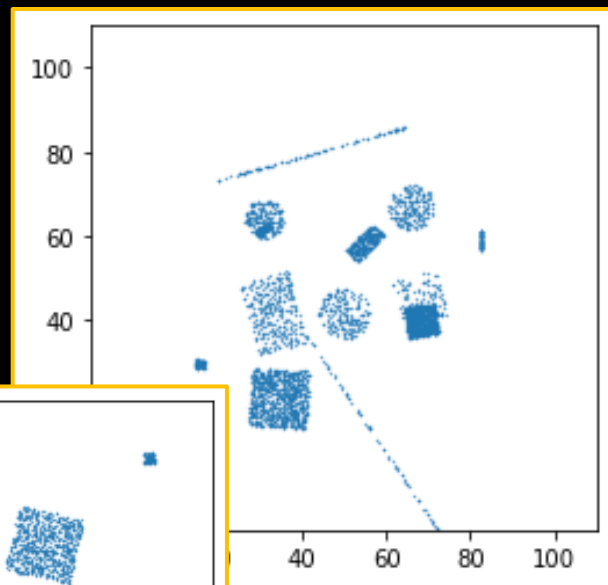
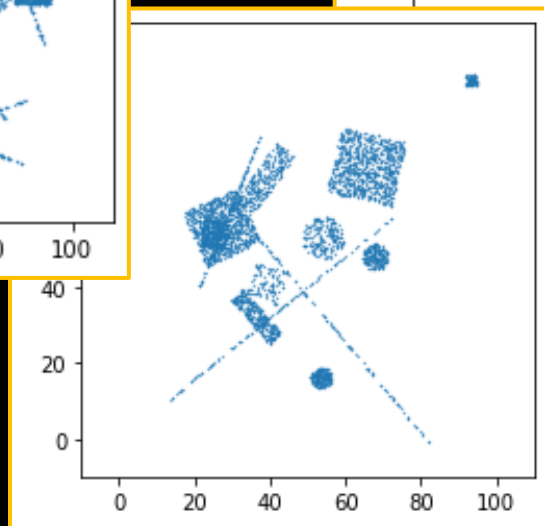
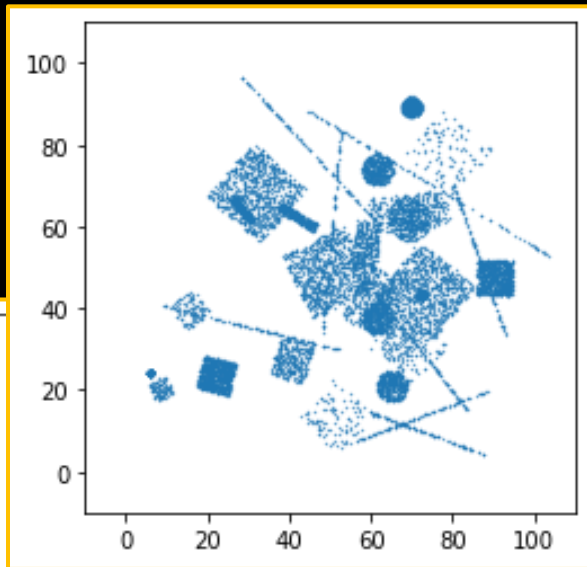
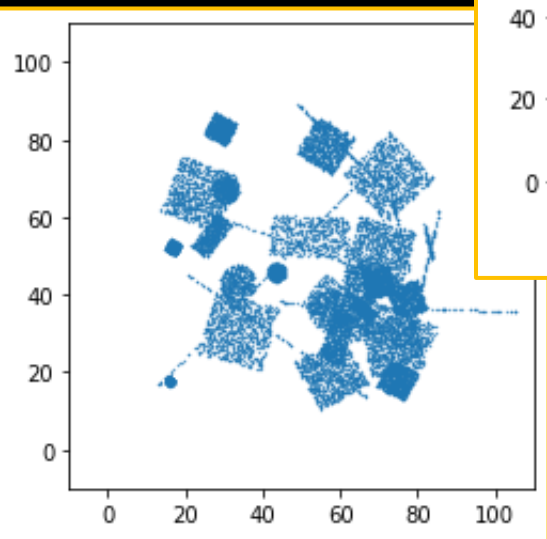
In `~urbanic/LargeScaleComputing/Spark/space.dat` on Bridges you will find the data as a list of 6 dimensional coordinates within a cube of side length 100.

It contains some interesting distributions. They may be simple and few:



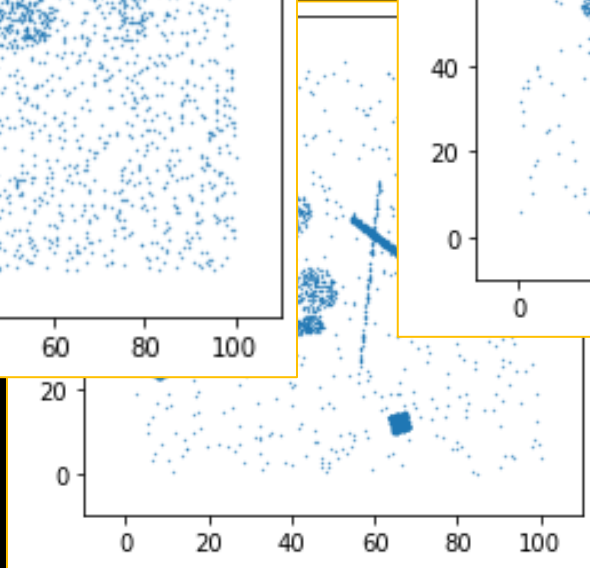
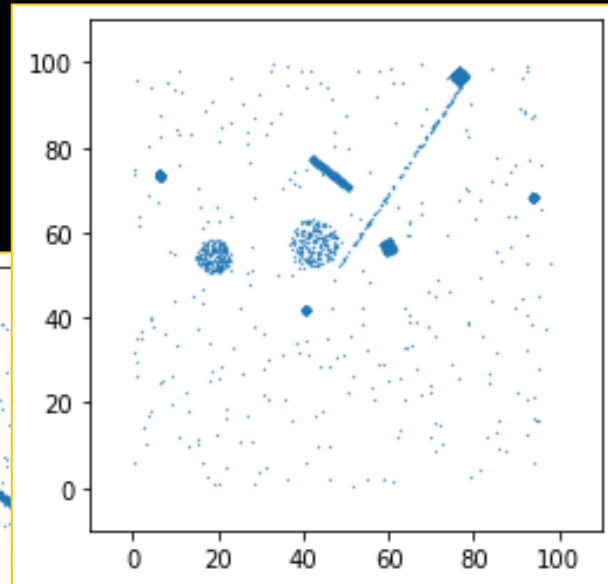
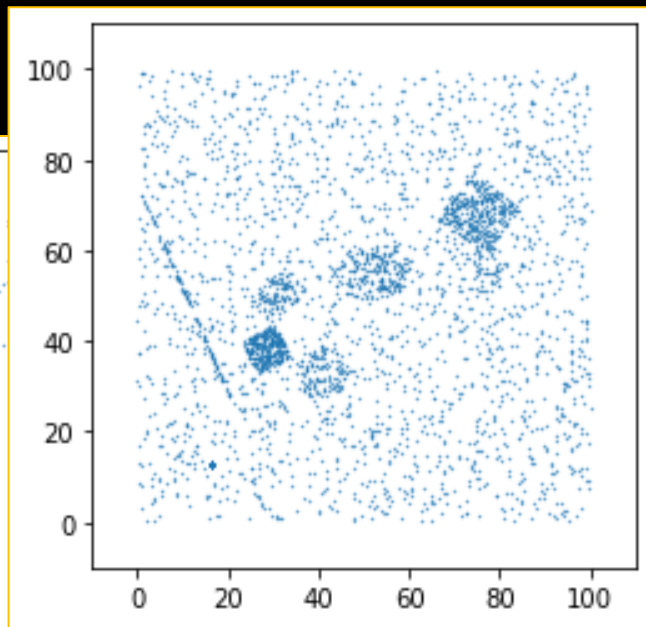
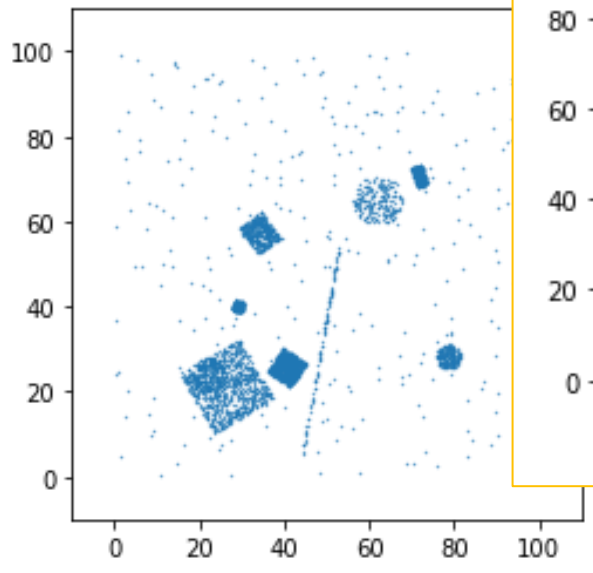
Assignment #4:

Or they may be more complex:



Assignment #4:

They might even have noise:



Assignment #4

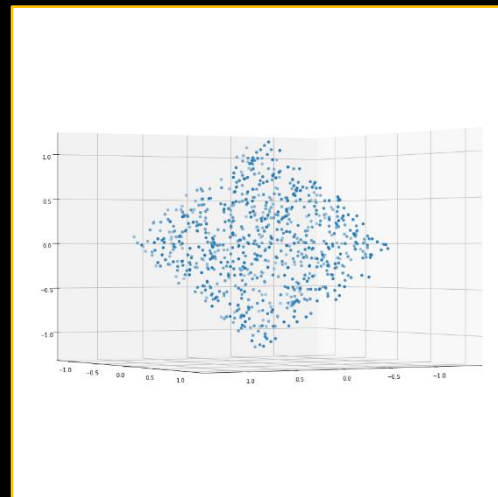
In 6 dimensions you will not be able to simply plot your way through this.

The distributions (objects) may each be any dimension up to 6. You may find lines, and you may find hypercubes or 6-balls.

They may be rotated in any direction. Make sure you appreciate this.

The good news is that the higher dimensionality lowers the chance that these objects will overlap significantly.

You can "extract" the 1 and 2 and 3 dimensional objects to plot:



Assignment #4

You should do all of this within Spark. This dataset may or may not be too big for other resources you have access to, but I could certainly create one that would be intractable without a large cluster. So work as though it is.

You will be expected to navigate the Spark documentation yourself. It is complete and comprehensive. There are also plenty of other online Spark resources.

Here are a few hints:

- We have discussed that the Spark DataFrame and RDD APIs are not always the same. Some provide functionality that the other version does not. You should be prepared to switch between them. For example, if one version of PCA does not provide the capability that you require, look at the other.
- DataFrames will require that you pay close attention to the datatypes required. You may find yourself having to use some of the more formal machine learning types here. This may seem like painful meticulousness, but these types and terminology are common to machine learning everywhere, so this is generally useful experience.
- There is a 2D dataset in the directory called `simple_2D_example.dat` that you can use to test your methods. I have put an image of that dataset there for your convenience, `simple_2D_example.png`.

Assignment #4

- When you save data using the convenient `saveAsTextFile` method with RDDs, you may find that your data has ended up not in the single file you hoped, but in several pieces in a directory. This is actually desirable behavior for enormous distributed files, but a little annoying here. If you use the `rdd.repartition(1)` transform on the RDD before you save it, it will keep your file in one piece (although still named something like "part-00000" in the named subdirectory).
- If you are using PCA to analyze the structure of data, it helps to normalize the data about 0. In the case of our spatial structures, you may want to recenter them to the origin before using any kind of PCA.
- As most of you are new to PCA, I will suggest that you will find more useful information applying it to any interesting objects you find *separately*. If you apply it to the whole dataset, you get results for all the pieces lumped together. And you may want to consider my previous hint with this as well.

Assignment #4

This is a new, and hopefully exciting, exercise. It is possible to extract all of the meaningful structure using the tools you have. Perhaps some of you will. We will certainly grade on a kind curve, and be flexible about what you report.

- You should submit a summary of what you have found. It could be a short paragraph, or a brief table, maybe even some matplotlib. Perhaps the answer is "There was one 2D square, centered at 20,20,20,20,20,20 with edge length 5", and that is all you have to say. Or maybe you found:

Object	Location	Size	Orientation	Points
6D Sphere	12,12,23,12,11,16	6	NA	155
Square	23,33,12,78,55,33	22	?	2000

- You must also include the Spark script that you used. Use RDD's. Do not write an unscalable generic python algorithm.
- It should be one script that I can run myself. It should read in the datafile and spit out all of the data that you used for your summary, like a serious production code. Inline comments are welcome.

You may ask questions at any point before this is due. I will not give you spoilers, but I will clarify anything I can, and give you general suggestions. If I feel the need to disclose anything particularly useful, I will make it an Announcement.

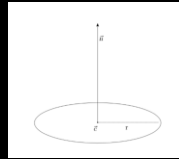
Assignment #4

Orientation?

This is an "optional" detail that you might explore with the objects you find. It is not essential. But, as scientists you should at least be thinking about how this concept applies here.

In 2D, you might specify the orientation of a square by the angle of one edge to some axis. Does that work for a disc? So, does that mean that a disc has no orientation?

How about in 3D. Does a disc have an orientation now?



Of the various methods that can be used to describe 3D orientations (Euler angles, quaternions, or my favorite: rotors from geometric algebra), we might pick a normal to a surface (and maybe a rotation about that normal, if necessary, for something like a square).

Does this work for a 3D ball (which we call a "3-ball" in this context, and it's surface is a "2-sphere"; our disc is a "2-ball")?

How about our 3-ball in 4 dimensions? Does it have an orientation now?

Think about how these concepts generalize to higher dimensions. Once you get it, it all seems pretty elegant!

Assignment #4

This is due anytime before we review answers in class on November 11th. Once we review, I can give no credit.

This is 15 points of your final grade.

The 15% of the final grade, and the due date, reflect the effort expected. This is not a good one to wait until the night before.

The open-ended and exploratory nature of this assignment is intended to be interesting. However, I recognize that is isn't fair to not know when you are "done". So I will bend my rule of determining grading only after I have reviewed the submissions to include this benchmark:

- | | |
|----|--|
| 13 | Distinguish <u>all</u> objects by some <u>structural</u> difference between them. Can be anything. |
| 14 | Describe all objects meaningfully. What are their shapes, etc. |
| 15 | TBD. There are plenty of extra possibilities here. |

There is all kinds of extra credit you might find, for example by noticing the orientation of some shape. So, even if you miss something you might make up for it by finding something else.

Mid-Term Exam

- Two parts: a closed-book paper exam, followed by an online programming problem.
- Total time is 80 minutes, but you can divide it up as you see fit. Turn in your paper exam and proceed to the online as you will. My intention is not to make this a time trial.
- Material will be drawn from everything we have covered thus far, but I am not out to get you with trivia from lecture modules. If you have done the assignments, you should be in pretty good shape to just take it right now.
- Show up on time!
- Bring your laptop and a pencil, or your finest quill.